

Алгоритмы машинного обучения

Вадим Смоляков



MANNING



Machine Learning Algorithms in Depth

VADIM SMOLYAKOV



MANNING
SHELTER ISLAND

Алгоритмы машинного обучения

ВАДИМ СМОЛЯКОВ



Санкт-Петербург • Москва • Минск

2026

ББК 32.813+ 32.973.2-018
УДК 004.85+ 004.021
С51

Смоляков Вадим

С51 Алгоритмы машинного обучения. — СПб.: Питер, 2026. — 336 с.: ил. — (Серия «Библиотека программиста»).

ISBN 978-5-4461-4257-6

Узнайте тонкости работы алгоритмов ML, чтобы эффективно решать задачи и повышать производительность используемых моделей. Познакомьтесь с фундаментальными математическими основами важнейших алгоритмов машинного обучения и вариантами их реализации на Python. Особое внимание уделяется вероятностным методам.

В книге анализируются и объясняются десятки алгоритмов, применяемых в различных сферах, в частности финансах, компьютерном зрении и обработке естественного языка. Каждый алгоритм сначала выводится математически, а потом иллюстрируется кодом на Python, снабженным подробными пояснениями и информативными графиками. Особую ценность представляет данная автором ясная интерпретация байесовских алгоритмов для моделей Монте-Карло и марковских цепей.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.813+ 32.973.2-018
УДК 004.85+ 004.021

Права на издание получены по соглашению с Manning Publications. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 978-1633439214 англ.

ISBN 978-5-4461-4257-6

Authorized translation of the English edition © 2024 Manning Publications.

This translation is published and sold by permission of Manning

Publications, the owner of all rights to publish and sell the same.

© Перевод на русский язык ООО «Прогресс книга», 2025

© Издание на русском языке, оформление ООО «Прогресс книга», 2025

© Серия «Библиотека программиста», 2025

Краткое содержание

Часть 1. Введение в алгоритмы машинного обучения

Глава 1. Алгоритмы машинного обучения.....	24
Глава 2. Марковские цепи Монте-Карло	37
Глава 3. Вариационный вывод	68
Глава 4. Программная реализация.....	83

Часть 2. Обучение с учителем

Глава 5. Алгоритмы классификации.....	98
Глава 6. Алгоритмы регрессии	132
Глава 7. Избранные алгоритмы обучения с учителем	149

Часть 3. Обучение без учителя

Глава 8. Основные алгоритмы обучения без учителя.....	194
Глава 9. Избранные алгоритмы обучения без учителя	215

Часть 4. Глубокое обучение

Глава 10. Фундаментальные алгоритмы глубокого обучения.....	248
Глава 11. Передовые алгоритмы глубокого обучения	281
Приложение А. Дополнительные материалы и веб-ресурсы	316
Приложение Б. Ответы на упражнения.....	320

Оглавление

От издательства.....	12
О научном редакторе русского издания	12
Предисловие	13
Благодарности	14
О книге	16
Для кого эта книга	16
Структура книги	17
О коде в книге	19
Форум liveBook	19
Об авторе.....	21
Иллюстрация на обложке.....	22

Часть 1. Введение в алгоритмы машинного обучения

Глава 1. Алгоритмы машинного обучения	24
1.1. Типы алгоритмов машинного обучения.....	25
1.2. Зачем изучать алгоритмы с нуля?	28
1.3. Математические основы	29
1.4. Байесовский вывод и глубокое обучение	30
1.4.1. Два основных направления байесовского вывода: МСМС и VI.....	32
1.4.2. Современные алгоритмы глубокого обучения	34

1.5. Реализация алгоритмов	34
1.5.1. Структуры данных.....	35
1.5.2. Парадигмы решения задач	35
Итоги.....	36
Глава 2. Марковские цепи Монте-Карло	37
2.1. Введение в марковские цепи Монте-Карло	38
2.1.1. Апостериорное распределение для подбрасываний монеты.....	39
2.1.2. Цепи Маркова для ранжирования веб-страниц	41
2.2. Оценка числа «пи».....	42
2.3. Модель биномиального дерева.....	45
2.4. Самоизбегающее случайное блуждание	48
2.5. Семплирование по Гиббсу.....	52
2.6. Алгоритм Метрополиса — Гастингса	56
2.7. Выборка по значимости	60
2.8. Упражнения.....	66
Итоги.....	67
Глава 3. Вариационный вывод	68
3.1. Вариационный вывод KL.....	69
3.2. Приближение среднего поля.....	72
3.3. Удаление шума на изображении в модели Изинга.....	74
3.4. Максимизация взаимной информации	81
3.5. Упражнения.....	82
Итоги.....	82
Глава 4. Программная реализация	83
4.1. Структуры данных.....	83
4.1.1. Линейные структуры данных	84
4.1.2. Нелинейные структуры данных	85
4.1.3. Вероятностные структуры данных	86
4.2. Парадигмы решения задач	87
4.2.1. Полный поиск.....	87
4.2.2. Жадный алгоритм	88
4.2.3. Разделяй и властвуй.....	90
4.2.4. Динамическое программирование	91

4.3. Исследования в области ML: методы семплирования и вариационный вывод	93
4.4. Упражнения.....	95
Итоги.....	96

Часть 2. Обучение с учителем

Глава 5. Алгоритмы классификации 98

5.1. Введение в задачу классификации.....	99
5.2. Перцептрон.....	100
5.3. Метод опорных векторов.....	106
5.4. Логистическая регрессия	112
5.5. Наивный байесовский классификатор.....	119
5.6. Дерево решений (CART).....	125
5.7. Упражнения.....	131
Итоги.....	131

Глава 6. Алгоритмы регрессии132

6.1. Введение в регрессию	133
6.2. Байесовская линейная регрессия	133
6.3. Иерархическая байесовская регрессия.....	137
6.4. Регрессия K ближайших соседей.....	141
6.5. Регрессия на основе гауссовского процесса	143
6.6. Упражнения.....	148
Итоги.....	148

Глава 7. Избранные алгоритмы обучения с учителем.....149

7.1. Марковские модели.....	150
7.1.1. Алгоритм ранжирования страниц.....	151
7.1.2. Скрытые марковские модели.....	154
7.2. Обучение на несбалансированных данных.....	160
7.2.1. Стратегии уменьшения выборки.....	161
7.2.2. Стратегии увеличения выборки	163
7.3. Активное обучение	166
7.3.1. Стратегии запроса.....	167

7.4. Выбор модели: настройка гиперпараметров.....	175
7.4.1. Байесовская оптимизация	176
7.5. Ансамблевые методы	179
7.5.1. Бэггинг	179
7.5.2. Бустинг	183
7.5.3. Стекинг	186
7.6. Исследования в области ML: алгоритмы обучения с учителем	189
7.7. Упражнения.....	191
Итоги.....	191

Часть 3. Обучение без учителя

Глава 8. Основные алгоритмы обучения без учителя	194
8.1. Метод К средних с процессом Дирихле.....	195
8.2. Модели гауссовой смеси.....	200
8.2.1. Алгоритм максимизации ожидания	201
8.3. Снижение размерности.....	208
8.3.1. Анализ главных компонент	208
8.3.2. Метод t-SNE-обучения на базе многообразий на примере изображений	211
8.4. Упражнения.....	214
Итоги.....	214
Глава 9. Избранные алгоритмы обучения без учителя	215
9.1. Латентное размещение Дирихле.....	216
9.1.1. Вариационный байесовский метод.....	217
9.2. Оценки плотности	224
9.2.1. Ядерная оценка плотности	225
9.2.2. Оптимизация тангенциального портфеля	227
9.3. Моделирование структуры.....	230
9.3.1. Алгоритм Чоу — Лю.....	231
9.3.2. Оценка обратной ковариационной матрицы.....	232
9.4. Метод имитации отжига.....	236
9.5. Генетический алгоритм	240
9.6. Исследования в области ML: обучение без учителя.....	243
9.7. Упражнения.....	245
Итоги.....	245

Часть 4. Глубокое обучение

Глава 10. Фундаментальные алгоритмы глубокого обучения	248
10.1. Многослойный перцептрон.....	249
10.2. Сверточные нейронные сети.....	254
10.2.1. LeNet на датасете MNIST	255
10.2.2. ResNet для поиска изображений	259
10.3. Рекуррентные нейронные сети	262
10.3.1. LSTM-классификация последовательностей.....	263
10.3.2. Модель с несколькими входами.....	267
10.4. Оптимизаторы нейронных сетей.....	273
10.5. Упражнения	279
Итоги.....	279
Глава 11. Передовые алгоритмы глубокого обучения	281
11.1. Автоэнкодеры.....	282
11.1.1. VAE: обнаружение аномалий во временных рядах	284
11.2. Амортизированный вариационный вывод	290
11.2.1. Сети смешанной плотности.....	290
11.3. Внимание и трансформеры	297
11.4. Графовые нейронные сети	305
11.5. Исследования в области ML: глубокое обучение.....	311
11.6. Упражнения	314
Итоги.....	314
Приложение А. Дополнительные материалы и веб-ресурсы.....	316
А.1. Спортивное программирование	316
А.2. Рекомендуемая литература.....	316
А.3. Научно-исследовательские конференции.....	318
Приложение Б. Ответы на упражнения	320

*Моим родителям, Сергею и Валерии,
за их постоянную любовь и поддержку.*

*Моей подруге Келли:
я тебя бесконечно люблю!*

От издательства

Мы выражаем огромную благодарность клубу рецензентов ИТ-литературы ReadIT Club за помощь в работе над русскоязычным изданием книги и вклад в повышение качества переводной литературы.

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

О научном редакторе русского издания

Валентина Бородина — руководитель проектов оптимизации и автоматизации бизнес-процессов в технологическом блоке крупного банка. Обладает опытом внедрения программных продуктов для коммерческого сегмента. Общий стаж работы в ИТ более 16 лет.

Предисловие

Я рад приветствовать читателя книги «Алгоритмы машинного обучения»! Идея ее написания появилась у меня во время учебы в аспирантуре. Тогда я сменил свою специальность с беспроводной связи на машинное обучение и обнаружил, что единственное, что осталось неизменным, — мое увлечение алгоритмами. Я хотел в мельчайших деталях изучить этот предмет и по-настоящему понять, как выводить формулы, реализовывать в коде и анализировать алгоритмы, отталкиваясь от базовых принципов. Мне посчастливилось найти место для своих исследований в группе Sensing, Learning, and Inference Group лаборатории Computer Science and Artificial Intelligence Laboratory (CSAIL) Массачусетского технологического института (MIT), где я познакомился с различными приложениями машинного обучения, в центре которых находился байесовский вывод. В то время технологии глубокого обучения находились в фазе бурного роста, и я погрузился в изучение и эксперименты с различными моделями нейронных сетей для компьютерного зрения и обработки естественного языка.

Меня всегда восхищали взаимодополняющие сильные стороны вероятностных графических моделей и моделей глубокого обучения, и я продумывал пути, как их можно было объединить. На протяжении всего моего обучения в аспирантуре я работал над различными задачами и в итоге создал библиотеку алгоритмов, реализованных с нуля. Я прочитал множество учебников по машинному обучению, а также выступил в роли научного редактора других подобных изданий. В результате я пришел к выводу, что существует пробел в литературе: в ней нет описания алгоритмов машинного обучения с нуля. Это был момент озарения, и он утвердил меня в моем намерении взяться за эту работу. Книга «Алгоритмы машинного обучения» поможет читателю пройти путь от вывода математических формул до программной реализации наиболее интересных алгоритмов. Цель, которую я преследовал при написании этой книги, заключалась в том, чтобы изложить научные основы машинного обучения так, чтобы передать интуитивное понимание и вдохновить читателя на самостоятельное изучение, инновации и развитие в этой области. Благодарю вас за проявленный интерес и добро пожаловать в мир алгоритмов машинного обучения!

Благодарности

Я хочу поблагодарить сотрудников издательства Manning, сделавших возможным выпуск этой книги: издателя Марджана Бейса (Marjan Bace) и всех специалистов редакторского и технического отделов, в том числе Патрика Барба (Patrick Barb), Элешу Хайд (Elesha Hyde) и многих других, чья работа осталась за кадром.

Выражаю огромную благодарность команде рецензентов под руководством Александра Драгосавлевича (Aleksandar Dragosavljević). Вот их имена: Абхилаш Бабу (Abhilash Babu), Аллан Макура (Allan Makura), Ариэль Гаминьо (Ariel Gamiño), Бин Ху (Bin Hu), Кристиан Саттон (Christian Sutton), Фернандо Гарсиа Седано (Fernando García Sedano), Харш Раваль (Harsh Raval), Джеймс Дж. Байлецки (James J. Byleckie), Джапнит Сингх (Japneet Singh), Джонни Хопкинс (Johnny Hopkins), Джордан Самек (Jordan Samek), Кай Геллиен (Kai Gellien), Кхай Вин (Khai Win), Кумар Абхишек (Kumar Abhishek), Мадхав Айягари (Madhav Ayiyagari), Мария Ана (Maria Ana), Марвин Шварце (Marvin Schwarze), Максим Волгин (Maxim Volgin), Мортеза Киади (Morteza Kiadi), Ор Голан (Or Golan), Рави Киран Бамиди (Ravi Kiran Bamidi), Садхана Ганапатираджу (Sadhana Ganapathiraju), Санкет Наик (Sanket Naik), Шреяс Б. Г. (Shreyas B. G.), Сулейман Саламе (Sleiman Salameh), Шрирам Мачарла (Sriram Macharla), Сумит Бхаттачарья (Sumit Bhattacharyya), Ватсал Десаи (Vatsal Desai) и Уолтер Александр Мата Лопес (Walter Alexander Mata Lopez), а также участникам форума. Рецензенты предоставляли отзывы по каждому разделу — от содержания до рисунков и реализации в коде — и сыграли важную роль в окончательном формировании рукописи.

Что касается содержательной стороны книги, то я выражаю особую благодарность Джунпену Лао (Junpeng Lao) — научному редактору. Джунпен Лао — старший аналитик данных в Google. Он получил степень PhD и как постдок работал в области когнитивной нейронауки. Большую часть своей университетской жизни он занимался бутстрэпом и перестановочными тестами, но потом увлекся байесовской статистикой и генеративным моделированием. Он также является

одним из основных разработчиков библиотек Blackjax, PyMC и TensorFlow Probability. Большое спасибо Хубину Кейо (Hubin Keio), который отредактировал эту книгу. В процессе работы я был по-настоящему впечатлен его вниманием к деталям и полезностью его отзывов.

Наконец, я хотел бы выразить благодарность моей подруге Келли за то, что она оказывала мне большую поддержку на протяжении всего времени создания этой книги.

О книге

Эта книга посвящена разработке алгоритмов машинного обучения (ML) с нуля и позволит вам развить математическую интуицию на материале классических и современных алгоритмов ML, изучать основы байесовского вывода и глубокого обучения, а также познакомиться со структурами данных и алгоритмическими парадигмами ML.

Глубокое понимание алгоритмов ML поможет вам в выборе правильных подходов для решения задач, объяснении полученных результатов, устранении сложных проблем, применении алгоритмов в новых сферах деятельности и повышении производительности существующих алгоритмов.

Что выделяет книгу из ряда подобных, так это анализ с чистого листа, в процессе которого достаточно подробно обсуждается, как и почему работают алгоритмы ML. Книга также содержит тщательно подобранный набор алгоритмов, которые я счел наиболее полезными и эффективными, когда изучал ML в аспирантуре. Здесь вы также найдете подробный вывод математических формул и реализацию алгоритмов ML с объяснениями, а также некоторые темы, не так часто встречающиеся в других книгах по ML.

После прочтения этой книги у вас появится сильная математическая интуиция в отношении классических и современных алгоритмов ML при обучении как с учителем, так и без учителя, а также опыт в сферах базового ML, обработки естественного языка, компьютерного зрения, оптимизации, вычислительной биологии и финансов.

Для кого эта книга

Эта книга предназначена для всех, кто хочет углубиться в изучение алгоритмов машинного обучения. Ее смогут оценить по достоинству самые разные категории читателей, в частности следующие:

- начинающие аналитики данных;
- аналитики данных всех уровней — от начального до продвинутого, включая стратегический;
- разработчики программного обеспечения, желающие развиваться в области анализа данных;
- дата-инженеры, стремящиеся углубить свои знания в области моделей ML;
- аспиранты, в круг научных интересов которых входит ML;
- студенты старших курсов, интересующиеся ML.

Для понимания материала этой книги требуется базовый уровень навыков программирования на Python, а также средний уровень понимания линейной алгебры, прикладной теории вероятностей и многомерного математического анализа.

Структура книги

Эта книга состоит из четырех частей. Если вы начинающий специалист в этой области, то рекомендуется читать главы в порядке нумерации. Однако если у вас имеется некоторый опыт, то можете смело переходить к конкретному алгоритму. Каждая глава завершается серией упражнений, которые помогут вам попрактиковаться с разобранными в ней инструментами. Приложение Б содержит ответы на все упражнения. Кроме того, в конце каждой части есть раздел, посвященный исследованиям в области машинного обучения, цель которого — ознакомить читателя с последними достижениями в этой быстро меняющейся области.

В части 1 на примере различных типов алгоритмов ML описываются базовые принципы их реализации, а также представлены два основных метода байесовского вывода: марковская цепь Монте-Карло и вариационный вывод.

- Глава 1 представляет собой введение в байесовский вывод и глубокое обучение, а также алгоритмические парадигмы и структуры данных, используемые в программной реализации алгоритмов машинного обучения.
- Глава 2 знакомит читателя с ключевыми байесовскими концепциями и обосновывает применение марковских цепей Монте-Карло на ряде примеров, начиная с оценки курса акций и заканчивая семплированием из многомерной гауссовой смеси по методу Метрополиса — Гастингса.
- Глава 3 посвящена вариационному выводу и, в частности, аппроксимации среднего поля для устранения шума на изображениях в модели Изинга.
- В главе 4 рассматриваются линейные, нелинейные и вероятностные структуры данных, а также следующие четыре алгоритмические парадигмы:

полный поиск, жадные алгоритмы, «разделяй и властвуй» и динамическое программирование.

Часть 2 посвящена алгоритмам обучения с учителем, которые имеют дело с размеченными примерами как частью обучающего набора данных и делятся на два основных класса: алгоритмы классификации и алгоритмы регрессии.

- Глава 5 фокусируется на алгоритмах классификации. Вы познакомитесь с выводом формул для нескольких классических алгоритмов, в том числе перцептрона, SVM, логистической регрессии, наивного байесовского алгоритма и деревьев решений.
- В центре внимания главы 6 — четыре увлекательных алгоритма регрессии: байесовская линейная регрессия, иерархическая байесовская регрессия, KNN-регрессия и регрессия на основе гауссовского процесса.
- В главе 7 представлены избранные алгоритмы обучения с учителем, в том числе марковские модели, такие как алгоритмы ранжирования страниц и скрытые марковские модели; стратегии обучения на несбалансированных данных; активное обучение; байесовская оптимизация для выбора гиперпараметров; а также ансамблевые методы.

В части 3 рассматриваются алгоритмы обучения без учителя, которое происходит, когда отсутствуют обучающие метки. В этом случае задачей часто является выявление закономерностей в данных и обучение представлением данных.

- Глава 8 начинается с рассмотрения байесовского непараметрического расширения алгоритма К средних, за которым следует ЕМ-алгоритм для моделей гауссовой смеси. Далее обсуждаются два отдельных метода уменьшения размерности, а именно PCA и t-SNE, в применении к обучению на базе многообразий изображений.
- В главе 9 продолжается обсуждение избранных алгоритмов обучения без учителя. Мы начинаем с рассмотрения латентного размещения Дирихле для обучения тематических моделей, далее переходим к оценке плотности и алгоритмам моделирования структуры данных, а завершаем методом имитации отжига и генетическими алгоритмами.

В части 4 рассматриваются алгоритмы глубокого обучения, которые произвели революцию в отрасли машинного обучения и позволили распространить использование ML на прежде считавшиеся неподвластными классическим алгоритмам исследовательские и бизнес-задачи.

- Глава 10 начинается с изложения основ алгоритмов глубокого обучения, таких как многослойный перцептрон и сверточная модель LeNet для классификации цифр MNIST, за которыми следуют более продвинутые задачи, например поиск изображений на базе сверточной нейронной сети ResNet50.

Вы погрузитесь в мир рекуррентных нейронных сетей, применяемых для классификации последовательностей с использованием LSTM, и познакомитесь с полноценной реализацией модели с несколькими входами для определения сходства последовательностей. Завершается глава сравнительным анализом различных алгоритмов оптимизации, используемых для обучения глубоких нейронных сетей.

- В главе 11 представлены более сложные алгоритмы глубокого обучения. Вы изучите генеративные модели, построенные на базе вариационных автоэнкодеров, и познакомитесь с полной реализацией детектора аномалий для данных временных рядов. Помимо этого, вы узнаете об интригующей комбинации нейронных сетей и вероятностных графовых моделей, а также о реализации с нуля сети смешанной плотности. Далее мы обсудим мощную архитектуру трансформера и применим ее к задаче классификации текстов. Наконец, мы рассмотрим графовые нейронные сети и используем одну из них для классификации узлов в графе цитирования.

О коде в книге

Книга содержит множество примеров исходного кода как в нумерованных листингах, так и в тексте. В обоих случаях исходный код форматируется **моноширинным шрифтом**, в отличие от обычного текста.

Во многих случаях оригинальная версия исходного кода переформатируется; добавляются разрывы строк и измененные отступы, чтобы код помещался на странице. Иногда даже этого оказывается недостаточно, и в листинги включаются маркеры продолжения строк (`>>>`). Также из исходного кода часто удаляются комментарии, если код описывается в тексте. Многие листинги снабжены примечаниями, выделяющими важные концепции.

Исполняемые фрагменты кода можно загрузить из версии liveBook (электронной) по адресу <https://livebook.manning.com/book/machine-learning-algorithms-in-depth>. Полный код примеров книги доступен для загрузки на сайте издательства Manning по адресам <https://www.manning.com/books/machine-learning-algorithms-in-depth> и https://github.com/vsmolyakov/ml_algo_in_depth.

Форум liveBook

Приобретая книгу «Алгоритмы машинного обучения», вы также получаете бесплатный доступ к платформе для онлайн-чтения liveBook издательства Manning (на английском языке). Эксклюзивные возможности liveBook позволяют оставлять комментарии как к книге в целом, так и к отдельным ее разделам или абзацам. Можно легко делать заметки для себя, задавать технические вопросы и отвечать на них, а также получать помощь от автора и других пользователей. Чтобы получить доступ к форуму, посетите страницу

<https://livebook.manning.com/book/machine-learning-algorithms-in-depth/discussion>. Информацию о форумах Manning и правилах поведения на них см. на <https://livebook.manning.com/discussion>.

В рамках своих обязательств издательство Manning предоставляет ресурс для содержательного общения читателей и автора. Это не подразумевает конкретную степень участия автора, которое остается добровольным (и неоплачиваемым). Задавайте автору хорошие вопросы, чтобы ему было интересно участвовать в диалоге! Форум и архивы обсуждений доступны на сайте Manning, пока книга продолжает издаваться.

Об авторе

Вадим Смоляков — data scientist научно-исследовательской группы Enterprise & Security DI компании Microsoft. Он окончил аспирантуру кафедры искусственного интеллекта лаборатории CSAIL Массачусетского технологического института. Его научные интересы связаны с байесовским выводом и глубоким обучением. До прихода в Microsoft Вадим разрабатывал методы решения задач с помощью машинного обучения в сфере электронной коммерции. На своей нынешней должности он занимается промышленной разработкой продуктов с использованием искусственного интеллекта и руководит несколькими подразделениями.

Иллюстрация на обложке

Иллюстрация под названием «Maraîchere» (что означает «огородница»), помещенная на обложку книги «Алгоритмы машинного обучения», взята из книги Луи Кюрмэ (Louis Curmer), опубликованной в 1841 году. Каждая иллюстрация этой книги тщательно прорисована и раскрашена от руки.

В прежние времена по одежде человека можно было легко определить, где он живет и какова его профессия или социальное положение. Издательство Manning приветствует изобретательность и инициативность — качества, присущие индустрии IT, — и в знак этого размещает на обложках изображения, которые демонстрируют богатое разнообразие региональных культур, запечатленное на старинных рисунках.

Часть 1

Введение в алгоритмы машинного обучения

Мы рады приветствовать читателя книги «Алгоритмы машинного обучения»! В первой части мы обсудим различные типы алгоритмов ML, подробно объясним их реализацию исходя из базовых принципов и представим два основных метода байесовского вывода: марковские цепи Монте-Карло и вариационный вывод.

В главе 1 мы поговорим о причинах, по которым стоит изучать алгоритмы ML с нуля, познакомимся с байесовским логическим выводом и глубоким обучением, а также обсудим алгоритмические парадигмы и структуры данных, используемые при программной реализации алгоритмов машинного обучения.

В главе 2 я познакомлю вас с ключевыми байесовскими концепциями и обосную использование марковских цепей Монте-Карло на нескольких примерах — от оценки курса акций до семплирования Метрополиса — Гастингса из многомерных гауссовых смесей.

В главе 3 мы сосредоточимся на вариационном выводе и, в частности, на аппроксимации среднего поля, применяемой для удаления шума с изображений в модели Изинга. Вы научитесь аппроксимировать полное апостериорное распределение, используя дивергенцию Кульбака — Лейблера, и максимизировать нижнюю вариационную границу.

В главе 4 мы обсудим линейные, нелинейные и вероятностные структуры данных, а также четыре алгоритмические парадигмы: полный поиск, жадные алгоритмы, «разделяй и властвуй» и динамическое программирование. Мы рассмотрим несколько примеров из каждой области и завершим главу обзором исследований ML в области методов семплирования и вариационного вывода.

1

Алгоритмы машинного обучения

В этой главе

- ✓ Типы алгоритмов машинного обучения
- ✓ Важность изучения алгоритмов с нуля
- ✓ Введение в байесовский вывод и глубокое обучение
- ✓ Программная реализация алгоритмов машинного обучения с нуля

Алгоритм — это последовательность шагов, необходимых для выполнения определенной задачи. Он получает входные данные, выполняет последовательность операций и выдает желаемый результат. Простейшим примером алгоритма является *сортировка* списка целых чисел. После выполнения набора операций мы получаем отсортированный список. Информация в таком списке приобретает более упорядоченный вид, и в ней становится легче находить ответы на интересующие нас вопросы.

Обычно алгоритм, в зависимости от размера входных данных n , характеризуется двумя показателями: тем, как быстро он работает (временная сложность алгоритма), и тем, сколько памяти ему требуется (пространственная сложность алгоритма). Например, сортировка на основе сравнения, как мы увидим позже, обладает временной сложностью $O(n \log n)$ и требует памяти $O(n)$.

Существует множество способов провести сортировку, но в каждом случае автор алгоритма, если он придерживается классической парадигмы, перечисляет набор инструкций. Представьте себе ситуацию, в которой вы можете *выучить*

эти инструкции, используя ряд доступных вам примеров входных и выходных данных. Именно так выглядит структура алгоритмической парадигмы машинного обучения. Это подобно тому, как учится наш мозг, когда мы играем в игру «соедини по точкам» или рисуем пейзаж: мы заполняем пропуски, сравнивая на каждом шаге желаемое и имеющееся. Так в общих чертах работают алгоритмы ML с учителем (supervised). В процессе обучения алгоритмы ML выучивают правила (например, границы классификации) на основе обучающих примеров путем оптимизации целевой функции (objective function). Во время тестирования алгоритмы ML применяют ранее выученные правила к новым входным данным и выдают прогноз, как показано на рис. 1.1.

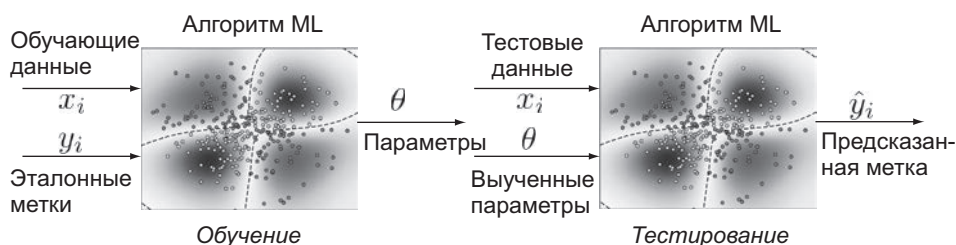


Рис. 1.1. Обучение с учителем: обучение (слева) и тестирование (справа)

1.1. Типы алгоритмов машинного обучения

Давайте немного раскроем смысл предыдущего абзаца и введем некоторые обозначения. Эта книга посвящена алгоритмам ML, которые можно сгруппировать по следующим категориям: обучение с учителем (supervised), обучение без учителя (unsupervised) и глубокое обучение (deep learning). При *обучении с учителем* задача состоит в том, чтобы получить отображение f входных данных x на выходные y при заданном обучающем наборе данных $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$, состоящем из n пар вход-выход. Другими словами, нам дается n примеров того, как должны выглядеть выходные данные при заданных входных. Выходное значение y также часто называют *меткой* (label). Оно представляет собой контрольный сигнал, который сообщает нашему алгоритму, каков правильный ответ.

Обучение с учителем, в зависимости от той величины, которую мы пытаемся предсказать, в свою очередь, можно разделить на *классификацию* и *регрессию*. Если наше предсказание y является дискретной величиной (например, K различных классов), то перед нами стоит задача классификации. В то же время если наше выходное значение y — непрерывная величина (к примеру, вещественное число вроде курса акций), то мы сталкиваемся с проблемой регрессии.

Таким образом, характер задачи меняется в зависимости от величины y , которую мы пытаемся предсказать. В любом случае мы хотим максимально приблизиться к эталонному (ground truth) значению y .

Обычно для оценки эффективности или близости к эталону используется *функция потерь* (loss function). Она позволяет рассчитать расстояние между предсказанной и истинной метками. Пусть $y = f(x; \theta)$ — наш алгоритм ML, который отображает входные примеры x на выходные метки y , параметризуемый θ , где θ содержит все требуемые параметры нашего алгоритма.

Теперь мы можем задать функцию потерь для задачи классификации, равную количеству неправильно классифицированных случаев, как показано в формуле 1.1:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n 1[y_i \neq f(x_i; \theta)]. \quad (1.1)$$

Здесь $1[]$ — это индикаторная функция, которая принимает значение 1, когда аргумент внутри скобок истинен, и 0 в противном случае. Выражение в формуле 1.1 означает, что мы суммируем все случаи, в которых наше предсказание $f(x_i; \theta)$ не соответствовало эталонной метке y_i , и делим сумму на общее количество примеров n . Иначе говоря, мы вычисляем среднюю долю случаев ошибочной классификации. Наша цель — минимизировать функцию потерь (то есть найти такой набор параметров θ , при котором коэффициент ошибочной классификации будет как можно ближе к нулю). Обратите внимание, что для задачи классификации существует множество альтернативных функций потерь, таких как кросс-энтропия, которые мы рассмотрим в последующих главах.

Для непрерывных меток, или выходных переменных, обычно в качестве функции потерь используют *среднеквадратичное отклонение* (mean squared error, MSE). Оно определяет, насколько наша оценка далека от эталона, как показано в формуле 1.2:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n [y_i - f(x_i; \theta)]^2. \quad (1.2)$$

Как видно из формулы, мы вычитаем наше предсказание из истинной метки, возводим разность в квадрат и усредняем результат по всем значениям. Возводя в квадрат, мы избавляемся от отрицательных значений и выбраковываем случаи с большим отклонением от эталона.

Одной из главных задач ML является умение делать обобщение примеров, которые не встречались до этого. Мы хотим добиться высокой доли верных результатов (ассигасу), то есть низких потерь, не только для обучающих данных (уже размеченных), но и для новых незнакомых тестовых примеров. Именно эта способность к обобщению делает ML таким привлекательным: если мы сможем разработать алгоритмы ML, которые смогут выходить за рамки своей обучающей программы, то мы станем на шаг ближе к общему (близкому к человеческому) искусственному интеллекту (artificial general intelligence, AGI).

При *обучении без учителя* не только отсутствуют метки y , но перед нами даже не стоит задача нахождения соответствия между входными и выходными примерами;

вместо этого мы хотим разобраться в самих данных, то есть обнаружить в них закономерности, а это проще сделать, если спроецировать многомерные данные в пространство меньшей размерности, как показано на рис. 1.2. Таким образом, в случае обучения без учителя набор обучающих данных состоит из n входных примеров без каких-либо меток y : $D = \{x_1, \dots, x_n\}$. Простейший пример обучения без учителя — это поиск кластеров в данных. Интуиция нам подсказывает, что точки данных, которые принадлежат к одному и тому же кластеру, имеют сходные характеристики. И действительно, центр кластера может выступать как типичный представитель точек этого кластера, заменяя их, что используется в алгоритмах сжатия. С другой стороны, анализ расстояния между кластерами в пространстве меньшей размерности может помочь нам оценить взаимосвязь между различными группами. Кроме того, точка, удаленная от всех существующих кластеров, может рассматриваться как аномалия, что подводит к алгоритму обнаружения аномалий. Как можно заметить, обучение без учителя имеет бесконечное число интересных вариантов использования, и на всем протяжении этой книги самые захватывающие алгоритмы из этой серии мы будем разбирать с нуля.

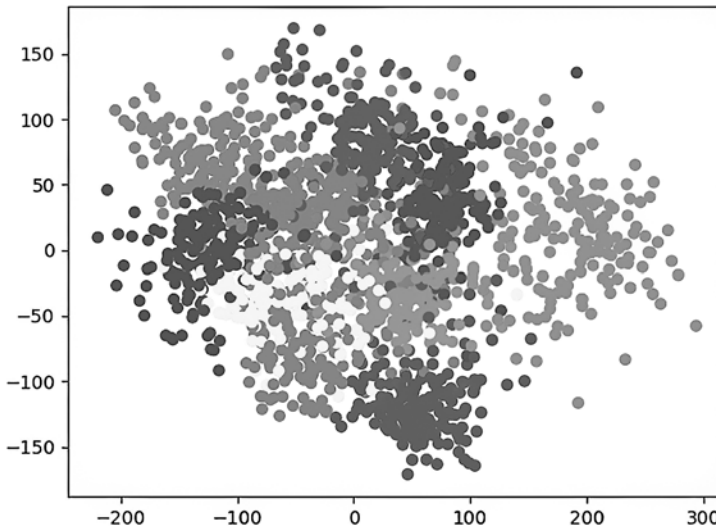


Рис. 1.2. Обучение без учителя: кластеры точек данных, спроецированные в двумерное пространство

Еще одной очень важной областью применения современных алгоритмов ML является *глубокое обучение*. Название происходит от стека вычислительных уровней, вместе образующих вычислительный граф. Глубина этого графа относится к последовательным вычислениям, а ширина — к параллельным.

Как мы увидим в дальнейшем, модели глубокого обучения постепенно уточняют свои параметры с помощью алгоритма обратного распространения ошибки

(back-propagation) до тех пор, пока не достигнут целевой функции. Такие модели получили широкое распространение в отрасли благодаря своей способности решать сложные задачи с высокой долей верных результатов. В качестве примера на рис. 1.3 изображена архитектура глубокого обучения для анализа тональности текста (sentiment analysis). В следующих главах мы узнаем больше о том, что представляет собой каждый из ее блоков.

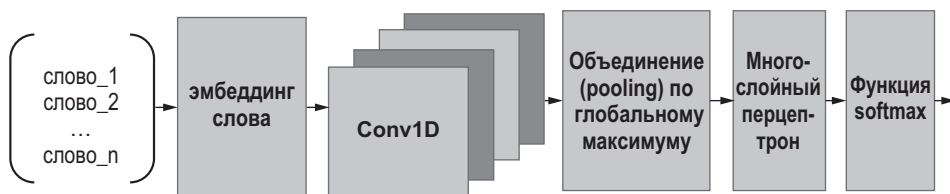


Рис. 1.3. Архитектура глубокой нейронной сети (DNN) для анализа тональности текста

Глубокое обучение — стремительно развивающаяся область исследований, которой в данной книге уделяется особое внимание. Например, при обучении с самоконтролем (self-supervised learning), используемым в трансформерных моделях, мы задействуем контекст и структуру естественного языка в качестве контрольного сигнала, тем самым извлекая метки из самих данных. В дополнение к классическим сферам применения глубокого обучения, таким как обработка естественного языка (natural language processing, NLP) и компьютерное зрение (computer vision, CV), мы также обсудим генеративные модели, научимся прогнозировать на основе данных временных рядов и работать с реляционными графами.

1.2. Зачем изучать алгоритмы с нуля?

Глубокое понимание алгоритмов ML даст читателю несколько ценных преимуществ. Во-первых, вы сможете подбирать правильный подход для решения поставленных задач. Зная внутреннее устройство алгоритма, вы сумеете понять его недостатки, допущения, сделанные при его создании, а также преимущества в различных сценариях обработки данных. Такой навык позволит вам взвешенно подходить к выбору правильного решения задачи и экономить время, исключая заведомо неподходящие методы.

Во-вторых, сможете объяснить результаты работы данного алгоритма заинтересованным сторонам. Умение разъяснять и представлять результаты аудитории в производственных или академических кругах — важный навык специалиста по ML.

В-третьих, будете использовать интуицию, развитую при чтении этой книги, для решения сложных задач ML. Для того чтобы разбить подобную задачу на более

мелкие части и понять, где таится корень проблемы, часто требуется глубокое понимание фундаментальных основ и алгоритмическая проницательность. Эта книга научит читателя создавать работающие образцы с минимальным набором функций, а также на основе существующих алгоритмов разрабатывать и уметь отлаживать более сложные модели.

В-четвертых, сможете совершенствовать алгоритмы, чтобы они соответствовали изменяющимся условиям, в частности, в ситуации, когда формулы из учебника или библиотеки не могут быть применены в исходном виде. Глубокое понимание темы, которое вы разовьете, читая эту книгу, поможет модифицировать существующие алгоритмы в соответствии с вашими потребностями.

И наконец, часто возникает необходимость повысить производительность имеющихся моделей. Концепции, рассматриваемые в книге, позволят читателю сделать это. Таким образом, освоение алгоритмов ML с нуля поможет вам выбирать правильный вариант для решения поставленной задачи, объяснять получаемые результаты, решать запутанные проблемы, развивать методологию и повышать производительность существующих моделей.

1.3. Математические основы

Прежде чем приступать к основательному изучению алгоритмов ML, будет полезным повторить основные положения прикладной теории вероятностей, математического анализа и линейной алгебры. Хороший обзор теории вероятностей содержится в книге Димитриса Берцекаса (Dimitri Bertsekas) и Джона Цициклиса (John Tsitsiklis) «Introduction to Probability» (Athena Scientific, 2002). Читатель должен быть знаком с непрерывными и дискретными случайными величинами, условными и частными распределениями, правилом Байеса, цепями Маркова и предельными теоремами.

Для повторения положений математического анализа я рекомендую книгу Джеймса Стюарта (James Stewart) «Calculus» (Thomson Brooks / Cole, 2007). Предполагается, что читатель владеет правилами дифференцирования и интегрирования, имеет представление о последовательностях и рядах, векторах и стереометрии, частных производных и многомерных интегралах.

И наконец, книга Гилберта Стрэнга (Gilbert Strang) «Introduction to Linear Algebra» (Wellesley-Cambridge Press, 2016) служит отличным введением в линейную алгебру. От читателя требуется знакомство с векторными пространствами, матрицами, собственными значениями и собственными векторами, матричными нормами и факторизациями, положительно определенными и полуопределенными матрицами, а также матричным исчислением. В дополнение к вышеупомянутым книгам, пожалуйста, ознакомьтесь с приложением А, где содержится список рекомендуемых источников, способных углубить ваше понимание алгоритмов, представленных в этой книге.

1.4. Байесовский вывод и глубокое обучение

Байесовский вывод (Bayesian inference) дает возможность обновить наши представления о мире на основе наблюдений. В нашем сознании существует множество ментальных моделей, объясняющих различные аспекты окружающей действительности, и принимая во внимание новые данные, мы можем скорректировать наши неявные представления и тем самым улучшить понимание реальности. Любая *вероятностная модель* описывается набором управляющих ее поведением параметров θ , понимаемых как случайные величины, а также связанным набором данных x .

Цель байесовского вывода состоит в том, чтобы найти апостериорное распределение $p(\theta|x)$ (распределение по параметрам при заданных данных), которое хорошо отражает конкретный аспект реальности. Апостериорное распределение пропорционально произведению правдоподобия $p(x|\theta)$ (распределение по данным при фиксированных параметрах) и априорной вероятности $p(\theta)$ (начальное распределение по параметрам), что следует из правила Байеса, представленного в формуле 1.3:

$$\begin{array}{c}
 \text{Правдо-} \quad \text{Априорная} \\
 \text{подобие} \quad \text{вероятность} \\
 \uparrow \quad \uparrow \\
 p(\theta|x) = \frac{p(x|\theta)p(\theta)}{p(x)} = \frac{p(x|\theta)p(\theta)}{\int p(x|\theta)p(\theta)d\theta} \sim p(x|\theta)p(\theta). \quad (1.3) \\
 \downarrow \quad \downarrow \quad \downarrow \\
 \text{Апостериорная} \quad \text{Достоверность} \quad \text{Функция разбиения } Z \\
 \text{вероятность}
 \end{array}$$

Априорное распределение $p(\theta)$ отражает наши исходные представления и может быть либо неинформативным (например, равномерным по всем возможным состояниям), либо информативным (например, основанным на опыте в определенной области). Более того, результаты нашего вывода зависят от выбранного априорного распределения: не только от значения параметров, но и от вида функциональной зависимости. Можно представить себе цепочку обновления наших представлений в виде байесовского механизма, в котором априорное распределение становится апостериорным по мере получения большего количества данных, как показано на рис. 1.4. Видно, как при поступлении новых данных априорное распределение превращается в апостериорное по правилу Байеса.



Рис. 1.4. Байесовский механизм, показывающий преобразование априорного распределения в апостериорное при получении новых данных

Априорные распределения, имеющие тот же вид, что и апостериорные, известны как *сопряженные априорные распределения*. Они, как правило, являются предпочтительными, поскольку упрощают вычисления благодаря выражениям в закрытой форме. Знаменатель $Z = p(x) = \int p(x|\theta)p(\theta)d\theta$ известен как *нормирующая постоянная*, или *функция распределения*, и он часто с трудом поддается вычислениям из-за интегрирования в пространстве параметров большой размерности. В этой книге мы рассмотрим несколько методов оценки Z .

Мы можем представить взаимосвязи между различными случайными величинами в нашей модели в виде графа, как показано на рис. 1.5, результатом чего является *вероятностная графовая модель* (ВГМ). Каждый узел графа представляет собой случайную величину (СВ), а каждое ребро — условную зависимость. Параметры модели изображены в виде прозрачных узлов, в то время как заштрихованные узлы представляют наблюдаемые данные, а прямоугольники обозначают копию или повторение случайной переменной. Сама топология графа меняется в зависимости от целей, которые вы преследуете при моделировании. Однако задачи байесовского вывода остаются прежними: найти апостериорное распределение по параметрам модели при фиксированных наблюдаемых данных.

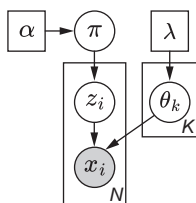


Рис. 1.5. Вероятностная графовая модель (ВГМ) для модели смеси нормальных распределений

В отличие от ВГМ, где связи задаются экспертами в предметной области, *модели глубокого обучения* автоматически изучают представления о мире с помощью алгоритма обратного распространения, который минимизирует целевую функцию. *Глубокие нейронные сети* (ГНС) (DNN, deep neural network) состоят из набора слоев, параметризованных матрицами весов и параметрами смещения. Математически DNN можно выразить как композицию функций отдельных слоев, как показано в формуле 1.4:

$$DNN = (x; \theta) = f_L \left(f_{L-1} \left(\dots \left(f_1(x; \theta_1) \right) \dots \right); \theta_L \right). \quad (1.4)$$

Здесь $f_1(x) = f(x; \theta_1)$ — функция в слое 1. Композиционная форма ГНС напоминает нам о цепном правиле для дифференцирования параметров в рамках стохастического градиентного спуска. На страницах этой книги мы будем рассматривать разные типы нейронных сетей: сверточные (convolutional neural networks, CNN), рекуррентные (recurrent neural networks, RNN), а также трансформеры и нейронные сети графовой структуры (graph neural networks, GNN), которые используются в самых различных областях, от компьютерного зрения до финансов.

1.4.1. Два основных направления байесовского вывода: МСМС и VI

Марковская цепь Монте-Карло (Markov chain Monte Carlo, МСМС) — это методология семплирования (sampling) в высокоразмерных пространствах параметров для аппроксимации апостериорного распределения $p(\theta|x)$. В статистике процесс *семплирования* означает генерацию случайной выборки значений из заданного распределения вероятностей. Существует множество подходов к семплированию в пространствах параметров большой размерности. Как мы увидим в последующих главах, в основе метода МСМС лежит построение цепи Маркова, стационарное распределение которой представляет собой интересующую нас целевую плотность вероятности (то есть апостериорное распределение). При случайном блуждании в пространстве состояний доля времени, которую мы проводим в каждом состоянии θ , будет пропорциональна $p(\theta|x)$. Как следствие, мы можем использовать интегрирование по методу Монте-Карло для получения интересующих величин, связанных с нашим апостериорным распределением.

Прежде чем мы обсудим высокоразмерные пространства параметров, давайте рассмотрим способы осуществлять выборку в случае низкоразмерных пространств. Наиболее популярный метод семплирования из одномерных распределений известен как *метод обратного преобразования* (inverse cumulative density function (CDF) method). Функция распределения определяется как $\text{CDF}_X(x) = P(X \leq x)$ (рис. 1.6).

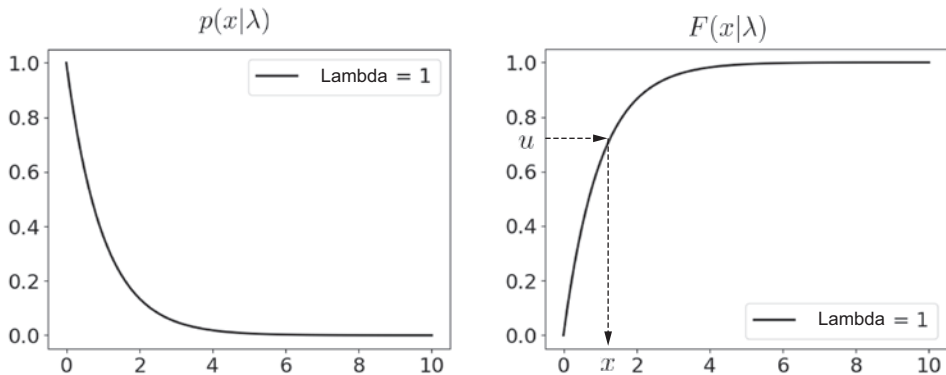


Рис. 1.6. Функция плотности вероятности (слева) и функция распределения (справа) экспоненциальной случайной величины (СВ)

Давайте для примера рассмотрим случай экспоненциальной случайной величины (СВ) с функцией плотности вероятности и функцией распределения, заданными формулами 1.5.

$$p(x|\lambda) = \lambda e^{-\lambda x}, \quad x \geq 0 \quad F(x|\lambda) = \int_0^x p(x|\lambda) dx = 1 - e^{-\lambda x}, \quad x \geq 0. \quad (1.5)$$

Обратная функция распределения описывается формулой 1.6:

$$F^{-1}(u) = -\frac{\ln(1-u)}{\lambda}. \quad (1.6)$$

Таким образом, чтобы сгенерировать выборку для экспоненциальной СВ, нам сначала нужно сгенерировать выборку из равномерного распределения $u \sim \text{Unif}(0, 1)$ и применить преобразование $-\ln(1-u)/\lambda$. Генерируя достаточное количество семплов, мы можем достичь необходимой доли верных результатов. Установление того, как *эффективно* генерировать выборки из высокоразмерных распределений, — одна из проблем метода MCMC. В этой книге мы рассмотрим два способа семплирования: схему Гиббса и алгоритм Метрополиса — Гастингса (Metropolis — Hastings, MH).

Вариационный вывод (variational inference, VI) — это подход к аппроксимации апостериорного распределения $p(x)$, основанный на оптимизации. Здесь мы упрощаем обозначения, придавая общему распределению $p(x)$ смысл апостериорного распределения. Основная идея VI заключается в том, чтобы выбрать приближенное распределение $q(x)$ из семейства удобных (tractable) распределений, а затем сделать это приближение как можно более близким к истинному апостериорному распределению $p(x)$. Под *удобным распределением* понимается всего лишь то, которое легко вычислить. Как мы увидим в разделе книги, посвященном среднему полю (mean-field), приближенная функция $q(x)$ может принимать вид полностью разложенного представления совместного апостериорного распределения. Такая факторизация значительно ускоряет вычисления.

Далее мы введем дивергенцию Кульбака — Лейблера (Kullback — Leibler, KL) и используем ее для измерения близости нашего приближенного распределения к истинному апостериорному. На рис. 1.7 показаны два варианта KL-дивергенции:

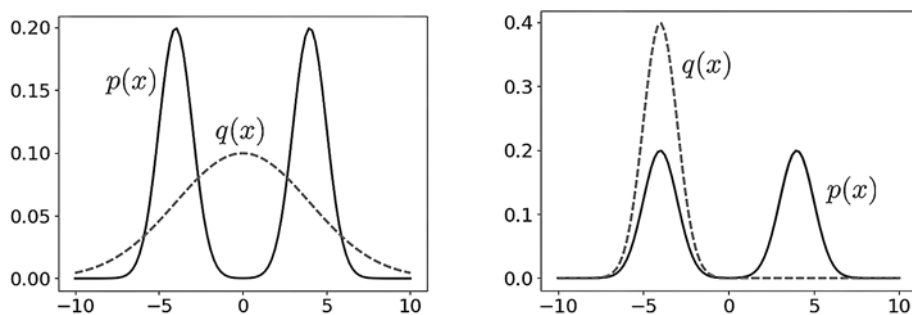


Рис. 1.7. Подгонка приближенного распределения $q(x)$ к гауссовой смеси $p(x)$ с использованием прямой (слева) и обратной дивергенции KL (справа)

Исходное распределение $p(x)$ представляет собой бимодальное гауссово распределение (оно же гауссова смесь с двумя компонентами), в то время как

приближенное распределение $q(x)$ является унимодальным гауссовым распределением. Как показано на рис. 1.7, распределение с двумя пиками мы можем аппроксимировать либо в центре с помощью $q(x)$, которое имеет высокую дисперсию, чтобы отразить наличие бимодального распределения, либо в одной из его мод (правая часть рис. 1.7). Это следует из определений прямой и обратной KL-дивергенции, данных в формуле 1.7. Как мы увидим в последующих главах, путем минимизации KL-дивергенции мы фактически сводим VI к задаче оптимизации.

$$KL(p \parallel q) = \sum p(x) \log \frac{p(x)}{q(x)} \quad KL(q \parallel p) = \sum q(x) \log \frac{q(x)}{p(x)}. \quad (1.7)$$

1.4.2. Современные алгоритмы глубокого обучения

За годы своего существования глубокие нейронные сети прошли путь от базовых блоков сверточной нейросети LeNet до архитектуры визуальных трансформеров. Стали появляться определенные элементы, такие как остаточные связи (residual connections) в модели ResNet, которые стали стандартным решением для современных нейронных сетей произвольной глубины. В последующих главах этой книги мы познакомимся с современными алгоритмами глубокого обучения, в том числе с трансформерами на основе механизма самовнимания (self-attention), с генеративными моделями, такими как вариационные автоэнкодеры, и нейронными сетями графовой структуры.

Мы также обсудим амортизированный вариационный вывод — интересную область исследований, сочетающую в себе выразительные возможности и способность к обучению представлениям, которыми обладают DNN, со знанием предметной области вероятностных графовых моделей. Мы познакомимся с одним из вариантов использования сетей смешанной плотности, где мы будем использовать нейронную сеть для отображения пространства наблюдения на параметры приближенного апостериорного распределения.

Большинство моделей глубокого обучения относятся к категории узкого искусственного интеллекта (narrow AI), демонстрирующего хорошие показатели на конкретном наборе данных. Хотя умение хорошо справляться с узким кругом задач — полезное свойство, мы хотели бы сделать необходимые преобразования, чтобы перейти от узкого ИИ к общему искусственному интеллекту (AGI).

1.5. Реализация алгоритмов

Ключевой частью изучения алгоритмов с нуля является программная реализация. Важно писать хороший код, который не только эффективно использует структуры данных, но также обладает низкой алгоритмической сложностью. Во всех главах мы будем группировать функциональные аспекты кода по классам

и реализовывать различные вычислительные методы с нуля. Тем самым вы познакомитесь с большим количеством методов объектно-ориентированного программирования (ООП), широко используемых в популярных библиотеках ML, в частности в `scikit-learn`. Хотя целью этой книги является написание всех примеров кода с нуля, мы все же будем иногда пользоваться библиотеками ML (той же `scikit-learn`, <https://scikit-learn.org/stable/>), в случае, если таковые имеются, для проверки результатов нашей реализации. На протяжении всей этой книги мы будем использовать язык Python.

1.5.1. Структуры данных

Структура данных — это способ хранения и систематизации данных. Каждая из них обладает своими особенностями в отношении производительности; и поэтому одни структуры больше подходят для данной конкретной задачи, чем другие. В нашей реализации алгоритмов ML мы преимущественно будем использовать *линейные структуры данных*, такие как массивы фиксированного размера, поскольку время доступа к элементу в таком массиве постоянно — $O(1)$. Мы также часто будем использовать динамически изменяемые массивы (например, списки Python), чтобы хранить данные на протяжении серии итераций.

Мы также будем постоянно пользоваться *нелинейными структурами данных* вроде ассоциативных массивов (словарей) и множеств, потому что они построены на основе самобалансирующихся деревьев бинарного поиска (binary search trees, BST), которые гарантируют операции вставки, поиска и удаления со сложностью $O(n \log n)$. И наконец, хеш-таблица, или неупорядоченный ассоциативный массив, — это еще одна широко используемая эффективная структура данных, которая нам пригодится. Она характеризуется временем доступа $O(1)$ при условии отсутствия коллизий.

1.5.2. Парадигмы решения задач

Большинство алгоритмов ML, описанных в этой книге, можно отнести к одной из четырех парадигм решения задач: полный поиск, жадные алгоритмы, «разделяй и властвуй» и динамическое программирование. *Полный поиск* — это метод, в котором решение задачи находится путем обхода всего пространства поиска. Примером задачи машинного обучения, в которой выполняется полный поиск, является точный вывод (exact inference) с помощью полного перечисления. Во время точного вывода необходимо полностью указать набор таблиц вероятностей для выполнения вычислений.

Жадный алгоритм на каждом шаге выбирает локально оптимальный вариант в надежде в итоге прийти к глобально оптимальному решению. Такие алгоритмы часто полагаются на «жадную» эвристику. Примером использования является размещение датчиков. Если, например, имеется помещение и некоторое

количество датчиков температуры, то задача состоит в том, чтобы разместить их таким образом, чтобы максимально увеличить охват помещения.

«Разделяй и властвуй» — это метод, который разбивает проблему на более мелкие, независимые подзадачи, а затем объединяет все их решения. Примером является алгоритм дерева решений CART. Как мы увидим в следующей главе, в CART оптимальный порог для разделения дерева решений определяется путем оптимизации цели классификации (например, индекса Джини). Та же процедура применяется к дереву глубиной на единицу больше, что приводит к рекурсивному алгоритму.

Последняя группа алгоритмов, *динамическое программирование* (ДП), — это метод, который разбивает задачу на более мелкие, перекрывающиеся подзадачи, вычисляет решение для каждой из них и сохраняет его в таблице ДП. Примером может служить обучение с подкреплением (reinforcement learning, RL) при поиске решения уравнений Беллмана. Для небольшого числа состояний мы можем вычислить Q-функцию в табличном виде, используя динамическое программирование.

Итоги

- Алгоритм — это последовательность шагов, необходимых для выполнения определенной задачи. Алгоритмы машинного обучения можно разбить на следующие категории: обучение с учителем, обучение без учителя и глубокое обучение.
- Доскональное понимание алгоритмов поможет выбрать наиболее подходящий для решения поставленной задачи, объяснить результаты, устранить сложные проблемы и повысить производительность существующих моделей.
- Байесовский вывод позволяет обновлять наши убеждения о мире на основе наблюдаемых данных, в то время как модели глубокого обучения изучают представления о мире с помощью алгоритма обратного распространения, который минимизирует целевую функцию. Существует два вида байесовского вывода: марковская цепь Монте-Карло (МСМС) и вариационный вывод (VI). Эти два вида сосредоточены вокруг семплирования и аппроксимации апостериорного распределения соответственно.
- Важно писать хороший код, который не только эффективен в использовании структур данных, но и обладает низкой алгоритмической сложностью. Большинство алгоритмов ML, описанных в этой книге, можно отнести к одной из четырех парадигм решения задач: полный поиск, жадные алгоритмы, «разделяй и властвуй» и динамическое программирование.

Марковские цепи Монте-Карло

В этой главе

- ✓ Введение в марковские цепи Монте-Карло
- ✓ Оценка числа «пи» при помощи интегрирования по методу Монте-Карло
- ✓ Моделирование методом Монте-Карло с использованием биномиального дерева
- ✓ Самоизбегающее случайное блуждание
- ✓ Алгоритм семплирования по Гиббсу
- ✓ Алгоритм Метрополиса — Гастингса
- ✓ Выборка по значимости

В предыдущей главе мы говорили о различных типах алгоритмов ML и их программной реализации. Теперь мы сосредоточимся на такой часто используемой категории, как марковская цепь Монте-Карло. Любая вероятностная модель, объясняющая какую-либо часть реальности, существует в высокоразмерном параметрическом пространстве, так как она описывается большим множеством параметров. *Марковская цепь Монте-Карло* (МСМС) — это методология семплирования в высокоразмерных пространствах параметров для аппроксимации апостериорного распределения $p(\theta|x)$. Этот метод был изначально разработан физиками, но впоследствии стал популярен в кругах специалистов, занимающихся

байесовской статистикой, поскольку позволяет при помощи семплирования оценивать апостериорные распределения высокой размерности. Основная идея МСМС заключается в построении цепи Маркова, стационарное распределение которой равняется целевому апостериорному распределению $p(\theta|x)$.

Другими словами, если мы перемещаемся случайным образом в параметрическом пространстве, то доля времени, которую мы проводим в определенном состоянии θ , пропорциональна $p(\theta|x)$.

Следующий раздел будет посвящен введению в метод МСМС. Мы начнем с трех ознакомительных примеров метода Монте-Карло (оценка числа «пи», модель биномиального дерева и самоизбегающее случайное блуждание), чтобы затем перейти к трем популярным алгоритмам генерации выборки (семплирование по Гиббсу, алгоритм Метрополиса — Хастингса и выборка по значимости). Ознакомительные алгоритмы этой главы подобраны таким образом, чтобы дать читателю адекватное представление о МСМС, а алгоритмы семплирования выбраны с учетом частоты их использования для генерации выборки в вероятностных графовых моделях (ВГМ).

2.1. Введение в марковские цепи Монте-Карло

Давайте начнем с рассмотрения высокоразмерных параметрических пространств на примере классификации ирисов. Рассматриваемый набор данных содержит информацию о цветках ириса трех видов: щетинистый (*setosa*), разноцветный (*versicolor*) и виргинский (*virginica*). Цветки ириса различаются длиной, а также шириной своих лепестков и чашелистиков — характеристиками, которые можно использовать в качестве признаков для определения вида растения. На рис. 2.1 показан парный график характеристик цветков ириса.

Парный график (pairplot) — это матрица диаграмм, где каждый внедиагональный элемент содержит диаграмму рассеяния (scatter plot) для одного признака (например, длины лепестка) относительно какого-либо другого признака, в то время как элементы главной диагонали содержат графики для отдельных признаков. На диаграммах кружками, квадратами и ромбами обозначены разные виды растений. Парный график отражает попарные взаимосвязи в наборе данных. Давайте сосредоточимся на элементах основной диагонали и попытаемся смоделировать изображенные данные. Поскольку нам даны три вида цветков ириса, мы можем смоделировать эти данные как смесь трех нормальных распределений. Гауссова смесь — это взвешенная сумма нормальных распределений вероятностей. Формула 2.1 представляет это в математических терминах:

$$p(x|\theta) = \sum_{k=1}^K \underbrace{N(x; \mu_k, \sigma_k^2)}_{\text{Гауссова СВ}} \underbrace{\pi_k}_{\text{Доли компонент}}. \quad (2.1)$$

$\uparrow$ $\sum_{k=1}^K$ $\uparrow$
 Сумма K гауссианов

$\downarrow$ $\downarrow$ $\uparrow$
 Гауссова смесь Гауссова СВ Доли компонент

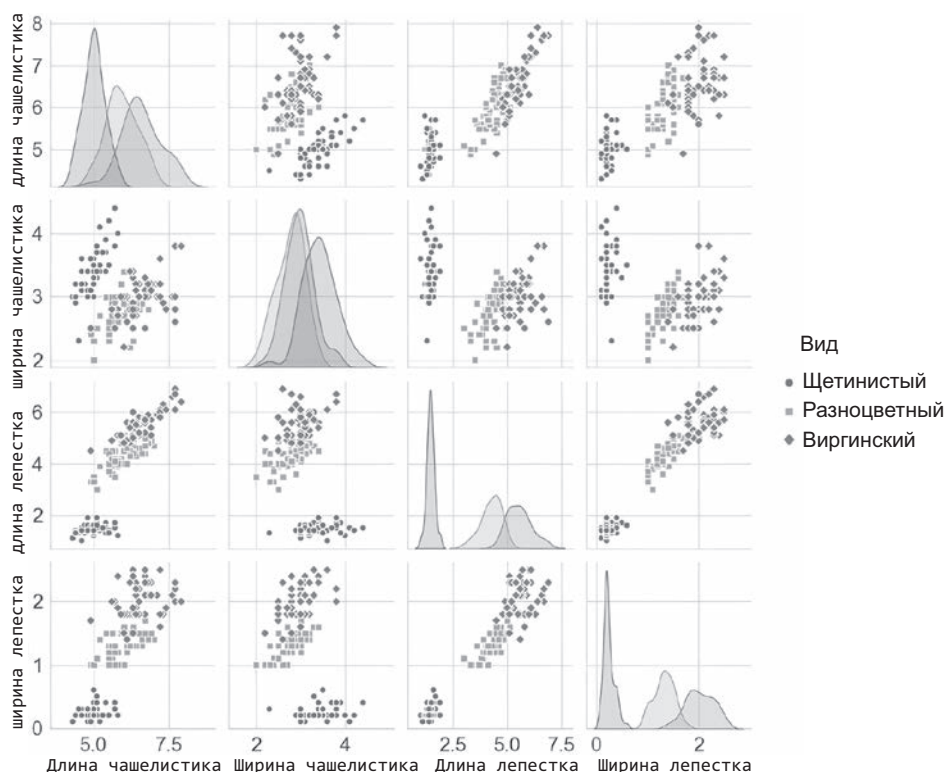


Рис. 2.1. Парный график характеристик цветков ириса: диаграммы рассеяния, в которых кружками, квадратами и ромбами обозначены виды

Смесь гауссианов является линейной комбинацией K нормальных СВ с положительными весами π_k , дающими в сумме 1: $\sum \pi_k = 1$, $\pi_k > 0$. Здесь мы имеем дело с параметрами высокой размерности, поскольку для подгонки модели гауссовой смеси нам нужно найти значения средних величин μ_k , ковариаций σ_k^2 и весов π_k . В наборе данных ирисов $K = 3$. Отсюда количество параметров для гауссовой смеси равно $9 - 1 = 8$ (в $\theta = \{\mu_k, \sigma_k^2, \pi_k\}_{k=1}^3$ есть 9 параметров, но мы вычитаем единицу благодаря нормировке суммы весов π_k на единицу). В одной из последующих глав мы увидим, как можно найти параметры гауссовой смеси с помощью алгоритма ожидания-максимизации (expectation-maximization, EM). Получив представление о параметрических пространствах, давайте теперь уточним наше понимание апостериорного распределения на примере подбрасываний монеты.

2.1.1. Апостериорное распределение для подбрасываний монеты

В рамках алгоритмов MCMC мы пытаемся аппроксимировать апостериорное распределение при помощи выборки. Большинство байесовских выводов

действительно хорошо подходят для аппроксимации апостериорного распределения. Но давайте сначала немного подробнее разберемся в том, что такое апостериорное распределение. Мы сталкиваемся с апостериорной вероятностью в тот момент, когда пытаемся подогнать параметры модели θ под наблюдаемые данные x . Иначе говоря, это условная вероятность параметров при заданных данных $p(\theta|x)$. Рассмотрим пример последовательного подбрасывания монеты, у которой вероятность выпадения орла равна θ , а решки — $1 - \theta$, как показано на рис. 2.2.

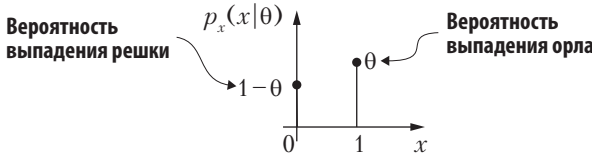


Рис. 2.2. Случайная величина Бернулли

На рис. 2.2 изображена функция вероятностного распределения случайной величины (СВ) Бернулли, моделирующей подбрасывание монеты. Если $\theta = 1/2$, то у нас имеется симметричная монета с равными шансами выпадения орла (1) или решки (0); во всех других случаях мы говорим, что монета несимметричная, причем степень ее несимметричности равна θ . Функцию вероятности СВ Бернулли можно записать следующим образом:

$$p_x(x|\theta) = \theta^x (1-\theta)^{1-x}, x \in \{0, 1\}. \quad (2.2)$$

Математическое ожидание СВ Бернулли равно $E[X] = \sum_x x p_x(x|\theta) = 0(1-\theta) + 1(\theta) = \theta$, а дисперсия равна $\text{VAR}(x) = E[x^2] - E[x]^2 = \theta - \theta^2 = \theta(1-\theta)$. Давайте вернемся к нашему примеру с подбрасыванием монеты. Пусть $D = \{x_1, \dots, x_n\}$ — последовательность независимых и одинаково распределенных подбрасываний монеты. Тогда правдоподобие можно вычислить следующим образом:

$$\begin{aligned} p_x(D|\theta) &= p_x(x_1, \dots, x_n|\theta) = \\ &= \prod_{i=1}^n p(x_i|\theta) = \prod_{i=1}^n \theta^{x_i} (1-\theta)^{1-x_i} = \\ &= \theta^{N_1} (1-\theta)^{N_0}. \end{aligned} \quad (2.3)$$

Здесь под $N_1 = \sum_{i=1}^n x_i$ мы понимаем общее количество случаев, в которых выпал орел, а под $N_0 = n - N_1$ — число случаев, когда выпала решка.

Формулы 2.2 и 2.3 имеют вид $p(x|\theta)$, что, по определению, представляет собой правдоподобие данных x при заданном параметре θ . Но что, если у нас есть некоторая априорная информация о параметре θ ? Другими словами, что, если мы знаем что-то о $p(\theta)$, например количество орлов и решек, которые мы получили в другом эксперименте с теми же монетами? Мы можем записать эту априорную информацию следующим образом:

$$\text{Beta}(\theta|a, b) \propto \theta^{a-1} (1-\theta)^{b-1}. \quad (2.4)$$

Здесь a — это количество орлов, а b — количество решек в нашем предыдущем эксперименте. Вычислить апостериорное распределение означает получить $p(\theta|x)$. С помощью правила Байеса из главы 1 мы можем записать апостериорное распределение как пропорциональное произведению правдоподобия и априорного распределения:

$$\begin{aligned} p(\theta|D) &\propto p(D|\theta)p(\theta) = \\ &= \theta^{N_1}(1-\theta)^{N_0}\theta^{a-1}(1-\theta)^{b-1} = \theta^{N_1+a-1}(1-\theta)^{N_0+b-1} \propto \\ &\propto \text{Beta}(\theta|N_1+a, N_0+b). \end{aligned} \quad (2.5)$$

Формула 2.5 позволяет вычислить апостериорное распределение $p(\theta|D)$, которое описывает вероятность выпадения орла в последовательности подбрасываний монеты. Как видно из формулы, оно пропорционально бета-распределению. Так как наше априорное распределение имеет тот же вид, то можно сказать, что оно является «сопряженным апостериорному». Другими словами, апостериорное распределение может быть вычислено аналитически путем обновления априорных значений наблюдаемыми данными.

2.1.2. Цепи Маркова для ранжирования веб-страниц

Прежде чем погружаться в алгоритмы МСМС, необходимо ввести понятие цепи Маркова. *Цепь Маркова* — это последовательность случайных событий, в которой вероятность текущего события зависит только от предшествующего события. Цепь Маркова первого порядка может быть представлена в виде формулы 2.6. Пусть $x_1, \dots, x_t$ — выборка из нашего апостериорного распределения. Тогда совместное распределение можно записать следующим образом:

$$\begin{aligned} p(x_1, \dots, x_t) &= p(x_1)p(x_2|x_1)p(x_3|x_2) \cdots p(x_t|x_{t-1}) = \\ &= p(x_1) \prod_{i=2}^t p(x_i|x_{i-1}). \end{aligned} \quad (2.6)$$

Обратите внимание, что совместное распределение раскладывается на произведение условных распределений, в которых каждое следующее состояние обусловлено только предыдущим состоянием. Любая марковская цепь характеризуется начальным распределением по состояниям $p(x_1=i)$ и матрицей перехода из состояния i в состояние j $A_{ij} = p(x_t=j|x_{t-1}=i)$.

Давайте рассмотрим цепи Маркова на примере ранжирования веб-страниц. Так, компания Google использует специальный алгоритм для ранжирования миллиардов веб-страниц. Набор из n веб-страниц можно представить в виде графа $G=(V, E)$. В нем каждая вершина $v \in V$ представляет собой веб-страницу, а каждое ребро $e \in E$ — ссылку с одной страницы на другую. Информация о связи между страницами позволяет нам построить гигантскую разреженную переходную

матрицу A , где A_{ij} — это вероятность перехода по ссылке со страницы i на страницу j , как показано на рис. 2.3.

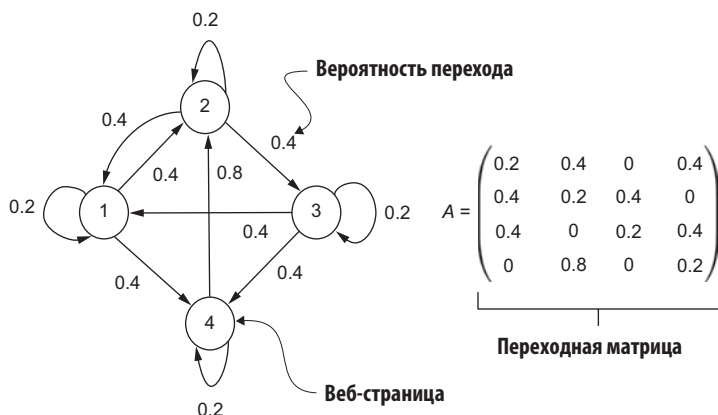


Рис. 2.3. Граф веб-страниц с вероятностями перехода

Обратите внимание, что каждая строка в матрице A в сумме дает 1, то есть $\sum_j A_{ij} = 1$. Матрица, обладающая таким свойством, называется *стохастической*. Заметьте также, что A_{ij} соответствует вероятности перехода из состояния i в состояние j . Как вы увидите позднее, ранг страницы представляет собой стационарное распределение по состояниям цепи Маркова. Мы выведем с нуля и реализуем в коде высокопроизводительный итерационный алгоритм, который можно будет масштабировать на миллиарды веб-страниц. Но перед этим в следующих нескольких разделах мы рассмотрим другие алгоритмы семплирования МСМС: семплирование по Гиббсу, алгоритм Метрополиса — Гастингса и выборку по значимости. Давайте начнем ознакомление с алгоритмами Монте-Карло с пары примеров.

2.2. Оценка числа « π »

Первый пример, с которым мы познакомимся, — это оценка значения числа π с помощью интегрирования по методу Монте-Карло. *Интегрирование методом Монте-Карло* (ММК) имеет преимущество перед численным интегрированием (которое вычисляет функцию по фиксированной сетке точек), состоящее в том, что функция вычисляется только в тех областях, где существует близкая к единице вероятность. Таким образом, интеграция ММК лучше подходит для задач высокой размерности.

Давайте рассмотрим математическое ожидание некоторой функции $f: \mathbb{R} \rightarrow \mathbb{R}$ от случайной величины $Z = f(Y)$. Мы можем аппроксимировать его, генерируя выборку СВ y с распределением $p(y)$ следующим образом:

$$E[f(Y)] = \int f(y)p(y)dy \approx \frac{1}{S} \sum_{s=1}^S f(y_s), \text{ где } y_s \sim p(y). \quad (2.7)$$

Другими словами, мы аппроксимируем математическое ожидание $E[f(y)]$ средним значением $f(y_s)$, где y_s — выборка из распределения $p(y)$. Давайте теперь используем тот же подход, но применительно к вычислению интеграла $I = \int f(x)dx$. Мы можем аппроксимировать интеграл следующим образом:

$$\begin{aligned} I &= \int_a^b f(x)dx = \int_a^b w(x)p(x)dx = E_p[w(x)] = \\ &= \frac{1}{n} \sum_{i=1}^N w(x_i), \text{ где } x_i \sim p(x). \end{aligned} \quad (2.8)$$

Здесь $p(x) \sim \text{Unif}(a, b) = 1/(b - a)$ — это плотность распределения равномерной случайной величины на интервале (a, b) , а $w(x) = f(x)(b - a)$ — масштабированная функция $f(x)$.

По мере увеличения размера выборки N наша эмпирическая оценка среднего значения становится более точной. И действительно, стандартная ошибка равна $\sigma/\sqrt{n}$, где σ — эмпирическое стандартное отклонение:

$$\sigma^2 = \frac{1}{N-1} \sum_{i=1}^N (f(x_i) - I)^2. \quad (2.9)$$

Получив представление о том, как работает интегрирование по методу Монте-Карло, давайте используем его для оценки числа π . Мы знаем, что площадь круга I радиусом r равна πr^2 . Кроме того площадь круга может быть вычислена как интеграл:

$$I = \int_{-r}^r \int_{-r}^r 1[x^2 + y^2 \leq r^2] dx dy = E_{x,y}[w(x, y)]. \quad (2.10)$$

Здесь $1[x^2 + y^2 \leq r^2]$ — индикаторная функция, равная 1 в случаях, когда точка находится внутри окружности радиуса r , и равная 0 в остальных случаях. Преобразуя исходную формулу для площади, получаем $\pi = I/r^2$. Чтобы вычислить значение I , воспользуемся следующим выражением:

$$\begin{aligned} w(x, y) &= (b_x - a_x)(b_y - a_y) 1[x^2 + y^2 \leq r^2] = \\ &= (2r)(2r) 1[x^2 + y^2 \leq r^2] = \\ &= 4r^2 1[x^2 + y^2 \leq r^2]. \end{aligned} \quad (2.11)$$

Мы можем обобщенно записать алгоритм оценки числа «пи» в виде псевдокода, показанного на рис. 2.4.

Мы генерируем выборку размером N для равномерного распределения на отрезке от $-R$ до R и вычисляем логическое выражение того, находится ли выборочное значение внутри круга или за его пределами. Если значение находится внутри, то оно учитывается при вычислении интеграла. Как только мы получаем оценку

44 Глава 2. Марковские цепи Монте-Карло

интеграла, мы делим результат на R^2 , чтобы вычислить нашу оценку числа «пи». Теперь мы готовы воплотить наш алгоритм в коде!

```
1: function pi_est(R, N):
2:   for i = 1 to N:
3:     X[i] ~ Unif(-R, R)
4:     Y[i] ~ Unif(-R, R)
5:     IN[i] = X[i]^2 + Y[i]^2 ≤ R^2
6:     S[i] = (2R) × (2R) × IN[i]
7:   end for
8:    $\hat{I} = \frac{1}{N} \sum_{i=1}^N S[i]$ 
9:    $\hat{\pi} = \frac{\hat{I}}{R^2}$ 
10:   $\hat{\pi}_{se} = \frac{\sigma_S}{\sqrt{N}}$ 
11:  return  $\hat{\pi} \pm \hat{\pi}_{se}$ 
```

Рис. 2.4. Псевдокод для оценки числа «пи»

Листинг 2.1. Оценка числа «пи»

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)  # ← задает начальное число для получения воспроизводимых результатов

def pi_est(radius=1, num_iter=int(1e4)):  # ← Параметр размера выборки N
    X = np.random.uniform(-radius, +radius, num_iter)
    Y = np.random.uniform(-radius, +radius, num_iter)

    R2 = X**2 + Y**2
    inside = R2 < radius**2
    outside = ~inside

    samples = (2*radius)*(2*radius)*inside

    I_hat = np.mean(samples)
    pi_hat = I_hat/radius ** 2  # ← Оценка числа «пи»
    pi_hat_se = np.std(samples)/np.sqrt(num_iter)  # ← Стандартное отклонение числа «пи»
    print("pi est: {} +/- {}".format(pi_hat, pi_hat_se))

    plt.figure()
    plt.scatter(X[inside], Y[inside], c='b', alpha=0.5)
    plt.scatter(X[outside], Y[outside], c='r', alpha=0.5)
    plt.show()

if __name__ == "__main__":
    pi_est()
```

Выполнение кода дает $\pi = 3,1348 \pm 0,0164$, что надо признать относительно неплохим результатом (в пределах погрешности). Попробуйте поэкспериментировать с разным размером выборки N , чтобы увидеть, как быстро мы сходимся к π в зависимости от N . На рис. 2.5 темно-серым цветом показаны выборочные

значения в методе Монте-Карло, находящиеся внутри круга, а светло-серым — отбракованные значения, лежащие за пределами круга.

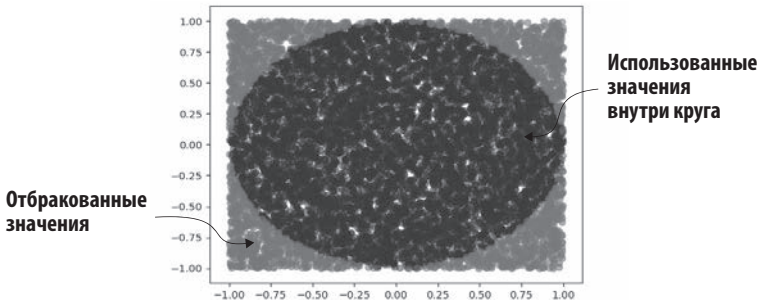


Рис. 2.5. Выборка, сгенерированная по методу Монте-Карло для оценки числа «пи»

В предыдущем примере мы исходили из того, что выборка генерируется для равномерного распределения $X, Y \sim \text{Unif}(-R, R)$. В следующем разделе рассмотрим моделирование цены акций на разных временных горизонтах с использованием модели биномиального дерева.

2.3. Модель биномиального дерева

Теперь давайте рассмотрим, как семплирование методом Монте-Карло может быть использовано в финансовой сфере. В модели биномиального дерева цен на акции мы предполагаем, что на каждом временном шаге акции могут либо расти, либо падать в цене, причем на разную величину, что характерно для рискованного актива. Предположим, что начальная цена акций в момент времени $t = 0$ равна 1 долл. Тогда на следующем временном шаге $t = 1$ цена будет равна u в состоянии роста (up) и d в состоянии снижения (down), причем вероятность перехода в состояние роста равна p . Модель биномиального дерева для первых двух временных шагов показана на рис. 2.6. Обратите внимание, что цена в каждом следующем состоянии «рост» («снижение») равна цене в предыдущем состоянии, умноженной на $u(d)$.

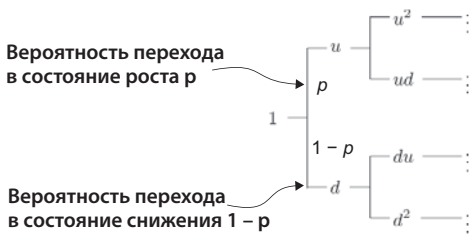


Рис. 2.6. Модель биномиального дерева

Мы можем использовать метод Монте-Карло для получения оценок неопределенности конечной цены акций на некотором временном горизонте T . При реализации биномиальной модели для описания процесса ценообразования

акций мы можем воспользоваться следующими формулами (вывод которых выходит за рамки данной книги):

$$u = \exp\left(\sigma\sqrt{\frac{T}{n}}\right) = \frac{1}{d};$$

$$p = \frac{1}{2} + \frac{1}{2}\left(\frac{\mu}{\sigma}\right)\sqrt{\frac{T}{n}}.$$
(2.12)

Здесь T — длина горизонта прогнозирования в годах, а n — количество временных шагов. Мы предполагаем, что 1 год равен 252 торговым дням, 1 месяц равен 21 дню, 1 неделя равна 5 рабочим дням, а 1 день — 8 часам. Давайте смоделируем цену акций, используя биномиальную модель с временным шагом в один день для двух разных временных горизонтов: 1 месяц и 1 год с сегодняшнего дня. Алгоритм можно обобщенно представить в виде псевдокода, показанного на рис. 2.7.

1: **function** binomial_tree($\mu, \sigma, S_0, N, T, \text{step}$):

2: $u = \exp\left(\sigma\sqrt{\frac{T}{n}}\right) \leftarrow \begin{array}{l} \text{Цена состояния «рост»} \end{array}$

3: $d = \frac{1}{u} \leftarrow \begin{array}{l} \text{Цена состояния «снижение»} \end{array}$

4: $p = \frac{1}{2} + \frac{1}{2}\left(\frac{\mu}{\sigma}\right)\sqrt{\frac{T}{n}} \leftarrow \begin{array}{l} \text{Вероятность перехода} \\ \text{в состояние «рост»} \end{array}$

5: $\text{up_times} = \text{Binomial}\left(\frac{T}{\text{step}}, p, N\right)$

6: $\text{down_times} = \frac{T}{\text{step}} - \text{up_times}$

7: $S_T = S_0 \times u^{\text{up_times}} \times d^{\text{down_times}}$

8: **return** S_T

Рис. 2.7. Псевдокод биномиального дерева

Мы начинаем с инициализации цены состояния «рост» u и цены состояния «снижение» d , а также вероятности перехода в состояние «рост» p . Затем мы моделируем все переходы в состояние роста путем семплирования биномиальной случайной величины с количеством испытаний, равным T/step (шаг), вероятностью успеха p и количеством итераций по методу Монте-Карло, равным N . Переходы в состояние снижения вычисляются как дополнение к переходам в состояние роста. В конечном итоге мы вычисляем цену актива, умножая начальную цену S_0 на цену всех переходов в состояния роста и снижения.

Листинг 2.2. Моделирование цен на акции при помощи биномиального дерева

```
import numpy as np

import seaborn as sns
import matplotlib.pyplot as plt
```

```

np.random.seed(42)

def binomial_tree(mu, sigma, S0, N, T, step):

    # вычисляем цены состояний и вероятность
    u = np.exp(sigma * np.sqrt(step))  ← Цена состояния роста
    d = 1.0/u  ← Цена состояния снижения
    p = 0.5+0.5*(mu/sigma)*np.sqrt(step)  ← Вероятность состояния роста

    # моделирование биномиального дерева
    up_times = np.zeros((N, len(T)))
    down_times = np.zeros((N, len(T)))
    for idx in range(len(T)):
        up_times[:,idx] = np.random.binomial(T[idx]/step, p, N)
        down_times[:,idx] = T[idx]/step - up_times[:,idx]

    # вычисляем конечную цену
    ST = S0 * u**up_times * d**down_times

    # создание графиков
    plt.figure()
    plt.plot(ST[:,0], color='b', alpha=0.5, label='Горизонт в 1 месяц')
    plt.plot(ST[:,1], color='r', alpha=0.5, label='Горизонт в 1 год')
    plt.xlabel('Временной шаг, сутки')
    plt.ylabel('Цена')
    plt.title('Моделирование биномиального дерева')
    plt.legend()
    plt.show()

    plt.figure()
    plt.hist(ST[:,0], color='b', alpha=0.5, label='Горизонт в 1 месяц')
    plt.hist(ST[:,1], color='r', alpha=0.5, label='Горизонт в 1 год')
    plt.xlabel('Цена')
    plt.ylabel('Число значений')
    plt.title('Моделирование биномиального дерева')
    plt.legend()
    plt.show()

if __name__ == "__main__":

    # параметры модели
    mu = 0.1  ← Среднее
    sigma = 0.15  ← Волатильность
    S0 = 1  ← Стартовая цена

    N = 10000  ← Число итераций
    T = [21.0/252, 1.0]  ← Временной горизонт, годы
    step = 1.0/252  ← Временной шаг, годы

    binomial_tree(mu, sigma, S0, N, T, step)

```

На рис. 2.8 видно, что наши годовые оценки имеют более высокую волатильность по сравнению с месячными оценками. Это вполне логично, поскольку мы ожидаем, что на длинных временных горизонтах мы столкнемся с большей

неопределенностью. В следующем разделе мы узнаем, как использовать метод Монте-Карло для моделирования самоизбегающих случайных блужданий.

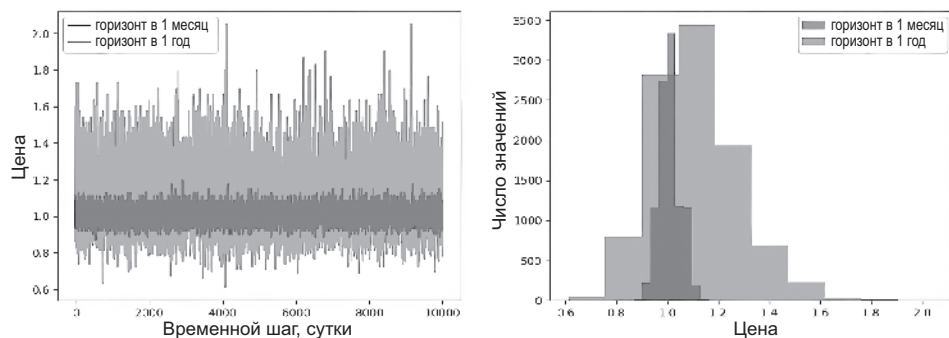


Рис. 2.8. Моделирование цен на акции при помощи биномиального дерева

2.4. Самоизбегающее случайное блуждание

Рассмотрим случайное блуждание по двумерной решетке. В каждой точке такой решетки мы делаем случайный шаг, тогда как все направления равновероятны. Подобное блуждание образует цепь Маркова $\{X_i\}_{i=1}^n$ на $\mathbb{Z} \times \mathbb{Z}$ с $X_0 = (0, 0)$ и вероятностями перехода, заданными по формуле 2.13:

$$P(X_i = (k, l) | X_{i-1} = (i, j)) = \begin{cases} \frac{1}{4}, & \text{если } |k-i| + |l-j| = 1 \\ 0, & \text{иначе.} \end{cases} \quad (2.13)$$

Другими словами, у нас есть равная $1/4$ вероятность перехода из одной точки решетки в любую из четырех соседних точек — верхнюю (up), нижнюю (down), левую (left) или правую (right). Кроме того, под *самоизбегающими* понимаются такие случайные блуждания, которые не проходят через одну точку дважды. Мы можем использовать метод Монте-Карло для моделирования самоизбегающего случайного блуждания (self-avoiding random walk), как показано в следующем псевдокоде (рис. 2.9).

Мы начинаем с инициализации решетки (lattice) нулевой матрицей. Далее помещаем начало случайного блуждания в центр решетки, как видно по переменным `xx` и `yy`. Заметьте, что `num_step` — это количество шагов в случайном блуждании. Мы начинаем каждую итерацию с вычисления переменных `up`, `down`, `left`, `right`, которые принимают значение 1, если соответствующая точка решетки занята, и 0 в противном случае. Таким образом, мы можем вычислить доступные направления (переменная `neighbors`), находя разность единицы и переменных направлений. Если сумма соседей (`neighbors`) равна нулю, то доступных направлений нет (все соседние ячейки решетки заняты), и случайное

блуждание становится самопересекающимся, на чем алгоритм завершается. В противном случае вычисляем весовой коэффициент значимости (importance weight) как произведение предыдущего значения веса на сумму соседей. Веса значимости используются для вычисления средневзвешенных квадратичных расстояний случайного блуждания.

```

1: function rand_walk(num_step, num_iter, moves):
2:   X, Y, lattice = 0, 0, 0
3:   weight = 1
4:   xx = num_step + 1 + X ← Середина оси x
5:   yy = num_step + 1 + Y ← Середина оси y
6:   lattice[xx, yy] = 1 ← Начальное положение решетки
7:   for i = 1 to num_step:
8:     up = lattice[xx, yy+1]
9:     down = lattice[xx, yy-1]
10:    left = lattice[xx-1, yy]
11:    right = lattice[xx+1, yy]
12:    neighbors = [1, 1, 1, 1] - [up, down, left, right] ← Доступные направления
13:    if sum(neighbors) == 0
14:      break ← Самозацикливание
15:    конец if
16:    weight = weight × sum(neighbors) ← Вычисление весов значимости
17:    direction ~ Cat( $\frac{\text{neighbors}}{\text{sum(neighbors)}}$ ) ← Разыгрывание направления движения
18:    X = X + moves[direction]
19:    Y = Y + moves[direction]
20:    //обновление координат решетки
21:    xx = num_step + 1 + X
22:    yy = num_step + 1 + Y
23:    lattice[xx, yy] = 1
24:  конец for
25:  return lattice

```

Рис. 2.9. Псевдокод случайного блуждания

Далее разыгрываем одно из возможных направлений движения при помощи категориальной случайной величины. Поскольку элементы массива `neighbors` могут принимать только значения 0 и 1, проводим выборку равновероятно по всем доступным направлениям.

Затем обновляем координаты решетки `xx` и `yy` и отмечаем занятость ячейки, присваивая `lattice[xx, yy]=1`. Хотя в псевдокоде показана только одна итерация, в листинге ниже алгоритм псевдокода помещен в дополнительный цикл `for` с количеством итераций `num_iter`. Каждая итерация — это эксперимент, или испытание, по генерации непересекающегося (то есть самоизбегающего) случайного блуждания. Квадратичное расстояние случайного блуждания

и весовые коэффициенты значимости регистрируются в каждом испытании. Теперь мы готовы взглянуть на код самоизбегающего случайного блуждания.

Листинг 2.3. Самоизбегающее случайное блуждание

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

np.random.seed(42)

def rand_walk(num_step, num_iter, moves):

    # статистика случайных блужданий
    square_dist = np.zeros(num_iter)
    weights = np.zeros(num_iter)

    for it in range(num_iter):

        trial = 0
        i = 1
        while i != num_step-1:
            # инициализация
            X, Y = 0, 0
            weight = 1
            lattice = np.zeros((2*num_step+1, 2*num_step+1))
            lattice[num_step+1, num_step+1] = 1
            path = np.array([0, 0])
            xx = num_step + 1 + X
            yy = num_step + 1 + Y

            print("iter: %d, trial %d" %(it, trial))

            for i in range(num_step):

                up = lattice[xx,yy+1]
                down = lattice[xx,yy-1]
                left = lattice[xx-1,yy]
                right = lattice[xx+1,yy]

                neighbors = np.array([1, 1, 1, 1]) -
                ➔ np.array([up, down, left, right])

                if (np.sum(neighbors) == 0):
                    i = 1
                    break
                # конец if

                weight = weight * np.sum(neighbors)

                direction = np.where(np.random.rand() <
                ➔ np.cumsum(neighbors/float(sum(neighbors))))

                X = X + moves[direction[0][0],0]
```

Повторяется до тех пор, пока мы не получим
непересекающееся случайное блуждание

Вычисление доступных
направлений

Для избежания самозакливания

Вычисление весов
значимости

Разыгрывание
направления
движения


```

Y = Y + moves[direction[0][0],1]

path_new = np.array([X,Y])
path = np.vstack((path,path_new))  ← Сохранение выбранного пути

# обновление координат решетки
xx = num_step + 1 + X
yy = num_step + 1 + Y
lattice[xx,yy] = 1
# конец for

trial = trial + 1
# конец while

square_dist[it] = X**2 + Y**2  ← Вычисление квадрата расстояния

weights[it] = weight
# конец for

mean_square_dist = np.mean(weights * square_dist)/np.mean(weights)
print("mean square dist: ", mean_square_dist)

# создание графиков
plt.figure()
for i in range(num_step-1):
    plt.plot(path[i,0], path[i,1], path[i+1,0], path[i+1,1], 'ob')
plt.title('Случайное блуждание без самопересечений')
plt.xlabel('X')
plt.ylabel('Y')
plt.show()

plt.figure()
sns.displot(square_dist)
plt.xlim(0,np.max(square_dist))
plt.title('Квадрат расстояния случайного блуждания')
plt.xlabel('Квадрат расстояния (X^2 + Y^2)')
plt.show()

if __name__ == "__main__":
    num_step = 150  ← Количество шагов в случайном блуждании
    num_iter = 100  ← Количество итераций для усреднения результатов
    moves = np.array([[0, 1],[0, -1],[-1, 0],[1, 0]])  ← Двумерные
                                                         передвижения

    rand_walk(num_step, num_iter, moves)

```

На рис. 2.10 показано самоизбегающее случайное блуждание, сгенерированное на последней итерации. На двумерной решетке возможно 4" случайных блуждания длиной n , и при больших n вероятность того, что подобное блуждание будет непересекающимся, очень мала. На рисунке также показана гистограмма квадрата расстояния, вычисленного для каждой итерации случайного блуждания. Гистограмма имеет положительный коэффициент асимметрии, что отражает меньшую вероятность блужданий на большие расстояния. В следующем разделе мы рассмотрим еще один популярный алгоритм генерации выборки: семплирование по Гиббсу.

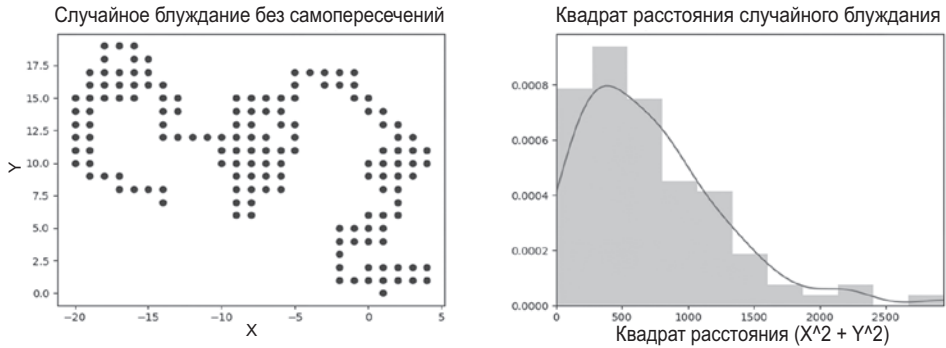


Рис. 2.10. Самоизбегающее случайное блуждание (слева) и квадратичное расстояние случайного блуждания (справа)

2.5. Семплирование по Гиббсу

В этом разделе вы познакомитесь с одним из основных алгоритмов МСМС, который называется *семплированием по Гиббсу*. Этот вид генерации выборки из многомерного распределения основан на идее семплирования по одной переменной за раз с использованием условных вероятностей для последних выборочных значений всех остальных переменных. Например, для трехмерного распределения $d = 3$, при наличии исходного набора значений $x^{(k)}$, мы генерируем следующий $x^{(k+1)}$, как показано ниже:

$$\begin{aligned} x_1^{k+1} &\sim p(x_1 | x_2^k, x_3^k); \\ x_2^{k+1} &\sim p(x_2 | x_1^{k+1}, x_3^k); \\ x_3^{k+1} &\sim p(x_3 | x_1^{k+1}, x_2^{k+1}). \end{aligned} \quad (2.14)$$

Распределения в формуле 2.14 называются *условными распределениями вероятностей* (fully conditional distribution). Также обратите внимание, что наивный алгоритм семплирования по Гиббсу является последовательным (с количеством шагов, пропорциональным размерности распределения) и подразумевает, что мы можем с легкостью семплировать условные распределения. Алгоритм семплирования по Гиббсу применим в тех случаях, когда условные распределения, указанные в формуле 2.14, легко поддаются вычислению.

Одним из примеров простых вычислений условных распределений является случай многомерного нормального распределения с плотностью вероятности, заданной следующим образом:

$$\mathcal{N}(x; \mu, \Sigma) = \frac{1}{(2\pi)^{\frac{d}{2}} |\Sigma|^{\frac{1}{2}}} \exp \left[-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right]. \quad (2.15)$$

Давайте разобьем гауссовский вектор на два подмножества $x\{1:D\} = (x_A, x_B)$, а параметры — как показано в формуле 2.16:

$$\begin{aligned}\mu &= \begin{pmatrix} \mu_A \\ \mu_B \end{pmatrix}; \\ \Sigma &= \begin{pmatrix} \Sigma_{AA} & \Sigma_{AB} \\ \Sigma_{BA} & \Sigma_{BB} \end{pmatrix}.\end{aligned}\tag{2.16}$$

Можно показать, как это сделано в разделе 2.3 книги Кристофера М. Бишоп (Christopher M. Bishop) «Pattern Recognition and Machine Learning»¹ (Springer, 2006), что условные вероятности описываются формулой 2.17:

$$\begin{aligned}p(x_A | x_B) &= \mathcal{N}(x_A | \mu_{A|B}, \Sigma_{A|B}); \\ \mu_{A|B} &= \mu_A + \Sigma_{AB} \Sigma_{BB}^{-1} (x_B - \mu_B); \\ \Sigma_{A|B} &= \Sigma_{AA} - \Sigma_{AB} \Sigma_{BB}^{-1} \Sigma_{BA}.\end{aligned}\tag{2.17}$$

Давайте разберемся с алгоритмом семплирования по Гиббсу, используя псевдокод на рис. 2.11.

```

1: class gibbs_gauss
2: function gauss_conditional(mu, Sigma, setA, x):
3: setU = set(range(len(mu))) ← Универсальное множество
4: setB =  $\frac{\text{setU}}{\text{setA}}$ 
5:  $x_B, \mu_A, \mu_B = x[\text{setB}], \mu[\text{setA}], \mu[\text{setB}]$ 
6:  $\mu_{A|B} = \mu_A + \Sigma_{AB} \Sigma_{BB}^{-1} (x_B - \mu_B)$ 
7:  $\Sigma_{A|B} = \Sigma_{AA} - \Sigma_{AB} \Sigma_{BB}^{-1} \Sigma_{BA}$ 
8: return  $\mu_{A|B}, \Sigma_{A|B}$ 
9: function sample(mu, Sigma, xinit, num_samples):
10:  $x = xinit$ 
11:  $\text{dim} = \text{len}(\mu)$ 
12: for  $s = 1$  to  $\text{num\_samples}$ :
13:   for  $d = 1$  to  $\text{dim}$ :
14:      $\mu_{A|B}, \Sigma_{A|B} = \text{gauss\_conditional}(\mu, \Sigma, \text{set}(d), x)$ 
15:      $x[d] \sim N(x; \mu_{A|B}, \Sigma_{A|B})$  ← Выборка по Гиббсу
16:   конец for
17:    $\text{samples}[s,:] = x$ 
18: конец for
19: return samples

```

Рис. 2.11. Псевдокод семплирования по Гиббсу

¹ Бишоп К. М. «Распознавание образов и машинное обучение».

Наш класс `gibbs_gauss` содержит две функции: `gauss_conditional` и `sample`. Функция `gauss_conditional` вычисляет условное гауссово распределение $p(x_A|x_B)$ для любых множеств переменных `setA` и `setB`. `setA` указывается на входе функции, а `setB` вычисляется как разница между универсальным множеством размерности D и `setA`. Напомню, что при семплировании по Гиббсу мы на каждом шагу генерируем случайную величину из какого-то одного измерения, при этом фиксируя значения всех остальных измерений. Другими словами, мы циклически перебираем все измерения и вычисляем условное распределение на каждой итерации. Именно это делает функция `sample`. Для каждого выборочного значения выполняем итерацию по всем измерениям и вычисляем среднее значение и ковариацию условного гауссовского распределения. Пользуясь последним, мы затем генерируем и запоминаем значение случайной величины. Повторяем этот процесс до тех пор, пока не будет достигнут максимальный размер выборки. Давайте посмотрим на алгоритм семплирования по Гиббсу в действии для выборки с двумерным гауссовым распределением.

Листинг 2.4. Семплирование по Гиббсу

```
import numpy as np
import matplotlib.pyplot as plt

import itertools
from numpy.linalg import inv
from scipy.stats import multivariate_normal

np.random.seed(42)

class gibbs_gauss:
    def gauss_conditional(self, mu, Sigma, setA, x):
        dim = len(mu)
        setU = set(range(dim))
        setB = setU.difference(setA)
        muA = np.array([mu[item] for item in setA]).reshape(-1,1)
        muB = np.array([mu[item] for item in setB]).reshape(-1,1)
        xB = np.array([x[item] for item in setB]).reshape(-1,1)

        Sigma_AA = []
        for (idx1, idx2) in itertools.product(setA, setA):
            Sigma_AA.append(Sigma[idx1][idx2])
        Sigma_AA = np.array(Sigma_AA).reshape(len(setA),len(setA))

        Sigma_AB = []
        for (idx1, idx2) in itertools.product(setA, setB):
            Sigma_AB.append(Sigma[idx1][idx2])
        Sigma_AB = np.array(Sigma_AB).reshape(len(setA),len(setB))

        Sigma_BB = []
        for (idx1, idx2) in itertools.product(setB, setB):
            Sigma_BB.append(Sigma[idx1][idx2])
        Sigma_BB = np.array(Sigma_BB).reshape(len(setB),len(setB))
```

Вычисляет $P(X_A|X_B=x) = N(\mu_{A|B}, \Sigma_{A|B})$

```

Sigma_BB_inv = inv(Sigma_BB)
mu_AgivenB = muA + np.matmul(np.matmul(Sigma_AB, Sigma_BB_inv),
    ➤ xB - muB)
Sigma_AgivenB = Sigma_AA - np.matmul(np.matmul(Sigma_AB,
    ➤ Sigma_BB_inv), np.transpose(Sigma_AB))

return mu_AgivenB, Sigma_AgivenB

def sample(self, mu, Sigma, xinit, num_samples):
    dim = len(mu)
    samples = np.zeros((num_samples, dim))
    x = xinit
    for s in range(num_samples):
        for d in range(dim):
            mu_AgivenB, Sigma_AgivenB = self.gauss_conditional(mu, Sigma,
                ➤ set([d]), x)
            x[d] = np.random.normal(mu_AgivenB, np.sqrt(Sigma_AgivenB))
        # конец for
        samples[s,:] = np.transpose(x)
    # конец for
    return samples

if __name__ == "__main__":

    num_samples = 2000
    mu = [1, 1]
    Sigma = [[2,1], [1,1]]
    xinit = np.random.rand(len(mu),1)
    num_burnin = 1000

    gg = gibbs_gauss()
    gibbs_samples = gg.sample(mu, Sigma, xinit, num_samples)

    scipy_samples = multivariate_normal.rvs
    ➤ (mean=mu,cov=Sigma,size=num_samples,random_state=42)

    plt.figure()
    plt.scatter(gibbs_samples[num_burnin:,0], gibbs_samples[num_burnin:,1],
        ➤ label='Выборка по Гиббсу')
    plt.grid(True); plt.legend(); plt.xlim([-4,5])
    plt.title("Семплирование по Гиббсу многомерного гауссиана");
    ➤ plt.xlabel("X1"); plt.ylabel("X2")
    plt.show()

    plt.figure()
    plt.scatter(scipy_samples[num_burnin:,0], scipy_samples[num_burnin:,1],
        ➤ label='Эталонная выборка')
    plt.grid(True); plt.legend(); plt.xlim([-4,5])
    plt.title("Эталонная выборка многомерного гауссиана");
    ➤ plt.xlabel("X1");
    plt.ylabel("X2")
    plt.show()

```

Из рис. 2.12 видно, что выборка по Гиббсу похожа на эталонную для двумерного нормального распределения с параметрами $\mu = [1, 1]^T$ и $\Sigma = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$. В следующем разделе мы рассмотрим более общий вариант МСМС-семплирования: алгоритм Метрополиса — Гастингса.

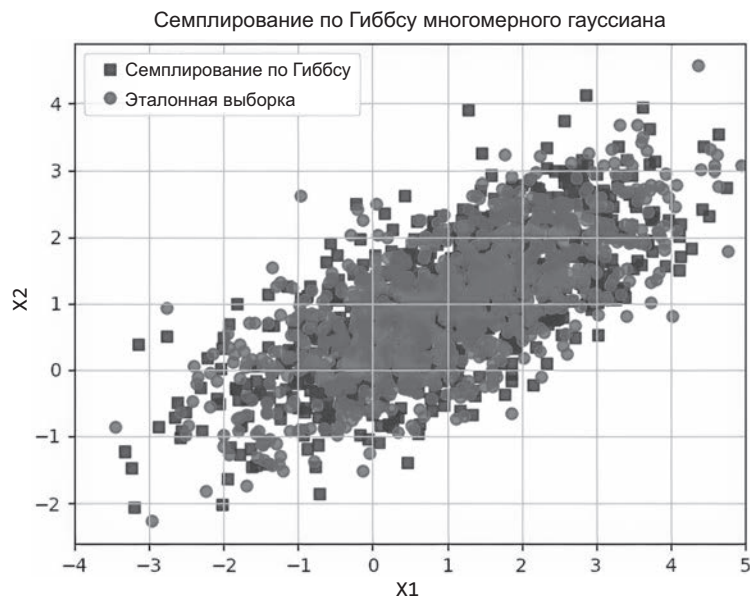


Рис. 2.12. Семплирование по Гиббсу для многомерного нормального распределения

2.6. Алгоритм Метрополиса — Гастингса

Давайте рассмотрим алгоритм семплирования МСМС, имеющий более общий характер. Наша цель — построить цепь Маркова, стационарное распределение которой равно нашему целевому распределению $p(x)$. Целевое распределение $p(x)$ — это распределение (обычно апостериорное $p(\theta|x)$ или функция плотности $p(\theta)$), для которого мы хотим получить выборку.

Основная идея алгоритма Метрополиса — Гастингса (МН, Metropolis–Hastings) состоит в том, чтобы на основе текущего состояния x сгенерировать новое x' согласно вспомогательному распределению $q(x'|x)$, а затем либо принять, либо отклонить предложенное состояние в соответствии с коэффициентом МН, обеспечивающим соблюдение детального равновесия, как показано в формуле 2.18:

$$p(x')q(x|x') = p(x)q(x'|x). \quad (2.18)$$

Согласно уравнению детального равновесия, вероятность перехода из состояния x равна вероятности перехода в состояние x . Чтобы вывести коэффициент МН, предположим, что равенство в уравнении детального равновесия не

выполняется. Тогда должен существовать такой поправочный коэффициент $r(x'|x)$, что обе стороны становятся равны. Решение получившегося уравнения приводит к следующей формуле для коэффициента МН:

$$\begin{aligned} p(x')q(x|x') &= r(x'|x)p(x)q(x'|x) \\ r(x'|x) &= \min \left[1, \frac{p(x')q(x|x')}{p(x)q(x'|x)} \right]. \end{aligned} \quad (2.19)$$

Алгоритм Метрополиса — Гастингса в обобщенном виде представлен в виде псевдокода на рис. 2.13.

1: Инициализировать x_0 случайным образом 2: for $k = 0, 1, 2, \dots$ do 3: предложить новое значение $x' \sim q(x' x_k)$ 4: вычислить коэффициент Метрополиса — Гастингса: 5: $r(x' x) = \min \left[1, \frac{p(x')q(x x')}{p(x)q(x' x)} \right]$ 6: присвоить $x_{k+1} = \begin{cases} x', & \text{с вероятн. } r(x' x) \\ x_k, & \text{с вероятн. } 1 - r(x' x) \end{cases}$ 7: конец for	Семплирование вспомогательного распределения Значение принимается Значение отклоняется
---	---

Рис. 2.13. Псевдокод алгоритма Метрополиса — Гастингса

Давайте реализуем алгоритм МН для многомерной гауссовой смеси в качестве целевого распределения и вспомогательного нормального распределения:

$$\begin{aligned} p(x) &= \sum_{k=1}^K \pi(k) \mathcal{N}(x; \mu_k, \Sigma_k); \\ q(x'|x) &= \mathcal{N}(x'|x, \Sigma). \end{aligned} \quad (2.20)$$

Рассмотрим код, приведенный в листинге 2.5 ниже. Класс `mh_gauss` содержит следующие компоненты. Метод `target_pdf` определяет целевое распределение (в нашем случае гауссову смесь $p(x)$). Метод `proposal_pdf` определяет вспомогательное распределение (многомерное нормальное). Метод `sample` разыгрывает новое состояние согласно вспомогательному распределению $q(x'|x_k)$, зависящему от предыдущего состояния x_k , вычисляет коэффициент Метрополиса — Гастингса и либо принимает новое значение с вероятностью $r(x'|x)$, либо отклоняет его с вероятностью $1 - r(x'|x)$.

Листинг 2.5. Семплирование Метрополиса — Гастингса

```
import numpy as np
import matplotlib.pyplot as plt

from scipy.stats import uniform
```

58 Глава 2. Марковские цепи Монте-Карло

```
from scipy.stats import multivariate_normal

np.random.seed(42)

class mh_gauss:

    def __init__(self, dim, K, num_samples, target_mu, target_sigma,
        ➤ target_pi, proposal_mu, proposal_sigma):
        self.dim = dim
        self.K = K
        self.num_samples = num_samples
        self.target_mu = target_mu
        self.target_sigma = target_sigma
        self.target_pi = target_pi

        self.proposal_mu = proposal_mu
        self.proposal_sigma = proposal_sigma

        self.n_accept = 0
        self.alpha = np.zeros(self.num_samples)
        self.mh_samples = np.zeros((self.num_samples,
            ➤ self.dim))

    def target_pdf(self, x):
        prob = 0
        for k in range(self.K):
            prob += self.target_pi[k]*\
                multivariate_normal.pdf(x, self
                ➤ .target_mu[:,k], self.target_sigma[:, :, k])
        # конец for
        return prob

    def proposal_pdf(self, x):
        return multivariate_normal.pdf(x, self
        ➤ .proposal_mu, self.proposal_sigma)

    def sample(self):
        x_init = multivariate_normal
        ➤ .rvs(self.proposal_mu, self.proposal_sigma, 1)
        self.mh_samples[0,:] = x_init

        for i in range(self.num_samples-1):
            x_curr = self.mh_samples[i,:]
            x_new = multivariate_normal.rvs(x_curr, self
            ➤ .proposal_sigma, 1)

            self.alpha[i] = self.proposal_pdf(x_new) /
            ➤ self.proposal_pdf(x_curr)
            self.alpha[i] = self.alpha[i] *
            ➤ (self.target_pdf(x_new)/self.target_pdf(x_curr))

            r = min(1, self.alpha[i])
            u = uniform.rvs(loc=0, scale=1, size=1)
            if (u <= r):
```

Параметры целевого распределения:
 $p(x) = \sum_k \pi(k) N(x; \mu_k, \Sigma_k)$

Параметры вспомогательного распределения: $q(x) = N(x; \mu, \Sigma)$

Параметры цепочки семплирования

Плотность целевого распределения:
 $p(x) = \sum_k \pi(k) N(x; \mu_k, \Sigma_k)$

Плотность вспомогательного распределения:
 $q(x) = N(x; \mu, \Sigma)$

Генерирует начальное значение для вспомогательного распределения

Коэффициент МН

Вероятность принятия МН


```

        self.n_accept += 1
        self.mh_samples[i+1,:] = x_new ← Принимаются
    else:
        self.mh_samples[i+1,:] = x_curr ← Отвергается
# конец for
print("MH acceptance ratio: ", self.n_accept/float(self.num_samples))
⇒ self.n_accept/float(self.num_samples))

if __name__ == "__main__":

    dim = 2
    K = 2
    num_samples = 5000
    target_mu = np.zeros((dim, K))
    target_mu[:,0] = [4,0]
    target_mu[:,1] = [-4,0]
    target_sigma = np.zeros((dim, dim, K))
    target_sigma[:, :,0] = [[2,1],[1,1]]
    target_sigma[:, :,1] = [[1,0],[0,1]]
    target_pi = np.array([0.4, 0.6])

    proposal_mu = np.zeros((dim,1)).flatten()
    proposal_sigma = 10*np.eye(dim)

    mhg = mh_gauss(dim, K, num_samples, target_mu, target_sigma, target_pi,
    ⇒ proposal_mu, proposal_sigma)
    mhg.sample()

    plt.figure()
    plt.scatter(mhg.mh_samples[:,0], mhg.mh_samples[:,1],
    ⇒ label='Выборочные значения МН')
    plt.grid(True); plt.legend()
    plt.title("Семплирование Метрополиса – Гастингса для двумерного нормального
    ⇒ распределения")
    plt.xlabel("X1"); plt.ylabel("X2")
    plt.show()

```

Из рис. 2.14 следует, что выборка МН похожа на эталонную смесь двух двумерных нормальных распределений со средними значениями $\mu_1 = [4, 0]$, $\mu_2 = [-4, 0]$, ковариациями $\Sigma_1 = [[2, 1], [1, 1]]$, $\Sigma_2 = [[1, 0], [0, 1]]$ и количественным соотношением $\pi = [0.4, 0.6]$.

Обратите внимание, что мы можем свободно выбирать любое вспомогательное распределение $q(x'|x)$, способствующее гибкости метода. Правильный выбор вспомогательного распределения означает высокий уровень принятия выборочных значений.

В нашей реализации мы выбрали симметричное нормальное распределение, центрированное относительно текущего состояния, как показано в формуле 2.21:

$$q(x'|x) = \mathcal{N}(x'|x, \Sigma). \quad (2.21)$$

Такой подход известен как *алгоритм случайного блуждания Метрополиса*. Если используется вспомогательное распределение вида $q(x'|x) = q(x')$, где новое состояние не зависит от старого, то мы получаем независимое семплирование (independence sampler), аналогичное выборке по значимости, которую мы рассмотрим в следующем разделе.

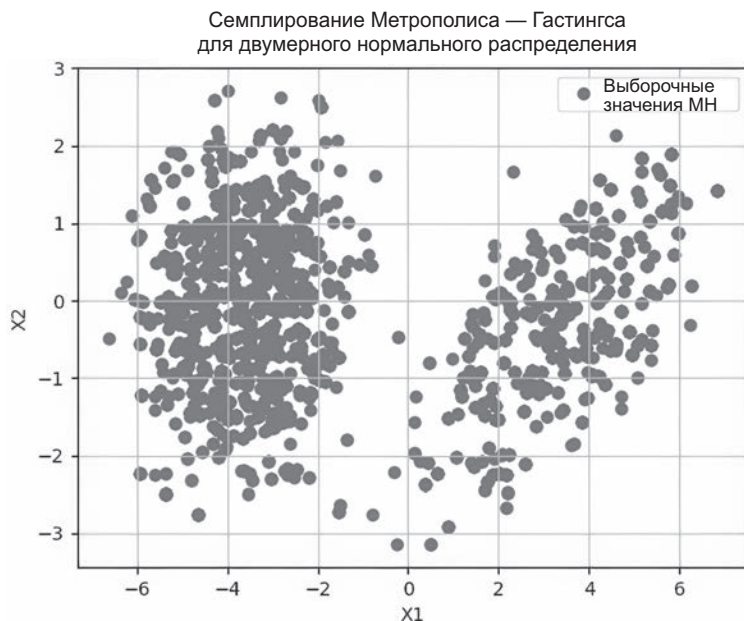


Рис. 2.14. Выборка Метрополиса — Гастингса для многомерной гауссовой смеси

Также обратите внимание, что для вычисления коэффициента МН нам не нужно знать нормирующие постоянные Z наших распределений, поскольку значения Z в формуле взаимно сокращаются. Существует связь между алгоритмом МН и семплированием по Гиббсу: вероятность принятия в алгоритме Гиббса всегда равна 1.

2.7. Выборка по значимости

Выборка по значимости (importance sampling, IS) — это алгоритм Монте-Карло для оценки интегралов следующего вида:

$$E[f(x)] = \int p(x)f(x)dx. \quad (2.22)$$

В основе метода выборки по значимости лежит идея о том, чтобы генерировать выборочные значения в «полезных» областях (то есть там, где и $p(x)$, и $|f(x)|$

велики). Отбор значений при этом осуществляется путем семплирования более простого вспомогательного распределения $q(x)$. Таким образом, мы можем вычислить математическое ожидание $f(x)$ относительно целевого распределения $p(x)$, семплируя вспомогательное распределение $q(x)$ и используя интегрирование по методу Монте-Карло:

$$E[f(x)] = \int f(x) \frac{p(x)}{q(x)} q(x) dx \approx \frac{1}{N} \sum_{i=1}^N w(x_i) f(x_i), \text{ где } x_i \sim q(x). \quad (2.23)$$

В формуле 2.23 мы определили весовые коэффициенты значимости как $w(x) = p(x)/q(x)$. Давайте рассмотрим пример, в котором мы воспользуемся выборкой по значимости для аппроксимации математического ожидания $f(x) = 2\sin(\pi x/1.5)$, $x \geq 0$. Нам требуется вычислить математическое ожидание относительно ненормированного хи-распределения, параметризованного нецелочисленным параметром степеней свободы k :

$$p(x) \sim x^{(k-1)} \exp\left\{-\frac{x^2}{2}\right\}, \quad x \geq 0. \quad (2.24)$$

Мы будем использовать более простой вариант семплирования вспомогательного распределения $q(x)$:

$$q(x) \sim N(x; 0.8; 1.5). \quad (2.25)$$

Давайте разберем псевдокод на рис. 2.15.

```

1: class importance_sampler
2:   function sample(N):
3:     for i = 1 to N:
4:        $x_i \sim q(x) = N(x; \mu, \sigma^2)$  ← Семплирование вспомогательного распределения
5:        $w(x_i) = \frac{p(x_i)}{q(x_i)}$  ← Вычисление весов значимости
6:     конец for
7:    $E[f(x)] = \frac{1}{N} \sum_{i=1}^N w(x_i) f(x_i)$ 
8:   return  $w(x), E[f(x)]$ 
```

Рис. 2.15. Псевдокод выборки по значимости

Класс `importance_sampler` содержит функцию `sample`, которая принимает размер выборки N в качестве входного аргумента. Внутри функции `sample` мы выполняем цикл по всей выборке. В каждой итерации мы производим семплирование вспомогательного распределения $q(x)$ и вычисляем весовой коэффициент значимости как отношение целевого распределения $p(x)$ к вспомогательному $q(x)$. В конечном итоге мы вычисляем интеграл по методу Монте-Карло, взвешивая рассматриваемую функцию $f(x)$ по весам значимости, суммируя по всей выборке и деля на N . Подробности реализации отражены в листинге ниже:

Листинг 2.6. Выборка по значимости

```

import numpy as np
import matplotlib.pyplot as plt

from scipy.integrate import quad
from scipy.stats import multivariate_normal

np.random.seed(42)

class importance_sampler:

    def __init__(self, k=1.5, mu=0.8, sigma=np.sqrt(1.5), c=3):
        self.k = k  # Целевые параметры p(x)

        self.mu = mu  # Параметры вспомогательного
        self.sigma = sigma  # распределения q(x)
        self.c = c  # Назначает c такое, что p(x) < c q(x)

    def target_pdf(self, x):
        return (x**(self.k-1)) * np.exp(-x**2/2.0)  # p(x) ~ Chi(k=1.5)

    def proposal_pdf(self, x):
        return self.c * 1.0/np.sqrt(2*np.pi*1.5) *
            np.exp(-(x-self.mu)**2/(2*self.sigma**2))  # q(x) ~ N(mu,sigma)

    def fx(self, x):
        return 2*np.sin((np.pi/1.5)*x)  # Рассматриваемая функция f(x), x >= 0

    def sample(self, num_samples):
        x = multivariate_normal.rvs(self.mu,
            np.sqrt(self.sigma), num_samples)  # Семплирование вспомогательного распределения

        idx = np.where(x >= 0)  # Отбрасываем отрицательные значения, поскольку f(x)
        x_pos = x[idx]  # определена для x >= 0

        isw = self.target_pdf(x_pos) /
            self.proposal_pdf(x_pos)  # Вычисление весов значимости

        fw = (isw/np.sum(isw))*self.fx(x_pos)  # Вычисление E[f(x)] = sum_i f(x_i) w(x_i),
        f_est = np.sum(fw)  # где x_i ~ q(x)

        return isw, f_est

if __name__ == "__main__":

    num_samples = [10, 100, 1000, 10000, 100000, 1000000]

    F_est_iter, IS_weights_var_iter = [], []
    for k in num_samples:
        IS = importance_sampler()
        IS_weights, F_est = IS.sample(k)
        IS_weights_var = np.var(IS_weights/np.sum(IS_weights))
        F_est_iter.append(F_est)

```

```

IS_weights_var_iter.append(IS_weights_var)

# эталонное значение (численное интегрирование)
k = 1.5
I_gt, _ = quad(lambda x: 2.0*np.sin((np.pi/1.5)*x)*(x**(k-1))*np
    ➤ .exp(-x**2/2.0), 0, 5)

# создание графиков
plt.figure()
xx = np.linspace(0,8,100)
plt.plot(xx, IS.target_pdf(xx), '-r', label='Плотность целевого
    ➤ распределения p(x)')
plt.plot(xx, IS.proposal_pdf(xx), '-b', label='Плотность вспомогательного
    ➤ распределения q(x)')
plt.plot(xx, IS.fx(xx) * IS.target_pdf(xx), '-k', label='Подынтегральное
    ➤ выражение p(x)f(x)')
plt.grid(True); plt.legend(); plt.xlabel("X1"); plt.ylabel("X2")
plt.title("Компоненты выборки по значимости")
plt.show()

plt.figure()
plt.hist(IS_weights, label = "Веса выборки по значимости")
plt.grid(True); plt.legend();
plt.title("График весов значимости")
plt.show()

plt.figure()
plt.semilogx(num_samples, F_est_iter, label = "Оценка
    ➤ по значимости E[f(x)]")
plt.semilogx(num_samples, I_gt*np.ones(len(num_samples)), label = "Эталонное
    ➤ значение")
plt.grid(True); plt.legend(); plt.xlabel('Итерации'); plt.ylabel("E[f(x)]
    ➤ оценка")
plt.title("Оценка по значимости E[f(x)]")
plt.show()

```

На рис. 2.16 (слева) показаны плотности распределения: целевая $p(x)$ и вспомогательная $q(x)$ — и подынтегральное выражение $p(x)f(x)$. Обратите внимание, что мы масштабировали вспомогательную плотность $q(x)$ с помощью константы c таким образом, чтобы $p(x) < cq(x)$ для всех $x \geq 0$. Кроме того, мы ограничивали выборку из $q(x)$ положительными значениями (то есть осуществляли семплирование усеченной гауссовой функции), чтобы соответствовать нашему ограничению $x \geq 0$ для $f(x)$. На рис. 2.16 показано также изменение в оценке $E[f(x)]$ в зависимости от размера выборки в сравнении с эталонным значением, вычисленным с помощью численного интегрирования.

В предыдущем примере мы использовали нормированное вспомогательное распределение. Однако в случаях более сложных распределений мы часто не знаем нормирующую постоянную (она же функция разбиения). Но поскольку мы работаем с отношениями плотностей, то, как будет видно далее, все вычисления справедливы и для ненормированных распределений. Кроме того, мы

рассмотрим способ оценки соотношения функций разбиения и обсудим свойства IS при производстве оценки.

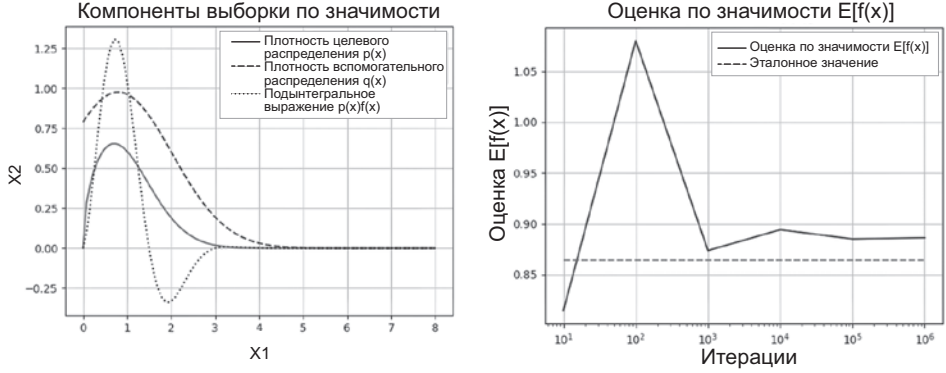


Рис. 2.16. Компоненты выборки по значимости (слева) и оценка матожидания по значимости (справа)

Давайте определим нормированные плотности распределения $p(x)$ и $q(x)$ следующим образом:

$$p(x) = \frac{1}{Z_p} \tilde{p}(x); \quad q(x) = \frac{1}{Z_q} \tilde{q}(x). \quad (2.26)$$

Здесь z_p и z_q — нормирующие постоянные для $p(x)$ и $q(x)$ соответственно, а $\tilde{p}(x)$ и $\tilde{q}(x)$ — ненормированные распределения. Мы можем записать нашу оценку следующим образом:

$$\begin{aligned} E[f(x)] &= \int f(x)p(x)dx = \\ &= \int f(x)\frac{q(x)}{q(x)}p(x)dx = \frac{Z_q}{Z_p} \int f(x)\frac{\tilde{p}(x)}{\tilde{q}(x)}q(x)dx \approx \\ &\approx \frac{Z_q}{Z_p} \frac{1}{S} \sum_{s=1}^S \tilde{w}(s)f(s), \text{ где } s \sim q(x) \text{ и } \tilde{w}(s) = \frac{\tilde{p}(s)}{\tilde{q}(s)}. \end{aligned} \quad (2.27)$$

Обратите внимание, что в уравнении 2.27 $\tilde{w}(s)$ — это ненормированные веса значимости. Мы можем вычислить отношение нормирующих постоянных следующим образом:

$$\frac{Z_p}{Z_q} = \frac{1}{Z_q} \int \tilde{p}(x)dx = \frac{1}{Z_q} \int \frac{\tilde{p}(x)}{q(x)}q(x)dx = \int \frac{\tilde{p}(x)}{\tilde{q}(x)}q(x)dx = \frac{1}{S} \sum_{s=1}^S \tilde{w}(s). \quad (2.28)$$

Объединив два приведенных выше выражения, мы получим такую формулу:

$$E[f(x)] \approx \frac{Z_q}{Z_p} \frac{1}{S} \sum_{s=1}^S \tilde{w}(s) f(s) = \frac{\frac{1}{S} \sum_s \tilde{w}(s) f(s)}{\frac{1}{S} \sum_s \tilde{w}(s)} = \sum_{s=1}^S w(s) f(s). \quad (2.29)$$

Формула 2.29 подтверждает наши вычисления в примере кода для оценки $E[f(x)]$. Давайте рассмотрим несколько свойств метода оценки по значимости (IS estimator) и найдем численные показатели, характеризующие производительность IS. Оценка по значимости может быть определена следующим образом:

$$\hat{a} = E[a(x)] = \frac{\frac{1}{n} \sum_{i=1}^n \tilde{w}^i a(x_i)}{\frac{1}{n} \sum_{i=1}^n \tilde{w}^i} = \frac{1}{n} \sum_{i=1}^n \tilde{w}_*^i a(x_i), \quad (2.30)$$

где $\tilde{w}_*^i = \frac{\tilde{w}^i}{\frac{1}{n} \sum_{i=1}^n \tilde{w}^i}$.

Согласно обычному закону больших чисел (weak law of large numbers, WLLN), подразумевающему независимость выборки x_i , известно, что среднее значение последовательности независимых и одинаково распределенных случайных величин сходится по вероятности к математическому ожиданию:

$$a = E[w_*(X_i) a(X_i)] \rightarrow b(\hat{a}) = E[\hat{a}] - a = 0. \quad (2.31)$$

Здесь предполагается, что X_i являются независимыми и одинаково распределенными случайными величинами, распределенными согласно $q(x)$. Как результат, выборка по значимости, в теории, дает несмещенную оценку. Дисперсия оценки по значимости вычисляется следующим образом:

$$\text{VAR}(\hat{a}) = \frac{\sigma^2}{n} = \frac{1}{n} \frac{\sum_{i=1}^n (\tilde{w}^i (a(x_i) - \hat{a}))^2}{\left(\sum_{i=1}^n \tilde{w}^i\right)^2}. \quad (2.32)$$

Чтобы убедиться, что оценка по значимости является состоятельной, нам нужно показать, что по мере увеличения размера выборки n оценка сходится по вероятности к истинному значению:

$$\lim_{n \rightarrow \infty} P(|\hat{a}_n - a| > \varepsilon) = 0. \quad (2.33)$$

Используя неравенство Чебышева, мы можем записать предел отклонения следующим образом:

$$\lim_{n \rightarrow \infty} P(|\hat{a}_n - a| > \varepsilon) \leq \frac{\text{VAR}(\hat{a})}{\varepsilon^2} = \frac{\sigma^2}{n\varepsilon^2} = 0. \quad (2.34)$$

Это означает, что оценка по значимости является состоятельной. Здесь нам не потребовалось предположение о независимости выборки, но мы использовали допущение, что дисперсия весов значимости конечна. Таким образом, ожидается, что дисперсия оценки будет уменьшаться по мере увеличения выборки. Одним из факторов, влияющих на дисперсию оценки, является величина весов значимости. Поскольку $w(x_i) = p(x_i)/q(x_i)$, то веса малы, когда $q(x_i)$ похожа на $p(x_i)$ и имеет тяжелые хвосты распределения. При подобных благоприятных обстоятельствах дисперсия нашей оценки становится мала.

Хотелось бы, однако, уметь распознавать те случаи, когда веса значимости могут представлять проблему (например, когда только пара весов доминирует во взвешенной сумме). Когда в выборке присутствует корреляция, дисперсия равна σ^2/n_{eff} , где n_{eff} — эффективный размер выборки. Чтобы получить выражение для эффективного размера выборки, приравняем дисперсию взвешенного среднего к дисперсии простого среднего значения:

$$\text{VAR}\left(\frac{1}{n}\sum_{i=1}^n a(x_i)\right) = \frac{\sigma^2}{n_{eff}} = \text{VAR}\left(\frac{\frac{1}{n}\sum_{i=1}^n \tilde{w}^i a(x_i)}{\frac{1}{n}\sum_{i=1}^n \tilde{w}^i}\right), \quad (2.35)$$

где $\text{VAR}(a(x_i)) = \sigma^2$.

В результате для n_{eff} мы получаем следующую формулу:

$$n_{eff} = \frac{\left(\sum_{i=1}^n \tilde{w}_i\right)^2}{\sum_{i=1}^n \tilde{w}_i^2} = \frac{n}{1 + \text{VAR}(\tilde{w}_i^*)}. \quad (2.36)$$

Мы можем использовать n_{eff} в качестве своего рода диагностического средства для выборки по значимости, где целевое значение n_{eff} зависит от решаемой задачи. Сфера применения выборки по значимости не ограничивается аппроксимацией линейных оценок, таких как математическое ожидание и интегралы, или нелинейных оценок. В области обработки сигналов выборку по значимости можно использовать для восстановления сигнала при малом количестве измерений.

2.8. Упражнения

2.1. Выведите условные распределения вероятностей $p(x_A|x_B)$ для многомерного распределения Гаусса, где A и B — подмножества совместно распределенных нормальных случайных величин $x_1, x_2, \dots, x_n$.

2.2. Выведите частные распределения $p(x_A)$ и $p(x_B)$ для многомерного нормального распределения, где A и B — подмножества совместно распределенных нормальных случайных величин $x_1, x_2, \dots, x_n$.

2.3. Пусть $y \sim N(\mu, \Sigma)$, где $\Sigma = LL^T$. Покажите, что выборку y можно получить следующим образом: $x \sim N(0, I)$; $y = Lx + \mu$.

Итоги

- Марковская цепь Монте-Карло — это методология семплирования в высоко-размерных пространствах параметров для аппроксимации апостериорного распределения $p(\theta|x)$. Интегрирование методом Монте-Карло имеет преимущество перед численным интегрированием (вычисляющим функцию по фиксированной сетке точек) благодаря тому, что функция рассчитывается только в тех областях, где вероятность велика.
- В модели биномиального дерева цен на акции предполагается, что на каждом временном шаге цена на акцию может либо расти, либо снижаться на разную величину, что типично для рискованных активов.
- Самоизбегающие случайные блуждания — это такие случайные блуждания, которые не проходят через одну точку дважды. Для моделирования такого блуждания можно использовать интеграцию по методу Монте-Карло.
- Семплирование по Гиббсу основано на идее выборки из многомерного распределения по одной переменной за раз при условии, что последние выборочные значения всех остальных переменных фиксированы.
- Основная идея алгоритма Метрополиса — Гастингса состоит в том, чтобы на основе текущего состояния x сгенерировать новое x' согласно вспомогательному распределению $q(x'|x)$, а затем либо принять, либо отклонить предложенное состояние в соответствии с коэффициентом МН, обеспечивающим соблюдение детального равновесия.
- В основе метода выборки по значимости лежит идея о том, чтобы генерировать выборочные значения в «полезных» областях (то есть там, где и $p(x)$, и $|f(x)|$ велики). Отбор значений при этом осуществляется путем семплирования более простого вспомогательного распределения $q(x)$.

3

Вариационный вывод

В этой главе

- ✓ Введение в вариационный вывод KL
- ✓ Приближение среднего поля
- ✓ Удаление шума с изображений в модели Изинга
- ✓ Максимизация взаимной информации

В предыдущей главе мы рассмотрели один из двух основных методов байесовского вывода — марковскую цепь Монте-Карло. Мы изучили различные алгоритмы семплирования и аппроксимировали апостериорное распределение при помощи выборки. В этой главе мы обсудим второй тип байесовского вывода — вариационный вывод. *Вариационный вывод* (variational inference, VI) — важный класс алгоритмов приближенного вывода. Его основная идея состоит в том, чтобы выбрать приближенное распределение $q(x)$ из семейства удобных или легко вычисляемых распределений с обучаемыми параметрами, а затем сделать это приближение как можно более близким к истинному апостериорному распределению $p(x)$.

Как мы увидим в разделе, посвященном среднему полю, приближенное распределение $q(x)$ может принимать вид полностью факторизованного представления совместного апостериорного распределения. Подобная факторизация значительно ускоряет вычисления. Мы введем KL-дивергенцию и будем использовать ее как меру близости нашего приближенного распределения к истинному апостериорному. Минимизация KL-дивергенции фактически сводит VI к задаче оптимизации.

В следующем разделе мы определим нижнюю вариационную границу (evidence lower bound, ELBO) и интерпретируем ее тремя различными способами, которые понадобятся при реализации алгоритма аппроксимации среднего поля для уменьшения шума на изображениях. Алгоритм устранения шума был выбран по той причине, что он наглядно иллюстрирует концепции, обсуждаемые в этом разделе.

3.1. Вариационный вывод KL

Мы можем использовать KL-дивергенцию для измерения расстояния между распределениями вероятностей. Это особенно полезно при аппроксимации целевого распределения, поскольку мы хотим выяснить, насколько близко наше приближение к истинному распределению. Пусть $q(x)$ — приближенное распределение, а $p(x)$ — целевое апостериорное распределение. Тогда обратная KL-дивергенция определяется следующим образом:

$$KL(q \parallel p) = \sum_x \overbrace{q(x)}^{\text{Приближенное распределение}} \log \frac{q(x)}{\underbrace{p(x)}_{\text{Логарифм отношения приближенной функции к истинной}}}. \quad (3.1)$$

Рассмотрим простой пример, в котором наше целевое распределение является стандартным одномерным нормальным распределением $p(x) \sim N(0, 1)$, а приближенное распределение — одномерным нормальным распределением со средним значением μ и дисперсией σ^2 : $q(x) \sim N(\mu, \sigma^2)$. Тогда мы можем записать $KL(q \parallel p)$ следующим образом:

$$\begin{aligned} KL(q \parallel p) &= \int q(x) \log \frac{q(x)}{p(x)} = \\ &= -\int q(x) \log p(x) + \int q(x) \log q(x) = \\ &= -\int q(x) \left[-\frac{1}{2} \log 2\pi - \frac{1}{2} x^2 \right] + \\ &\quad + \int q(x) \left[-\frac{1}{2} \log 2\pi\sigma^2 - \frac{1}{2\sigma^2} (x - \mu)^2 \right] = \\ &= \left[\frac{1}{2} \log 2\pi + \frac{1}{2} (\sigma^2 + \mu^2) \right] + \left[-\frac{1}{2} \log 2\pi\sigma^2 - \frac{1}{2} \right] = \\ &= -\frac{1}{2} (1 + \log \sigma^2 - \mu^2 - \sigma^2). \end{aligned} \quad (3.2)$$

Мы можем изобразить на графике то, как изменяется $KL(q \parallel p)$ при изменении параметров приближенного распределения $q(x)$. Давайте зафиксируем $\sigma^2 = 4$ и будем варьировать среднее значение $\mu \in [-4, 4]$. Тогда мы получим график, изображенный на рис. 3.1.

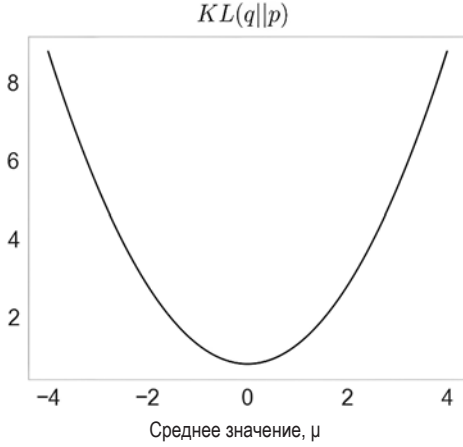


Рис. 3.1. $KL(q||p)$ для $p(x) \sim N(0,1)$ и $q(x) \sim N(\mu, 4)$

Обратите внимание, что KL-дивергенция неотрицательна и минимальна при $\mu = 0$ (то есть когда среднее значение приближенного распределения равно среднему значению целевого распределения $p(x) \sim N(0, 1)$). Мы можем интерпретировать KL-дивергенцию как меру расстояния между распределениями.

Пусть $\tilde{p}(x) = p(x)Z$ — ненормированное распределение. Тогда рассмотрим следующую целевую функцию:

$$\begin{aligned} J(q) &= KL(q||\tilde{p}) = \\ &= \sum_x q(x) \log \frac{q(x)}{p(x)Z} = \sum_x q(x) \log \frac{q(x)}{p(x)} - \log Z = \\ &= KL(q||p) - \log Z. \end{aligned} \quad (3.3)$$

Поскольку KL-дивергенция неотрицательна, $J(q)$ является верхней границей для предельного правдоподобия (marginal likelihood).

$$J(q) = KL(q||p) - \log Z \geq -\log Z = -\log p(D). \quad (3.4)$$

Если $q(x)$ совпадает с истинным апостериорным распределением $p(x)$, то KL-дивергенция сходит на нет. Оптимальное значение $J(q^*)$ равно логарифму функции распределения, а для всех остальных значений q оно дает границу. $J(q)$ называется *вариационной свободной энергией* (variational free energy) и может быть записана следующим образом:

$$\min_q J(q) = E_q[\log q(x)] + E_q[-\log \tilde{p}(x)] = -H(q) + E_q[E(x)]. \quad (3.5)$$

Целевая функция вариационного метода в формуле 3.5 имеет близкую аналогию с минимизацией энергии в статистической физике. Первый член максимизирует энтропию и потому работает как регуляризатор, а второй член представляет собой

ожидаемую энергию и способствует тому, чтобы вариационное распределение q лучше объясняло данные.

Обратная KL-дивергенция, которая выступает в качестве штрафного члена в целевой функции, также известна как *информационная проекция* (information projection), или *I-проекция*. При минимизации обратной KL-дивергенции $q(x)$, как правило, недооценивает носитель $p(x)$ и фиксируется на одной из его мод. Это связано с тем, что для того, чтобы KL-дивергенция оставалась конечной, $q(x) = 0$ во всех случаях, когда $p(x) = 0$. С другой стороны, результатом минимизации прямой KL-дивергенции, известной как *проекция момента* (moment projection), или *M-проекция*, будет $q(x)$, избегающее нулей и переоценивающее носитель $p(x)$, как показано на рис. 3.2.

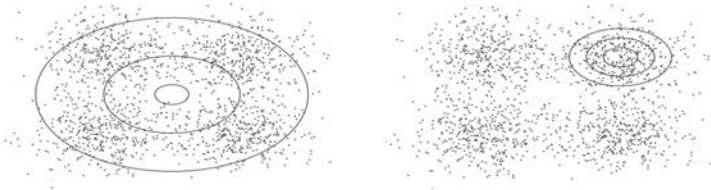


Рис. 3.2. $q(x)$ прямой KL-дивергенции (слева) переоценивает носитель, тогда как $q(x)$ обратной KL-дивергенции (справа) фиксируется на одной из мод

На рис. 3.2 показаны выборка из двумерной гауссовой смеси с четырьмя компонентами $p(x)$, а также эллипсы плотности приближенного распределения $q(x)$. Можно видеть, что оптимизация прямой KL-дивергенции приводит к тому, что $q(x)$ центрируется в нуле (в области низкой плотности), поскольку мы переоцениваем носитель $q(x)$. С другой стороны, оптимизация обратной KL-дивергенции приводит к тому, что $q(x)$ центрируется в одной из четырех мод гауссовой смеси.

Мы можем использовать неравенство Йенсена для получения ELBO — целевой функции, которую мы можем максимизировать, чтобы узнать вариационные параметры нашей модели. Пусть x — наши данные, а z — скрытые переменные. Тогда мы можем расписать целевую функцию ELBO следующим образом:

$$\begin{aligned}
 \log p(x) &= \log \sum_z p(x, z) = \log \sum_z \frac{q(z)}{q(z)} p(x, z) = \\
 &= \log E_{q(z)} \left[\frac{p(x, z)}{q(z)} \right] \geq E_{q(z)} \left[\log \frac{p(x, z)}{q(z)} \right] = \\
 &= E_{q(z)} [\log p(x, z)] - E_{q(z)} [\log q(z)] = ELBO.
 \end{aligned} \tag{3.6}$$

$\downarrow$
Энергия

$\downarrow$
Энтропия

Обратите внимание, что первый член — это средняя отрицательная энергия, а второй — энтропия. Хорошее апостериорное распределение благодаря этому

должно распределять бóльшую часть своей вероятностной меры по областям с низкой энергией (то есть с высокой общей плотностью вероятности), одновременно максимизируя энтропию $q(z)$. Таким образом, вариационный вывод, в отличие от MAP-оценки, предотвращает сжатие $q(z)$ в точку.

В следующем варианте записи ELBO делается акцент на том, что нижняя граница становится более плотной по мере того, как вариационное распределение становится ближе к апостериорному.

$$\begin{aligned} ELBO &= E_{q(z)} \left[\log \frac{p(x, z)}{q(z)} \right] = E_{q(z)} \left[\log \frac{p(z|x)p(x)}{q(z)} \right] = \\ &= -KL(q(z) \parallel p(z|x)) + \log p(x). \end{aligned} \quad (3.7)$$

↓
Расстояние между приближением $q(z)$
и апостериорной вероятностью $p(z|x)$

Как следствие, мы можем повысить ELBO, улучшив логарифм обоснованности (evidence) модели $\log p(x)$ с помощью априорного $p(z)$ или правдоподобия $p(z|x)$ или улучшив вариационное апостериорное приближение $q(z)$.

Наконец, мы можем записать ELBO следующим образом:

$$ELBO = \frac{1}{n} \sum_{i=1}^n [\underbrace{E_{q(z)} [\log p(x_i | z_i)]}_{\text{Выборочное правдоподобие}} - \overbrace{KL(q(z_i) \parallel p(z_i))}^{\text{Расстояние между приближением } q(z) \text{ и априорной} \\ \text{вероятностью } p(z) \text{ для выборочного значения}}]. \quad (3.8)$$

В этой формуле выделяется слагаемое правдоподобия для i -го наблюдения и слагаемое KL-дивергенции между каждым приближенным распределением и априорной вероятностью. Во всех предыдущих случаях математическое ожидание относительно $q(z)$ может быть вычислено путем семплирования приближенного распределения. В следующем разделе мы рассмотрим одно из наиболее часто используемых вариационных приближений.

3.2. Приближение среднего поля

Одной из самых популярных разновидностей вариационного вывода является *приближение среднего поля* (mean-field approximation), где мы предполагаем, что апостериорное распределение является полностью факторизованным приближением следующего вида:

$$q(x) = \prod_i q_i(x_i). \quad (3.9)$$

Здесь мы проводим оптимизацию по параметрам каждого частного распределения $q_i(x_i)$. Полностью факторизованное распределение можно наглядно представить так, как показано на рис. 3.3.

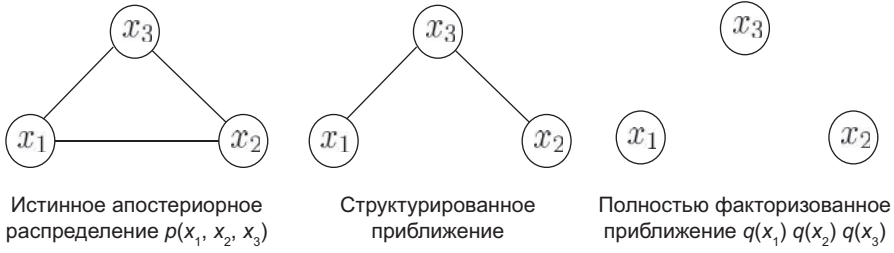


Рис. 3.3. Истинное апостериорное распределение (слева), структурированное приближение (посередине) и полностью факторизованная аппроксимация (справа)

Для распределения с тремя случайными величинами x_1, x_2 , и x_3 имеется истинное апостериорное распределение $p(x_1, x_2, x_3)$, которое мы пытаемся аппроксимировать при помощи полностью факторизованного распределения $q(x_1)q(x_2)q(x_3)$.

Наша цель — минимизировать вариационную свободную энергию $J(q)$ или, что эквивалентно, максимизировать нижнюю границу:

$$L(q) = -J(q) = \sum_x q(x) \log \frac{\tilde{p}(x)}{q(x)}. \quad (3.10)$$

Мы можем переписать целевую функцию для каждого частного распределения q_j при постоянстве остальных членов, как это показано в разделе 21.3 книги Кевина Мэрфи (Kevin Murphy) «Machine Learning: A Probabilistic Perspective» (MIT Press, 2012).

$$\begin{aligned}
 L(q_j) &= \sum_x \prod_i q_i(x_i) \left[\log \tilde{p}(x) - \sum_k \log q_k(x_k) \right] = \\
 &= \sum_{x_j} \sum_{x_{-j}} q_j(x_j) \prod_{i \neq j} q_i(x_i) \left[\log \tilde{p}(x) - \sum_k \log q_k(x_k) \right] \quad \leftarrow \begin{array}{l} \text{Выносим} \\ \text{за скобки } q_j \end{array} \quad (3.11) \\
 &= \sum_{x_j} q_j(x_j) \log f_j(x_j) - \\
 &\quad - \sum_{x_j} q_j(x_j) \sum_{x_{-j}} \prod_{i \neq j} q_i(x_i) \left[\sum_{k \neq j} \log q_k(x_k) + \log q_j(x_j) \right] = \\
 &= \sum_{x_j} q_j(x_j) \log f_j(x_j) - \sum_{x_j} q_j(x_j) \log q_j(x_j) + \text{const.} \quad \leftarrow \begin{array}{l} \text{Рассматриваем члены, отличные} \\ \text{от } q_j, \text{ как постоянные} \end{array}
 \end{aligned}$$

Здесь мы ввели следующее обозначение:

$$\log f_j(x_j) = \sum_{x_{-j}} \prod_{i \neq j} q_i(x_i) \log \tilde{p}(x) = E_{-q_j} [\log \tilde{p}(x)]. \quad (3.12)$$

Поскольку мы заменяем члены формулы на их среднее значение, этот метод известен как *метод среднего поля*. Мы можем переобозначить $L(q_j) = -KL(q_j \| f_j)$ и, следовательно, максимизировать целевую функцию, задав $q_j = f_j$ или, что равносильно, выражение из формулы 3.13:

$$\log q_j(x_j) = \log f_j(x_j) = E_{-q_j} [\log \tilde{p}(x)]. \quad (3.13)$$

Здесь функциональная форма q_j будет определяться типом переменных x_j и их вероятностной моделью. В следующем разделе мы будем использовать этот результат при разработке с нуля алгоритма устранения шума на изображении.

3.3. Удаление шума на изображении в модели Изинга

Модель Изинга является примером марковского случайного поля (Markov random field, MRF) и берет свое начало в статистической физике. Марковское случайное поле — это набор случайных величин с марковским свойством, описываемый неориентированным графом, в котором вершины представляют случайные величины, а ребра кодируют условную независимость. Модель Изинга предполагает, что у нас есть решетка, в которой каждый узел может находиться в одном из двух возможных состояний. Состояние каждого узла зависит от соседних узлов через потенциалы взаимодействия. В случае изображений это выражается в виде ограничения на гладкость (то есть пиксель «предпочитает» быть того же цвета, что и соседние пиксели). В задаче устранения шума на изображении мы исходим из того, что у нас есть двумерная сетка зашумленных пиксельных наблюдений некоторого истинного изображения и требуется восстановить истинное изображение.

Пусть y_i — зашумленные наблюдения двоичных скрытых переменных $x_i \in \{-1, +1\}$. Мы можем записать их совместное распределение следующим образом:

$$\begin{aligned} p(x, y) = p(x)p(y|x) &= \prod_{(s,t) \in E} \Psi_{st}(x_s, x_t) \prod_{i=1}^n p(y_i | x_i) = \\ &= \prod_{(s,t) \in E} \exp\{x_s w_{st} x_t\} \prod_{i=1}^n N(y_i | x_i, \sigma^2). \end{aligned} \quad (3.14)$$

В уравнении 3.14 потенциалы взаимодействия для каждой пары узлов x_s и x_t из множества ребер E обозначены как Ψ_{st} , а наблюдения y_i являются гауссовыми СВ со средним значением x_i и дисперсией σ^2 . Здесь w_{st} — сила связи, которая предполагается постоянной и равной $J > 0$, что указывает на предпочтение того же состояния, что и у соседей (то есть потенциал $\Psi(x_s, x_t) = \exp\{x_s J x_t\}$ тогда выше, когда x_s и x_t оба равны $+1$ или -1).

Чтобы подогнать параметры модели с помощью вариационного вывода, сначала нужно максимизировать ELBO:

$$\begin{aligned}
 ELBO &= E_{q(x)} [\log p(x, y)] - E_{q(x)} [\log q(x)] = \\
 &= E_{q(x)} \left[\sum_{(s,t) \in E} x_s w_{st} x_t + \sum_{i=1}^n \log N(x_i; \sigma^2) \right] - \\
 &\quad - \sum_{i=1}^n E_{q_i(x)} [\log q_i(x)].
 \end{aligned} \tag{3.15}$$

Здесь мы применяем допущение среднего поля о полностью факторизованном приближении $q(x)$:

$$q(x) = \prod_{i=1}^n q(x_i; \mu_i). \tag{3.16}$$

Используя результат, полученный в формуле 3.12, утверждаем, что $q(x_i; \mu_i)$, минимизирующая KL-дивергенцию, определяется следующим образом:

$$q_i(x_i) = \frac{1}{Z_i} \exp [E_{-q_i} \{ \log p(x) \}]. \tag{3.17}$$

Здесь E_{-q_i} обозначает математическое ожидание для каждого q_j , кроме случая $j = i$. Чтобы вычислить $q_i(x_i)$, понадобятся только те члены, которые включают x_i (мы можем выделить их так, как показано в формуле 3.18):

$$\begin{aligned}
 E_{-q_i} \{ \log p(x) \} &= E_{-q_i} \left\{ x_i \sum_{j \in N(i)} w_{ij} x_j + \log N(x_i, \sigma^2) + \text{const} \right\} = \\
 &= x_i \sum_{j \in N(i)} J \times \mu_j + \log N(x_i, \sigma^2) + \text{const}.
 \end{aligned} \tag{3.18}$$

Здесь $N(i)$ обозначает соседей узла i , а μ_j — среднее значение двоичной случайной величины:

$$\mu_j = E_{q_j} [x_j] = q_j(x_j = +1) \times (+1) + q_j(x_j = -1) \times (-1). \tag{3.19}$$

На рис. 3.4 аппроксимация среднего поля для модели Изинга представлена в параметрической форме.

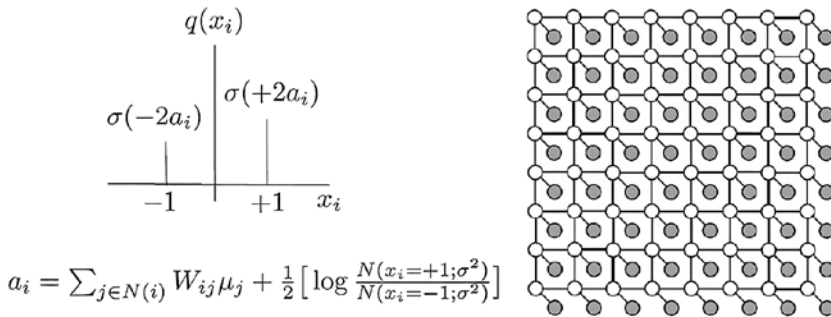


Рис. 3.4. Модель Изинга и ее приближенное распределение $q(x)$

Чтобы вычислить это среднее, нужно знать значения $q_j(x_j = +1)$ и $q_j(x_j = -1)$. Пусть $m_i = \sum_{j \in N(i)} w_{ij} \mu_j$ — среднее значение по соседним узлам и пусть $L_i^+ = N(x_i = +1, \sigma^2)$ и $L_i^- = N(x_i = -1, \sigma^2)$. Тогда можем вычислить среднее значение следующим образом:

$$\begin{aligned} q_i(x_i = 1) &= \frac{\exp\{m_i + L_i^+\}}{\exp\{m_i + L_i^+\} + \exp\{-m_i + L_i^-\}} = \\ &= \frac{1}{1 + \exp\{-2m_i + L_i^- - L_i^+\}} = \\ &= \frac{1}{1 + \exp\{-2a_i\}} = \sigma(2a_i). \end{aligned} \quad (3.20)$$

Здесь $a_i = m_i + \frac{1}{2}(L_i^+ - L_i^-)$, а $\sigma(x)$ — сигмоидная функция. Поскольку $q_i(x_i = -1) = 1 - q_i(x_i = +1) = 1 - \sigma(2a_i) = \sigma(-2a_i)$, можем записать среднее для нашего вариационного приближения $q_i(x_i)$ следующим образом:

$$\mu_i = E_{q_i}[x_i] = \sigma(2a_i) - \sigma(-2a_i) = \text{th}(a_i). \quad (3.21)$$

То есть обновления вариационных параметров μ_i на шаге k в приближении среднего поля вычисляются так, как показано ниже:

$$\begin{aligned} \mu_i^{(k)} &= \text{th} \left(\sum_{j \in N(i)} w_{ij} \mu_j^{(k-1)} + \frac{1}{2} \left[\log \frac{N(x_i = +1, \sigma^2)}{N(x_i = -1, \sigma^2)} \right] \right) \times \\ &\times \lambda + (1 - \lambda) \times \mu_i^{(k-1)}. \end{aligned} \quad (3.22)$$

Здесь мы добавили параметр скорости обучения (learning rate) $\lambda \in (0, 1]$. Далее мы видим, что можно почленно упростить выражение для вычисления ELBO, как показано в следующей формуле:

$$\begin{aligned} &\text{Первое слагаемое ELBO} \\ &\quad \uparrow \\ &\sum_{(s,t) \in E} E_{q(x)}[x_s w_{st} x_t] \\ &\quad \uparrow \text{По определению матожидания} \\ &= \frac{1}{2} \sum_{i=1}^n \sum_{j \in N(i)} \left(\sum_{x_i \in \{-1, +1\}} \sum_{x_j \in \{-1, +1\}} q_i(x_i) q_j(x_j) x_i x_j \right) \\ &= \frac{1}{2} \sum_{i=1}^n \sum_{j \in N(i)} \underbrace{(q_i(x_i = +1) JE[x_j] - q_i(x_i = -1) JE[x_j])}_{\substack{\downarrow \\ \text{После подстановки значений для } x_i \text{ и } x_j}} \\ &= \frac{1}{2} \sum_{i=1}^n \sum_{j \in N(i)} \underbrace{E[x_i] JE[x_j]}_{\substack{\downarrow \\ \text{По определению} \\ \text{матожидания}}} \end{aligned} \quad (3.23)$$

Формула 3.24 получается аналогично:

$$\begin{aligned}
 & \text{Второе слагаемое ELBO} \quad \uparrow \quad \text{По определению матожидания} \\
 E_{q(x)}[\log N(x_i, \sigma^2)] &= \sum_{i=1}^n \left[\sum_{x_i \in \{-1, +1\}} q_i(x_i) \log N(x_i, \sigma^2) \right] = \quad (3.24) \\
 &= \sum_{i=1}^n \left[\sigma(2a_i) \log N(x_i = +1, \sigma^2) + \sigma(-2a_i) \log N(x_i = -1, \sigma^2) \right]. \\
 & \quad \downarrow \\
 & \text{После суммирования}
 \end{aligned}$$

Чтобы лучше понять описываемый алгоритм, давайте изучим псевдокод на рис. 3.5.

```

1: class image_denoising
2:   function mean_field(σ, y, w, λ, max_iter):
3:     logp1 = log N(y; xi = +1, σ2)
4:     logm1 = log N(y; xi = -1, σ2)
5:     logodds = logp1 - logm1
6:     p1 = sigmoid(logodds) //init
7:     μ(0) = 2 × p1 - 1 //init
8:     for k = 1 to max_iter:
9:        $\tilde{S}_{ij} = \sum_{j \in N(i)} w_{ij} \mu_j^{(k-1)}$ 
10:       $\mu_i^{(k)} = \tanh(\tilde{S}_{ij} + \frac{1}{2} \logodds) \times \lambda + (1 - \lambda) \times \mu_i^{(k-1)}$ 
11:      ELBO[k] = ELBO[k] +  $\frac{1}{2} \left( \tilde{S}_{ij} \times \mu_i^{(k)} \right)$ 
12:      a = μ(k) +  $\frac{1}{2} \logodds$ 
13:      qxp1 = sigmoid(+2a)
14:      qxm1 = sigmoid(-2a)
15:      Hx = -qxm1 × log(qxm1) - qxp1 × log(qxp1)
16:      ELBO[k] = ELBO[k] +  $\sum_{i=1}^N (qxp1[i] \times \logp1[i] + qxm1[i] \times \logm1[i])$ 
        +  $\sum_{i=1}^N (Hx[i])$ 
17:   end for
18:   return μ(k)

```

Рис. 3.5. Псевдокод для вариационного вывода среднего поля в модели Изинга

В классе `image_denoising` у нас есть единственный метод `mean_field`, который принимает в качестве входных аргументов уровень шума σ , зашумленное двоичное изображение y , силу связи $w=J$, скорость обучения λ и максимальное количество итераций. Мы начинаем с вычисления логарифмического отношения шансов (то есть вероятности наблюдения пикселя изображения y при гауссовой случайной величине со средними значениями $+1$ и -1). Затем мы вычисляем сигмоидную функцию логарифмического отношения шансов и используем результат для инициализации переменной для среднего значения. Затем мы выполняем цикл с количеством итераций, равным заданному максимальному

значению, и на каждой итерации мы вычисляем влияние соседей S_{ij} , которое включаем в формулу для обновления параметров в приближении среднего поля. Затем мы вычисляем нашу целевую функцию ELBO и среднюю энтропию H_x , чтобы отслеживать сходимость алгоритма.

Теперь у нас есть все необходимые инструменты для реализации вариационного вывода среднего поля для модели Изинга в применении к подавлению шума на изображениях! В листинге ниже мы сначала считываем данные зашумленного изображения, а затем осуществляем вариационный вывод среднего поля на сетке пикселей, чтобы устранить шум.

Листинг 3.1. Вариационный вывод среднего поля в модели Изинга

```
import numpy as np
import pandas as pd

import seaborn as sns
import matplotlib.pyplot as plt

from PIL import Image
from tqdm import tqdm
from scipy.special import expit as sigmoid
from scipy.stats import multivariate_normal

np.random.seed(42)
sns.set_style('whitegrid')

class image_denoising:
    def __init__(self, img_binary, sigma=2, J=1):
        # параметры среднего поля
        self.sigma = sigma  # ← Уровень шума
        self.y = img_binary + self.sigma*np.random
        # → .randn(M, N)  ←  $y_i \sim N(x_i; \sigma^2)$ ;
        self.J = J  # ← Сила связи (wij)
        self.rate = 0.5  # ← Обновление скорости сглаживания
        self.maxiter = 15
        self.ELBO = np.zeros(self.max_iter)
        self.Hx_mean = np.zeros(self.max_iter)

    def mean_field(self):
        # Вариационный вывод среднего поля
        print("running mean-field variational inference...")
        logodds = multivariate_normal.logpdf(self.y.flatten(), mean=+1,
        # → cov=self.sigma**2) - \
            multivariate_normal.logpdf(self.y.flatten(), mean=-1,
            # → cov=self.sigma**2)
        logodds = np.reshape(logodds, (M, N))

        # инициализация
        p1 = sigmoid(logodds)
        mu = 2*p1-1  # ← Начальное значение  $\mu$ 

        a = mu + 0.5 * logodds
        qxp1 = sigmoid(+2*a) #  $q_i(x_i=+1)$ 
        qxm1 = sigmoid(-2*a) #  $q_i(x_i=-1)$ 
```

```

logp1 = np.reshape(multivariate_normal.logpdf(self.y.flatten(),
    ➡ mean=+1, cov=self.sigma**2), (M, N))
logm1 = np.reshape(multivariate_normal.logpdf(self.y.flatten(),
    ➡ mean=-1, cov=self.sigma**2), (M, N))

for i in tqdm(range(self.max_iter)):
    muNew = mu
    for ix in range(N):
        for iy in range(M):
            pos = iy + M*ix
            neighborhood = pos + np.array([-1,1,-M,M])
            boundary_idx = [iy!=0,iy!=M-1,ix!=0,ix!=N-1]
            neighborhood = neighborhood[np.where(boundary_idx)[0]]
            xx, yy = np.unravel_index(pos, (M,N), order='F')
            nx, ny = np.unravel_index(neighborhood, (M,N), order='F')

            Sbar = self.J*np.sum(mu[nx,ny])
            muNew[xx,yy] = (1-self.rate)*muNew[xx,yy] +
            ➡ self.rate*np.tanh(Sbar + 0.5*logodds[xx,yy])
            self.ELBO[i] = self.ELBO[i] + 0.5*(Sbar * muNew[xx,yy])
        # конец for
    # конец for
    mu = muNew

    a = mu + 0.5 * logodds
    qxp1 = sigmoid(+2*a) #q_i(x_i=+1)
    qxm1 = sigmoid(-2*a) #q_i(x_i=-1)
    Hx = -qxm1*np.log(qxm1+1e-10) - qxp1*np.log(qxp1+1e-10) # энтропия

    self.ELBO[i] = self.ELBO[i] + np.sum(qxp1*logp1 + qxm1*logm1)
    ➡ + np.sum(Hx)
    self.Hx_mean[i] = np.mean(Hx)
# конец for
return mu

if __name__ == "__main__":
    # загрузка данных
    print("loading data...")
    data = Image.open('./figures/bayes.bmp' )
    img = np.double(data)
    img_mean = np.mean(img)
    img_binary = +1*(img>img_mean) + -1*(img<img_mean)
    [M, N] = img_binary.shape

    mrf = image_denoising(img_binary, sigma=2, J=1)
    mu = mrf.mean_field()

    # создание графиков
    plt.figure()
    plt.imshow(mrf.y)
    plt.title("Зашумленное изображение")
    plt.show()

    plt.figure()
    plt.imshow(mu)
    plt.title("После %d итераций среднего поля" %mrf.max_iter)
    plt.show()

    plt.figure()

```

```
plt.plot(mrf.Hx_mean, color='b', lw=2.0, label='Средняя энтропия')
plt.title('Вариационный вывод в модели Изинга')
plt.xlabel('Итерации'); plt.ylabel('Средняя энтропия')
plt.legend(loc='upper right')
plt.show()

plt.figure()
plt.plot(mrf.ELBO, color='b', lw=2.0, label='ELBO')
plt.title('Вариационный вывод в модели Изинга')
plt.xlabel('Итерации'); plt.ylabel('Целевая функция ELBO')
plt.legend(loc='upper left')
plt.show()
```

На рис. 3.6 показаны экспериментальные результаты по устранению шума на бинарном изображении с помощью вариационного вывода среднего поля.

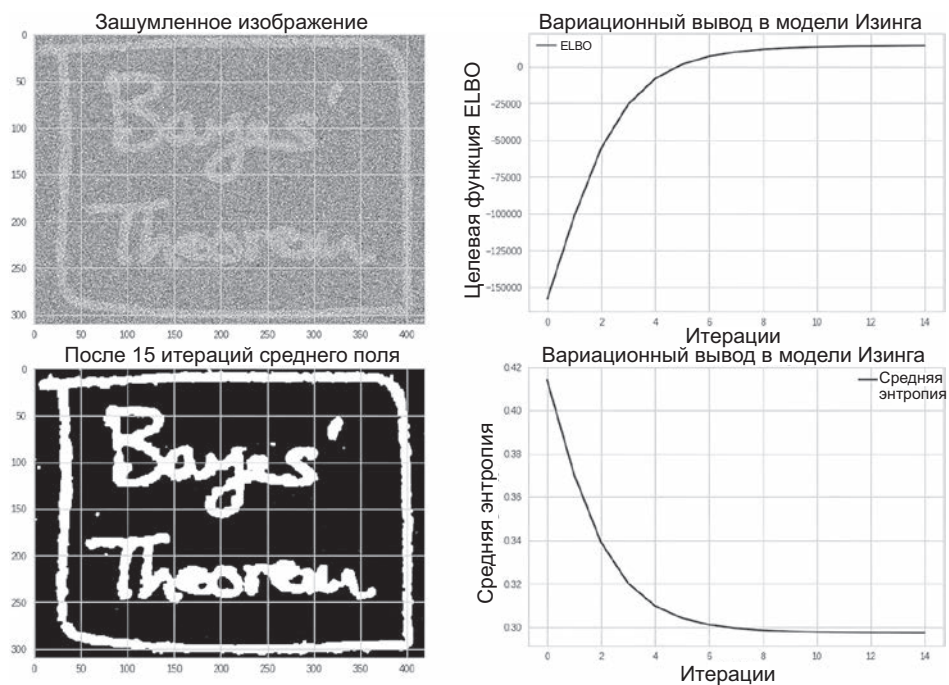


Рис. 3.6. Вариационный вывод среднего поля в модели Изинга для подавления шума на изображении

Зашумленное изображение, расположенное в левом верхнем углу рис. 3.6, получено путем добавления гауссовского шума ко всем пикселям и последующей бинаризации в соответствии со средним пороговым значением. Мы использовали следующие параметры вариационного вывода: сила связи $J = 1$, уровень шума $\sigma = 2$, скорость сглаживания $\lambda = 0,5$ и максимальное число итераций, равное 15. Очищенное от шума двоичное изображение показано в левом нижнем углу

рис. 3.6. Там же можно видеть как (в правом верхнем углу) растет целевая функция ELBO и (в правом нижнем углу) уменьшается средняя энтропия наших двоичных случайных величин $q_i(x_i)$, представляющих собой значение каждого пикселя, по мере увеличения числа итераций среднего поля. Двумерная модель Изинга может быть расширена разными способами, в частности, с использованием трехмерных сеток и K -состояний для каждого узла (она же модель Поттса).

3.4. Максимизация взаимной информации

В этом разделе мы разберем метод максимизации взаимной информации (mutual information, MI), который обычно используется при информационном планировании и передаче данных. Рассмотрим пример беспроводной связи. В нем мы передаем сигнал $x \sim p_x(x)$, который проходит через канал H с несколькими входами и выходами (multiple-input, multiple-output — МИМО). На выходе получаем сигнал $y = Hx + n$, где $n \sim N(0, \sigma^2 I)$ — аддитивный гауссовский шум. Мы хотели бы довести до максимума объем информации, передаваемой по данному беспроводному каналу. Другими словами, есть задача максимизировать пропускную способность, или взаимную информацию между передаваемым сигналом X и принятым сигналом Y : $C = \max I(X; Y)$, где максимизация берется по $p(x)$. Мы обсудим общую процедуру расчета пропускной способности канала на основе KL-дивергенции, позволяющей приближенно оценить максимальное значение взаимной информации:

$$I(X; Y) = H(X) - H(X | Y) = -E_{p(x)}[\log p(x)] - E_{p(x, y)}[\log p(x | y)]. \quad (3.25)$$

Пусть $q(x)$ — распределение, приближенное к $p(x)$. Рассмотрим KL-дивергенцию между апостериорными распределениями p и q , как показано в формуле 3.26:

$$D_{KL}(p(x | y) \| q(x | y)) \geq 0. \quad (3.26)$$

Мы хотели бы получить нижнюю границу для взаимной информации (MI). Раскрывая выражение в формуле 3.26, мы можем прийти к выражению, приведенному в формуле 3.27:

$$\sum_x p(x | y) \log p(x | y) - \sum_x p(x | y) \log q(x | y) \geq 0. \quad (3.27)$$

Умножив обе части на $p(y)$, мы получаем следующее:

$$\sum_{x, y} p(y) p(x | y) \log p(x | y) \geq \sum_{x, y} p(x, y) \log q(x | y). \quad (3.28)$$

Распознав в левой части $-H(X | Y)$, приходим к следующей формуле для нижней границы MI:

$$I(X; Y) = H(X) - H(X | Y) \geq H(X) - E_{p(x, y)}[\log q(x | y)] = \tilde{I}(X; Y). \quad (3.29)$$

Используя полученную формулу для нижней границы, алгоритм максимизации MI можно кратко представить следующим образом (рис. 3.7).

- 1: Выбрать семейство приближенных распределений $Q(x; \theta)$
- 2: Инициализировать θ
- 3: **повторить**
- 4: для фиксированного $q(x|y; \theta)$ найти
- 5: $\theta^{new} = \operatorname{argmax}_{\theta} \tilde{I}(X; Y)$
- 6: для фиксированного θ найти
- 7: $q_{new}(x|y; \theta) = \operatorname{argmax}_{q(x|y) \in Q} \tilde{I}(X; Y)$
- 8: **пока** не сойдется

Рис. 3.7. Псевдокод для максимизации взаимной информации

Алгоритм выше чередует поиск значений параметров, максимизирующих нижнюю границу MI, и нахождение приближенного распределения.

В этой главе мы рассмотрели, как можно использовать определения энтропии, взаимной информации и KL-дивергенции для получения нижней границы, которую, в свою очередь, можно итеративно максимизировать посредством обновления приближенного распределения q . В следующей главе мы посмотрим на ML с точки зрения computer science, а также освоим некоторые полезные структуры данных и алгоритмические парадигмы.

3.5. Упражнения

3.1. Получите выражение для KL-дивергенции между двумя одномерными гауссианами: $q(x) \sim N(\mu_1, \sigma_1^2)$ и $q(x) \sim N(\mu_2, \sigma_2^2)$.

3.2. Выведите $E[X]$, $\operatorname{Var}(X)$ и $H(X)$ для распределения Бернулли.

3.3. Выведите формулы для математического ожидания, моды и дисперсии распределения Beta(a, b).

Итоги

- Основная идея вариационного вывода заключается в том, чтобы выбрать приближенное распределение $q(x)$ из семейства удобных распределений, а затем сделать это приближение как можно более близким истинному апостериорному распределению $p(x)$.
- Нижняя вариационная граница — это целевая функция, которую мы стремимся максимизировать, чтобы найти вариационные параметры нашей модели.
- В приближении среднего поля предполагаем, что приближенное распределение $q(x)$ полностью факторизовано.
- Максимизация взаимной информации может быть осуществлена путем определения и максимизации ее нижней границы.

4

Программная реализация

В этой главе

- ✓ Структуры данных: линейные, нелинейные и вероятностные
- ✓ Парадигмы решения задач: полный поиск, жадный алгоритм, «разделяй и властвуй» и динамическое программирование
- ✓ Исследования в области ML: методы семплирования и вариационный вывод

В предыдущих главах мы рассмотрели два основных метода байесовского вывода: марковскую цепь Монте-Карло и вариационный вывод. В этой главе сделаем обзор концепций из области computer science, которые нам понадобятся для реализации алгоритмов с нуля. Чтобы писать качественный код, требуется хорошо понимать структуры данных и владеть азами теории алгоритмов. В этой главе вы познакомитесь с распространенными структурами данных и парадигмами решения задач. Многие из рассмотренных в главе концепций графически представлены в интерактивном режиме на веб-сайте VisuAlgo (<https://visualgo.net/en>).

4.1. Структуры данных

Структуры данных — это способы хранения и систематизации информации, поддерживающие ряд операций, такие как вставка, поиск, удаление и обновление. Правильный выбор структур данных предоставляет различные возможности в отношении производительности и может ускорить выполнение алгоритма. Все это говорит о том, как важно понимать работу структур данных.

4.1.1. Линейные структуры данных

Структура данных считается *линейной*, если ее элементы расположены последовательно. Простейшим примером линейной структуры данных является *массив фиксированного размера* (при этом размер такого массива может быть указан в качестве ограничивающего условия задачи). Время доступа к элементу в массиве является постоянным: $O(1)$. Если размер массива заранее неизвестен, лучше всего использовать *динамически изменяемый массив* (например, список в Python или вектор в C++), поскольку он изначально рассчитан на изменение своей длины.

Наиболее распространенные операции с массивами — поиск и сортировка. Самый простой поиск — это линейное сканирование по всем элементам, требующее $O(n)$ времени. Если массив отсортирован, то можно использовать двоичный поиск с быстройдействием $O(\log n)$. Такой поиск является примером алгоритма «разделяй и властвуй», который мы вскоре обсудим. Примитивные алгоритмы сортировки массивов, такие как сортировка методом выбора или вставкой, имеют сложность $O(n^2)$ и, следовательно, подходят только для небольших наборов данных. Обычно сортировка на основе сравнения, при которой элементы сравниваются попарно, такая как слияние, куча или быстрая сортировка, имеет время выполнения $O(n \log n)$, поскольку временную сложность можно представить как обход полного двоичного дерева, где каждый лист соответствует одному отсортированному упорядочению. В таком представлении высота дерева h равна времени исполнения алгоритма. Поскольку существует $n!$ возможных упорядочений (листовых узлов), можем оценить время выполнения следующим образом:

$$\begin{aligned} h &= \log n! = \log n(n-1)(n-2) \times \dots \times 1 = \\ &= \log n + \log(n-1) + \dots + \log 1 < \\ &< \log n + \log n + \dots + \log n = n \log n = O(n \log n). \end{aligned} \quad (4.1)$$

Если мы наложим дополнительные ограничения на входные данные, то сможем построить алгоритмы сортировки с линейным временем $O(n)$, такие как сортировка подсчетом, поразрядная сортировка и блочная сортировка. Например, сортировка подсчетом может быть применена к целым числам в небольшом диапазоне, а в поразрядной применяется сортировка подсчетом разряд за разрядом, как описано в книге Стивена Халима (Steven Halim) «Competitive Programming»¹ (lulu, 2013). Использование распределенных алгоритмов может еще больше сократить время выполнения, как описано в книге Мориса Херлихи (Maurice Herlihy) и Нира Шавита (Nir Shavit) «The Art of Multiprocessor Programming» (Morgan Kaufmann, 2012).

Связный список состоит из элементов, которые, начиная с головного элемента, хранят как значение, так и указатель на следующий элемент. Обычно им избегают

¹ Халим С., Халим Ф. «Спортивное программирование».

пользоваться из-за линейности времени поиска: $O(n)$. *Стек* обеспечивает $O(1)$ для вставки (push) и $O(1)$ для удаления (pop) элементов в порядке «последним пришел — первым ушел» (LIFO). Это свойство особенно полезно в алгоритмах, которые реализуются рекурсивно (например, определение парных скобок или топологическая сортировка). *Очередь* позволяет достичь $O(1)$ при вставке (постановке в очередь) элемента в конец и $O(1)$ при удалении из начала. Поэтому она соответствует модели «первый пришел — первый вышел» (FIFO). Очередь — широко используемая структура данных в таких алгоритмах, как поиск в ширину (breadth-first search, BFS), и алгоритмах, на нем основанных. В следующем разделе мы дадим определение и рассмотрим несколько примеров нелинейных структур данных.

4.1.2. Нелинейные структуры данных

Структура данных считается *нелинейной*, если ее элементы не расположены последовательно. Примерами нелинейных структур данных являются *ассоциативный массив* (словарь) и *множество*. Они относятся к данной категории потому, что упорядоченные словари и упорядоченные множества построены на самобалансирующемся двоичном дереве поиска, которое гарантирует $O(n \log n)$ время вставки, поиска и удаления. Двоичное дерево поиска обладает тем свойством, что значение корневого узла больше, чем значения узлов его левого поддерева, и меньше, чем значения узлов правого поддерева. Этим же качеством обладают и все поддерева. Самобалансирующееся дерево двоичного поиска обычно реализуется в виде дерева Адельсона-Вельского и Ландиса (АВЛ-дерево) или красно-черного дерева. Подробнее см. «Introduction to Algorithms»¹ Томаса Х. Кормена (Thomas H. Cormen), Чарльза Э. Лейзерсона (Charles E. Leiserson), Рональда Л. Ривеста (Ronald L. Rivest) и Клиффорда Штайна (Clifford Stein) (MIT Press, McGraw-Hill, 1990). Разница между упорядоченным ассоциативным массивом (словарем) и упорядоченным множеством заключается в том, что словарь хранит пары («ключ — значение»), тогда как множество хранит только ключи. *Куча* — это еще один способ организации данных в виде древовидной структуры. Например, массив $A = [2, 7, 26, 25, 19, 17, 1, 90, 3, 36]$ может быть представлен в виде дерева, изображенного на рис. 4.1.

Мы можем легко перемещаться по двоичной куче $A = [90, 36, 17, 25, 26, 7, 1, 2, 3, 19]$, стартуя с вершины i и используя простую индексную арифметику: $2i$ для доступа к левому потомку, $2i + 1$ для доступа к правому потомку, и $i/2$ для доступа к родительскому узлу. В отличие от деревьев двоичного поиска (binary search tree, BST), невозрастающая куча (max-heap) обладает свойством пирамиды: в каждом поддереве с корнем в x элементы в левом и правом поддеревьях x меньше (или равны) x . Это свойство гарантирует, что корень невозрастающей

¹ Кормен Т. Х., Лейзерсон Ч. И., Ривест Р. Л., Штайн К. «Алгоритмы: построение и анализ».

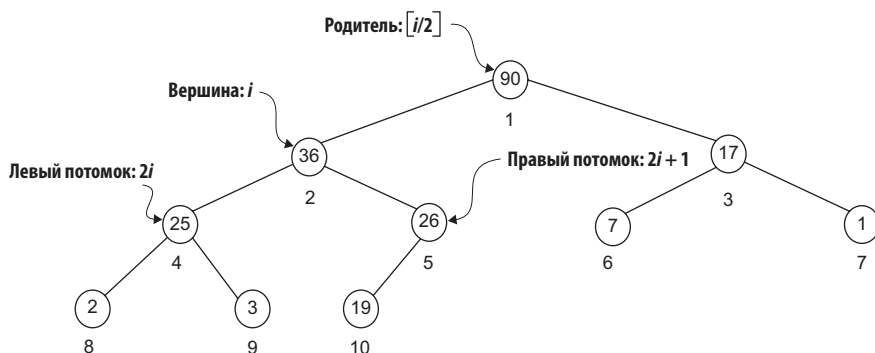


Рис. 4.1. Двоичная куча

кучи всегда является максимальным элементом. Таким образом, невозрастающая куча позволяет быстро извлекать наибольший элемент. Действительно, операции извлечения и вставки максимального элемента выполняются при обходе дерева за время $O(\log n)$, по мере необходимости совершая операции обмена (swapping) для сохранения свойства кучи. Куча лежит в основе *приоритетной очереди* (priority queue), которая играет важную роль в таких алгоритмах, как минимальное остовное дерево (minimum spanning tree, MST) Прима и Краскала, поиск кратчайших путей из одной вершины (single-source shortest path, SSSP) Дейкстры и поиск A^* . Наконец, *хеш-таблица*, или неупорядоченный ассоциативный массив, — это очень эффективная структура данных, с временем доступа $O(1)$ при условии отсутствия коллизий. Один из часто используемых классов хеш-таблиц — прямая адресация (direct addressing, DA), где сами ключи являются индексами. Цель хеш-функции — равномерно распределить элементы в таблице таким образом, чтобы свести к минимуму коллизии. С другой стороны, если вы хотите сгруппировать похожие элементы в один бакет, хеширование с учетом местоположения (locality-sensitive hashing, LSH) позволяет найти ближайших соседей, как описано в статье Александра Андони (Alexandr Andoni), Петра Индика (Piotr Indyk), Хи Ле Нгуена (Huy Le Nguyen) и Ильи Разенштейна (Ilya Razenshteyn) «Beyond Locality-Sensitive Hashing» (SODA, 2014).

4.1.3. Вероятностные структуры данных

Вероятностные структуры предназначены для обработки больших объемов данных. Они обеспечивают вероятностные гарантии (probabilistic guarantees) и приводят к существенной экономии памяти. Вероятностные структуры данных решают следующие распространенные проблемы, связанные с большими данными:

- Проверка принадлежности (membership querying): фильтр Блума, фильтр Блума с подсчетом, фильтр остатков (quotient filter) и кукушкин фильтр (cuckoo filter).

- Мощность: линейный подсчет, вероятностный подсчет, LogLog, HyperLogLog и HyperLogLog++.
- Частота: алгоритм большинства (majority algorithm), отсчетный скетч (count sketch или count-min sketch).
- Ранг: случайное семплирование, q -дайджест и t -дайджест.
- Сходство: LSH, MinHash и SimHash.

Подробное описание вероятностных структур данных можно найти в книге Андрея Гахова (Andrii Gakhov) «Probabilistic Data Structures and Algorithms for Big Data Applications» (BoD, 2022). В следующем разделе мы рассмотрим четыре основные алгоритмические парадигмы.

4.2. Парадигмы решения задач

Существуют четыре основные алгоритмические парадигмы: полный поиск, жадный алгоритм, «разделяй и властвуй» и динамическое программирование. В зависимости от поставленной задачи решение часто можно найти, обратившись к той или иной алгоритмической парадигме. В этом разделе мы обсудим каждую из четырех и для каждой приведем пример.

4.2.1. Полный поиск

Полный поиск (он же «грубая сила», или брутфорс) — это метод, в котором решение задачи находится путем обхода всего пространства поиска целиком. В процессе поиска мы можем отсечь те части области поиска, которые заведомо не приведут к нужному решению. Для примера рассмотрим задачу создания подмножеств. Мы можем использовать либо рекурсивное, либо итеративное решение. В обоих случаях завершаем работу при достижении требуемого размера подмножества. В следующем листинге реализуем стратегию полного поиска на примере генерации подмножеств.

Листинг 4.1. Создание подмножества

```
def search(k, n):
    if (k == n):
        print(subset)  # ← Обработка подмножества
    else:
        search(k+1, n)
        subset.append(k)
        search(k+1, n)
        subset.pop()
    # конец if

def bitseq(n):
    for b in range(1 << n):
        subset = []
```

```

        for i in range(n):
            if (b & 1 << i):
                subset.append(i)
        # конец for
        print(subset)
    # конец for

if __name__ == "__main__":
    n = 4
    subset = []
    search(0, n)    ← Рекурсивный

    subset = []
    bitseq(n)       ← Итеративный

```

Примером машинного обучения, в котором выполняется полный поиск, является точный вывод (exact inference) с помощью полного перечисления (complete enumeration) — подробнее см. главу 21 книги Дэвида Дж. С. Маккея (David J. C. MacKay) «Information Theory, Inference and Learning Algorithms». При заданной графовой модели мы хотели бы разложить совместное распределение на множители в соответствии с отношениями условной независимости и использовать правило Байеса для вычисления апостериорной вероятности определенных событий. В таком случае для проведения расчета нам потребуется заполнить все необходимые таблицы вероятностей полностью.

4.2.2. Жадный алгоритм

Жадный алгоритм на каждом шаге выбирает локально оптимальный вариант в надежде в итоге прийти к глобально оптимальному решению. Такие алгоритмы часто полагаются на «жадную» эвристику, и существует много примеров того, как им не удастся достичь глобального оптимума. В качестве примера рассмотрим задачу о рюкзаке с делимыми предметами (fractional knapsack problem). Цель задачи состоит в том, чтобы выбрать предметы для размещения в рюкзаке ограниченной вместимости W таким образом, чтобы максимизировать общую стоимость предметов рюкзака, где каждый предмет имеет соответствующий вес и ценность. Мы можем определить «жадную» эвристику как отношение стоимости предмета к его весу (то есть мы хотели бы «жадно» отбирать такие предметы, которые одновременно имеют высокую ценность и низкий вес, и сортировать их на основе этого критерия). В задаче о рюкзаке с делимыми предметами разрешается брать дробную часть от предмета (в отличие от задачи, в которой можно класть в рюкзак только вещь целиком). В следующем листинге реализуем жадную стратегию на примере задачи о рюкзаке с делимыми предметами.

Листинг 4.2. Рюкзак с делимыми предметами

```

class Item:
    def __init__(self, wt, val, ind):
        self.wt = wt

```

```

        self.val = val
        self.ind = ind
        self.cost = val // wt

    def __lt__(self, other):
        return self.cost < other.cost

class FractionalKnapSack:
    def get_max_value(self, wt, val, capacity):

        item_list = []
        for i in range(len(wt)):
            item_list.append(Item(wt[i], val[i], i))

        # сортировка предметов по критерию затрат (cost)
        item_list.sort(reverse = True) #O(nlogn)

        total_value = 0
        for i in item_list:
            cur_wt = int(i.wt)
            cur_val = int(i.val)
            if capacity - cur_wt >= 0:
                capacity -= cur_wt
                total_value += cur_val
            else:
                fraction = capacity / cur_wt
                total_value += cur_val * fraction
                capacity = int(capacity - (cur_wt * fraction))
                break
        return total_value

if __name__ == "__main__":
    wt = [10, 20, 30]
    val = [60, 100, 120]
    capacity = 50

    fk = FractionalKnapSack()
    max_value = fk.get_max_value(wt, val, capacity)
    print("greedy fractional knapsack")
    print("maximum value: ", max_value)

```

Поскольку сортировка является самой дорогостоящей операцией, алгоритм выполняется за время $O(n \log n)$. Можно заметить, что входные предметы сортируются по убыванию соотношения ценность/вес. После жадного выбора элементов 1 и 2 мы берем $2/3$ от элемента 3, получая общую стоимость $60 + 100 + (2/3)120 = 240$.

Примером жадного алгоритма в машинном обучении является размещение датчиков. Если имеется помещение и несколько датчиков температуры, то задача состоит в том, чтобы разместить датчики таким образом, чтобы максимально увеличить охват данного помещения. Простой «жадный» подход заключается в том, чтобы начать с пустого набора S_0 и на итерации i добавить датчик A , который максимизирует функцию приращений, такую как взаимная

информация: $F_M(A) = H(V \setminus A) - H(V \setminus A | A)$, где V — набор всех датчиков. Здесь мы использовали тождество $I(X; Y) = H(X) - H(X | Y)$. Оказывается, что взаимная информация является *субмодулярной* функцией, если наблюдаемые переменные независимы с учетом скрытого состояния, что приводит к эффективным алгоритмам «жадной» субмодулярной оптимизации с гарантиями производительности (см., например, PhD-диссертацию Андреаса Крауса (Andreas Kraus) «Optimizing Sensing: Theory and Applications» (Carnegie Mellon University, School of Computer Science, 2008)).

4.2.3. Разделяй и властвуй

«Разделяй и властвуй» (divide and conquer) — это метод, который разбивает задачу на более мелкие *независимые* подзадачи, а затем объединяет решения всех подзадач. В качестве примеров этого метода можно назвать такие алгоритмы, как быстрая сортировка, пирамидальная сортировка и сортировка слиянием, а также двоичный поиск. Классическим примером двоичного поиска является нахождение значения в отсортированном массиве. В таком алгоритме мы начинаем с проверки на наличие искомого элемента в середине массива. Если элемент найден или больше нет других элементов, то останавливаемся. В противном случае определяем, находится ли искомый элемент слева или справа от середины массива, и продолжаем поиск. Поскольку размер области поиска уменьшается вдвое после каждой проверки, сложность алгоритма равна $O(\log n)$. В следующем листинге реализуем стратегию «разделяй и властвуй» на примере двоичного поиска.

Листинг 4.3. Двоичный поиск

```
def binary_search(arr, l, r, x):
    # предполагается отсортированный массив
    if l <= r:
        mid = int(l + (r-l) / 2)

        if arr[mid] == x:
            return mid
        elif arr[mid] > x:
            return binary_search(arr, l, mid-1, x)
        else:
            return binary_search(arr, mid+1, r, x)
    # конец if
    else:
        return -1

if __name__ == "__main__":
    x = 5
    arr = sorted([1, 7, 8, 3, 2, 5])

    print(arr)
    print("binary search:")
```



```

result = binary_search(arr, 0, len(arr)-1, x)

if result != -1:
    print("element {} is found at index {}".format(x, result))
else:
    print("element is not found.")

```

Пример машинного обучения, использующего рассматриваемую парадигму, можно найти в алгоритме дерева решений CART, где пороговое разбиение выполняется по принципу «разделяй и властвуй», а узлы разделяются рекурсивно до тех пор, пока не будет достигнута максимальная глубина дерева. В алгоритме CART, как мы увидим в следующей главе, оптимальный порог находится «жадным» образом путем оптимизации цели классификации (например, индекса Джини), и та же процедура применяется к дереву на уровень глубже, в результате чего получается рекурсивный алгоритм.

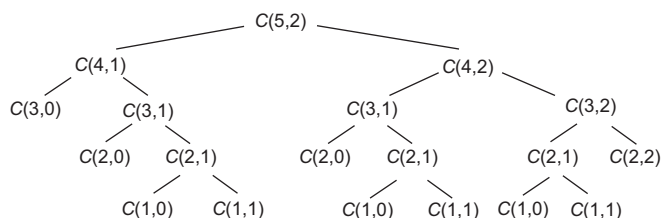
4.2.4. Динамическое программирование

Динамическое программирование (ДП) — это метод, который разбивает задачу на более мелкие, *перекрывающиеся* подзадачи, вычисляет решение для каждой из них и сохраняет его в ДП-таблице. Окончательное решение получается путем считывания из ДП-таблицы. Ключевые навыки в освоении динамического программирования включают способность определять состояния задачи (записи ДП-таблицы), а также отношения или переходы между этими состояниями. Затем, определив базовые случаи и рекуррентные соотношения, можно заполнить ДП-таблицу сверху вниз или снизу вверх. В нисходящем ДП таблица заполняется рекурсивно по мере необходимости, начиная с вершины и спускаясь к более мелким подзадачам. В восходящем ДП таблица заполняется итеративно, начиная с наименьших подзадач. На основе решения последних приходят к решению более крупных подзадач. В обоих случаях, если мы уже сталкивались с какой-то подзадачей, мы можем просто найти ее решение в таблице (вместо того чтобы повторно вычислять с нуля). Это значительно снижает вычислительные затраты.

Воспользуемся примером с биномиальными коэффициентами, чтобы проиллюстрировать использование нисходящего и восходящего ДП. Код ниже для вычисления биномиальных коэффициентов использует рекурсию с перекрывающимися подзадачами. Пусть $C(n, k)$ обозначает число сочетаний из n по k , и тогда можно записать следующие выражения:

$$\begin{aligned}
 \text{База} &: C(n, 0) = C(n, n) = 1; \\
 \text{Рекурсия} &: C(n, k) = C(n-1, k-1) + C(n-1, k).
 \end{aligned} \tag{4.2}$$

Обратите внимание, что здесь у нас имеется ряд перекрывающихся подзадач. Например, для $C(n=5, k=2)$ рекурсивное дерево выглядит так, как показано на рис. 4.2. Мы можем реализовать нисходящий и восходящий ДП-алгоритм так, как это сделано в листинге 4.4.

Рис. 4.2. Рекурсия для вычисления биномиального коэффициента $C(5,2)$ **Листинг 4.4. Биномиальные коэффициенты**

```

def binomial_coeffs1(n, k):
    # ДП сверху вниз
    if (k == 0 or k == n):
        return 1
    if (memo[n][k] != -1):
        return memo[n][k]
    memo[n][k] = binomial_coeffs1(n-1, k-1) + binomial_coeffs1(n-1, k)
    return memo[n][k]

def binomial_coeffs2(n, k):
    # ДП снизу вверх
    for i in range(n+1):
        for j in range(min(i,k)+1):
            if (j == 0 or j == i):
                memo[i][j] = 1
            else:
                memo[i][j] = memo[i-1][j-1] + memo[i-1][j]
        # конец if
    # конец for
    # конец for
    return memo[n][k]

def print_array(memo):
    for i in range(len(memo)):
        print('\t'.join([str(x) for x in memo[i]]))

if __name__ == "__main__":
    n = 5
    k = 2
    print("top down DP")
    memo = [[-1 for i in range(6)] for j in range(6)]
    nCk = binomial_coeffs1(n, k)
    print_array(memo)
    print("C(n={}, k={}) = {}".format(n,k,nCk))

    print("bottom up DP")
    memo = [[-1 for i in range(6)] for j in range(6)]
    nCk = binomial_coeffs2(n, k)
    print_array(memo)
    print("C(n={}, k={}) = {}".format(n,k,nCk))

```

Как временная, так и пространственная сложность алгоритма составляет $O(nk)$. В случае нисходящего ДП решения подзадач сохраняются (запоминаются) по

мере надобности, тогда как при восходящем ДП вычисляется вся таблица, начиная с базового случая.

Пример машинного обучения, в котором используется динамическое программирование, встречается в обучении с подкреплением (reinforcement learning, RL) при поиске решения уравнений Беллмана. Мы можем записать ценность состояния на основе его вознаграждения в момент времени t и суммы будущих дисконтированных вознаграждений, как показано в следующей формуле:

$$\begin{aligned} v_{\pi}(s) &= \sum_a \pi(a|s) \sum_{r,s'} p(r, s' | s, a) [r + \gamma v_{\pi}(s')] = \\ &= E_{\pi} [R_t + \gamma v_{\pi}(S_{t+1}) | S_t = s]. \end{aligned} \quad (4.3)$$

Формула 4.3 известна как уравнение оптимальности Беллмана для $v(s)$. Мы можем восстановить оптимальную стратегию, найдя решение для действия, которое максимизирует вознаграждение состояния, описываемое Q -функцией:

$$\pi(s) = \arg \max_a q(s, a). \quad (4.4)$$

Для небольшого числа состояний и действий можем вычислить Q -функцию в табличном виде, используя динамическое программирование. В обучении с подкреплением часто требуется найти баланс между исследованием (exploration) и использованием (exploitation). В таком случае берем предыдущее значение $\arg \max$ с вероятностью $1 - \epsilon$ и выполняем случайное действие с вероятностью ϵ .

4.3. Исследования в области ML: методы семплирования и вариационный вывод

В этом разделе мы сосредоточимся на актуальных исследованиях в сфере машинного обучения, о которых необходимо знать специалисту. Особое внимание уделим последним разработкам в области методов семплирования и вариационного вывода. Как мы уже отмечали в этой главе, многие современные алгоритмы ML используют изощренные приемы для аппроксимации трудновычислимых апостериорных плотностей вероятности. Трудность эта вызвана наличием сложных многомерных интегралов, задействованных в вычислении апостериорных распределений.

Стоит уделить время краткому обсуждению различий между методом МСМС и вариационным выводом. Преимущества вариационного вывода заключаются в том, что для задач среднего и небольшого масштаба он обычно выполняется быстрее, является детерминированным, позволяет легко определить момент завершения, а также часто обеспечивает нижнюю границу для логарифмического правдоподобия. Преимущества семплирования состоят в том, что его зачастую проще реализовать, оно применимо к более широкому кругу задач (например, задачам без удобных сопряженных априорных распределений), а также может обеспечивать лучшую производительность при применении к очень большим

моделям или наборам данных. Подробнее по этой теме см. главу 24 книги Кевина П. Мэрфи (Kevin P. Murphy) «Machine Learning: A Probabilistic Perspective» (MIT Press, 2012).

В дополнение к классическим алгоритмам МСМС-семплирования, которые мы изучали в предыдущих главах, есть несколько других, заслуживающих внимания. Среди них такие методы, как выборка по уровням (slice sampling; см. статью Рэдфорда М. Нила (Radford M. Neal) «Slice Sampling» в «Annals of Statistics», 2003), метод Монте-Карло с гамильтоновой динамикой (Hamiltonian Monte Carlo, HMC; см. статью Рэдфорда М. Нила (Radford M. Neal) «MCMC Using Hamiltonian Dynamics» в «arXiv», 2012) и семплер без разворотов (no-U-turn sampler, NUTS; см. статью Мэтью Д. Хоффмана (Matthew D. Hoffman) и Эндрю Гельмана (Andrew Gelman) «The No-U-Turn Sampler: Adaptively Setting Path Lengths in Hamiltonian Monte Carlo» в «Journal of Machine Learning Research», 2014). Новейший алгоритм NUTS широко используется в качестве метода МСМС-вывода для вероятностных графовых моделей и реализован в библиотеках вероятностного программирования PyMC3, Stan, TensorFlow Probability и Pyro.

Было предпринято несколько попыток масштабирования МСМС для больших данных, что привело к появлению стохастических методов Монте-Карло, которые в целом можно разделить на методы, основанные на стохастическом градиенте; методы, использующие приближенный метод Метрополиса — Гастингса с мини-пакетами случайной выборки (randomly sampled mini-batches); и методы аугментации данных. Другим популярным классом алгоритмов МСМС является потоковый (streaming) метод Монте-Карло, который аппроксимирует апостериорное распределение для байесовского вывода в реальном времени. Последовательный метод Монте-Карло (Sequential Monte Carlo, SMC) использует перевыборку (resampling) и распространение выборки с большим количеством частиц во времени.

Распараллеливание алгоритмов Монте-Карло — еще одна важная область исследований. Если группы независимых выборочных значений могут быть сгенерированы для апостериорного или вспомогательного распределения, алгоритм семплирования может быть распараллелен посредством запуска нескольких независимых генераторов выборки на отдельных машинах и последующего объединения результатов. К дополнительным методам относятся «разделяй и властвуй» и предварительная выборка (pre-fetching) (см. статью Чжуна Чжу (Jun Zhu), Цзяньфэй Чена (Jianfei Chen), Вэньбо Ху (Wenbo Hu) и Бо Чжана (Bo Zhang), «Big Learning with Bayesian Methods» в «National Science Review», 2017).

Среди последних достижений в области вариационного вывода (VI) стоит упомянуть следующие: масштабируемый (span-scalable) VI, который включает стохастические аппроксимации; универсальный (generic) VI, который расширяет применимость VI к несопряженным моделям; точный (accurate) VI, который включает вариационные модели, выходящие за рамки приближения среднего поля; и амортизированный (amortized) VI, который реализует вывод

по локальным скрытым переменным с помощью сетей вывода (см. статью Чэна Чжана (Cheng Zhang), Джудит Бутпейдж (Judith Butepage), Хедвиг Кьеллстром (Hedvig Kjellstrom) и Стефана Мандта (Stephan Mandt) «Advances in Variational Inference» в «IEEE Transactions on Pattern Analysis and Machine Intelligence», 2019). Масштабируемый VI включает в себя стохастический вариационный вывод (stochastic variational inference, SVI), который применяет методы стохастической оптимизации вариационной целевой функции. Эффективность стохастического градиентного спуска (stochastic gradient descent, SGD) зависит от дисперсии оценок градиента (меньший градиентный шум обеспечивает большую скорость обучения и приводит к более быстрой сходимости). Для ускорения сходимости используются такие методы, как адаптивная скорость обучения и размер мини-батча, а также уменьшение дисперсии, например контрольные переменные (control variates), неравномерное семплирование и другие подходы.

Помимо стохастической оптимизации, задействование структуры модели также может помочь достичь той же цели. Примерами могут служить свернутый (collapsed), разреженный (sparse), параллельный (parallel) и распределенный (distributed) вариационный вывод. Свернутый VI основан на идее аналитического интегрирования определенных параметров модели. При разреженном выводе используются либо рассредоточенные параметры, либо наборы данных, которые могут быть описаны небольшим числом репрезентативных точек. Кроме того, структурированный (structured) VI исследует вариационные распределения, которые не полностью разложены на множители, что приводит к более точным приближениям.

Наконец, амортизированный вариационный вывод — интересная область исследований, которая сочетает в себе вероятностные графовые модели и нейронные сети. Термин *амортизированный* относится к использованию результатов вывода из предыдущих вычислений для ускорения последующих расчетов. Амортизированный вывод стал популярным инструментом вывода в моделях с глубокими скрытыми переменными (deep latent variable model, DLVM), таких как вариационный автоэнкодер (VAE; см. Дидерик П. Кингма (Diederik P. Kingma), Макс Уэллинг (Max Welling) «Auto-Encoding Variational Bayes» (arXiv, 2013)). Аналогичным образом нейронные сети могут использоваться для обучения параметрам условных распределений в направленных (directed) вероятностных графовых моделях (см. Дидерик П. Кингма (Diederik P. Kingma) «Variational Inference and Deep Learning: A New Synthesis», докторская диссертация, 2017).

4.4. Упражнения

4.1. Докажите следующее биномиальное тождество: $C(n, k) = C(n - 1, k - 1) + C(n - 1, k)$.

4.2. Выведите неравенство Гиббса: $H(p, q) \geq H(q)$, где $H(p, q) = -\sum x p(x) \log q(x)$ — кросс-энтропия, а $H(q) = -\sum q(x) \log q(x)$ — энтропия.

4.3. Используйте неравенство Йенсена с $f(x) = \log(x)$ для доказательства неравенства $AM \geq GM$.

4.4. Докажите, что $I(x; y) = H(x) - H(x|y) = H(y) - H(y|x)$.

Итоги

- Структура данных — это способ хранения и систематизации информации. Структуры данных можно разделить на линейные, нелинейные и вероятностные.
- В этой главе мы рассмотрели четыре алгоритмические парадигмы: полный поиск, жадный алгоритм, «разделяй и властвуй» и динамическое программирование.
- Полный поиск (он же «грубая сила», или брутфорс) — это метод, в котором решение задачи ищется путем обхода всего пространства поиска. В процессе поиска мы можем отсечь те части области поиска, которые заведомо не приведут к нужному решению. Жадный алгоритм на каждом шаге выбирает локально оптимальный вариант, надеясь в итоге прийти к глобально оптимальному решению.
- «Разделяй и властвуй» — это метод, который разбивает задачу на более мелкие независимые подзадачи, а затем объединяет решения всех подзадач.
- Динамическое программирование (ДП) — метод, который разбивает задачу на более мелкие, перекрывающиеся подзадачи, вычисляет решение для каждой из них и сохраняет его в ДП-таблице. Окончательное решение получается путем считывания из ДП-таблицы.
- Совершенствование алгоритмических навыков и умения воплощать алгоритмы в коде может быть достигнуто с помощью практики спортивного программирования (источники информации указаны в приложении А).
- Овладение навыками машинного обучения требует глубокого понимания фундаментальных основ. Список рекомендованной литературы в приложении А может дать вам представление о том, как расширить и углубить свои знания.
- Область машинного обучения стремительно развивается, и поэтому лучший способ оставаться в курсе событий — читать материалы последних научных конференций.

Часть 2

Обучение с учителем

Во второй части книги мы рассмотрим алгоритмы машинного обучения с учителем. Они используют размеченные примеры данных из обучающего набора и, в зависимости от дискретного или непрерывного характера предсказываемой величины, делятся на два основных класса: классификация и регрессия. Алгоритмы обучения с учителем широко используются в ML. Мы проведем вывод ряда наиболее интересных из них с нуля, чтобы заложить фундамент и стимулировать разработку новых подходов.

В главе 5 мы сосредоточимся на классификации и выведем несколько классических алгоритмов, в том числе перцептрон, SVM, логистическую регрессию, наивный байесовский алгоритм и деревья решений.

В главе 6 рассмотрим четыре наиболее интересных алгоритма регрессии: байесовскую линейную регрессию, иерархическую байесовскую регрессию, KNN- и регрессию на основе гауссовского процесса. Мы проведем вывод всех рассматриваемых алгоритмов, отталкиваясь от базовых принципов, и воплотим их в коде, используя наиболее передовые методы.

В главе 7 разберем некоторые алгоритмы обучения с учителем, в частности марковские модели, такие как ранжирование страниц и скрытые марковские модели. Также обсудим стратегии обучения при несбалансированной выборке, активное обучение, байесовскую оптимизацию для выбора гиперпараметров и ансамблевые методы. В заключение проведем обзор исследований в области ML, уделив особое внимание обучению с учителем.

5

Алгоритмы классификации

В этой главе

- ✓ Введение в задачу классификации
- ✓ Алгоритм перцептрона
- ✓ Алгоритм SVM
- ✓ Логистическая регрессия SGD
- ✓ Наивный байесовский алгоритм Бернулли
- ✓ Алгоритм дерева решений (CART)

В предыдущей главе мы рассмотрели основной инструментарий, необходимый для реализации алгоритмов ML с нуля. В этой главе мы сосредоточимся на алгоритмах обучения с учителем. Классификация — одна из наиболее часто встречающихся задач. Мы выведем с нуля и воплотим в коде несколько решений, чтобы сформировать базовые навыки и стимулировать разработку новых подходов ML. Алгоритмы, представленные в этой главе, иллюстрируют важные концепции и знакомят читателя со сценариями, которые могут быть реализованы с нуля, в порядке возрастания сложности. Описываемые методы имеют широкое применение, в частности, в задачах обнаружения спама, классификации документов и разбиения клиентской базы на сегменты.

5.1. Введение в задачу классификации

При обучении с учителем имеется набор данных $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$, состоящий из кортежей признаков x и меток y . Цель алгоритма классификации состоит в том, чтобы научиться находить соответствие между входными данными x и выходными метками y , где y — дискретная величина (то есть $y \in \{1, \dots, K\}$). Если $K = 2$, то имеем дело с двоичной классификацией, тогда как при $K > 2$ — с многоклассовой.

Классификатор h можно рассматривать как отображение между d -мерным вектором признаков $\phi(x)$ и k -мерной выходной меткой y (то есть $h: R^d \rightarrow R^k$). У нас часто имеется несколько моделей на выбор; давайте назовем этот набор моделей классификаторов H . Таким образом, для данного $h \in H$ мы можем получить предсказание класса $y = h(\phi(x))$. Обычно требуется предсказание для новых или «незнакомых» нашей модели данных. Другими словами, наш классификатор h должен быть способен *обобщать* результаты на новые образцы данных.

Процесс поиска подходящего классификатора называется *выбором модели*. Мы хотим подобрать такую модель, которая имеет достаточное количество параметров (степеней свободы), чтобы избежать недообучения или переобучения на тренировочных данных, как показано на рис. 5.1.

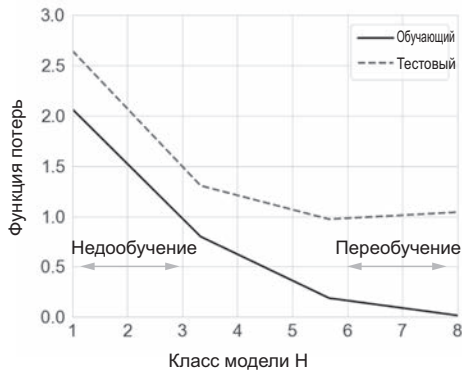


Рис. 5.1. Выбор модели для избежания переобучения или недообучения на тренировочных данных

Для моделей классов $H = [1, 2, 3]$ функция потерь при обучении и тестировании падающая, что указывает на нереализованный потенциал обучения. Как результат, эти модели недообучены. Для моделей классов $H = [6, 7, 8]$ функция потерь при обучении падает, тогда как при тестировании начинает расти, что указывает на то, что мы достигли переобучения.

5.2. Перцептрон

Давайте начнем с самой базовой модели классификации — *линейного классификатора*. Применим классификатор на основе перцептрона к набору данных цветков ириса. Линейный классификатор можно описать формулой 5.1:

$$\begin{aligned} h(x; \theta) &= \text{sign}(\theta_1 x_1 + \dots + \theta_d x_d + \theta_0) = \\ &= \text{sign}(\theta \cdot x + \theta_0) = \begin{cases} +1, & \text{если } \theta \cdot x + \theta_0 \geq 0 \\ -1, & \text{если } \theta \cdot x + \theta_0 < 0. \end{cases} \end{aligned} \quad (5.1)$$

Обратите внимание, как сигнум-функция (sign) от внутреннего произведения параметров θ и входных признаков x приводит его в соответствие меткам ± 1 . Если мыслить геометрически, то $\theta_x + \theta_0 = 0$ описывает гиперплоскость в d -мерном пространстве, однозначно определяемую вектором нормали θ . Точки, лежащие по одну сторону с нормалью θ , помечаются как $+1$, а точки с противоположной стороны — как -1 . Таким образом $\theta_x + \theta_0 = 0$ представляет собой *границу принятия решения* (decision boundary). Рисунок 5.2 иллюстрирует эти концепции в двумерном пространстве.

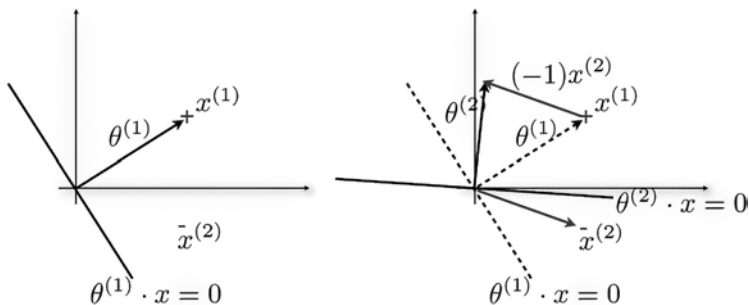


Рис. 5.2. Линейный классификатор: граница принятия решения

Как можно оценить эффективность такого классификатора? Один из способов — подсчитать количество ошибок, которые он допускает, по сравнению с эталонными метками y . Мы можем вычислить число ошибок следующим образом:

$$\varepsilon_n(\theta) = \frac{1}{n} \sum_{i=1}^n [[y_i \neq h(x_i; \theta)]] = \frac{1}{n} \sum_{i=1}^n [[y_i(\theta \cdot x_i + \theta_0) \leq 0]]. \quad (5.2)$$

Здесь $[[\cdot]]$ — это индикаторная функция, которая принимает значение 1, когда аргумент внутри скобок истинен, и 0 в противном случае. Обратите внимание, что в формуле 5.2 ошибка возникает всякий раз, когда метка $y_i \in [+1, -1]$ не соответствует предсказанию классификатора $h(x_i; \theta) \in [+1, -1]$ (то есть их произведение отрицательно).

Другим способом измерения качества двоичного классификатора является использование матрицы ошибок (confusion matrix).

На рис. 5.3 представлена таблица неточностей классификации, называемая *матрицей ошибок*. Прогноз точен, когда предсказанное значение совпадает с фактическим значением, как в случае истинно положительных (true positive, TP) и истинно отрицательных (true negative, TN) значений. По аналогии прогноз ошибочен, когда есть несоответствие между прогнозируемыми и фактическими значениями, как видно на примере ложноположительных (false positive, FP) и ложноотрицательных (false negative, FN) результатов. Изменяя порог классификации, мы получаем разные величины для числа TP, FP, FN и TN значений. Чтобы лучше представить себе работу классификатора при различных пороговых значениях, мы можем построить два дополнительных графика: ROC-кривую (receiver operating characteristic, ROC) и кривую точности — полноты (precision-recall curve; см. рис. 5.4).

		Фактический	
		$y = 1$	$y = 0$
Предсказанный	$\hat{y} = 1$	TP	FP
	$\hat{y} = 0$	FN	TN

Рис. 5.3. Матрица ошибок для двоичного классификатора

На графике ROC-кривой в левой части рис. 5.4 значение TPR соответствует доле истинно положительных случаев (true positive rate) и может быть вычислено следующим образом: $TPR = TP / (TP + FN)$. Мы можем также вычислить долю ложноположительных значений (false positive rate, FPR) как $FPR = FP / (FP + TN)$. Варьируя порог классификации, получаем разные точки на ROC-кривой. Идеальная классификация дает $TPR = 1$ и $FPR = 0$. На практике чем ближе точка лежит к верхнему левому углу, тем лучше будет классификатор. При случайном угадывании получаем диагональную линию $TPR = FPR$. Качество ROC-кривой часто описывается с помощью показателя, равного площади под кривой, или AUC (area under the curve). Чем выше показатель AUC, тем лучше, при максимальном значении AUC = 1.

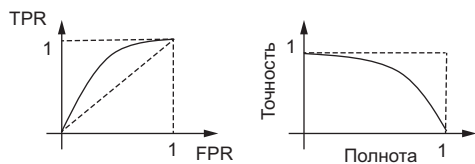


Рис. 5.4. График ROC-кривой (слева) и соотношения точность — полнота (справа)

В информационном поиске обычно используется график точность — полнота, изображенный в правой части рис. 5.4. Точность определяется как $Precision = TP / (TP + FP)$, а полнота равна $Recall = TP / (TP + FN)$. Кривая соотношения точности и полноты — это график, получаемый при варьировании порога классификации. Кривая может быть охарактеризована при помощи значения

средней точности, полученного путем усреднения по значениям полноты, что приблизительно соответствует площади под кривой. Кроме того, при фиксированном пороговом значении мы можем представить показатели в виде единой статистики, называемой *мерой F1*, которая равна среднему гармоническому значению точности и полноты: $F1 = 2PR/(P + R)$.

Перцептрон — это алгоритм, управляемый ошибками: он стартует с $\theta = 0$ и последовательно корректирует параметр θ с учетом каждого обучающего примера до тех пор, пока ошибки классификации не сойдут на нет, исходя из предположения, что данные линейно разделимы. Правило обновления перцептрона можно в общем виде представить следующим образом:

$$\begin{aligned} \text{если } y_i \neq h(x_i; \theta^{(k)}), \text{ то} \\ \theta^{(k+1)} &= \theta^{(k)} + y_i x_i; \\ \theta_0^{(k+1)} &= \theta_0^{(k)} + y_i. \end{aligned} \quad (5.3)$$

Здесь индекс k обозначает число обновлений параметра (оно же количество ошибок). Вы можете рассматривать обновление θ_0 как аналогичное обновлению θ , но с $x = 1$. Если обучающие примеры линейно разделимы, то алгоритм перцептрона в уравнении 5.3 сходится после конечного числа итераций. Обратите внимание, что порядок входных данных влияет на то, как обучается параметр θ . Поэтому имеет смысл рандомизировать (перетасовать) обучающий набор данных. В дополнение к этому, чтобы повлиять на сходимость θ , можем ввести параметр скорости обучения (learning rate), свойства которого мы обсудим позже. Алгоритм работы перцептрона в кратком виде представлен на рис. 5.5.

```

1: class perceptron
2:   function fit(X, y):
3:     k = 1
4:     for epoch = 1, 2, ..., num_epochs
5:       for i = 1, 2, ..., N
6:         if  $y_i(\theta \cdot x_i + \theta_0) \leq 0$ 
7:            $\eta = \frac{1}{k+1}$  | Обновление скорости обучения
8:            $k = k + 1$ 
9:            $\theta = \theta + \eta y_i x_i$ 
10:           $\theta_0 = \theta_0 + \eta y_i$  | Обновление  $\theta$ 
11:         конец if
12:       конец for
13:     конец for
14:   return  $\theta, \theta_0$ 
15:   function predict(X):
16:      $\hat{y} = \text{sign}(\theta \cdot X + \theta_0)$ 
17:   return  $\hat{y}$ 

```

Рис. 5.5. Псевдокод алгоритма перцептрона

Код состоит из класса `Perceptron` с двумя функциями: `fit` и `predict`. В функции `fit` мы принимаем обучающие данные X и метки y и в случае обнаружения ошибки (когда условие оператора `if` истинно) обновляем значения скорости обучения и параметров θ , как было показано ранее. Наконец, в функции `predict`, исходя из знака границы принятия решения, генерируем предсказание для тестовых данных.

Теперь у нас есть все составляющие для реализации алгоритма перцептрона с нуля! В исходном коде, приведенном ниже, мы классифицируем цветки ириса, обучая алгоритм перцептрона на обучающем наборе признаков и далее строя предсказание для тестовых данных.

Листинг 5.1. Алгоритм работы перцептрона

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from scipy.stats import randint
from sklearn.datasets import load_iris
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split

class perceptron:
    def __init__(self, num_epochs, dim):
        self.num_epochs = num_epochs
        self.theta0 = 0
        self.theta = np.zeros(dim)

    def fit(self, X_train, y_train):
        n = X_train.shape[0]
        dim = X_train.shape[1]

        k = 1
        for epoch in range(self.num_epochs):
            for i in range(n):
                idx = randint.rvs(0, n-1, size=1)[0]  ← Генерация случайной точки
                if (y_train[idx] * (np.dot(self.theta,
                    ⇒ X_train[idx,:]) + self.theta0) <= 0):  ← Кусочно-линейная
                                                                функция потерь
                    eta = pow(k+1, -1)  ← Обновление скорости обучения
                    k += 1
                    self.theta = self.theta + eta *
                    ⇒ y_train[idx] * X_train[idx, :]
                    self.theta0 = self.theta0 + eta *
                    ⇒ y_train[idx]
                # конец if

                print("epoch: ", epoch)
                print("theta: ", self.theta)
                print("theta0: ", self.theta0)
            # конец for
        # конец for
```

Обновление θ и θ_0

```

def predict(self, X_test):
    n = X_test.shape[0]
    dim = X_test.shape[1]

    y_pred = np.zeros(n)
    for idx in range(n):
        y_pred[idx] = np.sign(np.dot(self.theta, X_test[idx,:]) +
            ⇒ self.theta0)
    # конец for
    return y_pred

if __name__ == "__main__":

    iris = load_iris()      ← Загрузка набора данных
    X = iris.data[:100,:]
    y = 2*iris.target[:100] - 1  ← Приведение к меткам {+1,-1}

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
        ⇒ random_state=42)

    # (двоичный) классификатор на основе перцептрона
    clf = perceptron(num_epochs=5, dim=X.shape[1])
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)

    cmt = confusion_matrix(y_test, y_pred)
    acc = np.trace(cmt)/np.sum(np.sum(cmt))
    print("perceptron accuracy: ", acc)

    # создание графиков
    plt.figure()
    sns.heatmap(cmt, annot=True, fmt="d")
    plt.title("Матрица ошибок"); plt.xlabel("Предсказанный");
    ⇒ plt.ylabel("Фактический")
    plt.savefig("./figures/perceptron_acc.png")
    plt.show()

```

В результате исполнения алгоритма на тестовом наборе данных мы наблюдаем показатели верности классификации, представленные на рис. 5.6.

Давайте еще раз посмотрим на нашу реализацию и обсудим некоторые детали. Алгоритм перцептрона может быть сформулирован как стохастический градиентный спуск, который минимизирует кусочно-линейную функцию потерь (hinge loss). Рассмотрим функцию потерь, которая измеряет степень несоответствия $z_i = y_i(\theta \cdot x_i + \theta_0)$ между меткой y_i и предсказанием $h(x_i; \theta)$:

$$Loss_h(z) = \frac{1}{n} \sum_{i=1}^n \max\{1 - z_i, 0\} = \frac{1}{n} \sum_{i=1}^n \max\{1 - y_i(\theta \cdot x_i + \theta_0), 0\}. \quad (5.4)$$

Она называется кусочно-линейной функцией потерь и изображена на рис. 5.7.

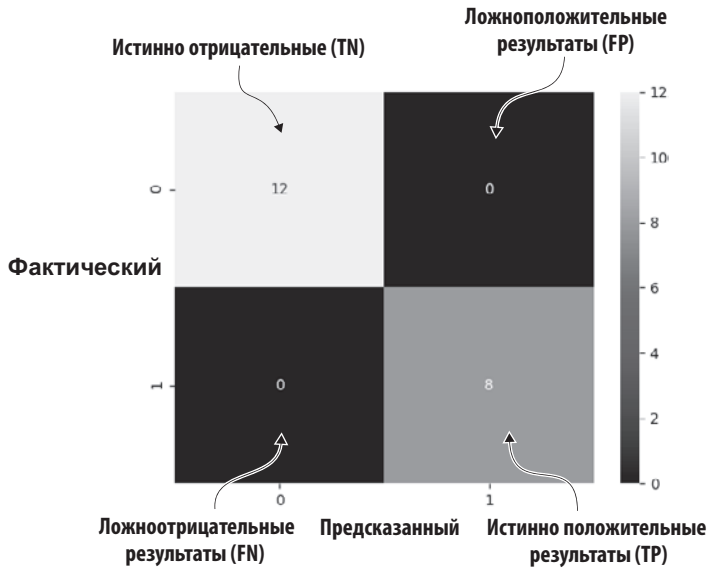


Рис. 5.6. Матрица ошибок двоичного классификатора на основе перцептрона (набор данных цветков ириса)

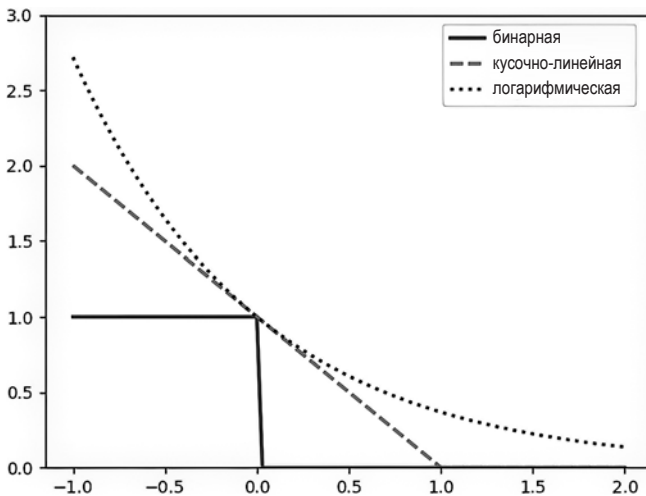


Рис. 5.7. Функции потерь: кусочно-линейная, бинарная (0–1) и логарифмическая

Стохастический градиентный спуск пытается свести к минимуму значение кусочно-линейной функции потерь, беря градиент по параметру θ . Однако оператор $\max$ недифференцируем при $z_i = 1$. На самом деле в этой точке у нас есть несколько возможных градиентов, которые собирательно называют *субдифференциалом*.

Поскольку функция потерь является кусочно-линейной, градиент при $z_i > 1$ равен нулю, тогда как градиент при $z_i \leq 1$ находится согласно формуле 5.5:

$$\begin{aligned}\nabla_{\theta} (1 - y_i(\theta \cdot x_i + \theta_0)) &= -y_i x_i; \\ \nabla_{\theta_0} (1 - y_i(\theta \cdot x_i + \theta_0)) &= -y_i.\end{aligned}\tag{5.5}$$

Объединив выражение из формулы 5.5 с формулой обновления стохастического градиентного спуска (где η — скорость обучения), мы приходим к формуле 5.6:

$$\theta^{(k+1)} = \theta^{(k)} - \eta_k \nabla_{\theta} \text{Loss}_h(y_i(\theta \cdot x_i + \theta_0)).\tag{5.6}$$

Это искомый алгоритм перцептрона! В следующем разделе мы поговорим о другом важном алгоритме классификации — *методе опорных векторов* (support vector machine, SVM).

5.3. Метод опорных векторов

В предыдущем разделе мы оценивали эффективность нашего классификатора, минимизировав функцию ожидаемых потерь (она же эмпирический риск). Но возникает следующий вопрос в связи с вышеизложенным. Предположим, что существует несколько различных классификаторов (несколько значений параметров θ и θ_0), которые позволяют достичь одного и того же значения эмпирического риска. Как нам тогда выбрать среди них лучшую модель и что означает в данной ситуации «лучшая»?

Одним из решений является регуляризация функции потерь в пользу малых значений параметров, как показано в формуле 5.7:

$$L_n(\theta, \theta_0) = \frac{\lambda}{2} \|\theta\|^2 + \frac{1}{n} \sum_{i=1}^n \text{Loss}(y_i(\theta \cdot x_i + \theta_0)).\tag{5.7}$$

Здесь регуляризация применяется к θ , но не к θ_0 . Причина в том, что θ определяет ориентацию границы принятия решения, тогда как θ_0 связана с ее смещением от начала координат, которое изначально неизвестно.

Давайте попробуем лучше представить себе границу принятия решения геометрически. Нам хотелось бы, чтобы граница принятия решения, во-первых, правильно классифицировала все точки данных и, во-вторых, была максимально удалена от всех обучающих примеров, то есть имела максимальный зазор (margin). Предположим, что первое условие выполнено, и для оптимизации в рамках второго условия нужно вычислить и максимизировать расстояние от каждого обучающего примера до границы принятия решения. Формула такого расстояния выглядит следующим образом:

$$\gamma_i = \frac{y_i(\theta \cdot x_i + \theta_0)}{\|\theta\|}.\tag{5.8}$$

Поскольку мы хотим максимизировать зазор, требуется максимизировать наименьшее расстояние до границы принятия решения по всем точкам данных (то есть найти $\max[\min_i \gamma_i]$). Это можно проще сформулировать как квадратичную программу с линейными ограничениями. *Квадратичная программа* (quadratic program, QP) — это тип математической задачи оптимизации, которая включает в себя оптимизацию квадратичной целевой функции с учетом линейных ограничений на переменные:

$$(\text{прямая задача}) \quad \min \frac{1}{2} \|\theta\|^2 \quad \text{при условиях } y_i(\theta \cdot x_i + \theta_0) \geq 1, \quad i = 1, \dots, n. \quad (5.9)$$

Здесь мы, по существу, минимизируем регуляризованную функцию потерь, подбирая θ по евклидовой норме l_2 , при условии соблюдения ограничения на верную классификацию каждого обучающего примера (рис. 5.8).

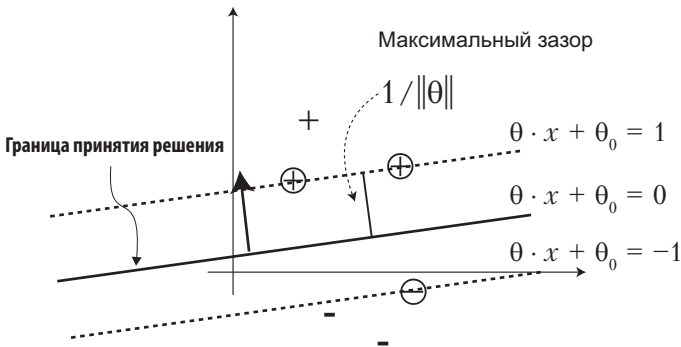


Рис. 5.8. Решение задачи определения максимального зазора SVM-классификатора

Обратите внимание, что по мере того как мы сводим к минимуму $\|\theta\|^2$, мы фактически увеличиваем расстояние $\gamma_i \propto 1/\|\theta\|$ между границей принятия решения и точками обучающих данных с индексом i . Геометрически мы отодвигаем границы зазора друг от друга, как показано на рис. 5.8. В какой-то момент они не могут быть дальше отодвинуты без нарушения ограничений классификации. Тогда границы фиксируются, давая уникальное решение задачи о максимальном зазоре. Точки обучающих данных, лежащие на границах зазора, становятся *опорными векторами*. Нам требуется только некоторое подмножество обучающих примеров (опорных векторов), чтобы полностью настроить параметры модели SVM.

Давайте посмотрим, сможем ли мы получить какую-то пользу от решения двойственной формы квадратичной программы. Напомним, что если прямая задача минимизационная, то двойственная — максимизационная (и наоборот). Кроме того, каждая переменная исходной (первичной) задачи становится ограничением в двойственной задаче, а каждое ограничение в первичной задаче — переменной в двойственной задаче.

Мы можем получить двойственную форму, расписав лагранжиан (то есть добавив ограничения к целевой функции с неотрицательными множителями Лагранжа):

$$\max_{\alpha \geq 0} L(\theta, \theta_0; \alpha) = \frac{1}{2} \|\theta\|^2 - \sum_{i=1}^n \alpha_i [y_i(\theta \cdot x_i + \theta_0) - 1]. \quad (5.10)$$

Теперь можем вычислить градиент относительно наших параметров:

$$\begin{aligned} \nabla_{\theta} L(\theta, \theta_0; \alpha) &= \theta - \sum_{i=1}^n \alpha_i y_i x_i = 0; \\ \frac{d}{d\theta_0} L(\theta, \theta_0; \alpha) &= - \sum_{i=1}^n \alpha_i y_i = 0. \end{aligned} \quad (5.11)$$

Подставляя выражение для θ обратно в наш лагранжиан, получаем следующее:

$$\begin{aligned} (\text{двойственная задача}) \quad \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j [x_i^T x_j] \\ & \text{при условиях } \alpha_i \geq 0, \quad \sum_{i=1}^n \alpha_i y_i = 0. \end{aligned} \quad (5.12)$$

Наиболее заметным изменением в двойственной задаче является то, что в ней d -мерные точки данных x_i и x_j взаимодействуют через внутреннее произведение. Такая запись имеет значительные вычислительные преимущества по сравнению с прямой задачей (в дополнение к более простым ограничениям в двойственной задаче).

Внутреннее произведение измеряет степень сходства между двумя векторами и может быть обобщено при помощи ядер $K(x_i, x_j)$. Ядра измеряют степень сходства между объектами без явного их представления в виде векторов признаков. Такой подход особенно удобен в тех случаях, когда у нас нет доступа к внутреннему устройству объектов или мы предпочитаем «внутри не заглядывать». Как правило, функция ядра, которая сравнивает два объекта $x_i, x_j \in X$, является симметричной $K(x_i, x_j) = K(x_j, x_i)$ и неотрицательной $K(x_i, x_j) \geq 0$. Существует большое разнообразие ядер, начиная с графовых, которые вычисляют сходство между графами, и заканчивая ядрами строк или документов. В нашей реализации SVM мы будем использовать известное ядро в виде радиальной базисной функции (radial basis function, RBF), представленное формулой ниже:

$$K(x_i, x_j) = \exp \left(- \frac{\|x_i - x_j\|^2}{2\sigma^2} \right). \quad (5.13)$$

Теперь можем самостоятельно реализовать двоичный SVM-классификатор, используя оптимизационный пакет CVXOPT. CVXOPT — это бесплатный программный пакет языка программирования Python для выпуклой оптимизации, который можно скачать по адресу cvxopt.org.

Стандартная форма квадратичной программы (QP), если пользоваться обозначениями библиотеки CVXOPT, приведена ниже:

$$\min_x \frac{1}{2} x^T P x + q^T x \text{ при условиях } Gx \leq h, Ax = b. \quad (5.14)$$

Обратите внимание, что эта целевая функция выпукла тогда и только тогда, когда матрица P положительно полуопределена. Функция из пакета CVXOPT для решения QP ожидает, что она будет представлена в виде формулы 5.14 с параметрами (P, q, G, h, A, b) . Давайте приведем нашу двойственную QP к этому виду. Пусть P — такая матрица, что справедливо следующее:

$$P_{ij} = y_i y_j [x_i^T x_j]. \quad (5.15)$$

Тогда задача оптимизации приобретает следующий вид:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \alpha^T P \alpha \text{ при условиях } \alpha_i \geq 0, \sum_{i=1}^n \alpha_i y_i = 0. \quad (5.16)$$

Мы можем дополнительно модифицировать QP путем умножения целевой функции и ограничения на -1 , тем самым превращая задачу в минимизационную и меняя неравенство на противоположное. Кроме того, мы можем преобразовать сумму по α в векторную форму, умножив α -вектор на вектор из единиц:

$$\min_{\alpha} \frac{1}{2} a^T P \alpha - 1^T \alpha \text{ при условиях } -\alpha_i \leq 0, y^T \alpha = 0. \quad (5.17)$$

Теперь мы можем воспользоваться пакетом CVXOPT для решения нашей SVM-квадратичной программы. Давайте начнем с обзора псевдокода на рис. 5.9:

```

1: class SupportVectorMachine
2:   function fit(X, y):
3:      $P_{ij} = y_i y_j K(x_i, x_j)$ 
4:      $q = -1$ 
5:      $G_{ij} = -1$ 
6:      $h = 0$ 
7:      $A = y$ 
8:      $b = 0$ 
9:      $\text{sol} = \text{cvxopt.solvers.qp}(P, q, G, h, A, b)$  ← Решение при помощи CVXOPT
10:     $\text{alphas} = \text{sol}[x]$ 
11:     $S = \text{alphas} > 1e-11$  ← Нахождение опорных векторов
12:     $\theta = \sum_{i=1}^n y_i \alpha_i x_i$  ← Нахождение вектора нормали
13:     $\theta_0 = y_s - \sum_{m \in S} \alpha_m y_m [x_m^T x_s]$  ← Нахождение вектора нормали
14:    return  $\theta, \theta_0$ 
15:   function predict(X,  $\theta, \theta_0$ ):
16:      $\hat{y} = \text{sign}(\theta^T X + \theta_0)$  ← Предсказание
17:   return  $\hat{y}$ 
```

Формулировка
 SVM-квадратичной программы

Рис. 5.9. Псевдокод машины опорных векторов

Класс SVM содержит две функции: `fit` и `predict`. Функция `fit` начинается с формулировки квадратичной задачи, которую необходимо решить с помощью пакета CVXOPT, и определения всех входных параметров: (P, q, G, h, A, b) . После вызова функции решения задачи (`solvers`) мы находим опорные векторы как α , большие нуля (с точностью до ошибки округления). Далее мы вычисляем вектор нормали и точку пересечения, как обсуждалось ранее. В функции `predict` мы используем вычисленные опорные векторы нормали и пересечения для предсказания меток тестовых данных.

Листинг 5.2. Алгоритм SVM

```
import cvxopt
import numpy as np

from sklearn.svm import SVC  ← Только для сравнения
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

def rbf_kernel(gamma, **kwargs):
    def f(x1, x2):
        distance = np.linalg.norm(x1 - x2) ** 2
        return np.exp(-gamma * distance)
    return f

class SupportVectorMachine(object):
    def __init__(self, C=1, kernel=rbf_kernel, power=4, gamma=None):
        self.C = C  ← Константа регуляризации
        self.kernel = kernel
        self.power = power
        self.gamma = gamma
        self.lagr_multipliers = None
        self.support_vectors = None
        self.support_vector_labels = None
        self.intercept = None

    def fit(self, X, y):
        n_samples, n_features = np.shape(X)

        if not self.gamma:
            self.gamma = 1 / n_features

        self.kernel = self.kernel(
            power=self.power,
            gamma=self.gamma
        )  ← Инициализация параметров метода ядра

        kernel_matrix = np.zeros((n_samples, n_samples))
        for i in range(n_samples):
            for j in range(n_samples):
                kernel_matrix[i, j] = self.kernel(X[i],
                    X[j])  ← Вычисление матрицы ядра
```

```

P = cvxopt.matrix(np.outer(y, y) * kernel_matrix,
    ⇒ tc='d')
q = cvxopt.matrix(np.ones(n_samples) * -1)
A = cvxopt.matrix(y, (1, n_samples), tc='d')
b = cvxopt.matrix(0, tc='d')

```

Определение квадратичной задачи оптимизации

```

if not self.C: # если пустая
    G = cvxopt.matrix(np.identity(n_samples) * -1)
    h = cvxopt.matrix(np.zeros(n_samples))
else:
    G_max = np.identity(n_samples) * -1
    G_min = np.identity(n_samples)
    G = cvxopt.matrix(np.vstack((G_max, G_min)))
    h_max = cvxopt.matrix(np.zeros(n_samples))
    h_min = cvxopt.matrix(np.ones(n_samples) * self.C)
    h = cvxopt.matrix(np.vstack((h_max, h_min)))

```

```

minimization = cvxopt.solvers
    ⇒ .qp(P, q, G, h, A, b)

```

Решение квадратичной задачи оптимизации с использованием cvxopt

```

lagr_mult = np.ravel(minimization['x'])

```

Множители Лагранжа

```

# Получение индексов ненулевых множителей Лагранжа
idx = lagr_mult > 1e-11
# Получение соответствующих множителей Лагранжа
self.lagr_multipliers = lagr_mult[idx]
# Получение примеров, выступающих в качестве опорных векторов
self.support_vectors = X[idx]
# Получение соответствующих меток
self.support_vector_labels = y[idx]

```

Извлечение опорных векторов

```

self.intercept = self.support_vector_labels[0]
for i in range(len(self.lagr_multipliers)):
    self.intercept -= self.lagr_multipliers[i] *
self.support_vector_labels[
    i] * self.kernel(self.support_vectors[i], self.support_vectors[0])

```

Вычисление точки пересечения с первым опорным вектором

```

def predict(self, X):
    y_pred = []
    for sample in X:
        prediction = 0
        # Определение метки примера по опорным векторам
        for i in range(len(self.lagr_multipliers)):
            prediction += self.lagr_multipliers[i] *
                ⇒ self.support_vector_labels[
                    i] * self.kernel(self.support_vectors[i], sample)
        prediction += self.intercept
        y_pred.append(np.sign(prediction))
    return np.array(y_pred)

```

Генерация предсказаний в цикле по списку примеров

```

def main():
    # Загрузка набора данных
    iris = load_iris()
    X = iris.data[:100,:]

```

```

y = 2*iris.target[:100] - 1  ← Приведение к меткам {+1, -1}
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4)
clf = SupportVectorMachine(kernel=rbf_kernel, gamma = 1)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print ("Accuracy (scratch):", accuracy)

clf_sklearn = SVC(gamma = 'auto')
clf_sklearn.fit(X_train, y_train)
y_pred2 = clf_sklearn.predict(X_test)
accuracy = accuracy_score(y_test, y_pred2)
print ("Accuracy :", accuracy)

if __name__ == "__main__":
    main()

```

Результаты выполнения показывают, что доля верных результатов SVM-классификации в нашей реализации соответствует верности модели из пакета sklearn!

5.4. Логистическая регрессия

Логистическая регрессия представляет собой алгоритм классификации. Прежде чем приступить к реализации, давайте сначала познакомимся с некоторыми теоретическими представлениями, лежащими в основании логистической регрессии. С вероятностной точки зрения на классификацию нам требуется посчитать вероятность метки класса C_k при учете входных данных x : $p(C_k|x)$. Рассмотрим два класса: C_1 и C_2 . Мы можем использовать правило Байеса для вычисления интересующей апостериорной вероятности:

$$p(C_1|x) = \frac{p(x|C_1)p(C_1)}{p(x|C_1)p(C_1) + p(x|C_2)p(C_2)}. \quad (5.18)$$

Здесь $p(C_k)$ — априорные вероятности классов. Разделив правую часть формулы на числитель, получаем следующее:

$$p(C_1|x) = \frac{1}{1 + \frac{p(x|C_2)p(C_2)}{p(x|C_1)p(C_1)}} + \frac{1}{1 + \exp(-a)} = \sigma(a). \quad (5.19)$$

Здесь мы ввели следующее обозначение:

$$a = \ln \frac{p(x|C_1)p(C_1)}{p(x|C_2)p(C_2)}. \quad (5.20)$$

В случае многоклассовой классификации ($K > 2$) мы имеем следующее:

$$p(C_k | x) = \frac{p(x | C_k)p(C_k)}{\sum_i p(x | C_i)p(C_i)} = \frac{\exp(a_k)}{\sum_i \exp(a_i)}. \quad (5.21)$$

В формуле 5.21 $a_k = \ln p(x | C_k)p(C_k)$. Это выражение также известно как многопеременная логистическая функция *softmax*. Она преобразует вектор вещественных чисел длины K в вектор из K значений, сумма которых равна 1. Поэтому результат функции softmax можно интерпретировать как распределение вероятностей. Теперь дело за выбором условных плотностей, которые хорошо моделируют данные. В случае двоичной логистической регрессии с параметрами θ и метками класса $y = C_k$ мы имеем следующее равенство:

$$p(C_k | x) = p(y | x, \theta) = \text{Ber}(y | \sigma(\theta^T x)). \quad (5.22)$$

Совместное распределение можно вычислить следующим образом:

$$p(x_i, y_i | \theta) = p(y_i | x_i, \theta)p(x_i | \theta) = \text{Ber}(y_i | \sigma(\theta^T x_i))p(x_i | \theta). \quad (5.23)$$

Поскольку мы не занимаемся моделированием распределения данных $p(x_i | \theta) = p(x_i)$, логарифмическое правдоподобие можно записать следующим образом:

$$\begin{aligned} \log p(D | \theta) &= \log \prod_{i=1}^n p(x_i, y_i | \theta) = \sum_{i=1}^n \log p(x_i, y_i | \theta) = \\ &= \sum_{i=1}^n \log \text{Ber}(y_i | \sigma(\theta^T x_i)) = \\ &= \sum_{i=1}^n \log \left[\sigma(\theta^T x_i)^{y_i} (1 - \sigma(\theta^T x_i))^{1-y_i} \right] = \\ &= \sum_{i=1}^n \left[y_i \log \sigma(\theta^T x_i) + (1 - y_i) \log (1 - \sigma(\theta^T x_i)) \right]. \end{aligned} \quad (5.24)$$

Заметьте, что нам нужно максимизировать логарифмическое правдоподобие или, что эквивалентно, минимизировать потери (loss) или отрицательную логарифмическую функцию правдоподобия (negative log likelihood, NLL):

$$\min_{\theta} \text{Loss}(\theta) = \min_{\theta} \text{NLL}(\theta) = \max_{\theta} \log p(D | \theta). \quad (5.25)$$

Мы будем минимизировать функцию потерь с помощью стохастического градиентного спуска (stochastic gradient descent, SGD), который можно представить в виде формулы ниже:

$$\theta_{k+1} = \theta_k - \eta_k g_k. \quad (5.26)$$

Здесь g_k — градиент, а η_k — размер шага. Чтобы обеспечить сходимость SGD, должны быть соблюдены следующие условия для скорости обучения, известные как *условия Роббинса — Монро*:

$$\begin{aligned} \sum_{k=1}^{\infty} \eta_k &= \infty; \\ \sum_{k=1}^{\infty} \eta_k^2 &< \infty. \end{aligned} \quad (5.27)$$

Данным условиям удовлетворяет следующий график изменения скорости обучения:

$$\eta_k = (\tau_0 + k)^{-\kappa}. \quad (5.28)$$

Здесь $\tau_0 \geq 0$ замедляет первые итерации алгоритма, а $\kappa \in (0.5, 1]$ управляет скоростью, с которой «забываются» старые значения. Чтобы вычислить направление наискорейшего спуска g_k , необходимо дифференцировать функцию потерь NLL(θ):

$$\begin{aligned} \frac{d}{d\theta} \log p(D|\theta) &= \sum_{i=1}^n \left[y_i \frac{d}{d\theta} \log \sigma(\theta^T x_i) + \right. \\ &\quad \left. + (1 - y_i) \frac{d}{d\theta} \log (1 - \sigma(\theta^T x_i)) \right] = \\ &= \sum_{i=1}^n \left[y_i \frac{\sigma(\theta^T x_i)(1 - \sigma(\theta^T x_i))}{\sigma(\theta^T x_i)} x_i + \right. \\ &\quad \left. + (1 - y_i) \frac{\sigma(\theta^T x_i)(1 - \sigma(\theta^T x_i))}{1 - \sigma(\theta^T x_i)} (-x_i) \right] = \\ &= \sum_{i=1}^n \left[y_i x_i (1 - \sigma(\theta^T x_i)) - (1 - y_i) x_i \sigma(\theta^T x_i) \right] = \\ &= \sum_{i=1}^n \left[y_i - \sigma(\theta^T x_i) \right] x_i = \\ &= \sum_{i=1}^n [y_i - \mu_i] x_i = -X^T (\mu - y). \end{aligned} \quad (5.29)$$

В формуле 5.29 мы воспользовались тем, что для распределения Бернулли $d/dx \sigma(x) = (1 - \sigma(x))\sigma(x)$ и математическое ожидание $\mu_i = \sigma(\theta^T x_i)$. Стоит заметить, что существует целый ряд библиотек для автоматического дифференцирования (autograd), позволяющих избежать необходимости выводить градиенты вручную. Мы можем также добавить регуляризацию для управления размером параметров. Регуляризованная целевая функция и градиент приобретают следующий вид:

$$\min_{\theta} Loss = \min_{\theta} [NLL(\theta) + \lambda \theta^T \theta]; \quad (5.30)$$

$$g_k = X^T(\mu - y) + 2\lambda\theta.$$

Теперь мы готовы реализовать алгоритм логистической регрессии со стохастическим градиентным спуском в коде. Давайте начнем с псевдокода, представленного на рис. 5.10.

```

1: class sgdlr
2:   function lr_objective(θ, X, y, λ)
3:     μi = sigmoid(θT Xi)
4:     cost = -∑i=1n [yi log μi + (1 - yi) log (1 - μi)] + λθT θ
5:     grad = XT(μ - y) + 2λθ ← Вычисление градиента
6:     return cost, grad
7:   function fit(X, y):
8:     ηi = (τ + i)-κ ← Задание скорости обучения
9:     for i = 1, 2, ... num_iter
10:       cost, grad = lr_objective(θ, X, y, λ)
11:       θ = θ - ηi grad ← Обновление θ
12:     конец for
13:   return θ
14:   function predict(X, θ):
15:     ŷ = sigmoid (θT X) ← Предсказание
16:   return ŷ

```

Рис. 5.10. Псевдокод алгоритма логистической регрессии

Класс `sgdlr` содержит три основные функции: `lr_objective`, `fit` и `predict`. Функция `lr_objective` вычисляет регуляризованную целевую функцию и градиент целевой функции описанным выше способом. В функции `fit` мы сначала задаем скорость обучения и на каждой итерации обновляем параметры θ в направлении, противоположном градиенту. Наконец, функция `predict` генерирует предсказание двоичной метки для тестовых данных. В коде ниже для обучения модели логистической регрессии используется синтезированный набор данных в виде гауссовой смеси.

Листинг 5.3. Логистическая регрессия со стохастическим градиентным спуском

```

import numpy as np
import matplotlib.pyplot as plt

def generate_data():
    n = 1000
    mu1 = np.array([1,1])
    mu2 = np.array([-1,-1])

```

116 Глава 5. Алгоритмы классификации

```
pik = np.array([0.4,0.6])

X = np.zeros((n,2))
y = np.zeros((n,1))

for i in range(1,n):
    u = np.random.rand()
    idx = np.where(u < np.cumsum(pik))[0]
    if (len(idx)==1):
        X[i,:] = np.random.randn(1,2) + mu1
        y[i] = 1
    else:
        X[i,:] = np.random.randn(1,2) + mu2
        y[i] = -1
return X, y

class sgdlr:

    def __init__(self):

        self.num_iter = 100
        self.lmbda = 1e-9

        self.tau0 = 10
        self.kappa = 1
        self.eta = np.zeros(self.num_iter)

        self.batch_size = 200
        self.eps = np.finfo(float).eps

    def fit(self, X, y):

        theta = np.random.randn(X.shape[1],1)  ← Случайная инициализация

        for i in range(self.num_iter):
            self.eta[i] = (self.tau0+i)*(-self.kappa)  ← Изменение скорости обучения

            batch_data, batch_labels = self.make_batches(
                X,y,self.batch_size)  ← Разбиение данных на пакеты
            num_batches = batch_data.shape[0]
            num_updates = 0

            J_hist = np.zeros((self.num_iter * num_batches,1))
            t_hist = np.zeros((self.num_iter * num_batches,1))

            for itr in range(self.num_iter):
                for b in range(num_batches):
                    Xb = batch_data[b]
                    yb = batch_labels[b]

                    J_cost, J_grad = self.lr_objective(theta, Xb, yb, self.lmbda)
                    theta = theta - self.eta[itr]*(num_batches*J_grad)

                    J_hist[num_updates] = J_cost
                    t_hist[num_updates] = np.linalg.norm(theta,2)
                    num_updates = num_updates + 1
                    print("iteration %d, cost: %f" %(itr, J_cost))

            y_pred = 2*(self.sigmoid(X.dot(theta)) > 0.5) - 1
```

```

y_err = np.size(np.where(y_pred - y)[0])/float(y.shape[0])
print("classification error:", y_err)
self.generate_plots(X, J_hist, t_hist, theta)
return theta

def make_batches(self, X, y, batch_size):
    n = X.shape[0]
    d = X.shape[1]
    num_batches = int(np.ceil(n/batch_size))

    groups = np.tile(range(num_batches),batch_size)
    batch_data=np.zeros((num_batches,batch_size,d))
    batch_labels=np.zeros((num_batches,batch_size,1))

    for i in range(num_batches):
        batch_data[i,:,:] = X[groups==i,:]
        batch_labels[i,:] = y[groups==i]
    return batch_data, batch_labels

def lr_objective(self, theta, X, y, lmbda):  ← Вычисление целевой функции

    n = y.shape[0]
    y01 = (y+1)/2.0

    mu = self.sigmoid(X.dot(theta))

    mu = np.maximum(mu,self.eps)  | Отступы от нуля
    mu = np.minimum(mu,1-self.eps) | и единицы

    cost = -(1/n)*np.sum(y01*np.log(mu)+
    ⇒ (1-y01)*np.log(1-mu))+np.sum(lmbda*theta*theta)  ← Вычисление потерь

    grad = X.T.dot(mu-y01) + 2*lmbda*theta  ← Вычисление градиента
                                         | целевой функции
    # вычисление гессиана целевой функции
    #H = X.T.dot(np.diag(np.diag( mu*(1-mu) ))).dot(X) +
    ⇒ 2*lmbda*np.eye(np.size(theta))

    return cost, grad

def sigmoid(self, a):
    return 1/(1+np.exp(-a))

def generate_plots(self, X, J_hist, t_hist, theta):

    plt.figure()
    plt.plot(J_hist)
    plt.title("Логистическая регрессия")
    plt.xlabel('Итерации')
    plt.ylabel('Потери')
    #plt.savefig('./figures/lrsgd_loss.png')
    plt.show()

    plt.figure()
    plt.plot(t_hist)

```

```

plt.title("LR theta l2 norm")
plt.xlabel('iterations')
plt.ylabel('theta l2 norm')
#plt.savefig('./figures/lrsgd_theta_norm.png')
plt.show()

plt.figure()
plt.plot(self.eta)
plt.title("LR learning rate")
plt.xlabel('iterations')
plt.ylabel('learning rate')
#plt.savefig('./figures/lrsgd_learning_rate.png')
plt.show()

plt.figure()
x1 = np.linspace(np.min(X[:,0])-1,np.max(X[:,0])+1,10)
plt.scatter(X[:,0], X[:,1])
plt.plot(x1, -(theta[0]/theta[1])*x1)
plt.title('Граница принятия решения')
plt.grid(True)
plt.xlabel('X1')
plt.ylabel('X2')
#plt.savefig('./figures/lrsgd_clf.png')
plt.show()

if __name__ == "__main__":

    X, y = generate_data()
    sgd = sgdlr()
    theta = sgd.fit(X,y)

```

Рисунок 5.11 демонстрирует стохастическую природу функции потерь, которая уменьшается с увеличением числа итераций, а также границу принятия решения, полученную с помощью двоичной логистической регрессии.

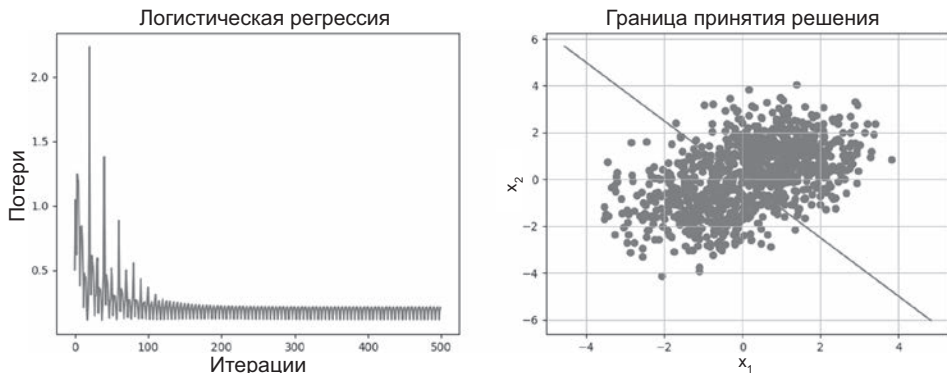


Рис. 5.11. Логистическая регрессия со стохастическим градиентным спуском: потери (слева) и граница принятия решения (справа)

Естественным обобщением двоичной логистической регрессии является мультиномиальная логистическая регрессия, которая подходит для случая более двух классов.

5.5. Наивный байесовский классификатор

Этот раздел посвящен описанию, выводу формул и реализации наивного байесовского алгоритма. Фундаментальное (наивное) предположение алгоритма заключается в том, что признаки являются условно независимыми при заданной метке класса. Это позволяет нам записать плотность условного распределения класса как произведение одномерных плотностей:

$$p(x_i | y = c, \theta) = \prod_{j=1}^D p(x_{ij} | y = c, \theta_{jc}). \quad (5.31)$$

Подобная модель называется наивной, потому что мы не ожидаем, что признаки действительно будут условно независимы. Однако даже если данное предположение ложно, модель, тем не менее, хорошо работает во многих случаях. Здесь мы сосредоточимся на наивной байесовской классификации Бернулли для документов, модель которой может быть представлена в виде графа, изображенного на рис. 5.12. Обратите внимание, что заштрихованные узлы представляют собой наблюдаемые переменные.

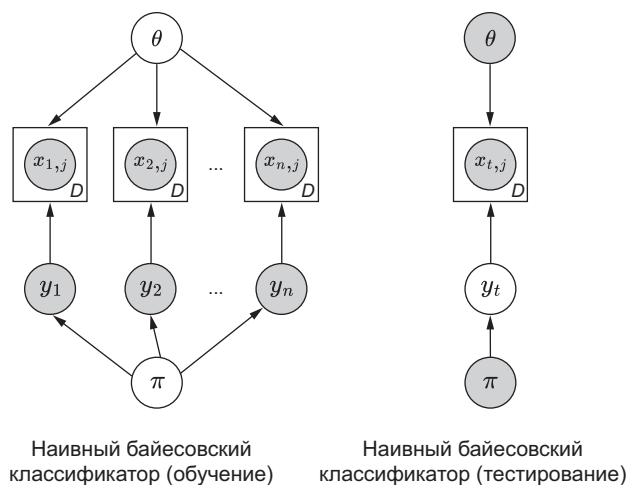


Рис. 5.12. Наивная байесовская вероятностная графическая модель

Выбор класса плотности условного распределения $p(x|y = c, \theta)$ определяет тип наивного байесовского классификатора, такой как гауссовский, бернуллиевский или мультиномиальный. В этом разделе мы сосредоточимся на методе наивной

байесовской классификации Бернулли из-за его высокой эффективности для классификации документов.

Пусть x_{ij} — случайные величины Бернулли, указывающие на наличие ($x_{ij} = 1$) или отсутствие ($x_{ij} = 0$) слова $j \in \{1, \dots, D\}$ в документе $i \in \{1, \dots, N\}$, параметризованные θ_{jc} для данной метки класса $y = c \in \{1, \dots, C\}$. Кроме того, пусть π — распределение Дирихле, представляющее априорное распределение для меток класса. Таким образом, общее количество доступных для обучения параметров равно $|\theta| + |\pi| = O(DC) + O(C) = O(DC)$, где D — размер словаря, а C — количество классов. Из-за небольшого числа параметров такая наивная байесовская модель устойчива к переобучению.

Мы можем записать плотность условного распределения класса так, как показано в формуле 5.32:

$$p(x | y = c, \theta) = \prod_{i=1}^n \prod_{j=1}^D p(x_{ij} | y = c, \theta_{jc}) = \prod_{i=1}^n \prod_{j=1}^D \text{Bernoulli}(\theta_{jc}). \quad (5.32)$$

Далее алгоритм наивного байесовского вывода можно получить, максимизируя логарифмическое правдоподобие. Рассмотрим слова x_i из документа i :

$$\begin{aligned} p(x_i, y_i | \theta) &= p(y_i | \pi) \prod_{j=1}^D p(x_{ij} | y_i, \theta) = \\ &= \prod_{c=1}^C \pi_c^{[y_i=c]} \prod_{j=1}^D \prod_{c=1}^C p(x_{ij} | \theta_{jc})^{[y_i=c]}. \end{aligned} \quad (5.33)$$

Используя наивное байесовское предположение, можем вычислить целевую функцию логарифмического правдоподобия:

$$\begin{aligned} \log p(D | \theta) &= \log \prod_{i=1}^n p(x_i, y_i | \theta) = \sum_{i=1}^n \log p(x_i, y_i | \theta) = \\ &= \sum_{c=1}^C N_c \log \pi_c + \sum_{j=1}^D \sum_{c=1}^C \sum_{i: y_i=c} \log p(x_{ij} | \theta_{jc}). \end{aligned} \quad (5.34)$$

Обратите внимание, что мы имеем дело с условной оптимизационной задачей, поскольку сумма вероятностей меток классов должна быть равна единице: $\sum \pi_c = 1$. Мы можем решить оптимизационную задачу формулы 5.34 с использованием лагранжиана, включив ограничение в целевую функцию и приравняв градиент лагранжиана $L(\theta, \lambda)$ относительно параметров модели к нулю:

$$L(\theta, \lambda) = \log p(D | \theta) + \lambda \left(1 - \sum_c \pi_c \right). \quad (5.35)$$

Дифференцируя по π_c , получаем следующее:

$$\frac{d}{d\pi_c} L(\theta, \lambda) = \frac{d}{d\pi_c} \log p(D | \theta) - \lambda = N_c \frac{1}{\pi} - \lambda = 0. \quad (5.36)$$

Отсюда легко получить выражение π_c через λ : $\pi_c = (1/\lambda) N_c$. Чтобы его решить относительно λ , используем наше ограничение на суммирование в единицу:

$$\sum_c \pi_c = 1 \rightarrow \sum_c \frac{1}{\lambda} N_c = 1 \rightarrow \lambda = \sum_c N_c. \quad (5.37)$$

Подставляя λ обратно в выражение для π_c , получаем $\pi_c = N_c / \sum N_c = N_c / N_{tot}$. Можем аналогично вычислить оптимальные значения параметров θ_{jc} , приравняв градиент целевой функции по θ_{jc} нулю:

$$\begin{aligned} \frac{d}{d\theta_{jc}} \log p(D|\theta) &= \frac{d}{d\theta_{jc}} \sum_{i: y_i=c} \left[x_{ij} \log(\theta_{jc}) + \right. \\ &\quad \left. + (1-x_{ij}) \log(1-\theta_{jc}) \right] = 0 = \\ &= \sum_{i: y_i=c} \left[\frac{x_{ij}}{\theta_{jc}} - \frac{1-x_{ij}}{1-\theta_{jc}} \right] = 0 \\ &\rightarrow \frac{N_{jc}}{\theta_{jc}} = \frac{1}{1-\theta_{jc}} [N_c - N_{jc}] \rightarrow N_{jc} = N_c \theta_{jc}. \end{aligned} \quad (5.38)$$

В результате получается оптимальное значение оценки максимального правдоподобия (maximum likelihood estimate, MLE): $\theta_{jc} = N_{jc} / N_c$, где $N_c = \sum 1[y_i = c]$. Подобная формула в виде отношения числа классов должна быть интуитивно понятна. Обратите внимание, что несложно добавить сопряженное априорное бета-распределение для случайных величин Бернулли и сопряженное априорное распределение Дирихле для плотности распределения классов, чтобы сгладить значения MLE:

$$\begin{aligned} p(\pi|D) &= Dir(N_1 + \alpha_1, \dots, N_c + \alpha_c); \\ p(\theta_{jc}|D) &= Beta([N_c - N_{jc}] + \beta_0, N_{jc} + \beta_1). \end{aligned} \quad (5.39)$$

Здесь мы используем *сопряженное априорное распределение* для удобства вычислений, поскольку апостериорное и априорное распределения имеют одинаковый вид, что позволяет проводить обновления в закрытой форме.

Во время тестирования мы хотели бы предсказать метку класса y , учитывая обучающие данные D и выученные параметры модели. Применяя правило Байеса, получаем следующее:

$$\begin{aligned} p(y=c | x_{i,1}, \dots, x_{i,D}, D) &\propto p(y=c | D) p(x_{i,1}, \dots, x_{i,D} | y=c, D) = \\ &= p(y=c | D) \prod_{j=1}^D p(x_{ij} | y=c, D). \end{aligned} \quad (5.40)$$

Подставляя распределения $p(y = c | D)$ и $p(x_{ij} | y = c, D)$ и беря логарифм, получаем следующее:

$$\log p(y = c | x, D) \propto \log \hat{\pi}_c + \sum_{j=1}^D \left(1[x_{ij} = 1] \log \hat{\theta}_{jc} + 1[x_{ij} = 0] \log(1 - \hat{\theta}_{jc}) \right). \quad (5.41)$$

Здесь π_c и θ_{jc} — оценки MLE, полученные в ходе обучения. Наивный алгоритм Байеса кратко представлен в виде псевдокода на рис. 5.13.

Временная сложность MLE-прогнозирования во время обучения равна $O(ND)$, где N — количество обучающих документов, а D — размер словаря. Временная сложность во время тестирования составляет $O(TCD)$, где T — количество тестовых документов, C — количество классов, а D — размер словаря. Аналогично, пространственная сложность, то есть размер массивов, необходимых для хранения параметров модели, возрастает как $O(DC)$. Теперь мы полностью готовы к реализации наивного байесовского алгоритма Бернулли.

```

1: Обучение:
2:  $N_c = 0, N_{jc} = 0$ 
3: for  $i = 1, 2, \dots, n$  do
4:    $c = y_i$  //метка класса для  $i$ -го примера
5:    $N_c = N_c + 1$ 
6:   for  $j = 1, \dots, D$  do
7:     if  $x_{ij} = 1$  then
8:        $N_{jc} = N_{jc} + 1$ 
9:   конец for
10: конец for
11:  $\hat{\pi}_c = \frac{N_c}{N}, \hat{\theta}_{jc} = \frac{N_{jc}}{N}$ 
12: return  $\hat{\pi}_c, \hat{\theta}_c$ 
13: Тестирование (для одного тестового документа):
14: for  $c = 1, 2, \dots, C$  do
15:    $\log p[c] = \log \pi_c$ 
16:   for  $j = 1, 2, \dots, D$  do
17:     if  $x_j = 1$  then
18:        $\log p[c] += \log \hat{\theta}_{jc}$ 
19:     else
20:        $\log p[c] += \log(1 - \hat{\theta}_{jc})$ 
21:   конец for
22: конец for
23:  $c = \arg \max_c \log p[c]$ 
24: return  $c$ 

```

Рис. 5.13. Псевдокод наивного алгоритма Байеса

Листинг 5.4. Наивный байесовский алгоритм Бернулли

```

import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from time import time
from nltk.corpus import stopwords
from nltk.tokenize import RegexpTokenizer

from sklearn.metrics import accuracy_score
from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer

sns.set_style("whitegrid")
tokenizer = RegexpTokenizer(r'\w+')
stop_words = set(stopwords.words('english'))
stop_words.update(['s', 't', 'm', '1', '2'])

class naive_bayes:
    def __init__(self, K, D):
        self.K = K      ← Количество классов
        self.D = D      ← Размер словаря

        self.pi = np.ones(K)      ← Априорные вероятности классов
        self.theta = np.ones((self.D, self.K))      ← Параметры алгоритма

    def fit(self, X_train, y_train):

        num_docs = X_train.shape[0]
        for doc in range(num_docs):

            label = y_train[doc]
            self.pi[label] += 1

            for word in range(self.D):
                if (X_train[doc][word] > 0):
                    self.theta[word][label] += 1
                # конец if
            # конец for

        # нормализация  $\pi$  и  $\theta$ 
        self.pi = self.pi/np.sum(self.pi)
        self.theta = self.theta/np.sum(self.theta, axis=0)

    def predict(self, X_test):

        num_docs = X_test.shape[0]
        logp = np.zeros((num_docs, self.K))
        for doc in range(num_docs):
            for kk in range(self.K):
                logp[doc][kk] = np.log(self.pi[kk])
                for word in range(self.D):

```

```

        if (X_test[doc][word] > 0):
            logp[doc][kk] += np.log(self.theta[word][kk])
        else:
            logp[doc][kk] += np.log(1-self.theta[word][kk])
        # конец if
    # конец for
# конец for
return np.argmax(logp, axis=1)

if __name__ == "__main__":

    import nltk
    nltk.download('stopwords')

    # загрузка данных
    print("loading 20 newsgroups dataset...")
    tic = time()
    classes = ['sci.space', 'comp.graphics', 'rec.autos', 'rec.sport.hockey']
    dataset = fetch_20newsgroups(shuffle=True, random_state=0,
    ➤ remove=('headers', 'footers', 'quotes'), categories=classes)
    X_train, X_test, y_train, y_test = train_test_split(dataset.data,
    ➤ dataset.target, test_size=0.5, random_state=0)
    toc = time()
    print("elapsed time: %.4f sec" %(toc - tic))
    print("number of training docs: ", len(X_train))
    print("number of test docs: ", len(X_test))

    print("vectorizing input data...")
    cnt_vec = CountVectorizer(tokenizer=tokenizer.tokenize, analyzer='word',
    ➤ ngram_range=(1,1), max_df=0.8, min_df=2, max_features=1000,
    ➤ stop_words=stop_words)
    cnt_vec.fit(X_train)
    toc = time()
    print("elapsed time: %.2f sec" %(toc - tic))
    vocab = cnt_vec.vocabulary_
    idx2word = {val: key for (key, val) in vocab.items()}
    print("vocab size: ", len(vocab))

    X_train_vec = cnt_vec.transform(X_train).toarray()
    X_test_vec = cnt_vec.transform(X_test).toarray()

    print("naive bayes model MLE inference...")
    K = len(set(y_train)) # количество классов
    D = len(vocab) # размер словаря
    nb_clf = naive_bayes(K, D)
    nb_clf.fit(X_train_vec, y_train)

    print("naive bayes prediction...")
    y_pred = nb_clf.predict(X_test_vec)
    nb_clf_acc = accuracy_score(y_test, y_pred)
    print("test set accuracy: ", nb_clf_acc)

```

Как свидетельствует вывод на экран, мы достигаем доли верных результатов в 82 % на тестовом наборе данных из двадцати групп новостей.

5.6. Дерево решений (CART)

В этом разделе мы обсудим алгоритм решения задачи классификации и регрессии путем построения дерева решений (classification and regression tree, CART). Древоподобные алгоритмы разбивают входное пространство на области, параллельные осям, таким образом, что каждый лист представляет определенную область. Затем это разбиение может быть использовано либо для отнесения области «большинством голосов» (majority vote) к тому или иному классу, либо для решения задачи регрессии путем предсказания ожидаемого значения сегмента. Древоподобные модели хорошо поддаются интерпретации и дают представление о важности того или иного признака. Они основаны на жадном рекурсивном алгоритме, так как оптимальное разделение пространства является NP-полным.

В древоподобных моделях задача состоит в том, чтобы в процессе обучения построить такое двоичное дерево, которое оптимизирует целевую функцию и не приводит к недо- или переобучению. Ключевым фактором построения дерева решений является выбор признака и порогового значения, которые будут использоваться при классификации точек данных. Рассмотрим матрицу входных данных $X_{n \times d}$, содержащую n точек размерности (количество признаков) d . Нам требуется выбрать оптимальный признак и его пороговое значение, которые приводят к разделению данных с минимальными потерями (cost). Пусть $j \in \{1, \dots, d\}$ представляет собой измерение признаков, а $t \in \tau_j$ — пороговое значение для признака j , выбранное из всех возможных пороговых значений τ_j (построенных путем вычисления средних значений наших данных x_{ij}). Тогда мы хотели бы рассчитать следующее:

$$j^*, t^* = \arg \min_{j \in \{1, \dots, d\}} \min_{t \in \tau_j} \text{cost}(\{x_i, y_i : x_{ij} \leq t\}) + \text{cost}(\{x_i, y_i : x_{ij} > t\}). \quad (5.42)$$

Прежде чем мы перейдем к примеру, давайте рассмотрим возможные варианты оценки потерь, которые можно использовать для оптимизации дерева классификации. Нам нужно выбрать такую функцию потерь (cost function), которая бы определяла качество разделения данных. Желательно, чтобы конечные узлы были чистыми (то есть не только содержали данные одного класса, но и были при этом способны распространять вывод на тестовые данные). Другими словами, хотелось бы ограничить глубину дерева (чтобы предотвратить переобучение), в то же время минимизируя неоднородность. Одним из показателей неоднородности является индекс Джини:

$$\sum_{k=1}^K \pi_k (1 - \pi_k) = \sum_k \pi_k - \sum_k \pi_k^2 = 1 - \sum_k \pi_k^2. \quad (5.43)$$

Здесь π_k — это доля точек данной области, принадлежащих кластеру k .

$$\pi_k = \frac{1}{|D|} \sum_{i \in D} 1[y_i = k]. \quad (5.44)$$

Обратите внимание, что поскольку π_k — это вероятность того, что случайная точка в листовом узле принадлежит классу k , а $1 - \pi_k$ — вероятность ошибки, то индекс Джини представляет собой ожидаемую частоту ошибок. Если лист дерева чистый ($\pi_k = 1$), то индекс Джини равен нулю. Таким образом, наша задача — минимизация индекса Джини.

Альтернативой целевой функции является энтропия (см. следующую формулу):

$$H(\pi) = - \sum_{k=1}^K \pi_k \log \pi_k. \quad (5.45)$$

Энтропия измеряет степень неопределенности. Если мы уверены, что листовый узел чистый (то есть $\pi_k = 1$), то энтропия равна нулю. Таким образом, в алгоритме CART мы также хотим минимизировать энтропию.

Теперь давайте рассмотрим одномерный пример выбора оптимального признака для разбиения и его порогового значения. Пусть $X = [1.5, 1.7, 2.3, 2.7, 2.7]$ и метки класса $y = [1, 1, 2, 2, 3]$. Поскольку данные имеют только одну координату, то нашей задачей будет найти такое пороговое значение, при котором разбиение X минимизирует индекс Джини. Если мы выберем среднее между 1.7 и 2.3 в качестве порогового значения $t_1 = 2$ и вычислим получающийся в результате индекс Джини, то получим следующее выражение:

$$G = \frac{2}{5} G_{left} + \frac{3}{5} G_{right} = \frac{2}{5} \times 0 + \frac{3}{5} \times \left(1 - \frac{2^2}{3} - \frac{1^2}{3} \right) = 0,27. \quad (5.46)$$

Здесь G_{left} — это индекс Джини для $\{x_i, y_i : x_{ij} \leq 2\}$, равный нулю, поскольку обе метки классов равны 1 (чистый лист). G_{right} — индекс Джини для $\{x_i, y_i : x_{ij} > 2\}$, содержащий метки классов $y_{right} = [2, 2, 3]$.

Ключевым моментом алгоритма CART является нахождение оптимального признака и порогового значения, при которых потери (например, индекс Джини) минимальны. Во время обучения нам нужно будет последовательно перебрать все признаки и вычислить индекс Джини при всех возможных пороговых значениях для каждого признака. Но как нам вычислить τ_j , множество всех возможных пороговых значений признака j ? Мы можем отсортировать обучающие данные $X[:, j]$ за время $O(\log n)$ и перебрать все средние точки между каждыми двумя соседними точками данных. Далее нам нужно будет вычислить индекс Джини для каждого порогового значения согласно формуле 5.47. В ней m — это размер узла, а m_k — количество точек в узле, принадлежащих классу k :

$$G = 1 - \sum_{k=1}^K \pi_k^2 = 1 - \sum_{k=1}^K \left(\frac{m_k}{m} \right)^2. \quad (5.47)$$

Можно выполнить итерацию по отсортированным порогам τ_j за время $O(n)$, вычисляя на каждой итерации индекс Джини с использованием порогового значения. Для i -го порога мы получаем следующее:

$$\begin{aligned} G_i &= \frac{i}{m} G_i^{left} + \frac{m-i}{m} G_i^{right}; \\ G_i^{left} &= 1 - \sum_k \left(\frac{m_k^{left}}{i} \right)^2; \\ G_i^{right} &= 1 - \sum_k \left(\frac{m_k^{right}}{m-i} \right)^2. \end{aligned} \quad (5.48)$$

Найдя оптимальный признак и пороговое значение, рекурсивно разбиваем каждый узел до тех пор, пока не будет достигнута максимальная глубина. Построив дерево в процессе обучения, на этапе тестирования просто проходим по дереву от корня до листа, который хранит метку нашего класса. Алгоритм CART в виде псевдокода представлен на рис. 5.14.

```

1: class TreeNode(gini, num_samples, num_samples_class, class_label):
2:     self.gini = gini // индекс Джини
3:     self.num_samples = num_samples // размер узла
4:     self.num_samples_class = num_samples_class // число точек с меткой k
5:     self.class_label = class_label // предсказанная метка класса
6:     self.feature_idx = 0 // индекс признака для разбиения
7:     self.threshold = 0 // лучший порог для разбиения
8:     self.left = None // указатель на левое поддерево
9:     self.right = None // указатель на правое поддерево
10: function grow_tree(X_train, y_train, depth)
11:     class_label = majority_vote(y_train)
12:     gini = compute_gini(y_train)
13:     node = new TreeNode(gini, class_label)
14:     //разбиваем рекурсивно до достижения максимальной глубины
15:     if depth < max_depth
16:         idx, threshold = best_split(X_train, y_train)
17:         if idx is not None:
18:             indices_left = X_train[:,idx] < threshold
19:             node.feature_index = idx
20:             node.left = grow_tree(X_train[indices_left], y_train[indices_left], depth + 1)
21:             node.right = grow_tree(X_train[~indices_left], y_train[~indices_left], depth + 1)
22:     return node

```

Рис. 5.14. Псевдокод алгоритма дерева решений CART

Как можно видеть из определения класса `TreeNode`, он хранит предсказанную метку класса, идентификатор признака и наилучшее пороговое значение для

разбиения, указатели на левое и правое поддеревья, а также индекс Джини и размер узла. Мы можем рекурсивно «наращивать» дерево решений, вызывая функцию `grow_tree`, если глубина дерева меньше установленного ранее максимального значения. Сначала определяем метку класса «большинством голосов» и вычисляем индекс Джини для меток обучающих данных. Затем устанавливаем наилучшее разбиение, перебирая все признаки (`feature`) и все возможные пороговые значения (`threshold`). Как только мы приходим к наилучшей комбинации признака и порогового значения для разбиения, инициализируем левый и правый указатели текущего узла новыми объектами `TreeNode`, которые содержат данные меньшие и большие величины порога соответственно. Повторяем эту процедуру до тех пор, пока не достигаем максимальной глубины дерева. Теперь мы готовы к воплощению в коде алгоритма CART.

Листинг 5.5. Алгоритм дерева решений CART

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

class TreeNode():
    def __init__(self, gini, num_samples, num_samples_class, class_label):
        self.gini = gini          ← Индекс Джини
        self.num_samples = num_samples ← Размер узла
        self.num_samples_class = num_samples_class ← Количество точек узла с меткой k
        self.class_label = class_label ← Предсказанная метка класса
        self.feature_idx = 0      ← Индекс признака для разбиения
        self.treshhold = 0       ← Лучший порог для разбиения
        self.left = None         ← Указатель на левое поддерево
        self.right = None        ← Указатель на правое поддерево

class DecisionTreeClassifier():
    def __init__(self, max_depth = None):
        self.max_depth = max_depth

    def best_split(self, X_train, y_train):
        m = y_train.size
        if (m <= 1):
            return None, None

        mk = [np.sum(y_train == k) for k in range(self
            ⇒ .num_classes)] ← Количество точек класса k

        best_gini = 1.0 - sum((n / m) ** 2 for n in mk) ← Индекс Джини
        best_idx, best_thr = None, None                ← текущего узла

        # цикл по всем признакам
        for idx in range(self.num_features):
```

```

thresholds, classes = zip(*sorted(zip(X[:,
    ➡ idx], y)))  ← Сортировка данных по выбранному признаку

num_left = [0]*self.num_classes
num_right = mk.copy()

for i in range(1, m):  ← Цикл по всем возможным точкам разбиения

    k = classes[i-1]

    num_left[k] += 1
    num_right[k] -= 1

    gini_left = 1.0 - sum(
        (num_left[x] / i) ** 2 for x in range(self.num_classes)
    )

    gini_right = 1.0 - sum(
        (num_right[x] / (m - i)) ** 2 for x in
    ➡ range(self.num_classes)
    )

    gini = (i * gini_left + (m - i) * gini_right) / m

    if thresholds[i] == thresholds[i - 1]:
        continue

    if (gini < best_gini):
        best_gini = gini
        best_idx = idx
        best_thr = (thresholds[i] +
    ➡ thresholds[i - 1]) / 2  ← Средняя точка
    # конец if
# конец for
# конец for
return best_idx, best_thr

def gini(self, y_train):
    m = y_train.size
    return 1.0 - sum((np.sum(y_train == k) / m) ** 2 for k in
    ➡ range(self.num_classes))

def fit(self, X_train, y_train):
    self.num_classes = len(set(y_train))
    self.num_features = X_train.shape[1]
    self.tree = self.grow_tree(X_train, y_train)

def grow_tree(self, X_train, y_train, depth=0):

    num_samples_class = [np.sum(y_train == k) for k in
    ➡ range(self.num_classes)]
    class_label = np.argmax(num_samples_class)

    node = TreeNode(

```

```

        gini=self.gini(y_train),
        num_samples=y_train.size,
        num_samples_class=num_samples_class,
        class_label=class_label,
    )
    if depth < self.max_depth:
        idx, thr = self.best_split(X_train, y_train)
        if idx is not None:
            indices_left = X_train[:, idx] < thr
            X_left, y_left = X_train[indices_left], y_train[indices_left]
            X_right, y_right = X_train[~indices_left],
            y_train[~indices_left]
            node.feature_index = idx
            node.threshold = thr
            node.left = self.grow_tree(X_left, y_left, depth + 1)
            node.right = self.grow_tree(X_right, y_right, depth + 1)

    return node

def predict(self, X_test):
    return [self.predict_helper(x_test) for x_test in X_test]

def predict_helper(self, x_test):
    node = self.tree
    while node.left:
        if x_test[node.feature_index] < node.threshold:
            node = node.left
        else:
            node = node.right
    return node.class_label
if __name__ == "__main__":

    # загрузка данных
    iris = load_iris()
    X = iris.data[:, [2,3]]
    y = iris.target

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ➡ random_state=42)

    print("decision tree classifier...")
    tree_clf = DecisionTreeClassifier(max_depth = 3)
    tree_clf.fit(X_train, y_train)

    print("prediction...")
    y_pred = tree_clf.predict(X_test)

    tree_clf_acc = accuracy_score(y_test, y_pred)
    print("test set accuracy: ", tree_clf_acc)

```

Как свидетельствует вывод на экран, мы достигаем уровня верности тестовой классификации в 80 % на наборе данных цветков ириса.

5.7. Упражнения

- 5.1. Даны точка $y \in R^d$ и гиперплоскость $\theta \cdot x + \theta_0 = 0$. Вычислите евклидово расстояние от точки до гиперплоскости.
- 5.2. Дана прямая задача линейного программирования (ЛП): $\min c^T x$ при условиях, что $Ax \leq b, x \geq 0$. Сформулируйте двойственную задачу ЛП.
- 5.3. Покажите, что использование радиальной базисной функции в качестве ядра эквивалентно вычислению меры сходства между двумя бесконечномерными векторами признаков.
- 5.4. Убедитесь, что график изменения скорости обучения $\eta_k = (\tau_0 + k)^{-\kappa}$ удовлетворяет условиям Роббинса — Монро.
- 5.5. Найдите производную сигмоидной функции $\sigma(a) = [1 + \exp(-a)]^{-1}$.
- 5.6. Оцените временную и пространственную сложность наивного байесовского алгоритма Бернулли.

Итоги

- Цель алгоритма классификации состоит в том, чтобы научиться находить соответствие между входными данными x и выходными метками y , где y — дискретная величина.
- Перцептрон — алгоритм классификации, который обновляет границу принятия решения до тех пор, пока ошибки классификации не сойдут на нет.
- SVM — это классификатор максимального зазора. Точки обучающих данных, лежащие на границах зазора, становятся опорными векторами.
- Логистическая регрессия — алгоритм классификации, который вычисляет условную плотность вероятности классов, используя функцию softmax.
- Наивный алгоритм Байеса предполагает, что признаки являются условно независимыми при заданной метке класса. Он широко применяется для классификации документов.
- Дерево решений CART — жадный рекурсивный алгоритм, который находит оптимальное разбиение признаков путем минимизации целевой функции, такой как индекс Джини.

6

Алгоритмы регрессии

В этой главе

- ✓ Введение в регрессию
- ✓ Байесовская линейная регрессия
- ✓ Иерархическая байесовская регрессия
- ✓ Регрессия K ближайших соседей
- ✓ Регрессия на основе гауссовского процесса

В предыдущей главе мы рассматривали алгоритмы обучения с учителем для решения задачи классификации. В этой главе сосредоточимся на обучении с учителем, в ходе которого мы стараемся предсказать непрерывную величину. Мы изучим четыре алгоритма регрессии, которые выведем, исходя из базовых принципов: байесовская линейная регрессия, иерархическая байесовская регрессия, регрессия K ближайших соседей и регрессия на основе гауссовского процесса. Выбор этих алгоритмов обусловлен тем, что они охватывают несколько важных сфер применения и иллюстрируют разнообразные математические концепции. Алгоритмы регрессии применяются для решения самых разных задач. Например, они могут быть использованы для прогнозирования цен на финансовые активы или предсказания содержания углекислого газа в атмосфере. Давайте начнем с рассмотрения основных составляющих задачи регрессии.

6.1. Введение в регрессию

При обучении с учителем у нас имеется набор данных $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$, состоящий из кортежей признаков x и меток y . Цель алгоритма регрессии состоит в том, чтобы найти соответствие между входными данными x и выходными данными y , где y — непрерывная величина (то есть $y \in R$).

Регрессор f можно рассматривать как отображение между d -мерным вектором признаков $\phi(x)$ и меткой y (а именно $f: R^d \rightarrow R$). Задачи регрессии обычно сложнее (с точки зрения достижения высокой доли верных результатов) в сравнении с задачами классификации, поскольку мы стремимся предсказать *непрерывную* величину. Более того, нас часто интересует прогнозирование *будущего* значения зависимой переменной y на основе обучающих данных из прошлого.

Одной из наиболее широко используемых моделей является *линейная регрессия*, которая моделирует зависимую переменную y как линейную комбинацию входных векторов признаков $\phi(x)$:

$$y(x) = \underbrace{w^T}_{\substack{\text{Коэффициенты} \\ \text{регрессии}}} \underbrace{\phi(x)}_{\substack{\text{Признаки}}} + \underbrace{\varepsilon}_{\substack{\text{Гауссов} \\ \text{шум}}} = \sum_{d=1}^D w_d \phi(x_d) + \varepsilon. \quad (6.1)$$

Здесь ε — остаточная ошибка, то есть разница между нашими линейными прогнозами и истинным значением. Мы можем оценить качество нашего регрессора при помощи среднеквадратичной ошибки (MSE):

$$MSE(w) = E \left[\underbrace{\left(\underbrace{y - f(x; w)}_{\substack{\text{Предсказание} \\ \text{Истинное} \\ \text{значение}}} \right)^2} \right] = \frac{1}{N} \sum_{i=1}^N \left(y_i - w^T \phi(x_i) \right)^2. \quad (6.2)$$

В этой главе мы рассмотрим несколько важных регрессионных моделей, начиная с байесовской и регрессии K ближайших соседей, их расширения в виде иерархических регрессионных моделей, и заканчивая регрессией на основе гауссовского процесса (Gaussian process, GP). Мы охватим как вопросы теории, так и реализации каждой модели собственными силами.

6.2. Байесовская линейная регрессия

Напомним, что линейную регрессию можно записать в виде $y(x) = w^T x + \varepsilon$. Если допустить, что $\varepsilon \sim N(0, \sigma^2)$ — это гауссова СВ с нулевым средним значением

и дисперсией σ^2 , то задачу линейной регрессии можно сформулировать следующим образом:

$$p(y | x, \theta) = N(y | w^T x, \sigma^2). \quad (6.3)$$

Здесь w — коэффициенты регрессии. Чтобы подогнать модель линейной регрессии к данным, мы минимизируем отрицательную логарифмическую функцию правдоподобия (NLL):

$$\begin{aligned} NLL(w, \sigma^2) &= -\log p(y | x, \theta) \leftarrow \text{Определение NLL} \\ &= -\log \prod_{n=1}^N \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[-\frac{1}{2\sigma^2} (y_n - w^T x)^2 \right] \right] = \\ &= -\sum_{n=1}^N \log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) + \sum_{n=1}^N \frac{1}{\sqrt{2\sigma^2}} (y_n - w^T x)^2 = \\ &= \frac{1}{\sqrt{2\sigma^2}} \sum_{n=1}^N (y_n - w^T x)^2 + \frac{N}{2} \log(2\pi\sigma^2). \end{aligned} \quad (6.4)$$

Приняв σ^2 за константу и дифференцируя по w , мы получаем следующее уравнение:

$$2X^T X w - 2X^T y + 2\lambda w = 0. \quad (6.5)$$

Отталкиваясь от формулы 6.5, получаем следующее:

$$\hat{w} = (X^T X + \lambda I)^{-1} X^T y. \quad (6.6)$$

Оценка из формулы 6.6 может приводить к переобучению, что является одной из проблем данного подхода. Чтобы сделать байесовскую линейную регрессию устойчивой к переобучению, можно способствовать тому, чтобы параметры были малы, установив в качестве априорного распределения нормальное с нулевым средним:

$$p(w) = \prod_d N(w_d | 0, \tau^2). \quad (6.7)$$

Таким образом, можем переписать нашу регуляризованную целевую функцию следующим образом:

$$\min_w NLL(w, \sigma^2) + \lambda \|w\|_2^2. \quad (6.8)$$

Снова находя решение относительно w , получаем следующие коэффициенты:

$$\hat{w}_{ridge} = (X^T X + \lambda I)^{-1} X^T y. \quad (6.9)$$

Мы можем обучить параметры модели w , используя градиентный спуск. Рассмотрим псевдокод на рис. 6.1.

```

1: class ridge_reg:
2:   function fit(X, y)
3:   for i = 1, 2, ..., num_iter
4:      $\hat{y} = w^T X$ 
5:      $grad = -(y - \hat{y})^T X + \lambda w$ 
6:      $w = w - \eta_i grad \leftarrow$  Обновление коэффициентов регрессии
7:   конец for
8:   return w
9:   function predict(w, X)
10:   $\hat{y} = w^T X \leftarrow$  Предсказание
11:  return  $\hat{y}$ 

```

Рис. 6.1. Псевдокод байесовской линейной регрессии

Класс `ridge_reg` содержит функции `fit` и `predict`. В функции `fit` мы вычисляем градиент целевой функции по w и обновляем коэффициенты регрессии (regression weights). В функции `predict` используем эти коэффициенты, чтобы сделать предсказание для тестовых данных.

Листинг 6.1. Байесовская линейная регрессия

```

import math
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing

class ridge_reg():

    def __init__(self, n_iter=20, learning_rate=1e-3, lambda=0.1):
        self.n_iter = n_iter
        self.learning_rate = learning_rate
        self.lambda = lambda

    def fit(self, X, y):
        X = np.insert(X, 0, 1, axis=1)  ← Вставка 1 для свободного члена

        self.loss = []
        self.w = np.random.rand(X.shape[1])

        for i in range(self.n_iter):
            y_pred = X.dot(self.w)
            mse = np.mean(0.5*(y - y_pred)**2 +
                ⇒ 0.5*self.lambda*self.w.T.dot(self.w))
            self.loss.append(mse)
            print("%d iter, mse: %.4f" %(i, mse))
            grad_w = - (y - y_pred).dot(X) + self
                ⇒ .lambda*self.w  ← Вычисление градиента NLL(w) по w
            self.w -= self.learning_rate * grad_w  ← Обновление коэффициентов

    def predict(self, X):

        X = np.insert(X, 0, 1, axis=1)  ← Вставка 1 для свободного члена

```

```

y_pred = X.dot(self.w)
return y_pred

if __name__ == "__main__":
    X, y = fetch_california_housing(return_X_y=True)
    X_reg = X[:,2].reshape(-1,1)
    X_std = (X_reg - X_reg.mean())/X.std()
    y_std = (y - y.mean())/y.std()
    X_std = X_std[:200,:]
    y_std = y_std[:200]

    rr = ridge_reg()
    rr.fit(X_std, y_std)
    y_pred = rr.predict(X_std)

    print(rr.w)
    plt.figure()
    plt.plot(rr.loss)
    plt.xlabel('Эпоха')
    plt.ylabel('Потери')
    plt.tight_layout()
    plt.show()

    plt.figure()
    plt.scatter(X_std, y_std)
    plt.plot(np.linspace(-1,1), rr.w[1]*np.linspace(-1,1)+rr.w[0], c='red')
    plt.xlim([-0.01,0.01])
    plt.xlabel("Масштабированное количество комнат")
    plt.ylabel("Масштабированная цена на жилье")
    plt.show()

```

Среднее количество
комнат

Стандартизация

На рис. 6.2 показаны результаты выполнения алгоритма байесовской линейной регрессии.

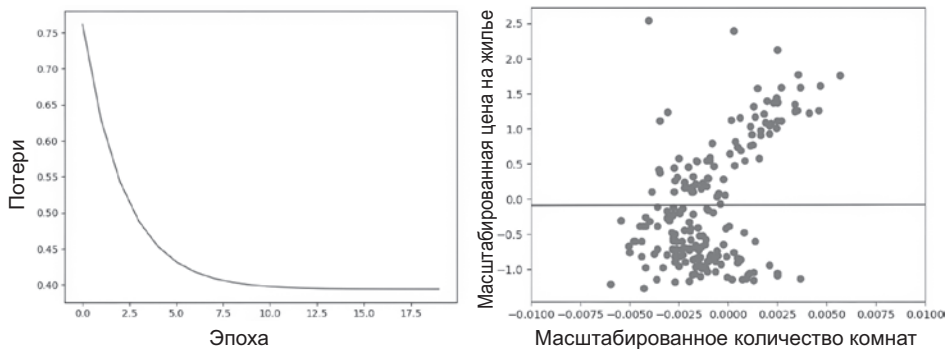


Рис. 6.2. Байесовская линейная регрессия: функция потерь (слева) и график (справа)

В левой части мы видим уменьшение функции потерь с течением времени, а в правой — соответствие прогнозу набора данных о ценах на жилье

в Калифорнии. Обратите внимание, что обе оси стандартизованы. Байесовская линейная регрессия успешно отражает тенденцию роста цен в зависимости от среднего количества комнат. В следующем разделе мы рассмотрим преимущества иерархической модели линейной регрессии.

6.3. Иерархическая байесовская регрессия

Иерархические модели обеспечивают совместное использование признаков несколькими группами. Сходство между группами моделируется при помощи предположительно общего распределения параметров модели. На рис. 6.3 изображены три сценария, иллюстрирующие преимущества иерархического моделирования. На диаграмме слева имеется единый набор параметров θ , моделирующих всю последовательность наблюдений. Такой сценарий называется *объединенной моделью* (pooled model). Здесь любые вариации в данных не описываются моделью в явном виде, поскольку мы исходим из предположения, что общий набор параметров описывает данные. В то же время у нас имеется пример использования *раздельных моделей* (unpooled model), в котором мы моделируем свой набор параметров для каждого наблюдения. В этом случае предполагаем, что разные группы наблюдений имеют разные параметры и все параметры независимы. Иерархическая модель сочетает в себе достоинства обоих подходов: она предполагает наличие общего распределения, которое моделирует отдельные параметры и, следовательно, отражает сходство между группами.

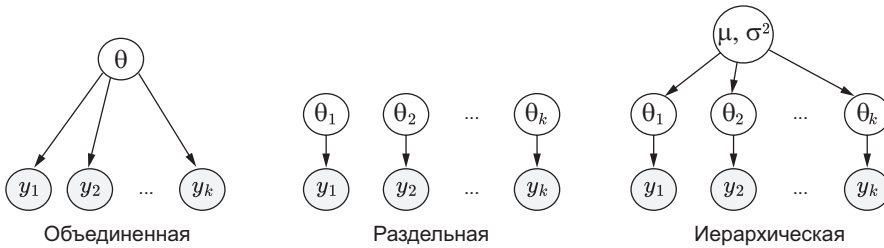


Рис. 6.3. Графовые модели: объединенная, раздельная, иерархическая

В байесовской иерархической регрессии мы можем задать априорные распределения для параметров модели и использовать МСМС-семплирование для прогнозирования апостериорных распределений. Воспользуемся набором данных о радиоактивном газе радоне для регрессии уровней содержания радона в домах разных округов (county) штата Миннесота в зависимости от номера этажа (и, в частности, от того, имеется ли у здания подвал или нет). Таким образом, наша регрессионная модель описывается следующим образом:

$$\begin{aligned} \alpha_c &\sim N(\mu_\alpha, \sigma_\alpha^2); \\ \beta_c &\sim N(\mu_\beta, \sigma_\beta^2); \\ \text{radon} &= \alpha_c + \beta_c \times \text{floor}_{i,c} + \varepsilon_c. \end{aligned} \tag{6.10}$$

Обратите внимание, что индекс s указывает на округ. Это означает, что для каждого округа находим точку пересечения и наклон, подчиняющиеся общему гауссовскому распределению. То есть мы используем иерархическую модель, в которой наши параметры (α_c и β_c) семплируются из общих распределений. В коде из листинга 6.2 мы задаем распределения вероятностей для коэффициентов регрессии и моделируем правдоподобие данных с помощью нормального распределения с единообразным стандартным отклонением. Сконфигурировав графовую модель с помощью библиотеки PyMC, можем провести вариационный вывод, используя семплер без разворотов (NUTS). PyMC — это библиотека вероятностного программирования на Python, которую можно загрузить по адресу <https://docs.pymc.io/>. Это отличное средство для байесовского моделирования, и его можно рассматривать как способ решения задачи с нуля, поскольку мы вначале определяем вероятностную графовую модель, а затем используем имеющиеся инструменты для семплирования апостериорного распределения. Если вы впервые сталкиваетесь с вероятностными языками программирования, я настоятельно рекомендую ознакомиться с имеющимися в Сети примерами использования библиотеки PyMC, чтобы узнать больше о ее возможностях.

Обратимся к псевдокоду на рис. 6.4.

```

1: function main(X, y):
2:   with pymc3.Model() as hierarchical_model:
3:      $\mu_a \sim N(0, 100^2)$ 
4:      $\sigma_a \sim \text{Unif}[0, 100]$ 
5:      $\mu_b \sim N(0, 100^2)$ 
6:      $\sigma_b \sim \text{Unif}[0, 100]$ 
7:      $a \sim N(\mu_a, \sigma_a^2)$ 
8:      $b \sim N(\mu_b, \sigma_b^2)$ 
9:      $\varepsilon \sim \text{Unif}[0, 100]$ 
10:     $y_{exp} = a + b \times X$ 
11:     $y_{lh} \sim N(X; y_{exp}, \varepsilon^2)$ 
12:  with hierarchical_model:
13:    mu, sds, elbo = pymc3.variational.advi(n = 100000)
14:    step = pymc3.NUTS(scaling = sds2, is_cov = True)
15:    trace = pymc3.sample(5000, step, start = mu)
16:  return trace

```

→ Гиперприоры
 ← Модель точки пересечения
 ← Модель угла наклона
 ← Модель ошибки
 ← Ожидаемое значение
 ← Правдоподобие данных
 → Вывод PyMC

Рис. 6.4. Псевдокод иерархической байесовской регрессии

Код состоит из единственной функции `main`. В первой части кода определяем вероятностную модель, а во второй используем библиотеку PyMC3 для вариационного вывода. Сначала мы определяем гиперприорные распределения среднего значения и дисперсии для моделей точки пересечения и наклона линии регрессии. Далее задаем модель точки пересечения и наклона как гауссову СВ, а модель ошибки — как СВ, имеющую равномерное распределение. Наконец,

вычисляем формулу регрессии и используем результат в качестве среднего значения в модели правдоподобия данных. Затем переходим к процедуре вывода с использованием NUTS, реализованной в PyMC.

Листинг 6.2. Иерархическая байесовская регрессия

```
import numpy as np
import pandas as pd

import seaborn as sns
import matplotlib.pyplot as plt

import pymc3 as pm

def main():

    data = pd.read_csv('./data/radon.txt')  ← Загрузка данных

    county_names = data.county.unique()
    county_idx = data['county_code'].values

    with pm.Model() as hierarchical_model:

        mu_a = pm.Normal('mu_alpha', mu=0., sd=100**2)
        sigma_a = pm.Uniform('sigma_alpha', lower=0,
                               ↳ upper=100)
        mu_b = pm.Normal('mu_beta', mu=0., sd=100**2)
        sigma_b = pm.Uniform('sigma_beta', lower=0,
                               ↳ upper=100)
        # Гиперприоры

        a = pm.Normal('alpha', mu=mu_a, sd=sigma_a,
                               ↳ shape=len(data.county.unique()))  ← Точка пересечения для каждого округа
        b = pm.Normal('beta', mu=mu_b, sd=sigma_b,
                               ↳ shape=len(data.county.unique()))  ← Угол наклона для каждого округа

        eps = pm.Uniform('eps', lower=0, upper=100)  ← Ошибка модели

        radon_est = a[county_idx] + b[county_idx] *
        ↳ data.floor.values  ← Ожидаемое значение

        y_like = pm.Normal('y_like', mu=radon_est, sd=eps,
                               ↳ observed=data.log_radon)  ← Правдоподобие данных

    with hierarchical_model:
        # Инициализация с помощью ADVI
        mu, sds, elbo = pm.variational.advi(n=100000)
        step = pm.NUTS(scoring=hierarchical_model.dict_to_array(sds)**2,
                               ↳ is_cov=True)
        hierarchical_trace = pm.sample(5000, step, start=mu)

    pm.traceplot(hierarchical_trace[500:])
    plt.show()

if __name__ == "__main__":
    main()
```

Из графиков трассировки на рис. 6.5 следует сходимость апостериорных распределений для α_c и β_c , что указывает на различие в значениях пересечения и угла наклона для разных округов.

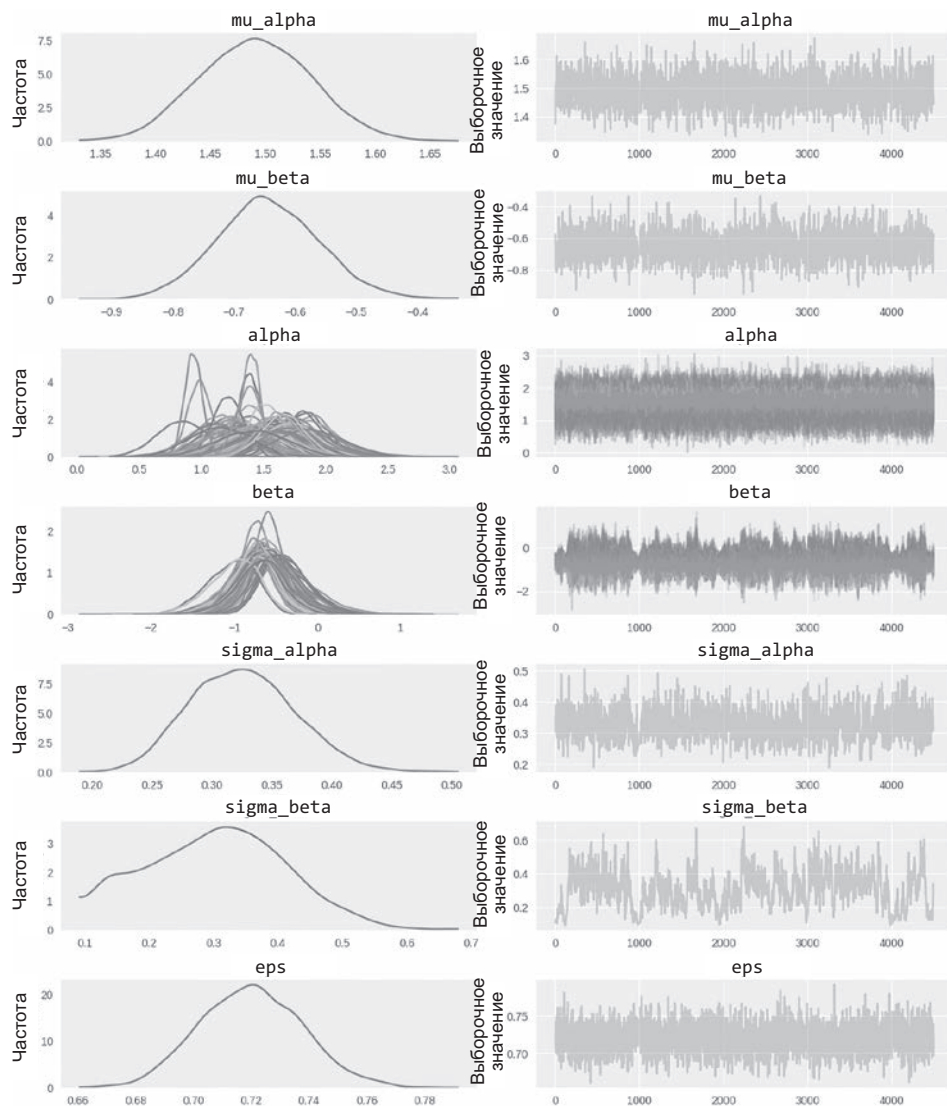


Рис. 6.5. Диаграммы трассировки МСМС для иерархической байесовской регрессии

Кроме того, мы также восстанавливаем апостериорное распределение общих параметров. Из графика μ_a видно, что групповое среднее логарифма уровня

радона близко к 1.5. Диаграмма μ_b , в свою очередь, показывает, что наклон отрицательный, со средним значением -0.65 , и, следовательно, отсутствие подвала снижает уровень радона. В следующем разделе мы рассмотрим алгоритм, подходящий для работы с нелинейными данными.

6.4. Регрессия К ближайших соседей

Регрессия К ближайших соседей (K nearest neighbors, KNN) — это пример непараметрической модели, в которой для запрашиваемой точки данных q ищутся k ее ближайших соседей в обучающем наборе и вычисляется среднее значение зависимой переменной y . В этом разделе мы будем вычислять среднее значение целевых KNN-меток для набора данных цветков ириса. Среднее значение берется по локальной окрестности K точек, ближайших к нашему запросу q :

$$y_q = \frac{1}{K} \sum_{i \in N_K(q, D)} y_i. \quad (6.11)$$

Здесь $N_K(q, D)$ обозначает локальную окрестность K ближайших соседей из обучающего набора данных D для запроса q . Чтобы найти локальную окрестность $N_K(q, D)$, мы можем вычислить расстояние между точкой q и каждой из точек обучающего набора $x_i \in D$, отсортировать эти значения в порядке возрастания и взять первые K точек данных. Из-за применения операции сортировки временная сложность такого подхода равна $O(n \log n)$, где n — размер обучающего набора данных. Теперь мы готовы приступить к написанию кода KNN-регрессора с нуля. В псевдокоде на рис. 6.6 результат KNN-регрессии вычисляется путем усреднения меток K ближайших соседей с использованием евклидова расстояния:

```

1: class KNN:
2:   function knn_search(K, X, y, Q)
3:   for query in Q:
4:     idx = argsort(euclidian_dist(query, X))[:K] ← KNN-индексы
5:     knn_labels = [y[i] for i in idx] ← KNN-метки
6:     y_pred = mean(knn_labels) ← KNN-регрессия
7:   конец for
8:   return y_pred

```

Рис. 6.6. Псевдокод регрессии К ближайших соседей

В функции `knn_search` для каждого запроса Q мы вычисляем евклидово расстояние между точкой запроса и всеми точками данных, сортируем результаты и выбираем индексы K точек с кратчайшим расстоянием. Затем формируем KNN-область, группируя метки с KNN-индексами. В конце вычисляем результат путем усреднения по KNN-меткам:

Листинг 6.3. Регрессия К ближайших соседей

```

import numpy as np
import matplotlib.pyplot as plt

from sklearn import datasets
from sklearn.model_selection import train_test_split

np.random.seed(42)

class KNN():

    def __init__(self, K):
        self.K = K

    def euclidean_distance(self, x1, x2):
        dist = 0
        for i in range(len(x1)):
            dist += np.power((x1[i] - x2[i]), 2)
        return np.sqrt(dist)

    def knn_search(self, X_train, y_train, Q):
        y_pred = np.empty(Q.shape[0])

        for i, query in enumerate(Q):
            idx = np.argsort([self.euclidean_distance(
                ➡ query, x) for x in X_train])[ :self.K]
            knn_labels = y_train[idx]
            y_pred[i] = np.mean(knn_labels)

        return y_pred

        # Получение K ближайших к точке запроса соседей
        # Извлечение KNN-меток обучающих данных
        # Вычисление среднего обучающих KNN-меток

if __name__ == "__main__":

    plt.close('all')

    # набор данных цветков ириса
    iris = datasets.load_iris()
    X = iris.data[:, :2]
    y = iris.target

    X_train, X_test, y_train, y_test = train_test_split(X, y,
        ➡ test_size=0.2, random_state=42)

    K = 4
    knn = KNN(K)
    y_pred = knn.knn_search(X_train, y_train, X_test)

    plt.figure(1)
    plt.scatter(X_train[:, 0], X_train[:, 1], s = 100, marker = 'x', color =
        ➡ 'r', label = 'data')
    plt.scatter(X_test[:, 0], X_test[:, 1], s = 100, marker = 'o', color =
        ➡ 'b', label = 'query')
    plt.title('K Nearest Neighbors (K=%d)' % K)
    plt.legend()

```

```
plt.xlabel('X1')
plt.ylabel('X2')
plt.grid(True)
plt.show()
```

Точное нахождение ближайших соседей в многомерном пространстве часто является трудноразрешимой задачей с вычислительной точки зрения, и потому существуют приближенные методы. Существует два класса таких методов. Одни разбивают пространство на области, например, при помощи k-d-деревьев, как это реализовано в библиотеке для быстрого поиска ближайших соседей FLANN (fast library for approximate nearest neighbors). Другие используют хеширование, например хеширование с учетом местоположения (locality-sensitive hashing, LSH). В следующем разделе мы обсудим еще один тип функциональной регрессии.

6.5. Регрессия на основе гауссовского процесса

Гауссовские процессы (ГП) задают априорное распределение на множестве функций, которое после поступления наблюдаемых данных может быть обновлено до апостериорного — см. книгу К. Э. Расмуссена (C. E. Rasmussen) и К. К. И. Уильямса (C. K. I. Williams) «Gaussian Processes for Machine Learning» (The MIT Press, 2006). В условиях обучения с учителем подобная функция является отображением между точками данных x_i и зависимой переменной y_i : $y_i = f(x_i)$. Гауссовские процессы позволяют рассчитать распределение по функциям при заданных данных $p(f|x, y)$, а затем с его использованием делать предсказания на новых данных. В ГП предполагается, что функция определена на конечном и произвольно выбранном множестве точек $x_1, \dots, x_n$, таком что $p(f(x_1), \dots, f(x_n))$ является совместным гауссовым распределением со средним значением $\mu(x)$ и ковариацией $\Sigma(x)$, где $\Sigma_{ij} = \kappa(x_i, x_j)$, и κ — положительно определенная функция ядра. Одним из примеров задачи, которую можно решить с помощью ГП-регрессии, является прогнозирование уровня углекислого газа на основе результатов измерений. В коде из листинга 6.4 мы используем синусоидальную модель и радиальную базисную функцию в качестве ядра.

Рассмотрим простую регрессионную задачу:

$$f(x) = x^T w; \quad y = f(x) + \varepsilon; \quad \varepsilon \sim N(0, \sigma_n^2). \quad (6.12)$$

В предположении, что шум описывается независимыми и одинаково распределенными СВ, можно записать следующую функцию правдоподобия:

$$p(y|X, w) = \prod_{i=1}^n p(y_i | x_i, w) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma_n^2}} \exp\left\{-\frac{(y_i - x_i^T w)^2}{2\sigma_n^2}\right\} \sim N(Xw, \sigma_n^2 I). \quad (6.13)$$

В байесовском подходе нужно задать априорное распределение для параметров: $w \sim N(0, \Sigma_p)$. Оставляя только зависящие от коэффициентов регрессии слагаемые для правдоподобия и априорного распределения, получаем следующую формулу:

$$\begin{aligned} p(w | X, y) &\propto \exp\left\{-\frac{1}{2\sigma_n^2}\|y - Xw\|^2\right\} \exp\left\{-\frac{1}{2}w^T \Sigma_p^{-1} w\right\} \propto \\ &\propto \exp\left\{-\frac{1}{2}(w - \bar{w})^T \left(\frac{1}{\sigma_n^2} XX^T + \Sigma_p^{-1}\right) (w - \bar{w})\right\} \sim \\ &\sim N\left(\frac{1}{\sigma_n^2} A^{-1} Xy, A^{-1}\right). \end{aligned} \quad (6.14)$$

Здесь мы предполагаем следующее:

$$A = \sigma_n^{-2} XX^T + \Sigma_p^{-1}. \quad (6.15)$$

Таким образом, имеем апостериорное распределение замкнутой формы по параметрам w . Чтобы генерировать предсказания с использованием этой формулы, нужно инвертировать матрицу A размером $p \times p$.

Исходя из предположения, что наблюдения не содержат шума, требуется предсказать выходные данные функции $y^* = f(x^*)$, где x^* — наши тестовые данные. Рассмотрим следующее совместное ГП-распределение:

$$\begin{pmatrix} f \\ f_* \end{pmatrix} \sim N\left(\begin{pmatrix} \mu \\ \mu_* \end{pmatrix}, \begin{pmatrix} K & K_* \\ K_*^T & K_{**} \end{pmatrix}\right). \quad (6.16)$$

Здесь $K = \kappa(X, X)$, $K^* = \kappa(X, X^*)$ и $K^{**} = \kappa(X^*, X^*)$, где X — обучающий набор данных, X^* — тестовый набор данных, а κ — ядро или функция ковариации. Используя стандартные правила для условных гауссовых распределений, приводим апостериорное распределение к следующему виду:

$$\begin{aligned} p(f_* | X_*, X, f) &\sim N(f_* | \mu_*, \Sigma_*); \\ \mu_* &= \mu(X_*) + K_*^T K^{-1} (f - \mu(X)); \\ \Sigma_* &= K_{**} - K_*^T K^{-1} K_*. \end{aligned} \quad (6.17)$$

В коде листинга 6.4 в качестве меры сходства используем ядро радиальной базисной функции, заданное по формуле 6.18. Теперь мы готовы приступить к реализации ГП-регрессии в коде (рис. 6.7).

Код содержит две основные функции: `kernel_func` и `compute_posterior`. Функция `kernel_func` возвращает значение ядерной радиальной базисной функции (*RBF*), которая измеряет сходство между двумя точками данных x и z . В функции `compute_posterior` мы сначала считаем необходимые составляющие, а затем вычисляем, как показано в тексте выше, и возвращаем апостериорное среднее и ковариацию.

```

1: class GPreg:
2:     function kernel_func(x, z)
3:         Kfn = exp  $\left\{-\frac{1}{2\sigma^2} \|x - z\|^2\right\}$  ← Ядро радиальной базисной функции
4:     return Kfn
5:     function compute_posterior(X)
6:         K = kernel_func(X_train, X_train)
7:         Ks = kernel_func(X_train, X_test)
8:         Kss = kernel_func(X_test, X_test)
9:          $\mu_{post} = \mu(X_{test}) + K_s^T K^{-1} (f - \mu(X_{train}))$ 
10:         $\Sigma_{post} = K_{ss} - K_s^T K^{-1} K_s$  → Апостериорное распределение гауссовского процесса
11:    return  $\mu_{post}, \Sigma_{post}$ 

```

Рис. 6.7. Псевдокод регрессии на основе гауссовского процесса**Листинг 6.4. Регрессия на основе гауссовского процесса**

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.spatial.distance import cdist

np.random.seed(42)

class GPreg:

    def __init__(self, X_train, y_train, X_test):

        self.L = 1.0
        self.keps = 1e-8

        self.muFn = self.mean_func(X_test)
        self.Kfn = self.kernel_func(X_test, X_test) + 1e-15*np.eye(
            ➡ np.size(X_test))

        self.X_train = X_train
        self.y_train = y_train
        self.X_test = X_test

    def mean_func(self, x):
        muFn = np.zeros(len(x)).reshape(-1,1)
        return muFn

    def kernel_func(self, x, z):
        sq_dist = cdist(x/self.L, z/self.L, 'euclidean')**2
        Kfn = 1.0 * np.exp(-sq_dist/2)
        return Kfn

    def compute_posterior(self):
        K = self.kernel_func(self.X_train, self.X_train)
        Ks = self.kernel_func(self.X_train, self.X_test)
        Kss = self.kernel_func(self.X_test, self.X_test) + self
            ➡ .keps*np.eye(np.size(self.X_test))
        Ki = np.linalg.inv(K) ← O(N_train^3)

```

```

postMu = self.mean_func(self.X_test) + np.dot(np.transpose(Ks),
⇒ np.dot(Ki, (self.y_train - self.mean_func(self.X_train))))
postCov = Kss - np.dot(np.transpose(Ks), np.dot(Ki, Ks))

self.muFn = postMu
self.Kfn = postCov

return None

def generate_plots(self, X, num_samples=3):
    plt.figure()
    for i in range(num_samples):
        fs = self.gauss_sample(1)
        plt.plot(X, fs, '-k')
        #plt.plot(self.X_train, self.y_train, 'xk')

    mu = self.muFn.ravel()
    S2 = np.diag(self.Kfn)
    plt.fill(np.concatenate([X, X[:-1]]), np.concatenate([mu -
⇒ 2*np.sqrt(S2), (mu + 2*np.sqrt(S2))[:-1]]), alpha=0.2, fc='b')
    plt.show()

def gauss_sample(self, n):
    A = np.linalg.cholesky(self.Kfn)
    Z = np.random.normal(loc=0, scale=1, size=(len(self.muFn),n))
    S = np.dot(A,Z) + self.muFn
    return S
    Возвращает выборку из многомерного
    гауссова распределения размера n

def main():
    # генерация обучающих данных без шума
    X_train = np.array([-4, -3, -2, -1, 1])
    X_train = X_train.reshape(-1,1)
    y_train = np.sin(X_train)

    # генерация тестовых данных
    X_test = np.linspace(-5, 5, 50)
    X_test = X_test.reshape(-1,1)

    gp = GPreg(X_train, y_train, X_test)
    gp.generate_plots(X_test,3)
    gp.compute_posterior()
    gp.generate_plots(X_test,3)

if __name__ == "__main__":
    main()

```

Выборка из априорного ГП-распределения

Выборка из апостериорного ГП-распределения

На рис. 6.8 показаны три функции, выбранные случайным образом из априорного (слева) и апостериорного (справа) ГП-распределений. Последнее получено в результате наблюдения пяти незашумленных точек данных. Заштрихованная область соответствует двукратному стандартному отклонению вокруг среднего значения (доверительная область 95 %). Видно, что модель

идеально интерполирует обучающие данные, а также что прогностическая неопределенность возрастает по мере того, как мы удаляемся от данных наблюдения.

Поскольку наш алгоритм определен в терминах внутренних произведений в пространстве входных данных, то его можно перенести в пространство признаков путем замены внутренних произведений на $k(x, x')$. Этот прием часто называют *трюком с ядром* (kernel trick):

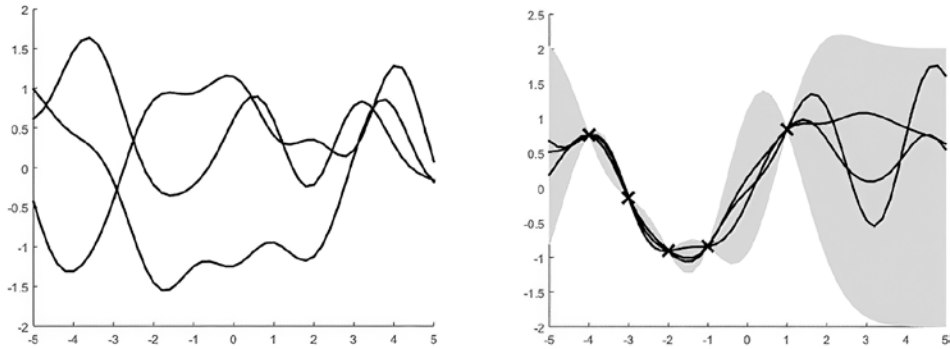


Рис. 6.8. Регрессия на основе гауссовского процесса: выборка из априорного (слева) и апостериорного (справа) распределений

Ядро измеряет сходство между объектами и не требует их предварительного преобразования в формат векторов признаков. Одной из широко распространенных функций ядра является *радиальная базисная функция*:

$$k(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right). \quad (6.18)$$

При использовании гауссова ядра карта признаков существует в бесконечно-мерном пространстве. В этом случае задача представить векторы признаков в явном виде становится неосуществимой.

Алгоритмы регрессии помогают предсказывать непрерывные величины. Исходя из характера данных (например, линейной или нелинейной зависимости между переменными), мы можем выбрать либо линейный алгоритм, такой как байесовская линейная регрессия, либо нелинейную KNN-регрессию. Нам также может понадобиться иерархическая модель, где некоторые признаки являются общими для всей популяции. Кроме того, в случае необходимости предсказать функциональную взаимосвязь между переменными в дело вступает регрессия на основе гауссовского процесса. В следующей главе мы обсудим более продвинутые алгоритмы обучения с учителем.

6.6. Упражнения

- 6.1. Оцените временную и пространственную сложность KNN-регрессора.
- 6.2. Выведите формулы обновления гауссовского процесса на основе правил для условного распределения многомерных нормальных случайных величин.

Итоги

- Цель алгоритма регрессии состоит в том, чтобы определить соответствие между входными данными x и выходной переменной y , где y — непрерывная величина.
- В байесовской линейной регрессии, определяемой как $y(x) = w^T x + \varepsilon$, мы предполагаем, что шумовая составляющая является гауссовой случайной величиной с нулевым средним значением.
- Иерархические модели позволяют совместно использовать признаки несколькими группами. Иерархическая модель предполагает наличие общего распределения, которое описывает индивидуальные параметры, и, следовательно, отражает сходство между группами.
- KNN-регрессия — это непараметрическая модель, в которой для заданной точки запроса q ищется K ее ближайших соседей в обучающем наборе данных и вычисляется среднее значение выходной переменной y .
- Гауссовские процессы (ГП) задают априорное распределение функций, которое может быть обновлено до апостериорного после получения данных наблюдения. В ГП предполагается, что функция определена на конечном и произвольно выбранном множестве точек $x_1, \dots, x_n$, так что $p(f(x_1), \dots, f(x_n))$ является совместным нормальным распределением со средним значением $\mu(x)$ и ковариацией $\Sigma(x)$, где $\Sigma_{ij} = k(x_i, x_j)$, а k — положительно определенная функция ядра.



Избранные алгоритмы обучения с учителем

В этой главе

- ✓ Марковские модели: ранжирование страниц и скрытые марковские модели
- ✓ Обучение на несбалансированных данных, стратегии уменьшения и увеличения выборки
- ✓ Активное обучение, в том числе семплирование по неуверенности (uncertainty sampling) и по несогласию в комитете (query by committee)
- ✓ Выбор модели, включая методы настройки гиперпараметров
- ✓ Ансамблевые методы: бэггинг, бустинг и стекинг
- ✓ Исследования в области ML, посвященные алгоритмам обучения с учителем

В предыдущих двух главах мы познакомились с такими алгоритмами машинного обучения с учителем, как классификация и регрессия. В этой главе сосредоточимся на некоторых других алгоритмах из этой же категории. Они были подобраны таким образом, чтобы дать представление о многообразии их использования: от моделей временных рядов, применяемых в вычислительных финансах, до обучения на несбалансированных данных, необходимого при выявлении мошенничества; от активного обучения, позволяющего сократить количество обучающих примеров, до выбора моделей и ансамблевых методов,

непрерывно присутствующих в соревнованиях по data science. В конце главы проведем обзор исследований по ML и я предложу несколько упражнений. Давайте начнем с рассмотрения основ марковских моделей.

7.1. Марковские модели

В этом разделе мы обсудим вероятностные модели для последовательностей наблюдений. Модели временных рядов имеют широкий спектр применения, в том числе в области вычислительных финансов, распознавания речи и вычислительной биологии. Мы начнем с рассмотрения двух популярных алгоритмов, в основании которых лежат свойства марковских цепей: алгоритма ранжирования страниц и ЕМ-алгоритма для скрытых марковских моделей (СММ).

Однако прежде чем погружаться в изучение отдельных алгоритмов, давайте познакомимся с основами. Марковская модель первого порядка для последовательности случайных величин $x_1, \dots, x_T$ представляет собой модель совместной вероятности, которую можно, как показано ниже, представить в виде произведения множителей:

$$\begin{aligned} p(x_1, \dots, x_T) &= p(x_1)p(x_2 | x_1) \cdots p(x_T | x_{T-1}) = \\ &= p(x_1) \prod_{t=2}^T p(x_t | x_{t-1}). \end{aligned} \quad (7.1)$$

Заметьте, что каждый множитель является условным распределением, зависящим от одной случайной величины (то есть каждый такой множитель зависит только от предыдущего состояния и, следовательно, имеет память объемом 1). Можно также сказать, что X_{t-1} служит достаточной статистикой для X_t . *Достаточная статистика* — это функция выборки данных (например, обобщающий показатель данных, такой как сумма или среднее значение), которая содержит всю необходимую информацию для оценки неизвестного параметра статистической модели.

Давайте рассмотрим более подробно вероятность перехода $p(x_t | x_{t-1})$. Для дискретной последовательности состояний $x_t \in \{1, \dots, K\}$ переходную вероятность можно представить в виде стохастической матрицы $K \times K$ (сумма каждой строки которой равна 1):

$$A_{ij} = p(X_t = j | X_{t-1} = i), \text{ где } \sum_j A_{ij} = 1 \forall i. \quad (7.2)$$

Пример простой марковской модели с двумя состояниями показан на рис. 7.1.

Здесь α — вероятность перехода из состояния 1, а $1 - \alpha$ — вероятность остаться в состоянии 1. Обратите внимание, что переходные вероятности для каждого состояния в сумме дают 1. Соответствующую матрицу переходных вероятностей можно записать следующим образом:

$$A = \begin{pmatrix} 1 - \alpha & \alpha \\ \beta & 1 - \beta \end{pmatrix}. \quad (7.3)$$

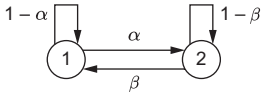


Рис. 7.1. Марковская модель с двумя состояниями

Если α и β не меняются с течением времени, другими словами, если переходная матрица A не зависит от времени, то такая цепь Маркова называется *стационарной*, или *инвариантной по времени*.

Обычно нас интересует поведение марковской цепи на длительных промежутках времени, то есть такое распределение по состояниям, которое построено по частоте посещений отдельных состояний во времени. Подобное распределение известно как *стационарное распределение*. Давайте выведем для него формулу. Пусть π_0 — начальное распределение по состояниям. Тогда после одного перехода мы имеем следующее:

$$\pi_1(j) = \sum_i \pi_0(i) A_{ij}. \quad (7.4)$$

Или если перевести в матричное представление, то так:

$$\pi_1 = \pi_0 A. \quad (7.5)$$

Если продолжить, то второй переход представим в следующем виде:

$$\pi_2 = \pi_1 A = \pi_0 A^2. \quad (7.6)$$

Очевидно, что возведение переходной матрицы A в степень n эквивалентно моделированию n шагов (или переходов) цепи Маркова. Через какое-то время мы достигаем такого состояния, когда умножение вектора-строки состояния π на матрицу A справа дает нам такой же вектор π :

$$\pi = \pi A. \quad (7.7)$$

Это означает, что мы нашли искомое стационарное распределение. Оно равно π , являющемуся собственным вектором матрицы A при собственном значении 1. Мы также постулируем здесь (оставив доказательство в качестве упражнения), что стационарное распределение существует тогда и только тогда, когда цепь является *возвратной* (имеющей возможность вернуться в любое из состояний с вероятностью 1) и *апериодической* (то есть неосциллирующей).

7.1.1. Алгоритм ранжирования страниц

Компания Google использует алгоритм ранжирования страниц (page rank algorithm) для упорядочивания результатов поиска по миллионам веб-страниц. Мы можем представить набор n веб-страниц в виде графа $G = (V, E)$. При этом каждая вершина $v \in V$ представляет собой веб-страницу, а каждое ребро

$e \in E$ — направленную ссылку с одной страницы на другую. Тогда G является разреженным графом с $|V| = n$. Интуитивно понятно, что веб-страница, на которую имеется много ссылок с авторитетных источников, должна быть продвинута на вершину рейтинга. Информация о том, как страницы связаны, позволяет нам построить гигантскую переходную матрицу A , где A_{ij} — это вероятность перехода по ссылке со страницы i на страницу j . Отталкиваясь от этой формулировки, запись π_j можно интерпретировать как значимость (importance), или ранг, страницы j . Учитывая также сказанное выше о марковских цепях, ранг страницы — это стационарное распределение π (то есть собственный вектор A , соответствующий собственному значению 1):

$$\pi_j = \sum_i A_{ij} \pi_i. \quad (7.8)$$

Для существования стационарного распределения π (то есть ранга страницы) необходимо, чтобы марковская цепь, описываемая матрицей переходов, была возвратной и апериодической. Посмотрим на то, как можно построить такую переходную матрицу. Примем следующую модель взаимодействия с веб-страницами. Если на определенной странице присутствуют исходящие ссылки, то с вероятностью $p > 0,5$ (p можно выбрать при помощи моделирования) мы переходим по одной из ссылок (выбираемой равномерно случайным образом), а с вероятностью $1 - p$ открываем новую страницу (тоже генерируемую равномерно случайным образом). Если исходящих ссылок нет, мы открываем новую страницу (выбираемую случайным образом из равномерного распределения). Эти два условия гарантируют, что марковская цепь является возвратной и апериодической, поскольку случайные переходы могут включать самопереходы. Таким образом, каждое состояние достижимо из любого другого состояния. Пусть G_{ij} — матрица смежности, а $d_j = \sum_i G_{ij}$ представляет собой полустепень исхода (out-degree) страницы j . Тогда если полустепень исхода равна 0, то мы переходим на случайную страницу с вероятностью $1/n$. Таким образом, с вероятностью p мы переходим по ссылке, разыгрываемой с вероятностью, обратно пропорциональной полустепени исхода $1/d_j$, а с вероятностью $1 - p$ мы переходим на случайную страницу. Давайте сведем воедино все случаи в виде переходной матрицы ниже:

$$A_{ij} = \begin{cases} \frac{p G_{ij}}{d_j} + \frac{1-p}{n}, & \text{если } d_j \neq 0; \\ \frac{1}{n}, & \text{если } d_j = 0. \end{cases} \quad (7.9)$$

Но как нам найти собственный вектор π , учитывая огромный размер нашей переходной матрицы A ? Для вычисления ранга страницы воспользуемся итеративным алгоритмом под названием *степенной метод* (power method). Пусть v_0 — произвольный вектор из области значений A ; мы можем инициализировать v_0 случайным образом. Представьте теперь многократное умножение v на A с последующей перенормировкой v :

$$\begin{aligned}
v_t &= A_{v_{t-1}}; \\
v_t &= \frac{v_t}{\|v_t\|}; \\
\lambda &= v_t^T A v_t.
\end{aligned} \tag{7.10}$$

Если повторять этот алгоритм до достижения сходимости ($\|v_t\| \approx \|v_{t-1}\|$) или пока не будет превышено максимальное число итераций T , то будет получено наше стационарное распределение $\pi = v_T$. Такой подход обусловлен тем, что можно переписать нашу матрицу как $A = U\Lambda U^T$. Тогда получаем следующее уравнение:

$$\begin{aligned}
v_t &= A v_{t-1} = A^t v_0 = (U\Lambda U^T)^t v_0 = U(\Lambda^t U^T v_0) = \\
&= a_1 \lambda_1^t u_1 + a_2 \lambda_2^t u_2 + \dots + a_n \lambda_n^t u_n = \\
&= \lambda_1^t \left(a_1 u_1 + a_2 \left(\frac{\lambda_2}{\lambda_1} \right)^t u_2 + \dots + a_n \left(\frac{\lambda_n}{\lambda_1} \right)^t u_n \right) \rightarrow \\
&\rightarrow \lambda_1^t a_1 u_1.
\end{aligned} \tag{7.11}$$

Равенство в формуле 7.11 выполняется для определенных значений коэффициентов a_i , и поскольку U является ортонормированной матрицей (то есть $U^T U = I$) и $|\lambda_k|/|\lambda_1| < 1$ для $k > 1$, то v_t сходится к u_1 , равному нашему стационарному распределению! Алгоритм ранжирования страниц можно схематично представить в виде псевдокода на рис. 7.2.

```

1: class page_rank
2:   function power_iteration(A):
3:      $v_0 \sim \text{Unif}[0, 1]^d$ 
4:     while (not converged) and (iter  $\leq$  max_iter):
5:        $v_t = A_{v_{t-1}}$ 
6:        $v_t = \frac{v_t}{\|v_t\|}$ 
7:        $\lambda = v_t^T A v_t$ 
8:       converged =  $\|v_t - v_{t-1}\| \leq \text{tolerance}$ 
9:     конец while
10:   return  $\lambda, v_t$ 

```

Рис. 7.2. Псевдокод ранжирования страниц

Мы начинаем с семплирования исходного значения вектора v из d -мерного равномерного распределения. Затем выполняем итерацию, умножая A на v с последующей перенормировкой v . Эти шаги повторяются до тех пор, пока не будет достигнута сходимость либо не будет превышено максимальное количество итераций. Теперь мы готовы реализовать в коде алгоритм степенного метода для вычисления ранга страницы (см. следующий листинг).

Листинг 7.1. Алгоритм ранжирования страниц

```

import numpy as np
from numpy.linalg import norm

np.random.seed(42)

class page_rank():

    def __init__(self):
        self.max_iter = 100
        self.tolerance = 1e-5

    def power_iteration(self, A):
        n = np.shape(A)[0]
        v = np.random.rand(n)
        converged = False
        iter = 0

        while (not converged) and (iter < self.max_iter):
            old_v = v
            v = np.dot(A, v)
            v = v / norm(v)
            lambd = np.dot(v, np.dot(A, v))
            converged = norm(v - old_v) < self.tolerance
            iter += 1
        # конец while

        return lambd, v

if __name__ == "__main__":

    X = np.random.rand(10,5)
    A = np.dot(X.T, X)

    pr = page_rank()
    lambd, v = pr.power_iteration(A)

    print(lambd)
    print(v)

    # сравнение с реализацией в np.linalg
    eigval, eigvec = np.linalg.eig(A)
    idx = np.argsort(np.abs(eigval))[:, -1]
    top_lambd = eigval[idx][0]
    top_v = eigvec[:, idx][0]

```

Создание для удобства симметричной матрицы случайных вещественных чисел

Сравнение с реализацией в np.linalg

Опыт показывает, что наша реализация успешно находит соответствующий собственному значению 1 собственный вектор A , который равен рангу страницы.

7.1.2. Скрытые марковские модели

Скрытые марковские модели (СММ) используются для представления данных временных рядов в различных приложениях — от прогнозирования фондового рынка

до секвенирования ДНК. СММ построены на базе марковских цепей с дискретным временем, скрытыми состояниями $z_t \in \{1, \dots, K\}$, матрицей переходов K и матрицей эмиссии (emission matrix) E , которая моделирует наблюдаемые данные X , испускаемые из каждого состояния. СММ изображена графически на рис. 7.3.

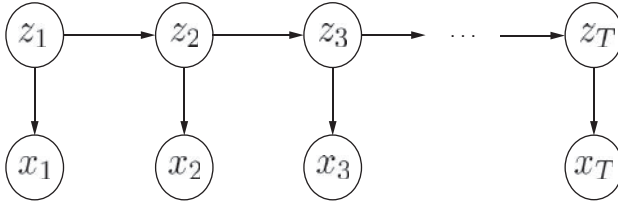


Рис. 7.3. Скрытая марковская модель

Совместную плотность вероятности можно записать следующим образом:

$$\begin{aligned}
 \overset{\text{Совместное распределение}}{p(z_{1:T}, x_{1:T} | \theta)} &= \overset{\text{Факторизация согласно СММ}}{p(z_{1:T} | \theta) p(x_{1:T} | z_{1:T}, \theta)} = \\
 &= \underset{\text{Распределение начального состояния}}{p(z_1 | \pi)} \left[\underset{\text{Распределение переходов между состояниями}}{\prod_{t=2}^T p(z_t | z_{t-1}, A)} \right] \left[\underset{\text{Распределение вероятности эмиссии для состояний}}{\prod_{t=1}^T p(x_t | z_t, E)} \right]. \quad (7.12)
 \end{aligned}$$

Здесь $\theta = \{\pi, A, E\}$ — параметры СММ, где π — начальное распределение состояний. Количество состояний K часто определяется конкретным приложением. Например, мы можем моделировать цикл сна — бодрствования, используя $K = 2$ состояния. Сами наблюдаемые данные могут быть как дискретными, так и непрерывными. Мы сосредоточимся на дискретном случае, в котором матрица эмиссии равна $E_{kl} = p(x_t = l | z_t = k)$. Предполагается, что переходная матрица не зависит от времени и равна $A_{ij} = p(z_t = j | z_{t-1} = i)$.

Обычно нас интересует предсказание ненаблюдаемого скрытого состояния z на основе наблюдаемых событий x в произвольный момент времени. На языке математики искомое выглядит как $p(z_t = j | x_{1:t})$. Обозначим эту величину $\alpha_t(j)$:

$$\alpha_t(j) = p(z_t = j | x_{1:t}). \quad (7.13)$$

Давайте выведем для него формулу. Обратите внимание на рекуррентное соотношение между последовательными значениями α . Распишем его подробнее:

$$\begin{aligned}
 p(z_t = j | x_{1:t-1}) &= \sum_{z_{t-1}} p(z_t = j, z_{t-1} | x_{1:t-1}) = \\
 &= \sum_i p(z_t = j | z_{t-1} = i) p(z_{t-1} = i | x_{1:t-1}) = \\
 &= \sum_i A(i, j) \alpha_{t-1}(i). \quad (7.14)
 \end{aligned}$$

На этапе предсказания мы суммируем по всем возможным состояниям i , умножая при этом α на матрицу перехода в состояние j . Используем результат предсказания на этапе обновления:

$$\begin{aligned}\alpha_t(j) &= p(z_t = j | x_{1:t}) = p(z_t = j | x_t, x_{1:t-1}) = \\ &= \frac{p(x_t | z_t = j, x_{1:t-1}) p(z_t = j | x_{1:t-1})}{p(x_t | x_{1:t-1})} = \\ &= \frac{p(x_t | z_t = j) p(z_t = j | x_{1:t-1})}{\sum_j p(x_t | z_t = j) p(z_t = j | x_{1:t-1})} = \\ &= \frac{1}{Z_t} E_t(j) \sum_i A(i, j) \alpha_{t-1}(i).\end{aligned}\tag{7.15}$$

Итог нашего вывода в матричном виде можно записать следующим образом:

$$\alpha_t \propto E_t(A^T \alpha_{t-1}).\tag{7.16}$$

Учитывая рекурсию в формуле для α и начальное условие, мы можем вычислить частное распределение для скрытого состояния. Этот алгоритм называется *прямым* (forward algorithm), поскольку для вычисления значений α используется прямой рекурсивный проход. Он работает в режиме реального времени и поэтому подходит (при использовании соответствующих признаков) для таких приложений, как распознавание рукописного ввода или речи. Однако мы можем улучшить наши оценки частных распределений, используя все имеющиеся данные к моменту времени T . Давайте посмотрим, как это можно сделать, добавляя обратный проход. Основная идея *алгоритма вперед-назад* (forward-backward algorithm) состоит в том, чтобы разделить марковскую цепь на прошлое и будущее путем введения условной вероятности z_t . Пусть T — конечное время нашего временного ряда. Теперь давайте определим частное распределение вероятности по всем наблюдаемым событиям следующим образом:

$$\begin{aligned}\gamma_t(j) &= p(z_t = j | x_{1:T}) = p(z_t = j | x_{1:T}, x_{t+1:T}) = \\ &= \frac{1}{Z_t} p(z_t = j | x_{1:t}) p(x_{t+1:T} | z_t = j) \propto \\ &\propto \alpha_t(j) \beta_t(j).\end{aligned}\tag{7.17}$$

Здесь мы ввели обозначение $\beta_t(j) = p(x_{t+1:T} | z_t = j)$ и использовали факт условной независимости прошлого и будущего марковской цепи при условии состояния z_t . Но вопрос остается открытым: как можно вычислить $\beta_t(j)$? Мы можем использовать аналогичную рекурсию, идя в обратном направлении от момента времени $t = T$:

$$\begin{aligned}
\beta_{t-1}(i) &= p(x_{t:T} \mid z_{t-1} = i) = \sum_j p(z_t = j, x_t, x_{t+1:T} \mid z_{t-1} = i) = \\
&= \sum_j p(x_{t+1:T} \mid z_t = j, x_t, z_{t-1} = i) p(z_t = j, x_t \mid z_{t-1} = i) = \\
&= \sum_j p(x_{t+1:T} \mid z_t = j) p(z_t = j, x_t \mid z_{t-1} = i) = \quad (7.18) \\
&= \sum_j p(x_{t+1:T} \mid z_t = j) p(x_t \mid z_t = j, z_{t-1} = i) p(z_t = j \mid z_{t-1} = i) = \\
&= \sum_j \beta_t(j) E_t(j) A(i, j).
\end{aligned}$$

В матричной записи полученный результат будет выглядеть следующим образом:

$$\beta_{t-1} = A(E_t \beta_t). \quad (7.19)$$

Здесь базовый случай представлен следующей формулой:

$$\beta_T(i) = p(X_{T+1:T} \mid z_T = i) = 1. \quad (7.20)$$

Выражение справедливо, поскольку последовательность заканчивается в момент времени T , а потому $X_{T+1:T}$ — это несуществующее событие с вероятностью 1. Вычислив как α -, так и β -сообщения, можем объединить их для получения сглаженных маргинальных распределений: $\gamma_t(j) \propto \alpha_t(j) \beta_t(j)$.

Давайте теперь посмотрим, как мы можем получить наиболее вероятную последовательность переходов между переменными скрытого состояния z_t . Другими словами, мы хотели бы найти следующее:

$$z^* = \arg \max_{z_{1:T}} p(z_{1:T} \mid x_{1:T}). \quad (7.21)$$

Пусть $\delta_t(j)$ — вероятность, что конечное состояние — это j при заданной наиболее вероятной последовательности путей $z_{1:t-1}^*$:

$$\delta_t(j) = \max_{z_1, \dots, z_{t-1}} p(z_t = j, z_{1:t-1} \mid x_{1:t}). \quad (7.22)$$

Можем представить ее в виде рекуррентной формулы:

$$\delta_t(j) = \max_i \delta_{t-1}(i) A(i, j) E_t(j). \quad (7.23)$$

Этот алгоритм известен как *алгоритм Витерби*. Давайте теперь сведем все, что мы узнали до сих пор, воедино — см. псевдокод на рис. 7.4.

```

1: class HMM
2:   function forward_backward:
3:     for  $t = 1$  to  $n$ :
4:        $\alpha_t = \text{normalize}(E_t A^T \alpha_{t-1})$ 
5:     конец for
6:     for  $t = n - 1$  to  $1$ :
7:        $\beta_t = \text{normalize}(A(E_t \beta_{t+1}))$ 
8:     конец for
9:      $\gamma = \alpha \cdot \beta$ 
10:    return  $\gamma, \alpha, \beta$ 
11:   function viterbi:
12:     for  $t = 1$  to  $n$ :
13:        $\delta_t[j] = \max_i (\delta_{t-1}[i] A[i, j] E_t[j])$ 
14:     конец for

```

Рис. 7.4. Псевдокод скрытой марковской модели

Код содержит две основные функции: `forward_backward` и `viterbi`. В функции `forward_backward` мы строим разреженную матрицу X индикаторов эмиссии и вычисляем прямые вероятности α с нормировкой на каждой итерации. Обратные вероятности β , как было показано ранее, вычисляются аналогично. Наконец, мы вычисляем маргинальные вероятности γ путем умножения α и β . В функции `viterbi` применяем логарифмическую шкалу для устойчивости численного решения и заменяем умножение на сложение, вычисляя выражение для δ по формуле, приведенной выше. Теперь мы готовы воплотить СММ-вывод в коде.

Листинг 7.2. Алгоритм вперед-назад СММ

```

import numpy as np
from scipy.sparse import coo_matrix
import matplotlib.pyplot as plt

np.random.seed(42)

class HMM():
    def __init__(self, d=3, k=2, n=10000):
        self.d = d      ← Размерность данных
        self.k = k      ← Размерность скрытого состояния
        self.n = n      ← Количество точек данных

        self.A = np.zeros((k,k)) ← Переходная матрица
        self.E = np.zeros((k,d)) ← Матрица эмиссии
        self.s = np.zeros(k)      ← Вектор начального состояния

        self.x = np.zeros(self.n) ← Наблюдаемые события

    def normalize_mat(self, X, dim=1):
        z = np.sum(X, axis=dim)
        Xnorm = X/z.reshape(-1,1)

```

```

    return Xnorm

def normalize_vec(self, v):
    z = sum(v)
    u = v / z
    return u, z

def init_hmm(self):
    # инициализация матриц случайными значениями
    self.A = self.normalize_mat(np.random.rand(self.k, self.k))
    self.E = self.normalize_mat(np.random.rand(self.k, self.d))
    self.s, _ = self.normalize_vec(np.random.rand(self.k))

    # генерация наблюдаемых значений CMM
    z = np.random.choice(self.k, size=1, p=self.s)
    self.x[0] = np.random.choice(self.d, size=1, p=self.E[z,:].ravel())
    for i in range(1, self.n):
        z = np.random.choice(self.k, size=1, p=self.A[z,:].ravel())
        self.x[i] = np.random.choice(self.d, size=1,
                                     p=self.E[z,:].ravel())
    # конец for

def forward_backward(self):
    # создание разреженной матрицы X индикаторов эмиссии
    data = np.ones(self.n)
    row = self.x
    col = np.arange(self.n)
    X = coo_matrix((data, (row, col)), shape=(self.d, self.n))

    M = self.E * X
    At = np.transpose(self.A)
    c = np.zeros(self.n) # нормировочные константы
    alpha = np.zeros((self.k, self.n)) # alpha = p(z_t = j | x_{1:T})
    alpha[:,0], c[0] = self.normalize_vec(self.s * M[:,0])
    for t in range(1, self.n):
        alpha[:,t], c[t] = self.normalize_vec(np
        ➡ .dot(At, alpha[:,t-1]) * M[:,t])
    # конец for

    beta = np.ones((self.k, self.n))
    for t in range(self.n-2, 0, -1):
        beta[:,t] = np.dot(self.A, beta[:,t+1] * M[:,t+1])/c[t+1]
    # конец for
    gamma = alpha * beta

    return gamma, alpha, beta, c

def viterbi(self):
    # создание разреженной матрицы X индикаторов эмиссии
    data = np.ones(self.n)
    row = self.x
    col = np.arange(self.n)

```

```

X = coo_matrix((data, (row, col)), shape=(self.d, self.n))

# логарифмическая шкала для численной устойчивости
s = np.log(self.s)
A = np.log(self.A)
M = np.log(self.E * X)

Z = np.zeros((self.k, self.n))
Z[:,0] = np.arange(self.k)
v = s + M[:,0]
for t in range(1, self.n):
    Av = A + v.reshape(-1,1)
    v = np.max(Av, axis=0)
    idx = np.argmax(Av, axis=0)
    v = v.reshape(-1,1) + M[:,t].reshape(-1,1)
    Z = Z[idx,:]
    Z[:,t] = np.arange(self.k)
# конец for
llh = np.max(v)
idx = np.argmax(v)
z = Z[idx,:]

return z, llh

if __name__ == "__main__":

    hmm = HMM()
    hmm.init_hmm()

    gamma, alpha, beta, c = hmm.forward_backward()
    z, llh = hmm.viterbi()

```

В результате выполнения кода мы получаем параметры α , β и γ для произвольно инициализированной СММ. Декодер Витерби возвращает максимально правдоподобную последовательность переходов между состояниями z .

В следующем разделе рассмотрим обучение на несбалансированных данных. В частности, мы сосредоточимся на стратегиях уменьшения и увеличения выборки для выравнивания объема данных между разными классами.

7.2. Обучение на несбалансированных данных

Большинство алгоритмов классификации будут работать оптимально, если количество примеров каждого класса приблизительно одинаково. Сильно асимметричные наборы данных, в которых класс меньшинства (minority class) значительно уступает по объему одному или нескольким другим классам, часто встречаются в задачах по выявлению мошенничества, в медицинской диагностике и вычислительной биологии. Один из способов решения этой проблемы состоит в повторном семплировании (resampling) данных таким образом, чтобы компенсировать подобный дисбаланс и в результате прийти к более надежной, дающей больший процент верных результатов, границе

принятия решения. Методы повторного семплирования можно в общих чертах разделить на четыре категории: уменьшение размера класса большинства (majority class), увеличение объема класса меньшинства, комбинирование предыдущих двух подходов, а также создание ансамбля сбалансированных наборов данных.

7.2.1. Стратегии уменьшения выборки

Процедуры уменьшения выборки (undersampling) приводят к удалению данных из класса большинства исходного набора данных, как показано на рис. 7.5. В методе *случайного уменьшения выборки* (random undersampling) точки данных класса большинства просто удаляются равномерно и случайным образом. *Кластерные центроиды* (cluster centroids) — это метод, в котором кластер примеров заменяется кластерным центроидом алгоритма K средних, где количество кластеров определяется процентом сокращения выборки.

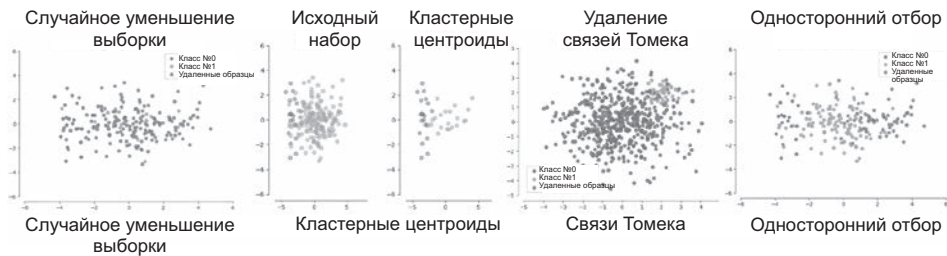


Рис. 7.5. Стратегии уменьшения выборки: случайная, кластерные центроиды, связи Томека и односторонний отбор

Другим эффективным способом уменьшения выборки является метод *связей Томека* (Tomek links), который устраняет нежелательное перекрытие между классами. Связи Томека устраняются до тех пор, пока все пары ближайших соседей, расположенных на минимальном расстоянии, не будут принадлежать к одному классу. Связь Томека определяется следующим образом. Дана пара примеров (x_i, x_j) , где $x_i \in S_{min}$, $x_j \in S_{maj}$ и $d(x_i, x_j)$ — это расстояние между x_i и x_j . Пара (x_i, x_j) называется связью Томека, если не существует такого примера x_k , что $d(x_i, x_k) < d(x_i, x_j)$ или $d(x_j, x_k) < d(x_j, x_j)$. Исходя из такого определения, если два примера образуют связь Томека, то либо один из этих примеров является шумом, либо оба находятся вблизи границы. Поэтому связи Томека можно использовать для устранения перекрытия между классами. Удалив перекрывающиеся примеры, можно сформировать ясно различимые кластеры в обучающем наборе и повысить эффективность классификации. В методе *одностороннего отбора* (one-sided selection, OSS) выбирают репрезентативное подмножество класса большинства E и объединяют его с множеством всех примеров класса меньшинства S_{min} , образуя $N = \{E \cup S_{min}\}$. Из сокращенного набора N затем удаляют все связи Томека класса большинства. Давайте, используя

библиотеку `imbalanced-learn`, поэкспериментируем с сокращением выборки методом связей Томека:

Листинг 7.3. Алгоритм связей Томека

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.utils import shuffle
from imblearn.under_sampling import TomekLinks

rng = np.random.RandomState(42)

def main():

    n_samples_1 = 500
    n_samples_2 = 50
    X_syn = np.r_[1.5 * rng.randn(n_samples_1, 2), 0.5 *
    ➤ rng.randn(n_samples_2, 2) + [2, 2]]  ← Генерация данных
    y_syn = np.array([0] * (n_samples_1) + [1] * (n_samples_2))
    X_syn, y_syn = shuffle(X_syn, y_syn)
    X_syn_train, X_syn_test, y_syn_train, y_syn_test =
    ➤ train_test_split(X_syn, y_syn)

    tl = TomekLinks(sampling_strategy='auto')
    X_resampled, y_resampled = tl.fit_resample(X_syn, y_syn)  ← Удаление
    idx_resampled = tl.sample_indices_                      связей Томека
    idx_samples_removed =
    ➤ np.setdiff1d(np.arange(X_syn.shape[0]), idx_resampled)

    fig = plt.figure()  ← Создание графиков
    ax = fig.add_subplot(1, 1, 1)

    idx_class_0 = y_resampled == 0
    plt.scatter(X_resampled[idx_class_0, 0], X_resampled[idx_class_0, 1],
    ➤ alpha=.8, label='Класс № 0')
    plt.scatter(X_resampled[~idx_class_0, 0], X_resampled[~idx_class_0, 1],
    ➤ alpha=.8, label='Класс № 1')
    plt.scatter(X_syn[idx_samples_removed, 0], X_syn[idx_samples_removed, 1],
    ➤ alpha=.8, label='Удаленные образцы')
    plt.title('Уменьшение выборки: связи Томека')
    plt.legend()
    plt.show()

if __name__ == "__main__":
    main()
```

Рисунок 7.6 демонстрирует полученный результат. Видно, что устранение нежелательного перекрытия классов может улучшить надежность границы принятия решений и повысить долю верных результатов классификации:



Рис. 7.6. Алгоритм связей Томека: количество удаленных примеров вблизи границы классификации

7.2.2. Стратегии увеличения выборки

Методы увеличения выборки увеличивают объем данных в классе меньшинства исходного набора, как показано на рис. 7.7. При использовании метода *случайного увеличения выборки* (random oversampling) новые точки данных в класс меньшинства добавляются равномерно и случайным образом. *Метод синтетического увеличения класса меньшинства* (synthetic minority oversampling technique, SMOTE) синтезирует примеры путем нахождения K ближайших соседей в пространстве признаков и создания новых точек данных вдоль отрезков прямых, соединяющих любые из K ближайших соседей класса меньшинства. Синтетические примеры генерируются таким образом: вычисляется разность

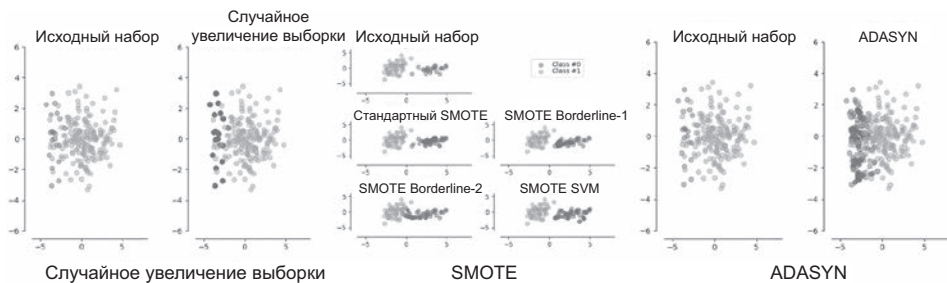


Рис. 7.7. Стратегии увеличения выборки

между рассматриваемым вектором признаков (примером) и его ближайшим соседом, она умножается на случайное число от 0 до 1, и результат прибавляется к рассматриваемому вектору признаков, пополняя набор данных новой точкой.

Адаптивное синтетическое семплирование (adaptive synthetic sampling, ADASYN) использует взвешенное распределение для примеров из класса меньшинства в соответствии с их уровнем сложности для обучения. При этом больше синтетических данных генерируется для примеров из класса меньшинства, которые сложнее выучить. Благодаря этому использование ADASYN улучшает обучение на несбалансированных наборах данных двояким образом: уменьшает вызванную дисбалансом классов погрешность и адаптивно смещает границу принятия классификационного решения в сторону сложных примеров. Давайте, используя библиотеку `imbalanced-learn`, поэкспериментируем с увеличением выборки при помощи SMOTE:

Листинг 7.4. Алгоритм SMOTE

```
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification
from sklearn.decomposition import PCA

from imblearn.over_sampling import SMOTE

def plot_resampling(ax, X, y, title):
    c0 = ax.scatter(X[y == 0, 0], X[y == 0, 1], label="Class #0", alpha=0.5)
    c1 = ax.scatter(X[y == 1, 0], X[y == 1, 1], label="Class #1", alpha=0.5)
    ax.set_title(title)
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    ax.get_xaxis().tick_bottom()
    ax.get_yaxis().tick_left()
    ax.spines['left'].set_position(('outward', 10))
    ax.spines['bottom'].set_position(('outward', 10))
    ax.set_xlim([-6, 8])
    ax.set_ylim([-6, 6])

    return c0, c1

def main():
    X, y = make_classification(n_classes=2, class_sep=2, weights=[0.3, 0.7],
                              n_informative=3, n_redundant=1, flip_y=0,
                              n_features=20, n_clusters_per_class=1,
                              n_samples=80,
                              random_state=10)  # ← Генерация набора данных

    pca = PCA(n_components=2)
    X_vis = pca.fit_transform(X)  # ← Уменьшение размерности
                                   # с помощью PCA для визуализации

    method = SMOTE()
    X_res, y_res = method.fit_resample(X, y)  # ← Стандартный SMOTE
    X_res_vis = pca.transform(X_res)
```

```

f, (ax1, ax2) = plt.subplots(1, 2)  ← Создание графиков
c0, c1 = plot_resampling(ax1, X_vis, y, 'Исходные данные')
plot_resampling(ax2, X_res_vis, y_res, 'SMOTE')
ax1.legend((c0, c1), ('Класс № 0', 'Класс № 1'))
plt.tight_layout()
plt.show()

if __name__ == "__main__":
    main()

```

В коде из листинга 7.4 мы генерируем набор данных высокой размерности с двумя несбалансированными классами. Затем мы применяем стандартный алгоритм увеличения выборки SMOTE и используем метод главных компонент (principal component analysis, PCA) с двумя компонентами для визуализации данных на плоскости. Рисунок 7.8 демонстрирует полученный результат.

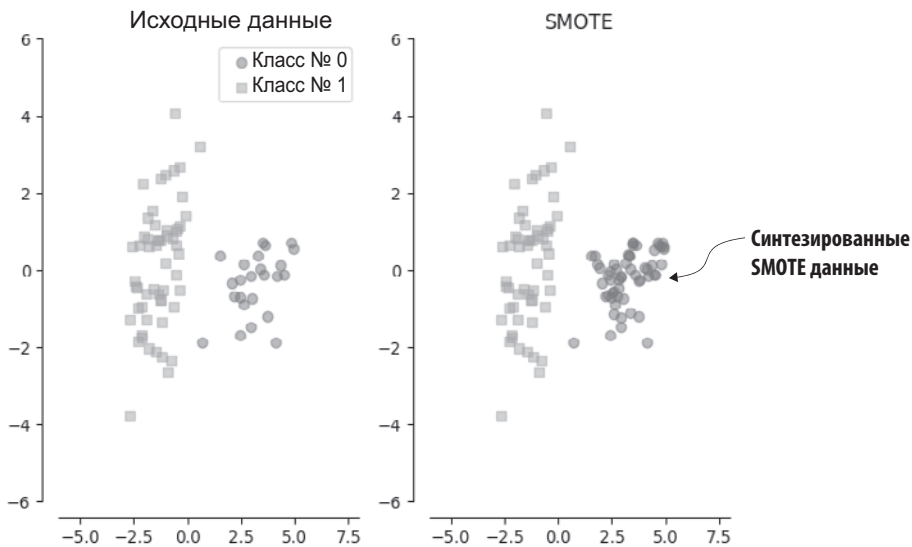


Рис. 7.8. Алгоритм SMOTE

Можно заметить, что SMOTE плотно заполняет класс меньшинства синтетическими данными. Объединение методов увеличения и уменьшения выборки дает нам гибридную стратегию. Типичными примерами являются объединение методов SMOTE и связей Томека или SMOTE и редактирования ближайших соседей (edited nearest neighbors, ENN). Дополнительные способы обучения на несбалансированных наборах данных включают взвешивание обучающих примеров, введение различных штрафов за неправильную классификацию положительных и отрицательных примеров и бутстрэппинг (bootstrapping). В следующем разделе мы сосредоточимся на очень важном принципе обучения с учителем, который может сократить количество требующихся учебных примеров, — активном обучении.

7.3. Активное обучение

Ключевая идея *активного обучения* заключается в том, что алгоритм может достичь большей достоверности результатов при меньшем количестве обучающих примеров, если ему будет позволено отбирать данные, на основе которых он учится. Активное обучение вполне оправданно использовать во многих современных задачах, где неразмеченных данных может быть в избытке, но их разметка обходится дорого. Активное обучение иногда называют *обучением с запросами* (query learning) или *оптимальным планированием эксперимента* (optimal experimental design), поскольку алгоритм активного обучения выдает запросы в виде неразмеченных экземпляров данных, которые должен снабдить меткой учитель (oracle). Таким образом, алгоритм активного обучения стремится, используя как можно меньше размеченных примеров, достичь высокой доли верных результатов. Существует обзор литературы по активному обучению: см. Берт Сеттлз (Burr Settles), «Active Learning Literature Survey» (University of Wisconsin-Madison, Department of Computer Sciences, 2009).

Мы же сфокусируемся на методе семплирования из пула данных (pool-based sampling), в котором предполагается наличие небольшого набора размеченных данных L и большого пула данных без меток U . Запросы выбираются из пула в соответствии с мерой информативности. Методы, основанные на пуле данных, чтобы выбрать наилучший запрос, производят ранжирование всей коллекции неразмеченных данных. Поэтому для очень больших наборов данных может оказаться более целесообразным семплирование из потока данных (stream-based sampling), при котором данные поступают последовательно и решения о запросе принимаются индивидуально. Рисунок 7.9 демонстрирует примеры активного обучения на основе пула данных с использованием двоичной классификации синтетического набора данных с двумя сбалансированными классами при помощи логистической регрессии (ЛР).

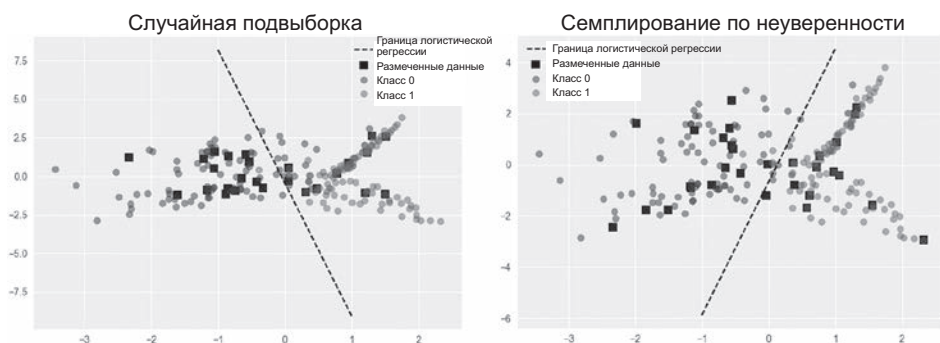


Рис. 7.9. Активное обучение для логистической регрессии

Слева можно видеть полученную в результате обучения на случайной подвыборке из 30 примеров границу принятия решения ЛР, которая позволяет достичь доли верно классифицированных примеров в 90 % на отложенных данных (held-out data). В правой части рисунка изображена граница принятия решения ЛР в результате обучения с 30 запросами, выбранными по критерию неопределенности на основе энтропии. Семплирование по неуверенности обеспечивает более высокую верность классификации: 92,5 % на отложенном наборе данных.

7.3.1. Стратегии запроса

Под стратегиями запроса (query strategies) подразумеваются критерии, которые используются для выбора подмножества обучающих примеров в рамках активного обучения. Здесь мы рассмотрим две стратегии запроса: семплирование по неуверенности и по несогласию в комитете.

СЕМПЛИРОВАНИЕ ПО НЕУВЕРЕННОСТИ

Одной из самых простых и наиболее часто используемых методик запроса является *семплирование по неуверенности* (uncertainty sampling). В этом подходе алгоритм активного обучения запрашивает метку для примера с минимальной степенью определенности. Например, в двоичной логистической регрессии выборка по неуверенности запрашивает класс точек, прилегающих к границе решения, вероятность положительного результата для которых близка к $1/2$. В задачах многоклассовой классификации алгоритм семплирования по неуверенности может запрашивать данные по точкам, относительно которых модель наименее уверена в своих предсказаниях:

$$x_{LC}^* = \arg \max_x 1 - P_\theta(\hat{y} | x). \quad (7.24)$$

Здесь $y \in \{1, \dots, K\}$ — это метка класса с наибольшей апостериорной вероятностью в рамках модели θ . Критерий наименьшей уверенности учитывает информацию только о наиболее вероятной метке. Мы можем использовать семплирование с максимальным зазором (max margin sampling), чтобы сохранить информацию о распределении по оставшимся меткам:

$$x_M^* = \arg \min_x P_\theta(\hat{y}_1 | x) - P_\theta(\hat{y}_2 | x). \quad (7.25)$$

Здесь y_1 и y_2 — это первая и вторая наиболее вероятные метки классов в рамках имеющейся модели. Интуитивно понятно, что примеры с большим зазором легко классифицировать, тогда как точки с небольшим зазором являются неоднозначными, а потому знание их меток помогло бы модели проводить различие с большей эффективностью. Однако для многоклассовых задач с очень большим количеством меток стратегия зазора по-прежнему игнорирует большую

часть распределения по оставшимся классам. Более универсальная стратегия семплирования по неуверенности использует энтропию:

$$x_H^* = \arg \max_x - \sum_i P_0(y_i | x) \log P_0(y_i | x). \quad (7.26)$$

Размечая примеры с наибольшей энтропией, можно уменьшить неопределенность в отношении меток. Семплирование по неуверенности также подходит для задач регрессии, в которых алгоритм запрашивает данные по точке с наибольшей дисперсией прогноза.

НЕСОГЛАСИЕ В КОМИТЕТЕ

Еще одной методикой выбора запросов является алгоритм несогласия в комитете (query by committee, QBC), который предполагает задействование «комитета» моделей $C = \{\theta^1, \dots, \theta^C\}$, обученных на текущем размеченном наборе L , но представляющих конкурирующие гипотезы. Каждому члену комитета разрешается проголосовать за метки примеров — кандидатов на запрос, и наиболее информативным затем считается тот запрос, который вызывает больше всего разногласий. Цель QBC — минимизировать набор гипотез, которые согласуются с текущими размеченными обучающими данными L . Для измерения уровня разногласий были предложены два основных подхода: энтропия голосов (vote entropy) и KL-дивергенция. Энтропия голосов определяется следующим образом:

$$x_{VE}^* = \arg \max_x - \sum_i \frac{V(y_i)}{C} \log \frac{V(y_i)}{C}. \quad (7.27)$$

Здесь, $y_i \in \{1, \dots, K\}$ — это метка класса, $V(y_i)$ — количество голосов, полученных меткой от членов комитета, а C — размер комитета. Обратите внимание на сходство с формулой 7.26. KL-дивергенция для QBC-голосования определяется следующим образом:

$$\begin{aligned} x_{KL}^* &= \arg \max_x \frac{1}{C} \sum_{c=1}^C KL(P_{\theta^{(c)}} \| P_C); \\ KL(P_{\theta^{(c)}} \| P_C) &= \sum_i P_{\theta^{(c)}}(y_i | x) \log \frac{P_{\theta^{(c)}}(y_i | x)}{P_C(y_i | x)}; \\ P_C(y_i | x) &= \frac{1}{C} \sum_{c=1}^C P_{\theta^{(c)}}(y_i | x). \end{aligned} \quad (7.28)$$

Здесь θ_c представляет собой модель — члена комитета, а $P_C(y_i | x)$ — вероятность консенсуса (consensus probability) о том, что y_i является предсказанной меткой. Согласно метрике KL-дивергенции, наиболее информативным является запрос с наибольшей средней разницей между распределением меток любого члена комитета и консенсусным распределением.

УМЕНЬШЕНИЕ ДИСПЕРСИИ

Ошибку обобщения модели можно уменьшить при помощи минимизации дисперсии выходных данных. Представьте регрессионную задачу, в которой целью обучения является минимизация среднеквадратичной ошибки. Пусть $\bar{\theta} = [\hat{\theta}]$ — математическое ожидание оценки параметра $\hat{\theta}$, а θ^* — истинное значение. Тогда мы получаем следующее выражение:

$$\begin{aligned}
 MSE &= E[(\hat{\theta} - \theta^*)^2] = E\left[\left[(\hat{\theta} - \bar{\theta}) + (\bar{\theta} - \theta^*)\right]^2\right] = \\
 &= E[(\hat{\theta} - \bar{\theta})^2] + 2(\bar{\theta} - \theta^*)E(\hat{\theta} - \bar{\theta}) + (\bar{\theta} - \theta^*)^2 = \\
 &= E[(\hat{\theta} - \bar{\theta})^2] + (\bar{\theta} - \theta^*)^2 = \\
 &= \text{VAR}[\hat{\theta}] + \text{bias}^2(\hat{\theta}).
 \end{aligned} \tag{7.29}$$

Эта запись называется *компромиссом между смещением и дисперсией* (bias–variance tradeoff). Таким образом, более низкого значения MSE с помощью смещенной оценки (biased estimator) можно добиться, если она уменьшает дисперсию. Теперь мы могли бы задаться вопросом, насколько низкой может быть дисперсия. Ответ дает нижняя граница Крамера — Рао, которая представляет собой нижнюю границу дисперсии любой несмещенной оценки (unbiased estimator).

Нижняя граница Крамера — Рао

При условии что $p(x|\theta)$ удовлетворяет условию регулярности, для дисперсии любой несмещенной оценки справедливо следующее неравенство:

$$\text{VAR}(\hat{\theta}) \geq \frac{1}{-E\left[\frac{\partial^2 \log p(x|\theta)}{\partial \theta^2}\right]} = \frac{1}{I(\theta)}. \tag{7.30}$$

Здесь $I(\theta)$ — информационная матрица Фишера. Таким образом, несмещенная оценка с минимальной дисперсией (minimum variance unbiased estimator, MVUE) обеспечивает минимальную дисперсию, равную обратной величине информационной матрицы Фишера. Чтобы минимизировать дисперсию оценок параметров, алгоритм активного обучения должен выбирать данные, которые максимизируют информацию Фишера. Для многомерных моделей с K параметрами информация Фишера принимает вид матрицы $K \times K$:

$$[I(\theta)]_{ij} = -E\left[\frac{\partial^2 \log p(x|\theta)}{\partial \theta_i \partial \theta_j}\right]. \tag{7.31}$$

В результате существует несколько вариантов минимизации обратного значения информационной матрицы: A -оптимальность минимизирует след, $T_r(I^{-1}(\theta))$; D -оптимальность минимизирует определитель, $|I^{-1}(\theta)|$; и E -оптимальность минимизирует наибольшее собственное значение, $\lambda_{\max}[I^{-1}(\theta)]$.

Однако у методов уменьшения дисперсии есть некоторые вычислительные недостатки. Оценка дисперсии выходных данных требует инвертирования матрицы $K \times K$ для каждого непомеченного экземпляра, что приводит к временной сложности, равной $O(UK^3)$, где U — размер пула запросов. Как следствие этого, методы уменьшения дисперсии на практике работают медленнее, чем более простые стратегии запроса, такие как семплирование по неуверенности. Давайте рассмотрим псевдокод для класса активного обучения (active learner) на рис. 7.10.

```

1: class ActiveLearner
2: function uncertainty_sampling(clf, X):
3:    $P_{\theta}(\hat{y}|x) = \text{clf.predict\_proba}(X)$ 
4:   if strategy = "least confident":
5:     return  $\arg \max_x 1 - P_{\theta}(\hat{y}|x)$ 
6:   else if strategy = "max margin":
7:     return  $\arg \min_x P_{\theta}(\hat{y}_1|x) - P_{\theta}(\hat{y}_2|x)$ 
8:   else if strategy = "entropy":
9:     return  $\arg \max_x - \sum_i P_{\theta}(y_i|x) \log P_{\theta}(y_i|x)$ 
10:  конец if
11: function query_by_committee(clf, X):
12:  if strategy = "vote entropy":
13:     $C = \text{len}(\text{clf})$ 
14:    for model in clf:
15:       $y_i = \text{clf.predict}$ 
16:    конец for
17:     $V(y_i) = \frac{1}{C} \sum_{i=1}^C 1[[y_1]]$ 
18:    return  $\arg \max_x - \sum_i \frac{V(y_i)}{C} \log \frac{V(y_i)}{C}$ 
19:  else if strategy = "average kl divergence":
20:     $P_C(y_i|x) = \frac{1}{C} \sum_{c=1}^C P_{\theta(c)}(y_i|x)$ 
21:     $KL(P_{\theta(c)}||P_C) = \sum_i P_{\theta(c)}(y_i|x) \log \frac{P_{\theta(c)}(y_i|x)}{P_C(y_i|x)}$ 
22:    return  $\arg \max_x \frac{1}{C} \sum_{c=1}^C KL(P_{\theta(c)}||P_C)$ 
23:  конец if
```

Рис. 7.10. Псевдокод активного обучения

У нас имеются две основные функции: `uncertainty_sampling` и `query_by_committee`. Функции `uncertainty_sampling` на входе передается модель классификатора `clf` и неразмеченные данные `X`. Затем предсказывается вероятность метки при заданной модели классификатора и обучающих данных, и это предсказание используется для вычисления одной из трех стратегий семплирования по неуверенности, как обсуждалось выше. В функции `query_by_committee` реализуются

два метода несогласия в комитете: энтропия голосов и средняя KL-дивергенция. Однако теперь мы передаем набор моделей `clf`, предсказания которых используются для голосования по предсказанной метке, формируя таким образом распределение для оценки энтропии. В случае KL-дивергенции используем средние значения предсказаний моделей при вычислении KL-дивергенции между предсказанием каждой модели и средним значением. Мы возвращаем обучающую выборку, которая максимизирует данную KL-дивергенцию. Теперь можем приступить к знакомству с кодом:

Листинг 7.5. Класс активного обучения

```
from __future__ import unicode_literals, division
from scipy.sparse import csc_matrix, vstack
from scipy.stats import entropy
from collections import Counter
import numpy as np

class ActiveLearner(object):

    uncertainty_sampling_frameworks = [
        'entropy',
        'max_margin',
        'least_confident',
    ]
    # Семплирование по неуверенности

    query_by_committee_frameworks = [
        'vote_entropy',
        'average_kl_divergence',
    ]
    # Отбор по несогласию в комитете

    def __init__(self, strategy='least_confident'):
        self.strategy = strategy

    def rank(self, clf, X_unlabeled, num_queries=None):

        if num_queries == None:
            num_queries = X_unlabeled.shape[0]

        elif type(num_queries) == float:
            num_queries = int(num_queries * X_unlabeled.shape[0])

        if self.strategy in self.uncertainty_sampling_frameworks:
            scores = self.uncertainty_sampling(clf, X_unlabeled)

        elif self.strategy in self.query_by_committee_frameworks:
            scores = self.query_by_committee(clf, X_unlabeled)

        else:
            raise ValueError("this strategy is not implemented.")

        rankings = np.argsort(-scores)[:num_queries]
        return rankings
```

```

def uncertainty_sampling(self, clf, X_unlabeled):
    probs = clf.predict_proba(X_unlabeled)

    if self.strategy == 'least_confident':
        return 1 - np.amax(probs, axis=1)  ← Стратегия наименьшей уверенности

    elif self.strategy == 'max_margin':
        margin = np.partition(-probs, 1, axis=1)
        return -np.abs(margin[:,0] - margin[:, 1])  ← Максимальный зазор

    elif self.strategy == 'entropy':
        return np.apply_along_axis(entropy, 1,
            ➡ probs)  ← Энтропия

def query_by_committee(self, clf, X_unlabeled):
    num_classes = len(clf[0].classes_)
    C = len(clf)
    preds = []

    if self.strategy == 'vote_entropy':
        for model in clf:
            y_out = map(int, model.predict(X_unlabeled))
            preds.append(np.eye(num_classes)[y_out])

        votes = np.apply_along_axis(np.sum, 0, np.stack(preds)) / C
        return np.apply_along_axis(entropy, 1,
            ➡ votes)  ← Энтропия голосов

    elif self.strategy == 'average_kl_divergence':
        for model in clf:
            preds.append(model.predict_proba(X_unlabeled))

        consensus = np.mean(np.stack(preds), axis=0)
        divergence = []
        for y_out in preds:
            divergence.append(entropy(consensus.T, y_out.T))

        return np.apply_along_axis(np.mean, 0, np
            ➡ .stack(divergence))  ← Средняя KL-дивергенция

```

В следующем коде применим наш алгоритм активного обучения к логистической регрессии:

Листинг 7.6. Активное обучение для логистической регрессии

```

import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from active_learning import ActiveLearner
from sklearn.metrics import accuracy_score
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

```

```

np.random.seed(42)

def main():

    num_queries = 30  ← Количество помеченных точек

    data, target = make_classification(n_samples=200, n_features=2,
    ⇒ n_informative=2,\
                                n_redundant=0,
    ⇒ n_classes=2, weights = [0.5, 0.5], random_state=0)  ← Генерация данных

    X_train, X_unlabeled, y_train, y_oracle = train_
    ⇒ test_split(data, target, test_size=0.2, random_state=0)  ← Разбиение
                                                                на размеченный
                                                                и неразмеченный
                                                                пулы

    rnd_idx = np.random.randint(0, X_train.shape[0],
    ⇒ num_queries)  ← Случайная подвыборка
    X1 = X_train[rnd_idx,:]
    y1 = y_train[rnd_idx]

    clf1 = LogisticRegression()
    clf1.fit(X1, y1)

    y1_preds = clf1.predict(X_unlabeled)
    score1 = accuracy_score(y_oracle, y1_preds)
    print("random subsampling accuracy: ", score1)

    # двумерная граница решения:  $w_2x_2 + w_1x_1 + w_0 = 0$ 
    w0 = clf1.intercept_
    w1, w2 = clf1.coef_[0]
    xx = np.linspace(-1, 1, 100)
    decision_boundary = -w0/float(w2) - (w1/float(w2))*xx

    plt.figure()
    plt.scatter(data[rnd_idx,0], data[rnd_idx,1], c='black', marker='s',
    ⇒ s=64, label='labeled')
    plt.scatter(data[target==0,0], data[target==0,1], c='blue', marker='o',
    ⇒ alpha=0.5, label='class 0')
    plt.scatter(data[target==1,0], data[target==1,1], c='red', marker='o',
    ⇒ alpha=0.5, label='class 1')
    plt.plot(xx, decision_boundary, linewidth = 2.0, c='black', linestyle =
    ⇒ '--', label='log reg boundary')
    plt.title("Random Subsampling")
    plt.legend()
    plt.show()

    AL = ActiveLearner(strategy='entropy')  ← Активное обучение
    al_idx = AL.rank(clf1, X_unlabeled, num_queries=num_queries)

    X2 = X_train[al_idx,:]
    y2 = y_train[al_idx]

    clf2 = LogisticRegression()
    clf2.fit(X2, y2)

    y2_preds = clf2.predict(X_unlabeled)

```

```

score2 = accuracy_score(y_oracle, y2_preds)
print("active learning accuracy: ", score2)

# двумерная граница решения:  $w_2x_2 + w_1x_1 + w_0 = 0$ 
w0 = clf2.intercept_
w1, w2 = clf2.coef_[0]
xx = np.linspace(-1, 1, 100)
decision_boundary = -w0/float(w2) - (w1/float(w2))*xx

plt.figure()
plt.scatter(data[al_idx,0], data[al_idx,1], c='black', marker='s',
    ➤ s=64, label='labeled')
plt.scatter(data[target==0,0], data[target==0,1], c='blue', marker='o',
    ➤ alpha=0.5, label='class 0')
plt.scatter(data[target==1,0], data[target==1,1], c='red', marker='o',
    ➤ alpha=0.5, label='class 1')
plt.plot(xx, decision_boundary, linewidth = 2.0, c='black', linestyle =
    ➤ '--', label='log reg boundary')
plt.title("Uncertainty Sampling")
plt.legend()
plt.show()

if __name__ == "__main__":
    main()

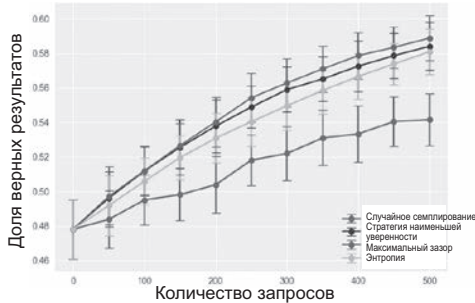
```

Как активное обучение, так и обучение с частичным привлечением учителя (semi-supervised learning) направлено на то, чтобы максимально эффективно использовать неразмеченные данные. Например, базовым методом частичного привлечения учителя является самообучение (self-training), при котором модель сначала обучается на небольшом количестве размеченных данных, а затем используется для классификации неразмеченных данных. Экземпляры неразмеченных данных, о классе которых модель обладает наибольшей уверенностью (most confident), вместе со своими предсказанными метками добавляются в обучающий набор, и процедура повторяется.

На рис. 7.11 сравниваются три метода семплирования по неуверенности: стратегия наименьшей уверенности, максимальный зазор и критерий на основе энтропии — с использованием случайной базовой подвыборки для набора данных из 20 групп новостей, классифицируемых с помощью логистической регрессии.

По сравнению со стандартными подходами все три метода обеспечивают более высокую долю верных результатов, тем самым подчеркивая преимущества активного обучения. На диаграмме справа можно видеть эффективность стратегии отбора по несогласию в комитете на примере набора данных MNIST. Комитет состоит из пяти экземпляров модели логистической регрессии. С базовым алгоритмом случайной подвыборки сравниваются два метода: энтропия голосов и средняя KL-дивергенция. Можно заметить, что средняя KL-дивергенция обеспечивает наибольшую долю верных результатов классификации. Каждый эксперимент повторялся 10 раз.

20 групп новостей: семплирование по не уверенности



MNIST: отбор по несогласию в комитете

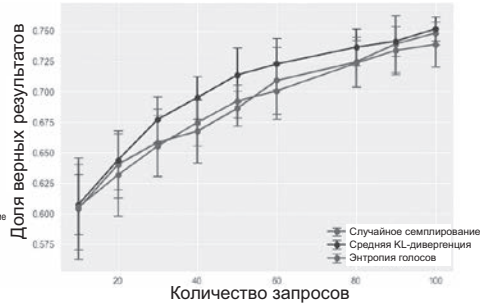


Рис. 7.11. Сравнение методов семплирования по не уверенности. Диаграмма слева показывает, что активное обучение работает лучше, чем случайное семплирование. График справа демонстрирует, что средняя KL-дивергенция показывает результаты лучшие, чем случайное семплирование

7.4. Выбор модели: настройка гиперпараметров

В процессе освоения машинного обучения мы часто сталкиваемся с разнообразием моделей. Выбор модели направлен на подбор оптимальной модели, которая имеет достаточное количество параметров (степеней свободы), чтобы избежать недообучения или переобучения на тренировочных данных. Его можно кратко охарактеризовать, используя принцип *бритвы Оккама*: выбирайте простейшую модель, которая хорошо объясняет данные. Другими словами, мы хотим сделать невыгодным усложнение модели во всех возможных вариантах решений.

Кроме того, мы хотели бы выйти за рамки описания исключительно обучающих данных: желательно, чтобы модель также умела объяснять будущие данные. Таким образом, наша цель — обучить модель, которая применима к новым и незнакомым данным. Поведение модели часто характеризуется ее гиперпараметрами, например количеством ближайших соседей или количеством кластеров в алгоритме *K* средних. Давайте рассмотрим несколько способов, с помощью которых можно найти оптимальные гиперпараметры для выбора модели.

Нам часто приходится оперировать в многомерном пространстве гиперпараметров, где требуется найти оптимальную точку, которая приводит к созданию модели, показывающей лучшие результаты на валидационном наборе данных. Например, в случае метода опорных векторов (SVM) можно попытаться применить различные ядра, параметры ядра и константы регуляризации.

Существуют различные стратегии настройки гиперпараметров. Наиболее простая из них называется *поиском по сетке* (grid search). Она представляет собой исчерпывающий поиск (полный перебор) по всем возможным комбинациям гиперпараметров. Используя поиск по сетке, чтобы найти правильный порядок величины, можно задавать значения гиперпараметров в логарифмической шкале

(например, 0.1, 1, 10, 100). Поиск по сетке хорошо работает для сравнительно небольших пространств гиперпараметров.

Альтернативой поиску по сетке является *случайный поиск* (random search) на основе семплирования гиперпараметров из соответствующих априорных распределений. Например, при нахождении числа кластеров K мы можем выбирать число K из дискретного экспоненциального распределения. Случайный поиск обладает вычислительным преимуществом, поскольку позволяет найти оптимальный набор параметров быстрее, чем полный перебор по сетке. Более того, для случайного поиска количество итераций может быть выбрано независимо от количества параметров, и поэтому добавление параметров, не влияющих на производительность, не снижает эффективность. Третья и самая интересная альтернатива, которую мы рассмотрим в следующем подразделе, называется *байесовской оптимизацией*.

7.4.1. Байесовская оптимизация

Вместо того чтобы исследовать пространство параметров случайным образом (в соответствии с выбранным распределением), было бы разумно применить активный подход к обучению, который выбирает непрерывные значения параметров таким образом, чтобы уменьшить неопределенность и обеспечить баланс между исследованием и использованием. Байесовская оптимизация гиперпараметров является автоматизированным байесовским подходом, использующим гауссовские процессы (ГП) для моделирования обобщающей способности алгоритма, — см. статью Джаспера Сноука (Jasper Snoek), Хьюго Ларошеля (Hugo Larochelle) и Райана П. Адамса (Ryan P. Adams) «Practical Bayesian Optimization of Machine Learning Algorithms» (Conference and Workshop on Neural Information Processing Systems, 2012).

Байесовская оптимизация исходит из предположения, что из ГП была выбрана подходящая функция эффективности (performance function), и поддерживает апостериорное распределение для этой функции по мере получения новых наблюдений: $f(x) \sim GP(m(x), \kappa(x, x'))$. Чтобы определиться, какие гиперпараметры исследовать далее, можно оптимизировать ожидаемое улучшение (expected improvement, EI) по сравнению с текущим наилучшим результатом или верхнюю доверительную границу гауссовского процесса (upper confidence bound, UCB). Оба подхода показали свою эффективность в минимизации количества вычислений функции, необходимых для нахождения глобального оптимума мультимодальных функций черного ящика (black-box functions).

Байесовская оптимизация, в отличие от использования локального градиента и аппроксимаций гессииана, задействует всю информацию, полученную из предыдущих оценок целевой функции. Эта особенность позволяет организовать автоматизированную процедуру поиска оптимума для невыпуклых функций, требующую относительно небольшого числа оценок функции за счет выполнения

дополнительных вычислений для определения следующей точки-кандидата. Преимущество такого решения особенно очевидно в тех случаях, когда выполнение вычислений требует больших затрат, например, при выборе гиперпараметров для глубоких нейронных сетей. Алгоритм байесовской оптимизации кратко представлен на рис. 7.12:

```

1: for  $n = 1, 2, \dots$  do
2:  выбор новой точки  $x_{n+1}$  путем оптимизации функции приобретения  $\alpha$ 
3:     $x_{n+1} = \arg \max_x \alpha(x; D_n, \theta)$ 
4:    запрос целевой функции для получения  $y_{n+1} = f(x_{n+1})$ 
5:    увеличение объема данных  $D_{n+1} = \{D_n, (x_{n+1}, y_{n+1})\}$ 
6:    обновление апостериорного распределения ГП и функции приобретения
7:  конец for

```

Рис. 7.12. Алгоритм байесовской оптимизации

В коде ниже байесовская оптимизация применяется для поиска гиперпараметров SVM и классификатора случайного леса (random forest classifier, RFC):

Листинг 7.7. Байесовская оптимизация для SVM и RFC

```

import numpy as np
import pandas as pd

import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier as RFC
from sklearn.svm import SVC

from bayes_opt import BayesianOptimization

np.random.seed(42)

# Загрузка набора данных и целевых значений
data, target = make_classification(
    n_samples=1000,
    n_features=45,
    n_informative=12,
    n_redundant=7
)
target = target.ravel()

def svccv(gamma):  ← SVM-классификатор
    val = cross_val_score(
        SVC(gamma=gamma, random_state=0),
        data, target, scoring='f1', cv=2
    ).mean()

    return val

```

```
def rfccv(n_estimators, max_depth):  ← Классификатор случайного леса
    val = cross_val_score(
        RFC(n_estimators=int(n_estimators),
            max_depth=int(max_depth),
            random_state=0
        ),
        data, target, scoring='f1', cv=2
    ).mean()
    return val

if __name__ == "__main__":

    gp_params = {"alpha": 1e-5}

    # SVM
    svcBO = BayesianOptimization(svccv,
        {'gamma': (0.00001, 0.1)})

    svcBO.maximize(init_points=3, n_iter=4, **gp_params)

    # Случайный лес
    rfcBO = BayesianOptimization(
        rfccv,
        {'n_estimators': (10, 300),
         'max_depth': (2, 10)
        }
    )
    rfcBO.maximize(init_points=4, n_iter=4, **gp_params)

    print('Final Results')
    print(svcBO.max)
    print(rfcBO.max)
```

Рисунок 7.13 демонстрирует байесовскую оптимизацию, примененную к SVM- и RF-классификаторам.

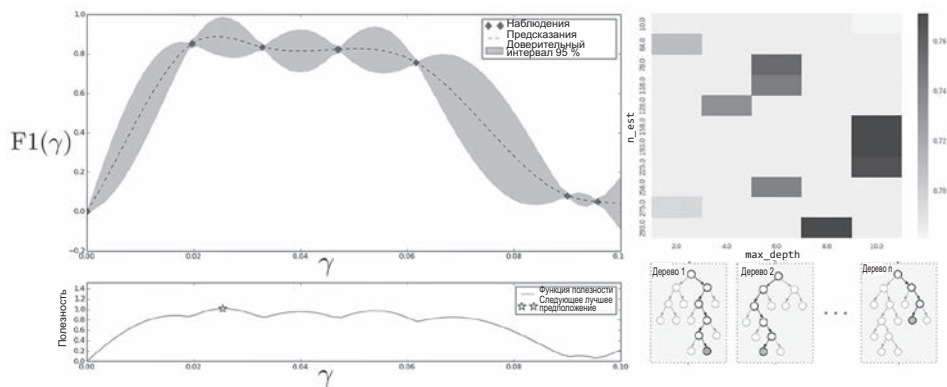


Рис. 7.13. Байесовская оптимизация в применении к SVM- и RF-классификаторам

В качестве целевой функции эффективности для задачи классификации была использована мера $F1$. На рисунке слева показана байесовская оптимизация меры $F1$ как функции параметра γ ядра радиальной базисной функции SVM: $K(x, x') = \exp\{-\gamma\|x - x'\|^2\}$, где γ — это точность, равная обратному значению дисперсии. Мы видим, что после всего семи итераций было найдено значение параметра γ , максимизирующее оценку $F1$. Пик функции полезности EI внизу указывает нам, какой эксперимент провести следующим. На рисунке справа показана байесовская оптимизация оценки $F1$ как функции максимальной глубины и количества оценщиков (estimator) классификатора по методу случайного леса. Исходя из тепловой карты, можно сказать, что максимальное значение $F1$ достигнуто для 158 оценщиков с глубиной, равной 10.

7.5. Ансамблевые методы

Ансамблевые методы — это метаалгоритмы, объединяющие несколько методов ML в одну прогнозирующую модель для уменьшения дисперсии (бэггинг), уменьшения смещения (бустинг) или улучшения точности предсказаний (стекинг). Ансамблевые методы можно разделить на две группы: *последовательные*, в которых базовые модели обучаются последовательно (например, AdaBoost), и *параллельные*, когда базовые модели обучаются параллельно (например, случайный лес). Основная идея последовательных методов заключается в том, чтобы использовать наличие зависимости между базовыми моделями, поскольку общая эффективность может быть повышена за счет присвоения ранее неправильно классифицированным примерам большего веса. Главная идея параллельных методов состоит в том, чтобы использовать независимость между базовыми моделями, так как ошибка может быть значительно уменьшена путем усреднения.

Большинство ансамблевых методов используют единый базовый алгоритм обучения, создающий однотипные базовые модели. Такие ансамбли известны как *однородные* (homogeneous). Также существуют отдельные методы, которые используют разнотипные модели. Эти ансамбли моделей называются *неоднородными* (heterogeneous). Чтобы ансамблевые методы были более эффективными, чем любая из своих составных частей, базовые модели должны быть как можно более точными и разнообразными.

7.5.1. Бэггинг

Название *бэггинг* происходит от словосочетания *бутстрэп-агрегирование* (bootstrap aggregation). Один из способов уменьшения дисперсии оценки — усреднение нескольких оценок. Например, мы можем обучить M различных деревьев решений f_m на разных подмножествах данных (выбранных случайным образом с возвращением) и вычислить среднее по ансамблю:

$$f(x) = \frac{1}{M} \sum_{m=1}^M f_m(x). \quad (7.32)$$

Чтобы получить подмножества данных для обучения базовых моделей, бэггинг использует бутстрэп-семплирование (bootstrap sampling). Чтобы объединить результаты базовых моделей, бэггинг для задачи классификации использует голосование, а для регрессии — усреднение. В следующем листинге показаны эксперименты по использованию бэггинга в качестве ансамблевого метаалгоритма:

Листинг 7.8. Ансамбль на основе бэггинга

```
import itertools
import numpy as np

import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

from sklearn import datasets

from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

from sklearn.ensemble import BaggingClassifier
from sklearn.model_selection import cross_val_score, train_test_split

from mlxtend.plotting import plot_learning_curves
from mlxtend.plotting import plot_decision_regions

def main():

    iris = datasets.load_iris()
    X, y = iris.data[:, 0:2], iris.target

    clf1 = DecisionTreeClassifier(criterion='entropy',
    ➤ max_depth=None)      ← Классификатор дерева решений
    clf2 = KNeighborsClassifier(n_neighbors=1)      ← KNN-классификатор

    bagging1 = BaggingClassifier(base_estimator =clf1, n_estimators=10,
    ➤ max_samples=0.8, max_features=0.8)
    bagging2 = BaggingClassifier(base_estimator=clf2, n_estimators=10,
    ➤ max_samples=0.8, max_features=0.8)

    label = ['Дерево решений', 'K-NN', 'Бэггинг-ансамбль деревьев решений',
    ➤ 'Бэггинг-ансамбль KNN' ]
    clf_list = [clf1, clf2, bagging1, bagging2]

    fig = plt.figure(figsize=(10, 8))
    gs = gridspec.GridSpec(2, 2)
    grid = itertools.product([0,1],repeat=2)

    for clf, label, grd in zip(clf_list, label, grid):
        scores = cross_val_score(clf, X, y, cv=3, scoring='accuracy')
        print("Accuracy: %.2f (+/- %.2f) [%s]" %(scores.mean(),
        ➤ scores.std(), label))
```

```

clf.fit(X, y)
ax = plt.subplot(gs[grd[0], grd[1]])
fig = plot_decision_regions(X=X, y=y, clf=clf, legend=2)
plt.title(label)

plt.show()
plt.savefig('./figures/bagging_ensemble.png')

# отрисовка кривой обучения
X_train, X_test, y_train, y_test = train_test_split(X, y,
    ➔ test_size=0.3, random_state=0)

plt.figure()
plot_learning_curves(X_train, y_train, X_test, y_test, bagging1,
    ➔ print_model=False, style='ggplot')
plt.show()
plt.savefig('./figures/bagging_ensemble_learning_curve.png')

# Размер ансамбля
num_est = list(map(int, np.linspace(1,100,20)))
bg_clf_cv_mean = []
bg_clf_cv_std = []
for n_est in num_est:
    print("num_est: ", n_est)
    bg_clf = BaggingClassifier(base_estimator=clf1, n_estimators=n_est,
        ➔ max_samples=0.8, max_features=0.8)
    scores = cross_val_score(bg_clf, X, y, cv=3, scoring='accuracy')
    bg_clf_cv_mean.append(scores.mean())
    bg_clf_cv_std.append(scores.std())

plt.figure()
(., caps, _) = plt.errorbar(num_est, bg_clf_cv_mean,
    ➔ yerr=bg_clf_cv_std, c='blue', fmt='-o', capsize=5)
for cap in caps:
    cap.set_markeredgewidth(1)
plt.ylabel('Доля верных результатов'); plt.xlabel('Размер ансамбля');
➔ plt.title('Бэггинг-ансамбль деревьев решений');
plt.show()
plt.savefig('./figures/bagging_ensemble_size.png')

if __name__ == "__main__":
    main()

```

На рис. 7.14 показана граница принятия решения для дерева решений и KNN-классификатора, а также их бэггинг-ансамбли в применении к набору данных iris. Дерево решений дает границы, параллельные осям координат, в то время как граница метода ближайших соседей при $k = 1$ близко прилегает к точкам данных. Бэггинг-ансамбли были обучены с использованием 10 базовых оценщиков с подвыборкой обучающих данных и подвыборкой признаков, равными 0.8. Бэггинг-ансамбль деревьев решений достиг более высокого уровня верных результатов, чем KNN-ансамбль, поскольку модель KNN менее чувствительна к возмущениям (perturbation) в обучающих данных. Вот почему такие алгоритмы

называются *устойчивыми* (stable). Объединение устойчивых алгоритмов менее выгодно, поскольку такой ансамбль не сможет улучшить обобщающую способность модели. Обратите также внимание, что граница принятия решения KNN-ансамбля выглядит аналогично границе принятия решения ансамбля деревьев как результата голосования. Рисунок также демонстрирует, как с увеличением размера ансамбля повышается доля верных результатов на тестовых данных. Из результатов кросс-валидации следует, что эффективность обучения растет примерно до 20 базовых оценщиков, а затем выходит на плато. Таким образом, увеличение числа базовых оценщиков свыше 20 только увеличивает вычислительную сложность, но не повышает верность результатов для набора данных iris. На рисунке также изображены кривые обучения для бэггинг-ансамбля деревьев. Они демонстрируют среднюю ошибку классификации в 0.15 на обучающих данных и U-образную кривую на тестовых данных. Наименьший зазор между ошибками на обучающих и тестовых данных достигается примерно при 80%-ном размере обучающего набора.

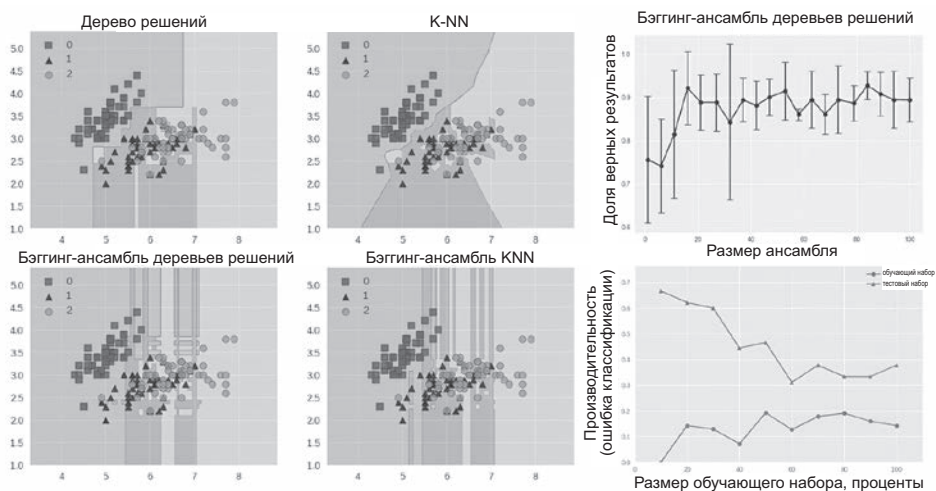


Рис. 7.14. Бэггинг-ансамбли в применении к набору данных цветков ириса по результатам работы кода выше

Широко используемый класс ансамблевых алгоритмов — это леса рандомизированных деревьев. В методе *случайного леса* (random forest) каждое дерево в ансамбле строится на основе выборки с возвращением (то есть бутстрэп-выборки) из обучающего набора. Кроме того, вместо использования всех признаков выбирается их случайное подмножество, что еще больше рандомизирует дерево. В результате смещение леса немного увеличивается, но из-за усреднения менее коррелированных деревьев его дисперсия уменьшается, что приводит к общему улучшению модели.

В *сверхслучайных деревьях* (extremely randomized trees) алгоритм становится еще более случайным: пороговые значения разделения рандомизируются. Вместо поиска оптимального порога для разделения пороговые значения определяются случайным образом для каждого признака-кандидата, и наилучшее из этих случайно сгенерированных пороговых значений выбирается в качестве правила разделения. Обычно это позволяет еще немного уменьшить дисперсию модели ценой несколько большего увеличения смещения.

7.5.2. Бустинг

Бустинг (boosting) относится к семейству метаалгоритмов, которые способны преобразовать слабые обучающие алгоритмы в сильные. Основная идея бустинга заключается в обучении последовательности слабых моделей (таких, которые лишь немного лучше случайного угадывания вроде небольших деревьев решений), используя взвешенные версии данных, где больший вес присваивается примерам, которые были неправильно классифицированы в предыдущих раундах. Затем предсказания для получения окончательного прогноза объединяются с помощью взвешенного большинства голосов (классификация) или взвешенной суммы (регрессия). Принципиальное различие между бустингом и методами комитета, такими как бэггинг, заключается в том, что базовые модели обучаются последовательно на взвешенной версии данных.

Алгоритм на рис. 7.15 описывает наиболее широко используемый вид бустинга — AdaBoost, который расшифровывается как адаптивный бустинг (adaptive boosting). Из него видно, что первый базовый классификатор $y_1(x)$ обучается с использованием одинаковых весовых коэффициентов w_n^1 . В последующих итерациях весовые коэффициенты w_n^m увеличиваются для неправильно классифицированных точек данных и уменьшаются для верно классифицированных точек. Величина ϵ_m представляет собой взвешенный коэффициент ошибок (error rate) каждого из базовых классификаторов. А потому весовые коэффициенты α_m придают больший вес более достоверным классификаторам:

- 1: Исходные веса данных $\{w_n\}$ to $\frac{1}{N}$
- 2: **for** $m = 1$ to M **do**
- 3: обучение классификатора $y_m(x)$ путем минимизации взвешенной функции ошибки J_m :
- 4: $J_m = \sum_{n=1}^N w_n^{(m)} 1[y_m(x_n) \neq t_n]$
- 5: вычисление $\epsilon_m = \sum_{n=1}^N w_n^{(m)} \frac{1[y_m(x_n) \neq t_n]}{\sum_{n=1}^N w_n^{(m)}}$
- 6: оценка $\alpha_m = \log\left(\frac{1-\epsilon_m}{\epsilon_m}\right)$
- 7: обновление весов данных: $w_n^{(m+1)} = w_n^{(m)} \exp\{\alpha_m 1[y_m(x_n) \neq t_n]\}$
- 8: **конец for**
- 9: Предсказание конечной модели: $Y_M(x) = \text{sign}\left(\sum_{m=1}^M \alpha_m y_m(x)\right)$

Рис. 7.15. Псевдокод алгоритма AdaBoost

Приведенный ниже код обучает классификатор AdaBoost с разным количеством моделей.

Листинг 7.9. Бустинг-ансамбль

```
import itertools
import numpy as np

import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

from sklearn import datasets

from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression

from sklearn.ensemble import AdaBoostClassifier
from sklearn.model_selection import cross_val_score, train_test_split

from mlxtend.plotting import plot_learning_curves
from mlxtend.plotting import plot_decision_regions

def main():

    iris = datasets.load_iris()
    X, y = iris.data[:, 0:2], iris.target

    # XOR-датасет
    # X = np.random.randn(200, 2)
    # y = np.array(map(int,np.logical_xor(X[:, 0] > 0, X[:, 1] > 0)))

    clf = DecisionTreeClassifier(criterion='entropy',
    ➤ max_depth=1)      ← Базовый классификатор

    num_est = [1, 2, 3, 10]
    label = ['AdaBoost (n_est=1)', 'AdaBoost (n_est=2)', 'AdaBoost
    ➤ (n_est=3)', 'AdaBoost (n_est=10)']

    fig = plt.figure(figsize=(10, 8))
    gs = gridspec.GridSpec(2, 2)
    grid = itertools.product([0,1],repeat=2)

    for n_est, label, grd in zip(num_est, label, grid):
        boosting = AdaBoostClassifier(base_estimator=clf,
        ➤ n_estimators=n_est)
        boosting.fit(X, y)
        ax = plt.subplot(gs[grd[0], grd[1]])
        fig = plot_decision_regions(X=X, y=y, clf=boosting, legend=2)
        plt.title(label)

    plt.show()
```

```

plt.savefig('./figures/boosting_ensemble.png')

# отрисовка кривой обучения
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    ➔ random_state=0)
boosting = AdaBoostClassifier(base_estimator=clf, n_estimators=10)

plt.figure()
plot_learning_curves(X_train, y_train, X_test, y_test, boosting,
    ➔ print_model=False, style='ggplot')
plt.show()
plt.savefig('./figures/boosting_ensemble_learning_curve.png')

num_est = list(map(int, np.linspace(1,100,20)))  ← Размер ансамбля
bg_clf_cv_mean = []
bg_clf_cv_std = []
for n_est in num_est:
    print("num_est: ", n_est)
    ada_clf = AdaBoostClassifier(base_estimator=clf, n_estimators=n_est)
    scores = cross_val_score(ada_clf, X, y, cv=3, scoring='accuracy')
    bg_clf_cv_mean.append(scores.mean())
    bg_clf_cv_std.append(scores.std())

plt.figure()
(, caps, _) = plt.errorbar(num_est, bg_clf_cv_mean,
    ➔ yerr=bg_clf_cv_std, c='blue', fmt='-o', capsize=5)
for cap in caps:
    cap.set_markeredgewidth(1)
plt.ylabel('Доля верных результатов'); plt.xlabel('Размер ансамбля');
    ➔ plt.title('Ансамбль AdaBoost');
plt.show()
plt.savefig('./figures/boosting_ensemble_size.png')

if __name__ == "__main__":
    main()

```

Результаты работы бустинг-ансамбля показаны на рис. 7.16. Каждая базовая модель является деревом решений глубиной 1, что позволяет классифицировать данные на основе порогового значения признака, которое делит пространство на две области, разделенные линейной поверхностью принятия решений, параллельной одной из осей. Рисунок демонстрирует, как повышается тестовая достоверность в зависимости от размера ансамбля. Также изображены кривые обучения на тестовых и обучающих данных.

Градиентный бустинг деревьев (gradient tree boosting) — это распространение бустинга на произвольные дифференцируемые функции потерь. Он может быть использован как для решения задач регрессии, так и для классификации. Градиентный бустинг строит модель последовательным образом:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x). \quad (7.33)$$

На каждом этапе дерево решений, $h_m(x)$, выбирается таким образом, чтобы минимизировать функцию потерь L при учете текущей модели $F_{m-1}(x)$:

$$F_m(x) = F_{m-1}(x) + \arg \min_h \sum_{i=1}^n L(y_i, F_{m-1}(x_i) + h(x_i)). \quad (7.34)$$

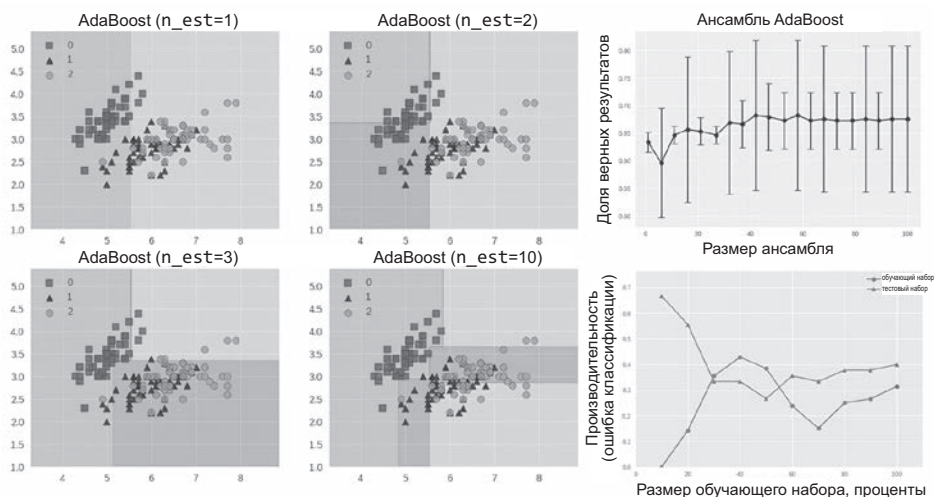


Рис. 7.16. Бустинг-ансамбль

Градиентный бустинг пытается решить эту минимизационную задачу численно с помощью кратчайшего спуска. Направление кратчайшего спуска — это отрицательный градиент функции потерь, рассчитанный при текущей модели F_{m-1} . Алгоритмы регрессии и классификации различаются по типу используемой функции потерь.

7.5.3. Стекинг

Стекинг — это метод ансамблевого обучения, который объединяет несколько классификационных или регрессионных моделей при помощи метаклассификатора, или метарегрессора. Модели базового уровня обучаются на полном обучающем наборе, а затем метамодель обучается на выходных данных модели базового уровня, рассматриваемых в качестве признаков. Базовый уровень часто состоит из разных алгоритмов обучения, поэтому стекинг-ансамбли часто неоднородны. Алгоритм на рис. 7.17 схематично показывает процесс стекинга:


```

1: Вход: обучающие данные  $D = \{x_i, y_i\}_{i=1}^m$ 
2: Выход: ансамблевый классификатор  $H$ 
3: Шаг 1: обучение классификаторов базового уровня
4: for  $t = 1$  to  $T$  do
5:   обучение  $h_t$  на  $D$ 
6: конец for
7: Шаг 2: создание нового набора данных предсказаний
8: for  $i = 1$  to  $m$  do
9:    $D_h = \{x'_i, y_i\}$ , where  $x'_i = \{h_1(x_i), \dots, h_T(x_i)\}$ 
10: конец for
11: Шаг 3: обучение метаклассификатора
12: обучение  $H$  на  $D_h$ 
13: return  $H$ 

```

Рис. 7.17. Псевдокод алгоритма стекинга

В следующем листинге стекинг-классификатор показан в действии:

Листинг 7.10. Стекинг-ансамбль

```

import itertools
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

from sklearn import datasets

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import RandomForestClassifier
from mlxtend.classifier import StackingClassifier

from sklearn.model_selection import cross_val_score, train_test_split

from mlxtend.plotting import plot_learning_curves
from mlxtend.plotting import plot_decision_regions

def main():

    iris = datasets.load_iris()
    X, y = iris.data[:, 1:3], iris.target

    clf1 = KNeighborsClassifier(n_neighbors=1)
    clf2 = RandomForestClassifier(random_state=1)
    clf3 = GaussianNB()
    lr = LogisticRegression()
    sclf = StackingClassifier(classifiers=[clf1, clf2, clf3],
                             meta_classifier=lr)  ← Стекинг-классификатор

```

```

label = ['KNN', 'Случайный лес', 'Наивный Байес',
        ➡ 'Стекинг-классификатор']
clf_list = [clf1, clf2, clf3, sclf]

fig = plt.figure(figsize=(10,8))
gs = gridspec.GridSpec(2, 2)
grid = itertools.product([0,1],repeat=2)

clf_cv_mean = []
clf_cv_std = []
for clf, label, grd in zip(clf_list, label, grid):

    scores = cross_val_score(clf, X, y, cv=3, scoring='accuracy')
    print("Accuracy: %.2f (+/- %.2f) [%s]" %(scores.mean(),
    ➡ scores.std(), label))
    clf_cv_mean.append(scores.mean())
    clf_cv_std.append(scores.std())

    clf.fit(X, y)
    ax = plt.subplot(gs[grd[0], grd[1]])
    fig = plot_decision_regions(X=X, y=y, clf=clf)
    plt.title(label)

plt.show()
#plt.savefig("./figures/ensemble_stacking.png")

#plot classifier accuracy
plt.figure()
(_, caps, _) = plt.errorbar(range(4), clf_cv_mean, yerr=clf_cv_std,
➡ c='blue', fmt='-o', capsize=5)
for cap in caps:
    cap.set_markeredgewidth(1)
plt.xticks(range(4), ['KNN', 'RF', 'NB', 'Stacking'],
            rotation='vertical')
plt.ylabel('Доля верных результатов'); plt.xlabel('Классификатор');
➡ plt.title('Стекинг-ансамбль');
plt.show()
#plt.savefig("./figures/stacking_ensemble_size.png")

# отрисовка кривой обучения
X_train, X_test, y_train, y_test = train_test_split(X, y,
➡ test_size=0.3, random_state=0)

plt.figure()
plot_learning_curves(X_train, y_train, X_test, y_test, sclf,
➡ print_model=False, style='ggplot')
plt.show()
#plt.savefig("./figures/stacking_ensemble_learning_curve.png")

if __name__ == "__main__":
    main()

```

Результаты работы стекинг-ансамбля показаны на рис. 7.18. Ансамбль состоит из трех базовых классификаторов — KNN, случайного леса и наивного

байесовского, прогнозы которых объединяются с помощью логистической регрессии в качестве метаклассификатора. Обращает на себя внимание смещение границ принятия решений, достигаемое стекинг-классификатором. Из рисунка также следует, что стекинг обеспечивает более высокую верность результатов, чем отдельные классификаторы, и, судя по кривым обучения, он не показывает признаков переобучения:

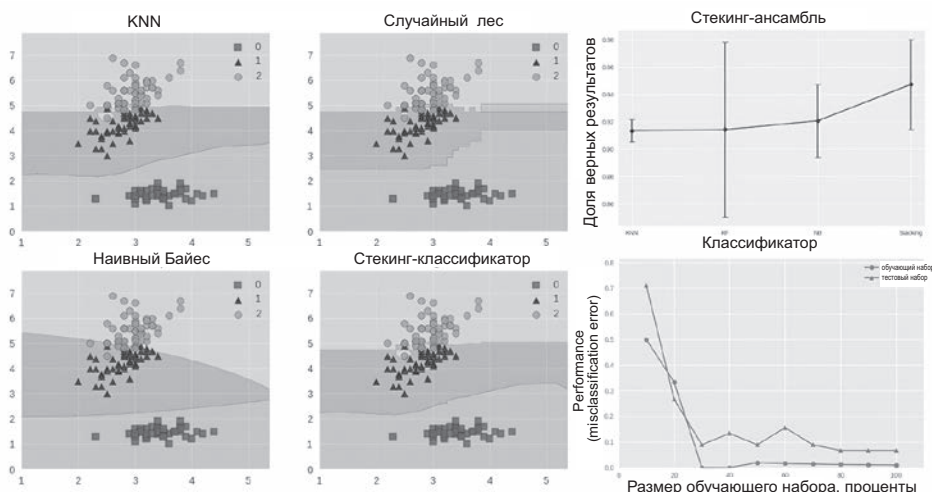


Рис. 7.18. Стекинг-ансамбль

7.6. Исследования в области ML: алгоритмы обучения с учителем

В этом разделе мы обсудим дополнительные соображения и материалы исследований по темам, представленным ранее в этой книге. В разделе, посвященном классификации, мы вывели несколько классических алгоритмов и заложили прочную основу для разработки новых алгоритмов. В случае перцептрона увидели, как можно конструировать алгоритм, вводя различные функции потерь, что приводит к слегка отличающимся правилам обновления. Более того, сама функция потерь может быть взвешена для учета несбалансированных наборов данных. Аналогично в случае SVM у нас есть свобода выбора подходящей функции ядра. Мы рассмотрели многоклассовую SVM, однако для обнаружения аномалий можно использовать одноклассовую SVM с ядром RBF.

Для логистической регрессии существует несколько дополнительных, отличных от SGD, алгоритмов оптимизации (solver), в частности liblinear (использующий координатный спуск), newton-cg (метод второго порядка, способный привести к более быстрой сходимости) и LBFGS (мощный метод оптимизации, требующий меньше памяти и вычислительных ресурсов). Похожим образом

существует несколько методов в развитие наивного байесовского алгоритма, в зависимости от типа моделируемых данных, таких как мультиномиальный наивный байесовский алгоритм и гауссовский наивный байесовский алгоритм. Их можно вывести по аналогии с наивным байесовским алгоритмом Бернулли, который мы подробно рассмотрели.

В разделе, посвященном регрессии, мы впервые столкнулись с непараметрической моделью, а именно с KNN-регрессором. Мы исследуем этот богатый набор моделей в главе 8, когда будем обсуждать байесовские непараметрические модели, в которых число параметров растет с увеличением объема данных. Например, в процессе Дирихле (Dirichlet process, DP) СММ имеется потенциально бесконечное число состояний, где вероятностная мера сконцентрирована на нескольких состояниях, подкрепленных данными, — см., например, статью Эмили Б. Фокс (Emily B. Fox) и др. «A Sticky HDP-HMM with Application to Speaker Diarization» (Annals of Applied Statistics, 2011).

Мы рассмотрели преимущества использования иерархических моделей. Эта методика может быть использована в тех случаях, когда различные группы данных имеют общие характеристики, что позволяет совместно использовать статистическую силу этих групп. Иерархические модели часто обеспечивают более высокую верность при меньшем количестве наблюдений, например, иерархические СММ и иерархические смеси моделей с процессом Дирихле (hierarchical Dirichlet process mixture model), — см., например, работу Джейсона Чанга (Jason Chang) и Джона У. Фишера III (John W. Fisher III) «Parallel Sampling of HDPs using Sub-Cluster Splits» (Conference and Workshop on Neural Information Processing Systems, 2014).

Мы коснулись масштабируемого ML, когда обсуждали ранжирование страниц, связанное с вычислениями на миллионах веб-страниц. Масштабируемое ML — это важная область исследований. В промышленных масштабах для обработки больших объемов данных обычно используется инструментарий Spark ML. Однако чтобы понять, как строятся распараллеливаемые алгоритмы, полезно попрактиковаться в их реализации с нуля. За дополнительной информацией о масштабируемых алгоритмах обращайтесь к работам Рона Беккермана (Ron Bekkerman), Михаила Биленко (Mikhail Bilenko) и Джона Лэнгфорда (John Langford) «Scaling Up Machine Learning: Parallel and Distributed Approaches» (2011).

Существует несколько обобщений СММ. Одним из примеров является скрытая полумарковская модель (hidden semi-Markov model), которая моделирует переходы между состояниями переменной длительности, обычно встречающиеся в геномных данных. Другим примером является иерархическая СММ, которая моделирует данные с иерархической структурой, часто встречающейся при работе с речью. Наконец, существуют факторные СММ (factorial HMM), которые состоят из множества взаимосвязанных цепей Маркова, работающих параллельно для отражения различных аспектов входного сигнала.

В рамках активного обучения есть несколько значимых областей исследований. Одной из них является обучение с частичным привлечением учителя, которое может быть использовано, например, при самообучении. Сначала модель проходит обучение на небольшом количестве размеченных данных, а затем расширяет свой собственный набор данных новыми, наиболее уверенно классифицированными учебными примерами. Другим направлением исследований является изучение компромисса между исследованием и использованием, который обычно присутствует в обучении с подкреплением. В частности, алгоритм активного обучения должен проявлять инициативу и разборчивость при исследовании примеров, в метке которых он не уверен. Наконец, еще одной важной областью исследований является субмодулярная оптимизация (submodular optimization) — см. книгу Андреаса Крауса (Andreas Kraus) «Optimizing Sensing: Theory and Applications» (Carnegie Mellon University, School of Computer Science, 2008). В задачах с фиксированным бюджетом на сбор данных целесообразно представить целевую функцию для отбора данных в виде субмодулярной функции, которая затем может быть оптимизирована с использованием жадных алгоритмов с гарантиями производительности.

В области выбора моделей с различным количеством параметров интересным решением является метод обратимого прыжка (reversible jump, RJ) марковской цепи Монте-Карло (MCMC) на основе семплирования. RJ-MCMC производит выборку в пространствах параметров разной размерности, например при выборе наилучшей модели гауссовой смеси с различным количеством кластеров K . Интересный момент возникает в семплировании по методу Метрополиса — Гастингса, когда мы проводим выборку из распределений с разными размерностями. RJ-MCMC дополняет пространство низкой размерности дополнительными случайными величинами, чтобы оба пространства имели общую меру, — см. «Tutorial on Transdimensional MCMC» Питера Дж. Грина (Peter J. Green) (Highly Structured Stochastic Systems, 2003). Наконец, ансамблевые методы в сочетании с выбором модели и настройкой гиперпараметров являются моделями-победителями любого соревнования по анализу данных!

7.7. Упражнения

7.1. Объясните, как работает температурный softmax для различных значений параметра температуры T .

7.2. В алгоритме вперед — назад СММ сохраните переменную скрытого состояния z (как часть класса НММ) и сравните выведенное значение z с эталонным.

Итоги

- Марковские модели обладают марковским свойством: будущие состояния обусловлены текущим состоянием, но не зависят от прошлых. Другими словами, текущее состояние служит достаточной статистикой для следующего состояния.

- Ранг страницы — это стационарное распределение цепи Маркова, описываемое переходной матрицей, которая является рекуррентной и апериодической. В промышленном масштабе алгоритм ранжирования страниц вычисляется с использованием метода степенных итераций (power iteration).
- Скрытые марковские модели моделируют данные временных рядов и состоят из марковской цепи скрытых состояний, матрицы переходов между скрытыми состояниями и матрицы эмиссии, которая моделирует наблюдаемые данные, производимые каждым состоянием. Логический вывод выполняется с использованием либо ЕМ-алгоритма, либо алгоритма вперед-назад с декодером последовательности максимального правдоподобия Витерби.
- Существуют две основные стратегии обучения на несбалансированных данных: уменьшение выборки класса большинства и увеличение выборки класса меньшинства. Среди дополнительных методов можно назвать введение в функцию потерь весовых коэффициентов классов.
- При активном обучении алгоритм ML выбирает образцы данных для обучения таким образом, чтобы максимально ускорить процесс обучения при использовании меньшего числа обучающих примеров. Существуют две основные стратегии запросов: семплирование по неувренности и запросы через комитет.
- Компромисс между смещением и дисперсией требует, чтобы среднеквадратичная ошибка была равна квадрату смещения плюс дисперсия. Минимальная дисперсия определяется величиной, обратной информации Фишера, которая измеряет кривизну логарифмического правдоподобия.
- При выборе модели мы склонны следовать принципу бритвы Оккама и выбирать наиболее простую модель, которая хорошо объясняет данные. При оптимизации гиперпараметров в дополнение к поиску по сетке и случайному поиску байесовская оптимизация использует метод активного обучения, который снижает неопределенность и обеспечивает баланс между исследованием и использованием.
- Ансамблевые методы — это метаалгоритмы, которые объединяют несколько методов машинного обучения в одну предсказательную модель для уменьшения дисперсии (бэггинг), уменьшения смещения (бустинг) или улучшения предсказаний (стекинг).

Часть 3

Обучение без учителя

В третьей части книги мы рассмотрим алгоритмы обучения без учителя. Обучение без учителя происходит, если отсутствуют обучающие метки. В этом случае нас часто интересует выявление закономерностей в данных и изучение представлений данных.

В главе 8 мы начнем с рассмотрения байесовского непараметрического расширения алгоритма K средних, вслед за которым обсудим EM-алгоритм для моделей гауссовой смеси. Затем рассмотрим два различных метода понижения размерности, а именно PCA и t-SNE в применении к обучению на базе многообразий (manifold learning) изображений.

В главе 9 продолжим обсуждение отдельных алгоритмов обучения без учителя. Мы начнем с рассмотрения латентного размещения Дирихле для обучения тематических моделей, далее перейдем к оценке плотности и алгоритмам структурного обучения, а завершим методом имитации отжига и генетическими алгоритмами. В заключение сделаем обзор исследовательских публикаций по ML, уделив особое внимание обучению без учителя.

8

Основные алгоритмы обучения без учителя

В этой главе

- ✓ Метод K средних с процессом Дирихле
- ✓ Модели гауссовой смеси
- ✓ Снижение размерности

В предыдущих главах мы рассматривали алгоритмы обучения с учителем для классификации и регрессии. В этой главе сосредоточимся на алгоритмах обучения без учителя. *Обучение без учителя* происходит тогда, когда отсутствуют обучающие метки. В данном случае мы заинтересованы в выявлении закономерностей в данных и изучении представлений данных. Области применения обучения без учителя охватывают задачи от кластеризации клиентских сегментов в электронной коммерции до извлечения признаков из данных изображений. В этой главе мы начнем с рассмотрения байесовского непараметрического расширения алгоритма K средних, за которым следует ЕМ-алгоритм для моделей гауссовой смеси (Gaussian mixture model, GMM). Затем рассмотрим два различных метода снижения размерности, а именно PCA и t-SNE, используемый для обучения на базе многообразий изображений. Алгоритмы, описанные в этой главе, были выбраны из-за их математической глубины и применимости к задачам из реальной жизни.

8.1. Метод К средних с процессом Дирихле

Метод К средних с процессом Дирихле (Dirichlet process, DP) является байесовским непараметрическим расширением алгоритма *К средних*. *Метод К средних* (K-means) — это алгоритм кластеризации, который может быть применен, например, для сегментации клиентской базы, когда клиенты группируются по истории покупок, интересам и географическому положению. Метод DP-means аналогичен K-means, за исключением того, что новые кластеры создаются всякий раз, когда точка данных находится достаточно далеко от всех существующих кластерных центроидов. Следовательно, количество кластеров растет вместе с объемом данных. DP-means сходится к локальному оптимуму похожей на K-means целевой функции, которая включает штраф за количество кластеров.

Мы выбираем кластер K таким образом, чтобы значение следующего выражения было минимальным:

$$\arg \min_k \left\{ \|x_i - \mu_1\|^2, \dots, \|x_i - \mu_k\|^2, \lambda \right\}. \quad (8.1)$$

Результирующее обновление аналогично этапу переназначения метода *К средних*, во время которого мы переназначаем точку кластеру, соответствующему ближайшему среднему значению, или создаем новый кластер, если квадрат евклидова расстояния больше λ . Алгоритм DP-means кратко представлен на рис. 8.1.

- 1: Иниц. $K = 1, l_1 = \{x_1, \dots, x_n\}$ и глобальное среднее μ_1
- 2: Иниц. меток $z_i = 1$ для всех $i = 1, \dots, n$
- 3: Иниц. $\lambda = \text{kpp_init}(X, K_{\text{init}})$
- 4: Повторять до сходимости:
- 5: **for each** (для каждого) x_i :
- 6: вычислить $d_{ic} = \|x_i - \mu_c\|^2$ for $c = 1, \dots, K$
- 7: если $\min_c d_{ic} > \lambda$, присвоить $K = K + 1$ and $\mu_k = x_i$
- 8: иначе присвоить $z_i = \arg \min_c d_{ic}$
- 9: для каждого кластера $l_j = \{x_i | z_i = j\}$, вычислить $\mu_j = \frac{1}{|l_j|} \sum_{x \in l_j} x$
- 10: Вычислить целевую функцию: $\sum_{c=1}^K \sum_{x \in l_c} \|x - \mu_c\|^2 + \lambda K$

Рис. 8.1. Псевдокод алгоритма *DP-means*

Для оценки эффективности кластеризации мы можем использовать следующие показатели: нормированная взаимная информация (normalized mutual information, *NMI*), вариация информации (variation of information, *VI*) и скорректированный индекс Рэнда (adjusted Rand index, *ARI*). Показатель *NMI* определяется следующим образом:

$$NMI(X, Y) = \frac{I(X; Y)}{\frac{H(X) + H(Y)}{2}}. \quad (8.2)$$

Здесь $H(X)$ — энтропия X , а $I(X; Y)$ — это взаимная информация между истинными метками X (когда они доступны) и вычисленными метками Y . Более подробное описание информационных мер см. в книге Томаса М. Кавера (Thomas M. Cover) и Джой А. Томас (Joy A. Thomas) «Elements of Information Theory» (Wiley, 2006).

Пусть $p_{XY}(I, j) = |x_i \cap y_j|/N$ — вероятность того, что метка принадлежит кластеру x_i в X и y_j в Y . Определим по аналогии $p_X(i) = |x_i|/N$ и $p_Y(j) = |y_j|/N$. Тогда мы получаем следующее уравнение:

$$I(X; Y) = \sum_i \sum_j p_{XY}(i, j) \log \frac{p_{XY}(i, j)}{p_X(i)p_Y(j)}. \quad (8.3)$$

Таким образом, значения NMI лежат в диапазоне от 0 до 1, причем более высокие показатели указывают на более сходные множества меток. Вариация информации (VI) определяется следующим образом:

$$VI(X; Y) = H(X) + H(Y) - 2I(X; Y) = H(X|Y) + H(Y|X). \quad (8.4)$$

Таким образом, VI уменьшается по мере увеличения совпадения наборов меток X и Y . ARI дает меру сходства между двумя кластерами, рассматривая все пары объектов в прогнозируемых и эталонных кластерах и подсчитывая пары, отнесенные к одному и тому же или разным кластерам:

$$ARI = \frac{RI - E[RI]}{\max RI - E[RI]}; \quad (8.5)$$

$$RI = \frac{TP + TN}{TP + FP + FN + TN}.$$

Таким образом, значения ARI приближаются к 1 для сходных кластерных разбиений. Давайте реализуем алгоритм K средних с использованием процесса Дирихле в коде!

Листинг 8.1. Метод K средних с процессом Дирихле

```
import numpy as np
import matplotlib.pyplot as plt

import time
from sklearn import metrics
from sklearn.datasets import load_iris

np.random.seed(42)

class dpmeans:
    def __init__(self, X):  ← Инициализация параметров DP-means
```

```

self.K = 1
self.K_init = 4
self.d = X.shape[1]
self.z = np.mod(np.random.permutation(X.shape[0]),self.K)+1
self.mu = np.random.standard_normal((self.K, self.d))
self.sigma = 1
self.nk = np.zeros(self.K)
self.pik = np.ones(self.K)/self.K

self.mu = np.array([np.mean(X,0)])  ← Инициализация среднего

self.Lambda = self.kpp_init(X,self.K_init)  ← Инициализация λ

self.max_iter = 100
self.obj = np.zeros(self.max_iter)
self.em_time = np.zeros(self.max_iter)

def kpp_init(self,X,k):  ← Инициализация K-means++

    [n,d] = np.shape(X)
    mu = np.zeros((k,d))
    dist = np.inf*np.ones(n)

    mu[0,:] = X[int(np.random.rand()*n-1),:]
    for i in range(1,k):
        D = X-np.tile(mu[i-1,:],(n,1))
        dist = np.minimum(dist, np.sum(D*D,1))
        idx = np.where(np.random.rand() <
np.cumsum(dist/float(sum(dist))))
        mu[i,:] = X[idx[0][0],:]
        Lambda = np.max(dist)

    print(" Lambda: ", Lambda)

    return Lambda  ← λ — это максимальное расстояние для K-means++

def fit(self,X):

    obj_tol = 1e-3
    max_iter = self.max_iter
    [n,d] = np.shape(X)

    obj = np.zeros(max_iter)
    em_time = np.zeros(max_iter)
    print('running dpmeans...')

    for iter in range(max_iter):
        tic = time.time()
        dist = np.zeros((n,self.K))

        for kk in range(self.K):
            Xm = X - np.tile(self.mu[kk,:],(n,1))
            dist[:,kk] = np.sum(Xm*Xm,1)

```

Шаг назначения

```

dmin = np.min(dist,1)
self.z = np.argmax(dist,1)
idx = np.where(dmin > self.Lambda)

if (np.size(idx) > 0):
    self.K = self.K + 1
    self.z[idx[0]] = self.K-1 # метки кластеров из [0,...,K-1]
    self.mu = np.vstack([self.mu,np.mean(X[idx[0],:],0)])
    Xm = X - np.tile(self.mu[self.K-1,:],(n,1))
    dist = np.hstack([dist, np.array([np.sum(Xm*Xm,1)]).T])

self.nk = np.zeros(self.K)
for kk in range(self.K):
    self.nk[kk] = self.z.tolist().count(kk)
    idx = np.where(self.z == kk)
    self.mu[kk,:] = np.mean(X[idx[0],:],0)

self.pik = self.nk/float(np.sum(self.nk))

for kk in range(self.K):
    idx = np.where(self.z == kk)
    obj[iter] = obj[iter] + np.sum(dist[idx[0],
    ➡ kk],0)
obj[iter] = obj[iter] + self.Lambda * self.K

if (iter > 0 and np.abs(obj[iter]-obj[iter-1]) <
    ➡ obj_tol*obj[iter]):
    print('converged in %d iterations\n'% iter)
    break
em_time[iter] = time.time()-tic
# конец for
self.obj = obj
self.em_time = em_time
return self.z, obj, em_time

def compute_nmi(self, z1, z2):
    n = np.size(z1)
    k1 = np.size(np.unique(z1))
    k2 = np.size(np.unique(z2))

    nk1 = np.zeros((k1,1))
    nk2 = np.zeros((k2,1))

    for kk in range(k1):
        nk1[kk] = np.sum(z1==kk)
    for kk in range(k2):
        nk2[kk] = np.sum(z2==kk)

    pk1 = nk1/float(np.sum(nk1))
    pk2 = nk2/float(np.sum(nk2))

    nk12 = np.zeros((k1,k2))
    for ii in range(k1):

```

Обновление меток

Шаг обновления

Вычисление целевой функции

Проверка сходимости

Вычисление нормированной взаимной информации

```

        for jj in range(k2):
            nk12[ii,jj] = np.sum((z1==ii)*(z2==jj))
        pk12 = nk12/float(n)

        Hx = -np.sum(pk1 * np.log(pk1 + np.finfo(float).eps))
        Hy = -np.sum(pk2 * np.log(pk2 + np.finfo(float).eps))

        Hxy = -np.sum(pk12 * np.log(pk12 + np.finfo(float).eps))

        MI = Hx + Hy - Hxy;
        nmi = MI/float(0.5*(Hx+Hy))

    return nmi

def generate_plots(self,X):

    plt.close('all')
    plt.figure(0)
    for kk in range(self.K):
        #idx = np.where(self.z == kk)
        plt.scatter(X[self.z == kk,0], X[self.z == kk,1], \
                    s = 100, marker= 'o', c = np.random.rand(3,), \
                    ⇨ label = str(kk))

    # конец for
    plt.xlabel('X1')
    plt.ylabel('X2')
    plt.legend()
    plt.title('Кластеры DP-means')
    plt.grid(True)
    plt.show()

    plt.figure(1)
    plt.plot(self.obj)
    plt.title('Целевая функция DP-means')
    plt.xlabel('Итерации')
    plt.ylabel('Квадрат расстояния с учетом штрафа l2')
    plt.grid(True)
    plt.show()

if __name__ == "__main__":

    iris = load_iris()
    X = iris.data
    y = iris.target

    dp = dpmeans(X)
    labels, obj, em_time = dp.fit(X)
    dp.generate_plots(X)

    nmi = dp.compute_nmi(y,labels)
    ari = metrics.adjusted_rand_score(y,labels)

    print("NMI: %.4f" % nmi)
    print("ARI: %.4f" % ari)

```

Результаты работы алгоритма DP-means показаны на рис. 8.2. Эффективность оценивалась на наборе данных *iris*, и значения *NMI* и *ARI* достаточно высокие. В следующем разделе мы обсудим популярный алгоритм кластеризации, который моделирует каждый кластер как гауссово распределение, принимая во внимание не только среднее значение, но и информацию о ковариации.

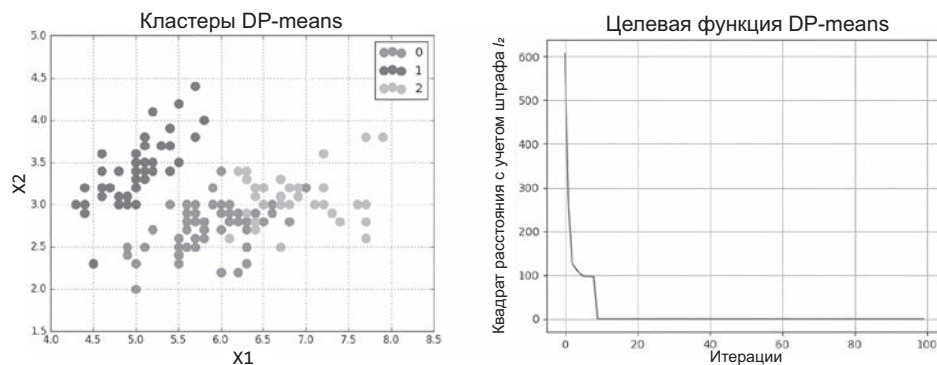


Рис. 8.2. DP-means: центры кластеров (слева) и целевая функция (справа) для набора данных *iris*

8.2. Модели гауссовой смеси

Смесовые модели (mixture models) обычно используются для моделирования сложных вероятностных распределений. Например, предметом вашего интереса могут быть закономерности в данных переписи населения, состоящих из информации о возрасте человека, доходе, роде занятий и других параметров. Если построить график полученных данных в многомерном пространстве, то скорее всего, обнаружится неоднородность плотности, представленная группами или кластерами точек данных. Мы можем смоделировать каждый кластер, используя базовое распределение вероятностей. Смесовая модель является выпуклой комбинацией K базовых моделей. В случае моделей гауссовой смеси базовое распределение является гауссовым и может быть записано следующим образом:

$$p(x_i | \theta) = \sum_{k=1}^K \pi_k p_k(x_i | \theta) = \sum_{k=1}^K \pi_k \mathcal{N}(x_i | \mu_k, \Sigma). \quad (8.6)$$

Здесь π_k — веса компонент, удовлетворяющие условиям $0 \leq \pi_k \leq 1$ и $\sum \pi_k = 1$. В отличие от алгоритма K средних, который моделирует только кластерные средние, GMM также моделирует кластерную ковариацию. Таким образом, GMM способен более точно описывать данные.

8.2.1. Алгоритм максимизации ожидания

Алгоритм максимизации ожидания (expectation maximization, EM) предоставляет способ вычисления ML/MAP-оценок, когда имеются ненаблюдаемые скрытые переменные или отсутствующие данные. EM использует тот момент, что если бы данные были полностью доступны, то ML/MAP-оценки было бы легко вычислить. В частности, EM является итеративным алгоритмом, который чередует вывод скрытых переменных с учетом параметров (Е-шаг) и последующую оптимизацию параметров с учетом полученных данных (М-шаг).

В EM-алгоритме мы определяем полное логарифмическое правдоподобие $l_c(\theta)$, где x_i — наблюдаемые случайные величины, а z_i — скрытые переменные. Поскольку мы не знаем z_i , то мы не можем вычислить $p(x_i, z_i | \theta)$, но у нас есть возможность вычислить ожидаемое значение $l_c(\theta)$ относительно параметров $\theta^{(k-1)}$ из предыдущей итерации:

$$l_c(\theta) = \sum_{i=1}^N \log p(x_i, z_i | \theta). \quad (8.7)$$

Целью Е-шага является вычисление $Q(\theta, \theta^{(k-1)})$, от которого зависят ML/MAP-оценки. Цель М-шага состоит в повторном вычислении θ путем нахождения ML/MAP-оценок:

$$\begin{aligned} \text{Е-шаг} : Q(\theta, \theta^{(k-1)}) &= E_{\theta^{(k-1)}} [l_c(\theta) | D, \theta^{(k-1)}]. \\ \text{М-шаг} : \theta^{(k)} &= \arg \max_{\theta} Q(\theta, \theta^{(k-1)}) + \log p(\theta). \end{aligned} \quad (8.8)$$

Чтобы вывести формулу EM-алгоритма для GMM, надо сначала получить формулу для ожидаемого значения функции логарифмического правдоподобия наших данных:

$$\begin{aligned} Q(\theta, \theta^{(k-1)}) &= E \left[\sum_i \log p(x_i, z_i | \theta) \right] = \\ &= \sum_i E \left[\log \left[\prod_{k=1}^K (\pi_k p(x_i | \theta_k))^{1[z_i=k]} \right] \right] = \\ &= \sum_i \sum_k E[1[z_i=k]] \log [\pi_k p(x_i | \theta_k)] = \\ &= \sum_i \sum_k p(z_i=k | x_i, \theta^{(k-1)}) \log [\pi_k p(x_i | \theta_k)] = \\ &= \sum_i \sum_k r_{ik} \log \pi_k + \sum_i \sum_k r_{ik} \log p(x_i | \theta_k). \end{aligned} \quad (8.9)$$

Здесь $r_{ik} = p(z_i = k | x_i, \theta^{(k-1)})$ — «мягкое» назначение (soft assignment) точки x_i кластеру k .

Теперь мы переходим к Е-шагу. При имеющихся $\theta^{(k-1)}$ мы хотим вычислить «мягкие» назначения:

$$\begin{aligned} r_{ik} &= p(z_i = k | x_i, \theta^{(k-1)}) = \\ &= \frac{p(z_i = k, x_i | \theta^{(k-1)})}{\sum_{k=1}^K p(z_i = k, x_i | \theta^{(k-1)})} = \\ &= \frac{p(x_i | z_i = k, \theta^{(k-1)})\pi_k}{\sum_{k=1}^K p(x_i | z_i = k, \theta^{(k-1)})\pi_k}. \end{aligned} \quad (8.10)$$

В формуле 8.10 $\pi_k = p(z_i = k)$ — пропорции смеси.

На М-шаге мы максимизируем Q по отношению к параметрам модели π и θ_k . Во-первых, давайте найдем π , которое максимизирует лагранжиан:

$$\begin{aligned} \frac{\partial Q}{\partial \pi_k} &= \frac{\partial}{\partial \pi_k} \left[\sum_i \sum_k r_{ik} \log \pi_k + \lambda \left(1 - \sum_k \pi_k \right) \right] = \\ &= \sum_i r_{ik} \frac{1}{\pi_k} - \lambda = 0. \end{aligned} \quad (8.11)$$

Подставляя это выражение в ограничивающее условие, мы получаем следующее уравнение:

$$\sum_k \pi_k = 1 \Rightarrow \sum_k \frac{1}{\lambda} \sum_i r_{ik} = 1 \Rightarrow \lambda = \sum_i \sum_k r_{ik} = \sum_i 1 = N. \quad (8.12)$$

Следовательно, $\pi_k = 1/\lambda \sum_i r_{ik} = 1/N \sum_i r_{ik}$. Чтобы найти оптимальные параметры $\theta_k = \{\mu_k, \Sigma_k\}$, требуется оптимизировать члены Q , которые зависят от θ_k :

$$\begin{aligned} l(\mu_k, \Sigma_k) &= \sum_i r_{ik} \log p(x_i | \theta_k) \propto \\ &\propto -\frac{1}{2} \sum_i r_{ik} \left[\log |\Sigma_k| + (x_i - \mu_k)^T \Sigma_k^{-1} (x_i - \mu_k) \right]. \end{aligned} \quad (8.13)$$

Чтобы найти оптимальное значение μ_k , продифференцируем это выражение. Сначала сосредоточим внимание на втором слагаемом внутри суммы. Мы можем произвести подстановку $y_i = x_i - \mu_k$:

$$\begin{aligned} \frac{\partial}{\partial \mu_k} (x_i - \mu_k)^T \Sigma_k^{-1} (x_i - \mu_k) &= \frac{\partial}{\partial y_i} y_i^T \Sigma^{-1} y_i \frac{\partial y_i}{\partial \mu_k} = \\ &= -1 \times (\Sigma^{-1} + \Sigma^{-T}) y_i. \end{aligned} \quad (8.14)$$

Подставляя выражение в формулу 8.14, мы получаем следующую запись:

$$\begin{aligned} \frac{\partial}{\partial \mu_k} l(\mu_k, \Sigma_k) &\propto -\frac{1}{2} \sum_i r_{ik} [-2\Sigma^{-1}(x_i - \mu_k)] = \\ &= \Sigma^{-1} \sum_i r_{ik} (x_i - \mu_k) = 0. \end{aligned} \quad (8.15)$$

Из этого следует следующее уравнение:

$$\sum_i r_{ik} (x_i - \mu_k) = 0 \Rightarrow \mu_k = \frac{\sum_i r_{ik} x_i}{\sum_i r_{ik}}. \quad (8.16)$$

Чтобы вычислить оптимальное значение Σ_k , мы можем использовать тождество для следа матрицы:

$$x^T A x = \text{tr}(x^T A x) = \text{tr}(x x^T A) = \text{tr}(A x x^T). \quad (8.17)$$

Используя обозначение $\lambda = \Sigma - 1$, получаем следующее:

$$\begin{aligned} l(\Delta) &\propto -\frac{1}{2} \sum_i r_{ik} \log |\Lambda| - \frac{1}{2} \sum_i r_{ik} \text{tr}[(x_i - \mu)(x_i - \mu)^T \Lambda] = \\ &= -\frac{1}{2} \sum_i r_{ik} \log |\Lambda| - \frac{1}{2} \text{tr}(S_\mu \Lambda). \end{aligned} \quad (8.18)$$

Беря производную от матрицы, мы получаем следующее уравнение:

$$\begin{aligned} \frac{\partial l(\Lambda)}{\partial \Lambda} &= -\frac{1}{2} \sum_i r_{ik} \Lambda^{-T} - \frac{1}{2} S_\mu^T = 0; \\ \Lambda^{-1} \sum_i r_{ik} &= S_\mu^T \Rightarrow \Sigma = \frac{S_\mu^T}{\sum_i r_{ik}}; \\ \Sigma &= \frac{\sum_i r_{ik} (x_i - \mu_k)(x_i - \mu_k)^T}{\sum_i r_{ik}} = \frac{\sum_i r_{ik} x_i x_i^T}{\sum_i r_{ik}} - \mu_k \mu_k^T. \end{aligned} \quad (8.19)$$

Эти уравнения интуитивно понятны: среднее значение кластера k взвешивается при помощи среднего значения r_{ik} всех точек, назначенных кластеру k , тогда как ковариация представляет собой взвешенную эмпирическую матрицу рассеяния. Теперь мы готовы реализовать ЕМ-алгоритм для GMM с нуля. Давайте начнем с обзора псевдокода на рис. 8.3.

Класс **GMM** состоит из трех основных функций: **gmm_em**, **estep** и **mstep**. В **gmm_em** мы инициализируем средние значения, используя алгоритм K средних, и инициализируем ковариации единичными матрицами. Далее мы выполняем итерацию, вызывая поочередно две следующие функции. **estep** вычисляет значения ответственности (responsibility), или веса мягкого назначения, точки данных i для каждого из k кластеров. **mstep** принимает значения ответственности

в качестве входных данных и вычисляет параметры модели: среднее значение, ковариацию и пропорции смеси для каждого кластера в модели гауссовой смеси (GMM). Код функций `estep` и `mstep` соответствует формулам в тексте. В коде следующего листинга мы используем синтетический набор данных для сравнения истинных параметров GMM с параметрами, полученными с помощью EM-алгоритма.

```

1: class GMM
2: function gmm_em(X, k):
3:    $\pi_{init} = \frac{1}{K}$ 
4:    $\mu_{init} = \text{KMeans}(X, k) \leftarrow$  Инициализация с помощью K-means
5:    $\Sigma_{init} = I_{d \times d}$ 
6:   for iter = 1, 2, ..., max_iter
7:      $r_{ik} = \text{estep}(\pi_k, \mu_k, \Sigma_k, X) \leftarrow$  Вычисление ответственности
8:      $\pi_k, \mu_k, \Sigma_k = \text{mstep}(r_{ik}, X) \leftarrow$  Вычисление параметров модели
9:   end for
10:  return  $\pi_k, \mu_k, \Sigma_k$ 
11: function estep( $\pi_k, \mu_k, \Sigma_k, X$ ):
12:   $r_{ik} = \frac{N(x_i | \mu_k, \Sigma_k) \pi_k}{\sum_{k=1}^K N(x_i | \mu_k, \Sigma_k) \pi_k}$ 
13:  return  $r_{ik}$ 
14: function mstep( $r_{ik}, X$ ):
15:   $\pi_k = \frac{1}{N} \sum_i r_{ik}$ 
16:   $\mu_k = \frac{\sum_i r_{ik} x_i}{\sum_i r_{ik}}$ 
17:   $\Sigma_k = \frac{\sum_i r_{ik} x_i x_i^T}{\sum_i r_{ik}} - \mu_k \mu_k^T$ 
18:  return  $\pi_k, \mu_k, \Sigma_k$ 

```

Рис. 8.3. Псевдокод EM-алгоритма модели гауссовой смеси

Листинг 8.2. Метод максимизации ожидания для моделей гауссовой смеси

```

import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl

from sklearn.cluster import KMeans
from scipy.stats import multivariate_normal
from scipy.special import logsumexp
from scipy import linalg

np.random.seed(3)

class GMM:

    def __init__(self, n=1e3, d=2, K=4):
        self.n = int(n)   $\leftarrow$  Количество точек данных
        self.d = d         $\leftarrow$  Размерность данных
        self.K = K         $\leftarrow$  Количество кластеров

```

```

self.X = np.zeros((self.n, self.d))

self.mu = np.zeros((self.d, self.K))
self.sigma = np.zeros((self.d, self.d, self.K))
self.pik = np.ones(self.K)/K

def generate_data(self):  ← Генеративная GMM-модель

    alpha0 = np.ones(self.K)
    pi = np.random.dirichlet(alpha0)

    # истинные значения  $\mu$  и  $\Sigma$ 
    mu0 = np.random.randint(0, 10, size=(self.d, self.K)) -
    ⇒ 5*np.ones((self.d, self.K))
    V0 = np.zeros((self.d, self.d, self.K))
    for k in range(self.K):
        eigen_mean = 0
        Q = np.random.normal(loc=0, scale=1, size=(self.d, self.d))
        D = np.diag(abs(eigen_mean + np.random.normal(loc=0, scale=1,
        ⇒ size=self.d)))
        V0[:, :, k] = abs(np.transpose(Q)*D*Q)

    # выборочные данные
    for i in range(self.n):
        z = np.random.multinomial(1, pi)
        k = np.nonzero(z)[0][0]
        self.X[i, :] = np.random.multivariate_normal(mean=mu0[:, k],
        ⇒ cov=V0[:, :, k], size=1)

    plt.figure()
    plt.scatter(self.X[:, 0], self.X[:, 1], color='b', alpha=0.5)
    plt.title("Ground Truth Data"); plt.xlabel("X1"); plt.ylabel("X2")
    plt.show()

    return mu0, V0

def gmm_em(self):

    kmeans = KMeans(n_clusters=self.K,
    ⇒ random_state=42).fit(self.X)  ← Инициализация  $\mu$  (мю) при помощи K-means
    self.mu = np.transpose(kmeans.cluster_centers_)

    # инициализация  $\Sigma$ 
    for k in range(self.K):
        self.sigma[:, :, k] = np.eye(self.d)

    # EM-алгоритм
    max_iter = 10
    tol = 1e-5
    obj = np.zeros(max_iter)
    for iter in range(max_iter):
        print("EM iter ", iter)
        # E-шаг
        resp, llh = self.estimate()  ← E-шаг
        # M-шаг

```

```

        self.mstep(resp)  ← M-шаг
        # проверка сходимости
        obj[iter] = llh
        if (iter > 1 and obj[iter] - obj[iter-1] < tol*abs(obj[iter])):
            break
        # конец if
    # конец for
    plt.figure()
    plt.plot(obj)
    plt.title('Целевая функция EM-GMM'); plt.xlabel("Итерации");
    ➡ plt.ylabel("Логарифмическое правдоподобие")
    plt.show()

def estep(self):
    log_r = np.zeros((self.n, self.K))
    for k in range(self.K):
        log_r[:,k] = multivariate_normal.logpdf(self.X,
            ➡ mean=self.mu[:,k], cov=self.sigma[:,:,k])
    # конец for
    log_r = log_r + np.log(self.pik)
    L = logsumexp(log_r, axis=1)
    llh = np.sum(L)/self.n # логарифмическое правдоподобие
    log_r = log_r - L.reshape(-1,1) # нормировка
    resp = np.exp(log_r)
    return resp, llh

def mstep(self, resp):
    nk = np.sum(resp, axis=0)
    self.pik = nk/self.n
    sqrt_resp = np.sqrt(resp)
    for k in range(self.K):
        # обновление  $\mu$ 
        rx = np.multiply(resp[:,k].reshape(-1,1), self.X)
        self.mu[:,k] = np.sum(rx, axis=0) / nk[k]

        # обновление  $\Sigma$ 
        Xm = self.X - self.mu[:,k]
        Xm = np.multiply(sqrt_resp[:,k].reshape(-1,1), Xm)
        self.sigma[:,:,k] = np.maximum(0, np.dot(np.transpose(Xm), Xm)
            ➡ / nk[k] + 1e-5 * np.eye(self.d))
    # конец for

if __name__ == '__main__':
    gmm = GMM()
    mu0, V0 = gmm.generate_data()
    gmm.gmm_em()

    for k in range(mu0.shape[1]):
        print("cluster ", k)
        print("-----")
        print("ground truth means:")
        print(mu0[:,k])
        print("ground truth covariance:")

```

```

    print(V0[:, :, k])
# конец for

for k in range(mu0.shape[1]):
    print("cluster ", k)
    print("-----")
    print("GMM-EM means:")
    print(gmm.mu[:, k])
    print("GMM-EM covariance:")
    print(gmm.sigma[:, :, k])

plt.figure()
ax = plt.axes()
plt.scatter(gmm.X[:, 0], gmm.X[:, 1], color='b', alpha=0.5)

for k in range(mu0.shape[1]):

    v, w = linalg.eigh(gmm.sigma[:, :, k])
    v = 2.0 * np.sqrt(2.0) * np.sqrt(v)
    u = w[0] / linalg.norm(w[0])

    # отрисовка эллипса для демонстрации гауссовой компоненты
    angle = np.arctan(u[1] / u[0])
    angle = 180.0 * angle / np.pi # перевод в градусы
    ell = mpl.patches.Ellipse(gmm.mu[:, k], v[0], v[1], 180.0 + angle,
        ➤ color='r', alpha=0.5)
    ax.add_patch(ell)

    # отрисовка кластерных центроидов
    plt.scatter(gmm.mu[0, k], gmm.mu[1, k], s=80, marker='x', color='k',
        ➤ alpha=1)
plt.title("Модель гауссовой смеси"); plt.xlabel("X1"); plt.ylabel("X2")
plt.show()

```

Как мы можем видеть из выходных данных, кластерные средние значения и ковариации точно соответствуют своим эталонным значениям.

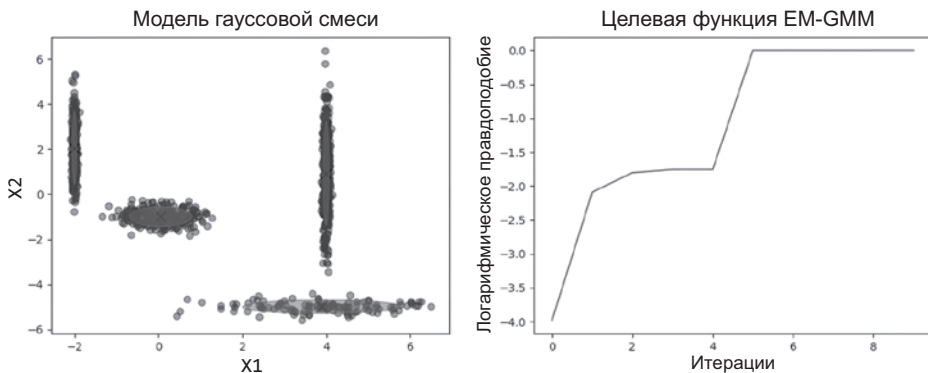


Рис. 8.4. Результаты EM-GMM-кластеризации (слева) и целевая функция логарифмического правдоподобия (справа)

На рис. 8.4 (слева) изображена модель гауссовой смеси, наложенная на данные. Видим, что гауссовы эллипсы точно соответствуют полученным данным. Это также подтверждается монотонным ростом целевой функции логарифмического правдоподобия, показанной на рис. 8.4 (справа). В следующем разделе мы рассмотрим два популярных метода уменьшения размерности: анализ главных компонент и стохастическое вложение соседей с t -распределением (t -distributed stochastic neighbor embedding, t-SNE).

8.3. Снижение размерности

Нам может пригодиться навык проецировать многомерные данные $x \in R^D$ в подпространство меньшей размерности $z \in R^L$ таким образом, чтобы сохранялись уникальные характеристики данных. Другими словами, желательно уметь схватывать суть данных в низкоразмерной проекции. Например, если нужно выучить векторные представления слов на корпусе текстов из «Википедии» и требуется понять взаимосвязь между различными векторами, то было бы намного проще визуализировать эти отношения в двумерном пространстве, применяя снижение размерности. В этом разделе мы рассмотрим два метода ML для снижения размерности: анализ главных компонент (principal component analysis, PCA) и t-SNE.

8.3.1. Анализ главных компонент

В анализе главных компонент (PCA) нам требуется спроецировать вектор данных $x \in R^D$ на вектор меньшей размерности $z \in R^L$ с $L < D$ таким образом, чтобы дисперсия спроецированных данных была максимальной. Максимизация дисперсии спроецированных данных является основным принципом PCA, который позволяет сохранять уникальные характеристики наших данных. Мы можем измерить качество нашей проекции как ошибку восстановления (reconstruction error):

$$E = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2 = \|X - WZ\|_F^2. \quad (8.20)$$

Здесь $\hat{x}_i$ размером $D \times 1$ — это восстановленный вектор, перенесенный в пространство более высокой размерности, z_i размером $L \times 1$ — это вектор главной компоненты меньшей размерности; а W — ортонормированная матрица размером $D \times L$. Напомним, что ортонормированная матрица — это вещественная квадратная матрица, транспонированная версия которой равна ее обратному значению (то есть $W^T W = W W^T = I$). Другими словами, если PCA-проекция равна $z_i = W^T x_i$, то $\hat{x}_i = W z_i$. Мы покажем, что оптимальная проекционная матрица W (та, которая максимизирует дисперсию спроецированных данных) равна матрице из L собственных векторов, соответствующих наибольшим собственным значениям эмпирической ковариационной матрицы $\hat{\Sigma} = 1/N \sum x_i x_i^T$.

Можем записать дисперсию спроецированных данных следующим образом:

$$\frac{1}{N} \sum_{i=1}^N z_i^2 = \frac{1}{N} \sum_{i=1}^N w^T x x^T w = w^T \hat{\Sigma} w. \quad (8.21)$$

Требуется максимизировать эту величину с учетом ограничения ортонормированности, то есть $\|w\|^2 = 1$. Лагранжиан можно представить в следующем виде:

$$\max J(w) = w^T \hat{\Sigma} w + \lambda(w^T w - 1). \quad (8.22)$$

Дифференцируя и приравнявая результат нулю, получаем следующее выражение:

$$\begin{aligned} \frac{\partial}{\partial w} J(w) &= 2\hat{\Sigma}w - 2\lambda w = 0; \\ \hat{\Sigma}w &= \lambda w. \end{aligned} \quad (8.23)$$

Таким образом, направление, которое максимизирует дисперсию, является собственным вектором ковариационной матрицы. Умножая слева на w и используя ограничение ортонормированности, получаем $w^T \hat{\Sigma} w = \lambda$. Таким образом, чтобы максимизировать дисперсию для первой главной компоненты, нужно выбрать собственный вектор, соответствующий наибольшему собственному значению. Если повторить описанную выше процедуру, вычтя первую главную компоненту из x , то обнаружим, что $\hat{\Sigma} w^2 = \lambda w^2$. Используя индукцию, можем показать, что PCA-матрица WDL состоит из L собственных векторов, соответствующих наибольшим собственным значениям эмпирической ковариационной матрицы $\hat{\Sigma}$. Теперь мы готовы воплотить алгоритм PCA в коде. PCA-алгоритм в схематичном виде представлен на рис. 8.5:

```

1: class PCA
2:   function transform(X, K):
3:     Σ = covariance_matrix(X) ← Вычисление эмпирической ковариации
4:     V, λ = eig(Σ) ← Разложение по собственным значениям
5:     idx = argsort(λ) ← Сортировка в порядке убывания
6:     V_pca = V[idx][:K] | Выбор K верхних собственных векторов
7:     λ_pca = λ[idx][:K] | и собственных значений
8:     X_pca = X V_pca ← Проекция данных на главные компоненты
9:   return X_pca

```

Рис. 8.5. Псевдокод метода главных компонент

Основной функцией класса PCA является функция `transform`. В ней мы сначала вычисляем эмпирическую ковариационную матрицу, пользуясь матрицей данных X высокой размерности. Затем вычисляем разложение ковариационной матрицы по собственным значениям. Мы сортируем собственные значения в порядке от наибольшего к наименьшему и используем отсортированный

индекс для выбора K самых больших собственных значений и соответствующих им собственных векторов. Наконец, вычисляем PCA-представление наших данных, умножая матрицу данных на матрицу из K верхних собственных значений. В коде ниже мы проецируем случайную d -мерную матрицу на две основные компоненты:

Листинг 8.3. Анализ главных компонент

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

class PCA():

    def __init__(self, n_components = 2):
        self.n_components = n_components

    def covariance_matrix(self, X, Y=None):
        if Y is None:
            Y = X
        n_samples = np.shape(X)[0]
        covariance_matrix = (1 / (n_samples-1)) * (X - X.mean(axis=0))
        ➡ .T.dot(Y - Y.mean(axis=0))
        return covariance_matrix

    def transform(self, X):
        Sigma = self.covariance_matrix(X)
        eig_vals, eig_vecs = np.linalg.eig(Sigma)

        idx = eig_vals.argsort()[::-1]
        eig_vals = eig_vals[idx][:self.n_components]
        eig_vecs = np.atleast_1d(eig_vecs[:,idx])[:, :self.n_components]
        X_transformed = X.dot(eig_vecs)
        ➡
        return X_transformed

if __name__ == "__main__":

    n = 20
    d = 5
    X = np.random.rand(n,d)

    pca = PCA(n_components=2)
    X_pca = pca.transform(X)

    print(X_pca)

    plt.figure()
    plt.scatter(X_pca[:,0], X_pca[:,1], color='b', alpha=0.5)
    plt.title("Анализ главных компонент"); plt.xlabel("X1");
    ➡ plt.ylabel("X2")
    plt.show()
```

Сортировка собственных значений в порядке убывания

Проекция данных на главные компоненты

На рис. 8.6 показана диаграмма первых двух главных компонент, примененных к случайной матрице. Напомню, что в PCA дисперсия проецируемых данных максимизируется, а потому есть возможность обнаружить тенденции и закономерности в прогнозируемых данных:

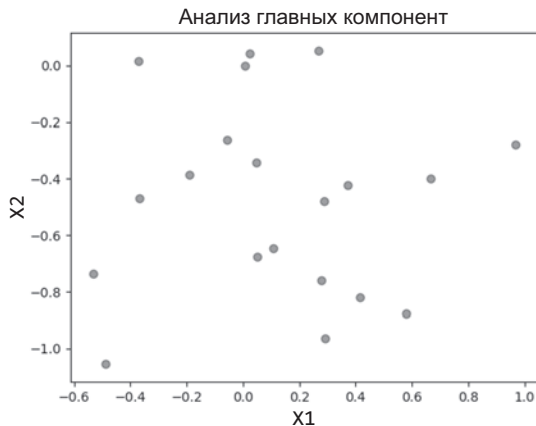


Рис. 8.6. PCA-проекция случайной матрицы

8.3.2. Метод *t-SNE*-обучения на базе многообразий на примере изображений

Изображения — это объекты высокой размерности, которые живут на многообразиях. *Многообразие* — это топологическое пространство, которое локально напоминает евклидово. Моделируя пространства изображений как многообразия, мы можем изучать их геометрические свойства. Мы можем визуализировать объекты высокой размерности с помощью вложений (embedding). Посмотрим в действии два метода таких вложений — *t-SNE* и *Isomap* — на наборе данных рукописных цифр MNIST.

Листинг 8.4. *t-SNE*-многообразие на наборе данных цифр MNIST

```
import numpy as np
import matplotlib.pyplot as plt

from time import time
from sklearn import manifold

from sklearn.datasets import load_digits
from sklearn.neighbors import KDTree

def plot_digits(X):
    n_img_per_row = np.amin((20, np.int(np.sqrt(X.shape[0]))))
    img = np.zeros((10 * n_img_per_row, 10 * n_img_per_row))
```

212 Глава 8. Основные алгоритмы обучения без учителя

```
for i in range(n_img_per_row):
    ix = 10 * i + 1
    for j in range(n_img_per_row):
        iy = 10 * j + 1
        img[ix:ix + 8, iy:iy + 8] = X[i * n_img_per_row +
                                         j].reshape((8, 8))

plt.figure()
plt.imshow(img, cmap=plt.cm.binary)
plt.xticks([])
plt.yticks([])
plt.title('Пример из 64-мерного набора данных цифр')

def mnist_manifold():

    digits = load_digits()

    X = digits.data
    y = digits.target

    num_classes = np.unique(y).shape[0]

    plot_digits(X)

    # TSNE
    # Алгоритм Барнса – Хата:  $O(d N \log N)$ , где  $d$  – размер, а  $N$  – объем выборки
    # Точный метод:  $O(d N^2)$ 
    t0 = time()
    tsne = manifold.TSNE(n_components = 2, init = 'pca', method =
                          'barnes_hut', verbose = 1)
    X_tsne = tsne.fit_transform(X)
    t1 = time()
    print('t-SNE: %.2f sec' %(t1-t0))
    tsne.get_params()

    plt.figure()
    for k in range(num_classes):
        plt.plot(X_tsne[y==k,0], X_tsne[y==k,1], 'o')
    plt.title('t-SNE-вложение набора данных цифр')
    plt.xlabel('X1')
    plt.ylabel('X2')
    axes = plt.gca()
    axes.set_xlim([X_tsne[:,0].min()-1,X_tsne[:,0].max()+1])
    axes.set_ylim([X_tsne[:,1].min()-1,X_tsne[:,1].max()+1])
    plt.show()

    # ISOMAP
    # 1. Поиск ближайших соседей:  $O(d \log k N \log N)$ 
    # 2. Поиск кратчайшего пути на графе:  $O(N^2(k+\log(N)))$ 
    # 3. Частичное разложение по собственным значениям:  $O(dN^2)$ 

    t0 = time()
    isomap = manifold.Isomap(n_neighbors = 5, n_components = 2)
    X_isomap = isomap.fit_transform(X)
    t1 = time()
```

```

print('Isomap: %.2f sec' %(t1-t0))
isomap.get_params()

plt.figure()
for k in range(num_classes):
    plt.plot(X_isomap[y==k,0], X_isomap[y==k,1], 'o', label=str(k),
             linewidth = 2)
plt.title('Isomap embedding of the digits dataset')
plt.xlabel('X1')
plt.ylabel('X2')
plt.show()

# K-d-дерево для поиска k ближайших соседей изображения
kdt = KDTree(X_isomap)
Q = np.array([[ -160, -30], [ -102, 14]])
kdt_dist, kdt_idx = kdt.query(Q,k=20)
plot_digits(X[kdt_idx.ravel(),:])

if __name__ == "__main__":
    mnist_manifold()

```

На рис. 8.7 в двух измерениях показано t-SNE-вложение с 10 кластерами, где каждый кластер соответствует цифре от 0 до 9. Видим, что без задействования обучающих данных мы можем обнаружить 10 кластеров с использованием t-SNE-вложения в двумерное пространство. Более того, мы ожидаем, что соседние группы цифр будут похожи друг на друга. На изображении в правой части рис. 8.7 показаны примеры цифр из двух соседних кластеров (цифра 0 и цифра 6). Чтобы найти K ближайших к точке запроса соседей, можем визуализировать соседние кластеры, построив k-d-дерево:

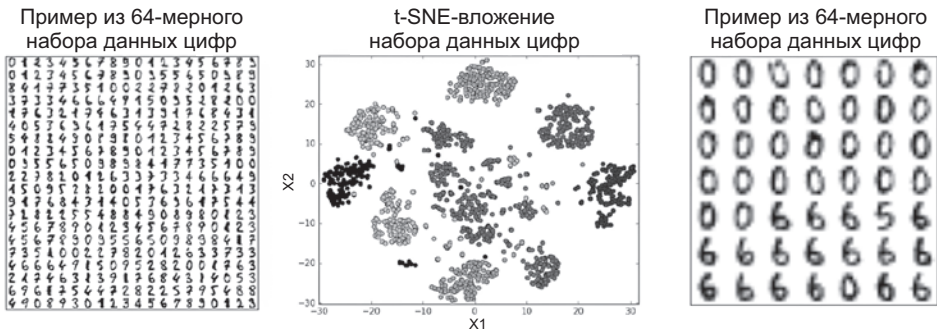


Рис. 8.7. t-SNE-вложение, показывающее в двух измерениях 10 кластеров, где каждый кластер соответствует цифре от 0 до 9

Важно иметь представление о некоторых подводных камнях t-SNE. Например, результаты могут меняться в зависимости от гиперпараметра перплексии (perplexity). Также алгоритм t-SNE не всегда выдает одинаковые результаты при последовательных запусках и имеет дополнительные гиперпараметры,

связанные с процессом оптимизации. Более того, размеры кластеров (измеряемые по стандартному отклонению) и расстояния между кластерами могут не иметь никакого смысла. Таким образом, высокая гибкость t-SNE также затрудняет интерпретацию получаемых результатов.

8.4. Упражнения

8.1. Покажите, что распределение Дирихле $\text{Dir}(\theta|\alpha)$ является сопряженным априорным распределением для мультиномиального правдоподобия, получив формулу для апостериорного распределения. Как изменяется вид апостериорного распределения в зависимости от числа событий?

8.2. Объясните принцип, лежащий в основе инициализации метода K-means++.

8.3. Докажите свойство циклической перестановки следа: $\text{tr}(ABC) = \text{tr}(BCA) = \text{tr}(CAB)$.

8.4. Оцените временную сложность алгоритма анализа главных компонент (PCA).

Итоги

- Обучение без учителя происходит в случае, если отсутствуют обучающие метки.
- Метод K средних с процессом Дирихле — это байесовское непараметрическое расширение алгоритма K средних, в котором число кластеров растет вместе с объемом данных.
- Модели гауссовой смеси обычно используются для моделирования сложных распределений плотности. Параметры GMM (средние значения, ковариации и пропорции смеси) могут быть определены с помощью алгоритма максимизации ожидания.
- EM — это итеративный алгоритм, который чередует вывод недостающих значений с учетом параметров (E-шаг) и последующую оптимизацию параметров с учетом полученных данных (M-шаг).
- В методах снижения размерности мы проецируем многомерные данные в подпространство меньшей размерности таким образом, чтобы сохранить уникальные характеристики данных.
- В алгоритме анализа главных компонент максимизируется дисперсия спроецированных данных.
- Среди недочетов алгоритма t-SNE можно отметить следующие: изменчивость результатов из-за гиперпараметра перплексии, разброс выходных данных при последовательных запусках, недостаточная интерпретируемость размеров кластеров и большие расстояния между кластерами.

Избранные алгоритмы обучения без учителя

В этой главе

- ✓ Латентное размещение Дирихле для тематического моделирования
- ✓ Методы оценки плотности в вычислительной биологии и финансах
- ✓ Моделирование структуры реляционных данных
- ✓ Метод имитации отжига для минимизации энергии
- ✓ Генетический алгоритм в эволюционной биологии
- ✓ Исследования в области ML: обучение без учителя

В предыдущей главе мы рассмотрели алгоритмы ML без учителя, которые помогают выявлять закономерности в данных. Эта глава продолжает ту же тему, уделяя особое внимание отдельным избранным алгоритмам, которые призваны продемонстрировать широту спектра методов обучения без учителя. Они достойны вашего внимания, поскольку охватывают широкий диапазон применения — от вычислительной биологии до физики и финансов. Мы начнем с рассмотрения латентного размещения Дирихле (latent Dirichlet allocation, LDA) для обучения тематических моделей, далее перейдем к алгоритмам оценки плотности и моделирования структуры данных, а завершим методом имитации отжига (simulated annealing, SA) и генетическими алгоритмами (genetic algorithms, GA).

9.1. Латентное размещение Дирихле

Тематическая модель (topic model) — это модель со скрытыми переменными для дискретных данных, таких как текстовые документы. Латентное размещение Дирихле (LDA) — это тематическая модель, которая представляет каждый документ в виде конечного набора тем, где тема является распределением по словам. Цель данного метода состоит в том, чтобы найти распределение общих тем и их пропорции для каждого документа. LDA построена на модели мешка слов (bag of words), в которой порядок слов не важен и, как следствие, структура предложения не фиксируется (то есть значение имеет только количество употреблений слов). Таким образом, каждый документ сводится к вектору V частоты слов из словаря, и весь корпус документов D представляется в виде *терм-документной* (term-document) матрицы A_{VD} . LDA можно рассматривать как задачу неотрицательного матричного разложения, в которой матрица термин-документ преобразуется в произведение тем W_{VK} и пропорций тем H_{KD} : $A = W_H$.

Частота термина — обратная частота документа (term frequency — inverse document frequency, tf-idf) — популярный метод корректировки показателя количества употреблений слов, поскольку он логарифмически сводит к нулю количество употреблений слов, часто встречающихся в документах: $A \log(D/n_t)$, где D — общее количество документов в корпусе, а n_t — число документов, в которых присутствует термин t . Сглаживание количества употреблений слов с помощью tf-idf дает возможность выделить наборы характерных для документов слов и поэтому приводит к повышению производительности модели. Можно ввести обобщение терм-документной матрицы путем замены количества употреблений отдельных слов (униграмм) на частоту более крупных структурных единиц, таких как n -граммы. В случае использования n -грамм для устранения недостатка наблюдений в очень большом пространстве признаков используются различные методы сглаживания количества употреблений слов (например, сглаживание по Лапласу). На рис. 9.1 показана графическая модель LDA.

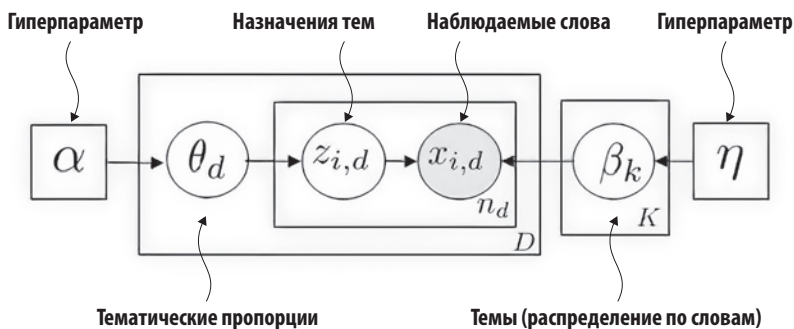


Рис. 9.1. Графическая модель латентного размещения Дирихле

Тематическая модель LDA каждому слову $x_{i,d}$ ставит в соответствие метку темы $z_{i,d} \in \{1, 2, \dots, K\}$. Каждый документ связывается с тематическими пропорциями θ_d , которые могут быть использованы для определения сходства документов. Темы β_k являются общими для всех документов. Гиперпараметры α и η отражают наши априорные знания о пропорциях тем и самих темах соответственно (например, из предшествующего онлайн-обучения модели). Полная генеративная модель может быть определена следующим образом:

$$\begin{aligned}\theta_d &| \alpha \sim \text{Dir}(\alpha); \\ z_{i,d} &| \theta_d \sim \text{Cat}(\theta_d); \\ \beta_k &| \eta \sim \text{Dir}(\eta); \\ x_{i,d} &| z_{i,d} \sim k, \beta \sim \text{Cat}(\beta_k).\end{aligned}\tag{9.1}$$

Совместное распределение для одного документа d можно представить в виде формулы ниже — см. работу Дэвида М. Блея (David M. Blei), Эндрю Ю. Нг (Andrew Y. Ng) и Майкла И. Джордана (Michael I. Jordan) «Latent Dirichlet Allocation» (Journal of Machine Learning Research, 2003):

$$p(x, z, \theta, | \alpha, \beta) = p(\theta_d | \alpha) \prod_{i=1}^{n_d} p(z_{i,d} | \theta_d) p(x_{i,d} | z_{i,d}, \beta).\tag{9.2}$$

α и β являются параметрами уровня корпуса, переменная θ_d семплируется один раз для каждого документа, а $z_{i,d}$ и $x_{i,d}$ — это переменные уровня слов, которые выбираются единожды для каждого слова в каждом документе. В отличие от модели мультиномиальной кластеризации, где каждый документ связан с одной темой, LDA представляет каждый документ как совокупность тем.

9.1.1. Вариационный байесовский метод

Ключевой проблемой вывода, которую нам нужно решить перед применением LDA, является вычисление апостериорного распределения скрытых переменных для данного документа: $p(\theta, z | x, \alpha, \beta)$. Апостериорное распределение может быть аппроксимировано следующим вариационным распределением:

$$q(\theta, z | \gamma, \phi) = q(\theta | \gamma) \prod_{i=1}^n q(z_i | \phi_i).\tag{9.3}$$

Вариационные параметры оптимизируются таким образом, чтобы максимально увеличить нижнюю вариационную границу (ELBO). Напомним, что в главе 3 ELBO была определена как разность между слагаемым энергии и слагаемым энтропии:

$$\begin{aligned}\log p(x | \alpha, \eta) &\geq L(x, \phi, \gamma, \lambda) = \\ &= E_q[\log p(x, z, \theta, \beta | \alpha, \eta)] - E_q[\log q(z, \theta, \beta)].\end{aligned}\tag{9.4}$$

Выбираем полностью факторизованное распределение q следующего вида:

$$q(z_{id} = k) = \phi_{dwk}; \quad q(\theta_d) \sim \text{Dir}(\theta_d | \gamma_d); \quad q(\beta_k) \sim \text{Dir}(\beta_k | \lambda_k). \quad (9.5)$$

Согласно публикации Дэвида М. Блея (David M. Blei), Эндрю Ю. Нг (Andrew Y. Ng) и Майкла И. Джордана (Michael I. Jordan) «Latent Dirichlet Allocation» (Journal of Machine Learning Research, 2003), нижнюю границу можно расширить при помощи факторизации p и q :

$$\begin{aligned} L(\gamma, \phi; \alpha, \beta) = & E_q[\log p(\theta | \alpha)] + E_q[\log p(z | \theta)] + \\ & + E_q[\log p(x | z, \beta)] - \\ & - E_q[\log q(\theta)] - E_q[\log q(z)]. \end{aligned} \quad (9.6)$$

Каждое из пяти слагаемых в $L(\gamma, \phi; \alpha, \beta)$ может быть разложено следующим образом:

$$\begin{aligned} L(\gamma, \phi; \alpha, \beta) = & \log \Gamma\left(\sum_{j=1}^k \alpha_j\right) - \sum_{i=1}^k \log \Gamma(\alpha_i) + \\ & + \sum_{i=1}^k (\alpha_i - 1) \left(\Psi(\gamma_i) - \Psi\left(\sum_{j=1}^k \gamma_j\right) \right) + \\ & + \sum_{n=1}^N \sum_{i=1}^k \phi_{ni} \left(\Psi(\gamma_i) - \Psi\left(\sum_{j=1}^k \gamma_j\right) \right) + \\ & + \sum_{n=1}^N \sum_{i=1}^k \sum_{j=1}^V \phi_{ni} x_n^j \log \beta_{ij} - \\ & - \log \Gamma\left(\sum_{j=1}^k \gamma_j\right) + \sum_{i=1}^k \log \Gamma(\gamma_i) - \\ & - \sum_{i=1}^k (\gamma_i - 1) \left(\Psi(\gamma_i) - \Psi\left(\sum_{j=1}^k \gamma_j\right) \right) - \\ & - \sum_{n=1}^N \sum_{i=1}^k \phi_{ni} \log \phi_{ni}. \end{aligned} \quad (9.7)$$

Здесь $\Psi(x) = d/dx \log \Gamma(x)$ — дигамма-функция. $L(\gamma, \phi; \alpha, \beta)$ можно максимизировать, используя координатный подъем по вариационным параметрам ϕ, γ, α :

$$\begin{aligned} \phi_{dwk} & \propto \exp\{E_q[\log \theta_{dk}] + E_q[\log \beta_{kw}]\}; \\ \gamma_{dk} & = \alpha + \sum_w n_{dw} \phi_{dwk}; \\ \lambda_{kw} & = \eta + \sum_d n_{dw} \phi_{dwk}. \end{aligned} \quad (9.8)$$

Здесь математическое ожидание по q для логарифмов θ и β выглядит следующим образом:

$$\begin{aligned} E_q[\log \theta_{dk}] &= \Psi(\gamma_{dk}) - \Psi\left(\sum_{i=1}^K \gamma_{di}\right); \\ E_q[\log \beta_{kw}] &= \Psi(\gamma_{kw}) - \Psi\left(\sum_{i=1}^W \lambda_{ki}\right). \end{aligned} \quad (9.9)$$

Вариационные обновления параметров можно применять в онлайн-режиме, для чего не требуется полное прохождение всего корпуса на каждой итерации. Онлайн-обновление вариационных параметров позволяет проводить тематический анализ для очень больших наборов данных, включая потоковые данные. Онлайн-вариант вариационного байесовского вывода для LDA представлен на рис. 9.2.

```

1: По определению  $\rho_t = (\tau_0 + t)^{-\kappa}$ 
2: Инициализировать  $\lambda$  случайным образом
3: for  $t = 1$  to  $\infty$  do
4:   E-шаг:
5:   Инициализировать  $\gamma_{tk} = 1$ 
6:   repeat
7:     Присвоить  $\phi_{twk} \propto \exp\{E_q[\log \theta_{tk}] + E_q[\log \beta_{kw}]\}$  ← Назначения тем
8:     Присвоить  $\gamma_{tk} = \alpha + \sum_w \phi_{twk} n_{tw}$  ← Тематические пропорции
9:   until  $\frac{1}{K} \sum_k |\Delta \gamma_{tk}| < \varepsilon$ 
10:  M-шаг:
11:  Вычислить  $\tilde{\lambda}_{kw} = \eta + D n_{tw} \phi_{twk}$ 
12:  Присвоить  $\lambda = (1 - \rho_t) \lambda + \rho_t \tilde{\lambda}$ 
13: конец for
```

Рис. 9.2. Псевдокод алгоритма LDA

После поступления t -го вектора количества употреблений слов n_t выполняем E -шаг, чтобы найти локально оптимальные значения γ_t и ϕ , сохраняя λ фиксированным. Затем вычисляем $\tilde{\lambda}$, которое было бы оптимальным, если бы весь наш корпус состоял только из одного документа n , повторяющегося D раз. Затем мы обновляем λ как взвешенное среднее его предыдущего значения и $\tilde{\lambda}$, где вес задается параметром обучения $\rho_t \in (0.5, 1]$, контролирующим скорость, с которой забываются старые значения λ . Теперь мы можем реализовать вариационный байесовский алгоритм для LDA с нуля — см. следующий листинг:

Листинг 9.1. Вариационный байесовский метод для латентного размещения Дирихле

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_20newsgroups
```

220 Глава 9. Избранные алгоритмы обучения без учителя

```
from sklearn.feature_extraction.text import TfidfVectorizer
from wordcloud import WordCloud
from scipy.special import digamma, gammaln

np.random.seed(12)

class LDA:
    def __init__(self, A, K):
        self.N = A.shape[0]  ← Размер словаря
        self.D = A.shape[1]  ← Количество документов
        self.K = num_topics  ← Количество тем

        self.A = A  ← Терм-документная матрица

        # инициализация распределения слов  $\beta$ 
        self.eta = np.ones(self.N)  ← Равномерное априорное распределение Дирихле слов
        self.beta = np.zeros((self.N, self.K))  ← Матрица тем  $N \times K$ 
        for k in range(self.K):
            self.beta[:,k] = np.random.dirichlet(self.eta)
            self.beta[:,k] = self.beta[:,k] + 1e-6 # чтобы избежать нулевых элементов
            self.beta[:,k] = self.beta[:,k]/np.sum(self.beta[:,k])
        # конец for

    # Равномерное априорное распределение Дирихле тем
    # инициализация тематических пропорций  $\theta$  и кластерных назначений  $z$ 
    self.alpha = np.ones(self.K)
    self.z = np.zeros((self.N, self.D))  ← Кластерные назначения  $z$ 
    for d in range(self.D):
        theta = np.random.dirichlet(self.alpha)
        wdn_idx = np.nonzero(self.A[:,d])[0]
        for i in range(len(wdn_idx)):
            z_idx = np.argmax(np.random.multinomial(1, theta))
            self.z[wdn_idx[i],d] = z_idx # id темы
        # конец for
    # конец for

    # инициализация вариационных параметров
    self.gamma = np.ones((self.D, self.K))  ← Тематические пропорции
    for d in range(self.D):
        theta = np.random.dirichlet(self.alpha)
        self.gamma[d,:] = theta
    # конец for

    self.lmbda = np.transpose(self.beta)
    #np.ones((self.K, self.N))/self.N  ← Частота встречаемости слов

    self.phi = np.zeros((self.D, self.N, self.K))  ← Назначения
    for d in range(self.D):
        for w in range(self.N):
            theta = np.random.dirichlet(self.alpha)
            self.phi[d,w,:] = np.random.multinomial(1, theta)
```

```

        # конец for
    # конец for

def variational_inference(self, var_iter=10):

    llh = np.zeros(var_iter)
    llh_delta = np.zeros(var_iter)
    for iter in range(var_iter):
        print("VI iter: ", iter)
        J_old = self.elbo_objective()
        self.mean_field_update()
        J_new = self.elbo_objective()

        llh[iter] = J_old
        llh_delta[iter] = J_new - J_old
    # конец for

    # обновление  $\alpha$  и  $\beta$ 
    for k in range(self.K):
        self.alpha[k] = np.sum(self.gamma[:,k])
        self.beta[:,k] = self.lmbda[k,:] / np.sum(self.lmbda[k,:])
    # конец for

    # обновление назначения тем
    for d in range(self.D):
        for i in range(len(wdn_idx)):
            wdn_idx = np.nonzero(self.A[:,d])[0]
            for i in range(len(wdn_idx)):
                z_idx = np.argmax(self.phi[d,wdn_idx[i],:])
                self.z[wdn_idx[i],d] = z_idx # id темы
            # конец for
        # конец for

    plt.figure()
    plt.plot(llh); plt.title('Вариационный вывод LDA');
    plt.xlabel('Итерации среднего поля'); plt.ylabel("ELBO")
    plt.show()

    return llh

def mean_field_update(self):
    ndw = np.zeros((self.D, self.N))
    for d in range(self.D):
        doc = self.A[:,d]
        wdn_idx = np.nonzero(doc)[0]

        for i in range(len(wdn_idx)):
            ndw[d,wdn_idx[i]] += 1
        # конец for

    # обновление  $\gamma$ 
    for k in range(self.K):

```

Количество употреблений слов
для каждого документа

```

        self.gamma[d,k] = self.alpha[k] + np.dot(ndw[d,:],
        ➡ self.phi[d,:,k])
    # конец for

    # обновление φ
    for w in range(len(wdn_idx)):
        self.phi[d,wdn_idx[w],:] = np.exp(digamma(self.gamma[d,:])
        ➡ - digamma(np.sum(self.gamma[d,:])) + digamma(self.lmbda[:,wdn_idx[w]]))
        ➡ - digamma(np.sum(self.lmbda, axis=1)))
        if (np.sum(self.phi[d,wdn_idx[w],:]) > 0): # чтобы избежать 0/0
            self.phi[d,wdn_idx[w],:] = self.phi[d,wdn_idx[w],:] /
            ➡ np.sum(self.phi[d,wdn_idx[w],:]) # нормировка φ
        # конец if
    # конец for

# конец for

# обновление λ для всех документов при заданной ndw
for k in range(self.K):
    self.lmbda[k,:] = self.eta
    for d in range(self.D):
        self.lmbda[k,:] += np.multiply(ndw[d,:], self.phi[d,:,k])
    # конец for
# конец for

def elbo_objective(self):
    # см. Блей и др., 2003

    T1_A = gammaln(np.sum(self.alpha)) - np.sum(gammaln(self.alpha))
    T1_B = 0
    for k in range(self.K):
        T1_B += np.dot(self.alpha[k]-1, digamma(self.gamma[:,k]) -
        ➡ digamma(np.sum(self.gamma, axis=1)))
    T1 = T1_A + T1_B

    T2 = 0
    for n in range(self.N):
        for k in range(self.K):
            T2 += self.phi[:,n,k] * (digamma(self.gamma[:,k]) -
            ➡ digamma(np.sum(self.gamma, axis=1)))

    T3 = 0
    for n in range(self.N):
        for k in range(self.K):
            T3 += self.phi[:,n,k] * np.log(self.beta[n,k])

    T4 = 0
    T4_A = -gammaln(np.sum(self.gamma, axis=1)) +
    ➡ np.sum(gammaln(self.gamma), axis=1)
    T4_B = 0
    for k in range(self.K):
        T4_B = -(self.gamma[:,k]-1) * (digamma(self.gamma[:,k]) -
        ➡ digamma(np.sum(self.gamma, axis=1)))
    T4 = T4_A + T4_B

```

```

T5 = 0
for n in range(self.N):
    for k in range(self.K):
        T5 += -np.multiply(self.phi[:,n,k], np.log(self.phi[:,n,k]
            ➤ + 1e-6))

T15 = T1 + T2 + T3 + T4 + T5
J = sum(T15)/self.D # среднее по документам
return J

if __name__ == "__main__":

    # Параметры LDA
    num_features = 1000 # размер словаря
    num_topics = 4 # фиксировано для LD

    # Датасет 20 новостных групп
    categories = ['sci.crypt', 'comp.graphics', 'sci.space',
        ➤ 'talk.religion.misc']

    newsgroups = fetch_20newsgroups(shuffle=True, random_state=42,
        ➤ subset='train',
            remove=('headers', 'footers', 'quotes'),
            ➤ categories=categories)

    vectorizer = TfidfVectorizer(max_features = num_features, max_df=0.95,
        ➤ min_df=2, stop_words = 'english')
    dataset = vectorizer.fit_transform(newsgroups.data)
    A = np.transpose(dataset.toarray()) # терм-документная матрица

    lda = LDA(A=A, K=num_topics)
    llh = lda.variational_inference(var_iter=10)
    id2word = {v:k for k,v in vectorizer.vocabulary_.items()}

    # вывод тем на экран
    for k in range(num_topics):
        print("topic: ", k)
        print("-----")
        topic_words = ""
        top_words = np.argsort(lda.lmbda[k,:])[-10:]
        for i in range(len(top_words)):
            topic_words += id2word[top_words[i]] + " "
        print(id2word[top_words[i]])

    wordcloud = WordCloud(width = 800, height = 800,
        background_color = 'white',
        min_font_size = 10).generate(topic_words)

    plt.figure()
    plt.imshow(wordcloud)
    plt.axis("off")
    plt.tight_layout(pad = 0)
    plt.show()

```

Рисунок 9.3 демонстрирует увеличение показателя ELBO по мере роста числа итераций среднего поля. На рис. 9.4 показано полученное распределение тем, представленное в виде облака слов:

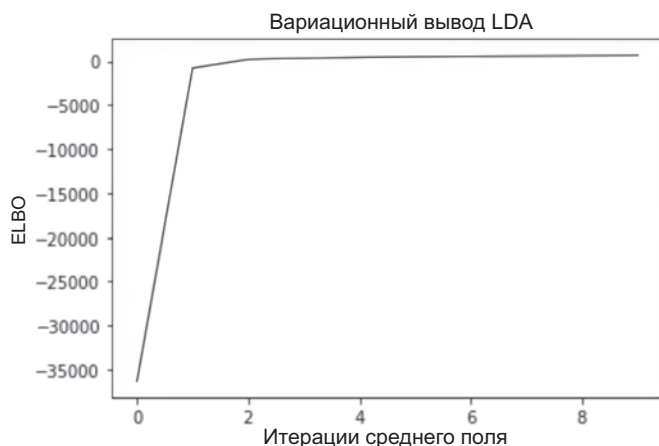


Рис. 9.3. Увеличение ELBO с ростом количества итераций среднего поля



Рис. 9.4. Распределение тем, полученное с помощью вариационного вывода среднего поля LDA

Как можно видеть из представленных выходных данных, ключевые слова для каждой темы соответствуют категориям в наборе данных 20 групп новостей. В следующем разделе мы рассмотрим несколько методов для моделирования плотности вероятности данных на примере вычислительной биологии и финансов.

9.2. Оценки плотности

Целью *оценки плотности* является моделирование плотности вероятности данных. В этом разделе мы обсудим методы ядерной оценки плотности (kernel density estimator, KDE), применяемые в вычислительной биологии, и рассмотрим, как можно оптимизировать портфель акций, используя теорию тангенциального портфеля (tangent portfolio theory).

9.2.1. Ядерная оценка плотности

Подход, альтернативный модели К-компонентной смеси, — *ядерная оценка плотности* (kernel density estimator, KDE), которая выделяет один кластерный центр для каждой точки данных. KDE является случаем применения ядерного сглаживания (kernel smoothing), использующего ядра в качестве весов для оценки плотности вероятности. В случае гауссова ядра мы имеем следующее:

$$p(x | D) = \frac{1}{N} \sum_{i=1}^N \mathcal{N}(x | x_i, \sigma^2 I). \quad (9.10)$$

Обратите внимание, что усредняется N гауссовых распределений, причем каждый гауссиан центрирован в точке данных x_i . Мы можем распространить выражение из формулы 9.10 на случай произвольного ядра $\kappa(x)$:

$$p(x | D) = \frac{1}{N} \sum_{i=1}^N \kappa_h(x - x_i). \quad (9.11)$$

Преимущество KDE перед параметрическими моделями, такими как смеси распределений, заключается в том, что не требуется подгонка модели (за исключением точной настройки параметра ширины полосы h) и нет необходимости выбирать количество смесей K . Недостатком является как большой расход памяти для хранения модели, так и высокая временная сложность для оценки. Другими словами, KDE подходит для тех случаев, когда требуется точная оценка плотности для относительно небольшого набора данных (небольшое количество точек N). Рассмотрим пример, в котором анализируются данные РНК-секвенирования для оценки скорости работы промотора T7:

Листинг 9.2. Ядерная оценка плотности

```
import numpy as np
import matplotlib.pyplot as plt

class KDE():
    Длина генома в парах оснований (п. о.)
    def init(self):
        # Гистограмма и оценка с гауссовым ядром, используемые для
        # анализа данных РНК-секвенирования с целью определения скорости работы
        # промотора T7
        self.G = 1e9
        self.C = 1e3  ← Количество уникальных молекул
        self.L = 100  ← Длина прочтения, п. о.
        self.N = 1e6  ← Количество прочтений длиной L п. о.
        self.M = 1e4  ← Количество уникальных прочтений, п. о.
        self.LN = 1000
        self.FDR = 0.05 ← Частота ложных обнаружений
```

Общая длина всех собранных или сопоставленных прочтений РНК-секвенирования

```

# равномерное семплирование (модель Пуассона)
self.lmbda = (self.N * self.L) / self.G  ← Ожидаемое количество охваченных оснований
self.C_est = self.M / (1 - np.exp(-self.lmbda))  ← Оценка размера библиотеки
self.C_cvr = self.G - self.C_est *
    ⇒ np.exp(-self.lmbda)  ← Охват оснований
self.N_gaps = self.N * np.exp(-self.lmbda)  ← Количество разрывов (неохваченных оснований)

# гамма-семплирование априора (отрицательная биномиальная модель)
#X = "number of failures before rth success"
self.k = 0.5  ← Параметр дисперсии (подгонка под данные)
self.p = self.lmbda / (self.lmbda + 1 / self.k)  ← Вероятность успеха
self.r = 1 / self.k  ← Количество успехов

# Данные о связывании РНКП (РНК-секвенирование)
self.data = np.random.negative_binomial(self.r, self.p, size=self.LN)

def histogram(self):
    self.bin_delta = 1  ← Параметр сглаживания
    self.bin_range = np.arange(1, np.max(self.data), self.bin_delta)
    self.bin_counts, _ = np.histogram(self.data, bins=self.bin_range)

    # оценка плотности на базе гистограммы
    # P = интеграл_R p(x) dx, где X лежит в R^3
    # p(x) = K / (N * V), где K = количество точек в области R
    # N = общее количество точек, V = объем области R

    rnap_density_est = self.bin_counts / (sum(self.bin_counts) *
    ⇒ self.bin_delta)
    return rnap_density_est

def kernel(self):
    # Оценка плотности гауссового ядра с параметром сглаживания h
    # сумма N гауссианов, центрированных в каждой точке данных,
    ⇒ параметризованных с помощью общего стандартного отклонения h

    x_dim = 1  ← Размерность x
    h = 10  ← Стандартное отклонение

    rnap_density_support = np.arange(np.max(self.data))
    rnap_density_est = 0
    for i in range(np.sum(self.bin_counts)):
        rnap_density_est += (1 / (2 * np.pi * h ** 2)) ** (x_dim / 2.0) * np.exp(-
        ⇒ (rnap_density_support - self.data[i]) ** 2 / (2.0 * h ** 2))
    # конец for

    rnap_density_est = rnap_density_est / np.sum(rnap_density_est)
    return rnap_density_est

if __name__ == "__main__":
    kde = KDE()
    est1 = kde.histogram()
    est2 = kde.kernel()

```



```
plt.figure()
plt.plot(est1, c='b', label='Гистограмма')
plt.plot(est2, c='r', label='Гауссово ядро')
plt.title("Оценка плотности РНК-секвенирования на базе отрицательной
    ➔ биномиальной модели")
plt.xlabel("Длина считывания, пары оснований"); plt.ylabel("Плотность");
    ➔ plt.legend()
plt.show()
```

На рис. 9.5 показана оценка плотности РНК-секвенирования с использованием отрицательной биномиальной модели. Он демонстрирует оценку плотности на базе гистограммы и оценку плотности с гауссовым ядром. Можно заметить, что гауссова оценка дает гораздо более плавную плотность, а в оценке гистограммы можно дополнительно подобрать размер интервала в качестве параметра сглаживания:

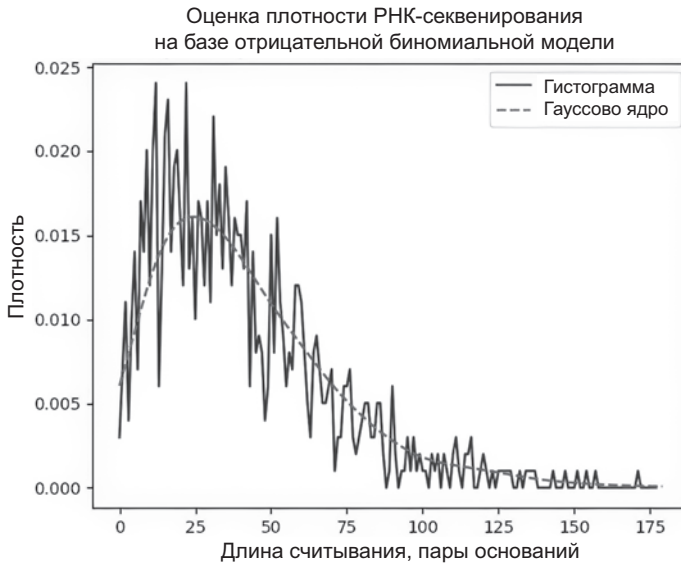


Рис. 9.5. Оценка плотности РНК-секвенирования с помощью гистограммы и гауссового KDE

9.2.2. Оптимизация тангенциального портфеля

Целью анализа средней доходности и вариации является максимизация ожидаемой доходности портфеля при заданном уровне риска, измеряемом стандартным отклонением прошлых показателей доходности. Варьируя пропорции каждого актива, можно достичь различных соотношений между риском и доходностью.

В коде листинга 9.3 мы сначала извлекаем данные о ценах закрытия списка акций. Затем мы исследуем корреляцию цен на акции с помощью матрицы диаграмм рассеяния. После этого мы создаем случайный портфель и вычисляем его риск. Вслед за этим мы генерируем 1000 портфелей, взвешенных случайным образом, и вычисляем их стоимость и риск. Наконец, мы выбираем веса ближайших соседей портфеля таким образом, чтобы свести к минимуму стандартное отклонение и максимизировать стоимость портфеля:

Листинг 9.3. Оптимизация тангенциального портфеля

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.neighbors import KDTree
from pandas.plotting import scatter_matrix
from scipy.spatial import ConvexHull
import pandas_datareader.data as web
from datetime import datetime
import pytz

STOCKS = ['SPY', 'LQD', 'TIP', 'GLD', 'MSFT']

np.random.seed(42)

if __name__ == "__main__":

    plt.close("all")

    # загрузка данных
    # год, месяц, день, час, минута, секунда, микросекунда
    start = datetime(2012, 1, 1, 0, 0, 0, 0, pytz.utc)
    end = datetime(2017, 1, 1, 0, 0, 0, 0, pytz.utc)

    data = pd.DataFrame()
    series = []
    for ticker in STOCKS:
        price = web.DataReader(ticker, 'stoq',
                               ➡ start, end)    ◀ Загрузка данных
        series.append(price['Close'])

    data = pd.concat(series, axis=1)
    data.columns = STOCKS
    data = data.dropna()

    scatter_matrix(data, alpha=0.2, diagonal='kde') ◀ Построение диаграмм
    plt.show()                                     корреляции

    cash = 10000
    num_assets = np.size(STOCKS)    ◀ Получение текущего портфеля
    cur_value = (1e4-5e3)*np.random.rand(num_assets,1) + 5e3
    tot_value = np.sum(cur_value)
```

```

weights = cur_value.ravel()/float(tot_value)

Sigma = data.cov().values
Corr = data.corr().values
volatility = np.sqrt(np.dot(weights.T,
    ➡ np.dot(Sigma, weights)))  ← Вычисление риска портфеля

plt.figure()
plt.title('Correlation Matrix')
plt.imshow(Corr, cmap='gray')
plt.xticks(range(len(STOCKS)),data.columns)
plt.yticks(range(len(STOCKS)),data.columns)
plt.colorbar()
plt.show()

num_trials = 1000
W = np.random.rand(num_trials, np.size(weights))  ← Генерация случайных
                                                    вешов портфеля
W = W/np.sum(W,axis=1).reshape(num_trials,1)  ← Нормировка

pv = np.zeros(num_trials)  ← Стоимость портфеля: w'v
ps = np.zeros(num_trials)  ← Стандартное отклонение портфеля: sqrt(w'Sw)

avg_price = data.mean().values
adj_price = avg_price

for i in range(num_trials):
    pv[i] = np.sum(adj_price * W[i,:])
    ps[i] = np.sqrt(np.dot(W[i,:].T, np.dot(Sigma, W[i,:])))

points = np.vstack((ps,pv)).T
hull = ConvexHull(points)

plt.figure()
plt.scatter(ps, pv, marker='o', color='b', linewidth = 3.0, label =
    ➡ 'Тангенциальный портфель')
plt.scatter(volatility, np.sum(adj_price * weights), marker = 's',
    ➡ color = 'r', linewidth = 3.0, label = 'Текущий')
plt.plot(points[hull.vertices,0], points[hull.vertices,1], linewidth =
    ➡ 2.0)
plt.title('Ожидаемая доходность в сравнении с волатильностью')
plt.ylabel('Ожидаемая цена')
plt.xlabel('Стандартное отклонение портфеля')
plt.legend()
plt.grid(True)
plt.show()

# запрос ближайших соседей портфеля
knn = 5
kdt = KDTree(points)
query_point = np.array([2, 115]).reshape(1,-1)
kdt_dist, kdt_idx = kdt.query(query_point,k=knn)
print("top-%d closest to query portfolios:" %knn)
print("values: ", pv[kdt_idx.ravel()])
print("sigmas: ", ps[kdt_idx.ravel()])

```

На рис. 9.6 (слева) показаны результаты регрессии между парами портфельных активов. Заметьте, например, что SPY не коррелирует с TIP и отрицательно коррелирует с GLD. Кроме того, диагональные графики являются мультимодальными и демонстрируют отрицательную асимметрию для более рискованных активов (ср., например, SPY и LQD). Рисунок 9.6 (справа) также показывает соотношение ожидаемой доходности и риска для набора случайно сгенерированных портфелей.

Граница эффективности определяется набором портфелей в верхней части кривой, которая соответствует максимальной ожидаемой доходности при заданном стандартном отклонении. Добавляя безрисковый актив, можем выбрать портфель по касательной линии с наклоном, равным коэффициенту Шарпа. В следующем разделе вы узнаете, как можно выявлять структуру в реляционных данных:

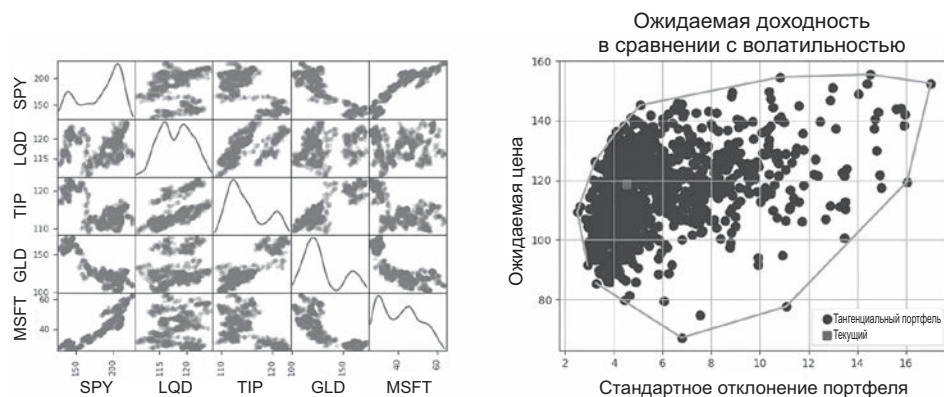


Рис. 9.6. График парных отношений (слева) и тангенциальный портфель (справа)

9.3. Моделирование структуры

В этом разделе мы будем определять структуру графа при заданных реляционных данных. Другими словами, мы хотели бы оценить вероятность графа $G = (V, E)$ при условии наблюдаемых данных D . При выводе структуры графа проблеме представляет экспоненциальная зависимость числа возможных вариантов графа. Например, в ориентированном графе G двум вершинам из V могут соответствовать два возможных ребра: поскольку каждое ребро имеет два возможных направления, у нас потенциально есть $O(2^V V(V-1)/2)$ графов. В силу того что задача моделирования структуры (structure learning) общих графов является NP -трудной, сосредоточимся на приближенных методах. Более конкретно, рассмотрим алгоритм Чоу — Лю (Chow — Liu algorithm) для древовидных графов и оценку обратной ковариационной матрицы для общих графов.

9.3.1. Алгоритм Чоу — Лю

Совместную вероятностную модель для дерева T можно определить следующим образом:

$$p(x|T) = \prod_{t \in V} p(x_t) \prod_{(s,t) \in E} \frac{p(x_s, x_t)}{p(x_s)p(x_t)}. \quad (9.12)$$

Здесь $p(x_t)$ — это частное распределение вершины, а $p(x_s, x_t)$ — частное распределение ребра. Например, для V -образного неориентированного дерева с $|V| = 3$ вершинами мы имеем следующее:

$$\begin{aligned} p(x_1, x_2, x_3 | T) &= p(x_1)p(x_2)p(x_3) \frac{p(x_1, x_2)p(x_2, x_3)}{p(x_1)p(x_2)p(x_2)p(x_3)} = \\ &= \frac{p(x_1, x_2)p(x_2, x_3)}{p(x_2)} = p(x_2)p(x_1 | x_2)p(x_3 | x_2). \end{aligned} \quad (9.13)$$

При выводе алгоритма Чоу — Лю можно использовать древесную декомпозицию в формуле (9.13), чтобы получить выражение для правдоподобия:

$$\begin{aligned} \log P(D | \theta, T) &= \sum_t \sum_k N_{tk} \log p(x_t = k | \theta) + \\ &+ \sum_{s,t} \sum_{j,k} N_{stjk} \log \frac{p(x_s = j, x_t = k | \theta)}{p(x_s = j | \theta)p(x_t = k | \theta)}. \end{aligned} \quad (9.14)$$

Здесь N_{stjk} — это количество раз, когда вершина s находится в состоянии j , а вершина t — в состоянии k , а N_{tk} — это количество раз, когда вершина t находится в состоянии k . Мы можем переписать N_{tk} как $N \times p(x_t = k)$ и, аналогично, N_{stjk} как $N \times p(x_s = j, x_t = k)$. Если мы проведем эти замены в нашем выражении для логарифмического правдоподобия, то получим следующее:

$$\begin{aligned} \frac{1}{N} \log P(D | \theta, T) &= \sum_{t \in V} \sum_k \hat{p}(x_t = k) \log \hat{p}(x_t = k) + \\ &+ \sum_{(s,t) \in E} I(x_s, x_t | \hat{\theta}_{st}). \end{aligned} \quad (9.15)$$

Здесь $I(x_s, x_t | \theta)$ — это взаимная информация между x_s и x_t . Как результат, древовидная топология, максимизирующая логарифмическое правдоподобие, может быть вычислена с помощью обладающего максимальным весом остовного дерева, где веса ребер представляют собой попарную взаимную информацию $I(x_s, x_t | \theta)$. Алгоритм, описываемый уравнением из формулы 9.15, носит имена *Чоу и Лю*. Обратите внимание, что для вычисления максимального остовного дерева мы можем использовать либо алгоритм Прима, либо алгоритмы Крускала, которые могут быть выполнены за $O(E \log V)$ времени. В следующем разделе мы

рассмотрим способ вывода структуры общего графа на основе оценки обратной ковариационной матрицы.

9.3.2. Оценка обратной ковариационной матрицы

Выявление кластеров на фондовом рынке помогает находить похожие компании, что может быть полезно для сравнительного анализа или разработки стратегии парного трейдинга. Похожие кластеры можно найти, произведя оценку обратной ковариационной матрицы (матрицы точности). Последняя может использоваться для построения графа зависимостей с учетом правила, что нули в матрице в точности соответствуют отсутствию ребер в построенном графе. Давайте представим искомую структуру графа в виде гауссовой графовой модели (Gaussian graphical model). Пусть $\Lambda = \Sigma^{-1}$ представляет собой матрицу точности многомерного нормального распределения. Тогда логарифмическое правдоподобие Λ может быть получено следующим образом:

$$l(\Lambda) = \log \det \Lambda - \text{tr}[S\Lambda]. \quad (9.16)$$

Здесь S — эмпирическая ковариационная матрица:

$$S = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)(x_i - \mu)^T. \quad (9.17)$$

Чтобы способствовать разреженности структуры, можно добавить штрафное слагаемое для ненулевых элементов матрицы точности. Таким образом, наша целевая функция отрицательного логарифмического правдоподобия алгоритма графового лассо (graph lasso) выглядит следующим образом:

$$NLL(\Lambda) = -\log \det \Lambda + \text{tr}[S\Lambda] + c \|\Lambda\|_1. \quad (9.18)$$

В коде листинга 9.4 будем использовать разницу между дневными ценами открытия и закрытия для вычисления эмпирической ковариации, которая, в свою очередь, задействуется в алгоритме графового лассо для оценки разреженной матрицы точности. Для кластеризации рынка акций используется метод распространения сродства (affinity propagation), а для отображения многомерных данных на плоскости — линейное вложение (linear embedding):

Листинг 9.4. Оценка обратной ковариационной матрицы

```
import numpy as np
import pandas as pd
from scipy import linalg

from datetime import datetime
import pytz
```

```

from sklearn.datasets import make_sparse_spd_matrix
from sklearn.covariance import GraphicalLassoCV, ledoit_wolf
from sklearn.preprocessing import StandardScaler
from sklearn import cluster, manifold

import seaborn as sns
import matplotlib.pyplot as plt
from matplotlib.collections import LineCollection

import pandas_datareader.data as web

np.random.seed(42)

def main():

    # генерация данных (синтетических)
    # num_samples = 60
    # num_features = 20
    # prec = make_sparse_spd_matrix(num_features, alpha=0.95,
    #                               smallest_coef=0.4, largest_coef=0.7)
    # cov = linalg.inv(prec)
    # X = np.random.multivariate_normal(np.zeros(num_features), cov,
    #                                   size=num_samples)
    # X = StandardScaler().fit_transform(X)

    # генерация данных (фактических)
    STOCKS = {
        'SPY': 'S&P500',
        'LQD': 'Bond_Corp',
        'TIP': 'Bond_Treas',
        'GLD': 'Gold',
        'MSFT': 'Microsoft',
        'XOM': 'Exxon',
        'AMZN': 'Amazon',
        'BAC': 'BoFA',
        'NVS': 'Novartis'}

    symbols, names = np.array(list(STOCKS.items())).T

    # загрузка данных
    # год, месяц, день, час, минута, секунда, микросекунда
    start = datetime(2015, 1, 1, 0, 0, 0, 0, pytz.utc)
    end = datetime(2017, 1, 1, 0, 0, 0, 0, pytz.utc)

    qopen, qclose = [], []
    data_close, data_open = pd.DataFrame(), pd.DataFrame()
    for ticker in symbols:
        price = web.DataReader(ticker, 'stooq', start, end)
        qopen.append(price['Open'])
        qclose.append(price['Close'])

    data_open = pd.concat(qopen, axis=1)
    data_open.columns = symbols

```

```

data_close = pd.concat(qclose, axis=1)
data_close.columns = symbols

variation = data_close - data_open
variation = variation.dropna()

X = variation.values
X /= X.std(axis=0)

graph = GraphicalLassoCV()
graph.fit(X)

gl_cov = graph.covariance_
gl_prec = graph.precision_
gl_alphas = graph.cv_alphas_
gl_scores = graph.cv_results_['mean_test_score']

plt.figure()
sns.heatmap(gl_prec, xticklabels=names, yticklabels=names)
plt.xticks(rotation=45)
plt.yticks(rotation=45)
plt.tight_layout()
plt.show()

plt.figure()
plt.plot(gl_alphas, gl_scores, marker='o', color='b', lw=2.0,
        ➤ label='GraphLassoCV')
plt.title("Graph Lasso Alpha Selection")
plt.xlabel("alpha")
plt.ylabel("score")
plt.legend()
plt.show()

_, labels = cluster.affinity_propagation(gl_cov)
num_labels = np.max(labels)

for i in range(num_labels+1):
    print("Cluster %i: %s" %((i+1), ', '.join(names[labels==i])))

node_model = manifold.LocallyLinearEmbedding(
    ➤ n_components=2, n_neighbors=6, eigen_solver='dense')
embedding = node_model.fit_transform(X.T).T

# создание графиков
plt.figure()
plt.clf()
ax = plt.axes([0.,0.,1.,1.])
plt.axis('off')

partial_corr = gl_prec
d = 1 / np.sqrt(np.diag(partial_corr))
non_zero = (np.abs(np.triu(partial_corr, k=1)) >
    ➤ 0.02)

```

Дневное изменение цены
для каждого биржевого символа

Нормировка для задействования
корреляции вместо ковариации

Оценка обратной ковариационной
матрицы

Кластеризация по принципу
распространения средства

Поиск низкораз-
мерного вложения
для визуализации

Матрица связности


```

# отрисовка вершин
plt.scatter(embedding[0], embedding[1], s = 100*d**2, c = labels, cmap
↳ = plt.cm.Spectral)

# отрисовка ребер
start_idx, end_idx = np.where(non_zero)
segments = [[embedding[:,start], embedding[:,stop]] for start, stop in
↳ zip(start_idx, end_idx)]
values = np.abs(partial_corr[non_zero])
lc = LineCollection(segments, zorder=0, cmap=plt.cm.hot_r,
↳ norm=plt.Normalize(0,0.7*values.max()))
lc.set_array(values)
lc.set_linewidths(2*values)
ax.add_collection(lc)

# отрисовка меток
for index, (name, label, (x,y)) in enumerate(zip(names, labels,
↳ embedding.T)):
    plt.text(x,y,name,size=12)

plt.show()

if __name__ == "__main__":
    main()

```

На рис. 9.7 показана разреженная матрица точности, рассчитанная с помощью алгоритма графового лассо.

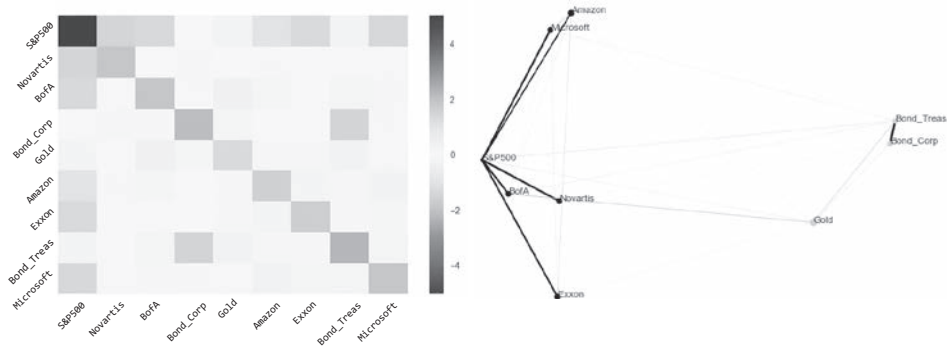


Рис. 9.7. Полученная с помощью метода графового лассо матрица точности (слева) и кластеры акций (справа)

Значения по краям матрицы точности на рис. 9.7, превышающие пороговое значение, соответствуют связанным компонентам, на основе которых можно вычислить кластеры фондового рынка, как показано в правой части рисунка. В следующем разделе вы познакомитесь с алгоритмом минимизации энергии, который называется имитацией отжига.

9.4. Метод имитации отжига

Имитация отжига (simulated annealing, SA) — это эвристический метод поиска, который для избежания попадания в локальный оптимум разрешает производить нерегулярные переходы в менее благоприятные состояния. SA можно сформулировать как задачу минимизации энергии с температурным параметром T следующим образом:

$$\alpha = \exp\left\{\frac{E_{old} - E_{new}}{T}\right\} \quad (9.19)$$

$$p = \min(1, \alpha).$$

При переходе в новое состояние желательно, чтобы энергия в новом состоянии была ниже. В таком случае $\alpha > 1$ и $p = 1$, это означает, что мы принимаем переход в другое состояние с вероятностью $p = 1$. С другой стороны, если энергия нового состояния выше, то $\alpha < 1$, и в этом случае мы принимаем переход с вероятностью $p = \alpha$. Другими словами, мы принимаем невыгодные переходы с вероятностью, пропорциональной разнице энергий между состояниями и обратно пропорциональной температурному параметру T . Изначально температура T высока, что позволяет осуществлять множество случайных переходов. По мере снижения температуры, в соответствии с режимом охлаждения (cooling schedule), разница в энергии становится более заметной. Схема, описанная формулами 9.19, позволяет методу имитации отжига избегать локальных оптимумов. Теперь мы готовы рассмотреть псевдокод, представленный на рис. 9.8.

В классе `simulated_annealing` основной функцией является `run`. Мы начинаем с инициализации температуры отжига T и вычисления в исходном положении целевой функции `target`. Напомню, что нас интересует нахождение точки минимума в сложном энергетическом ландшафте, представленном целевой функцией. Мы делаем выборку из нашего вспомогательного (proposal) распределения, чтобы получить новый набор координат и оценить энергию предложенных координат. Затем, чтобы принять решение о выходе из цикла или его продолжении, проверяем сходимости. После этого вычисляем вероятность перехода α как разницу между старым и новым энергетическими состояниями, деленную на температуру T . В случае $r > 1$ принимаем низкоэнергетическое состояние с вероятностью 1, а в случае $0 < r < 1$ (когда энергия нового состояния выше) принимаем переход с вероятностью $r = \alpha$. В качестве последнего шага регулируем температуру в соответствии с режимом охлаждения и при наличии сходимости возвращаем оптимальные координаты (те, при которых достигается минимальная энергия, полученная при моделировании отжига). Теперь мы готовы к знакомству с полной реализацией алгоритма имитации отжига, представленной в следующем листинге:

```

1: class simulated_annealing
2: function run(x_init, y_init):
3: converged = False
4: T = 1
5: x_old, y_old = x_init, y_init
6: energy_old = target(x_init, y_init)
7: while not converged:
8:     x_new, y_new = proposal(x_old, y_old) ← Семплирование
9:     energy_new = target(x_new, y_new) ← вспомогательного распределения
10:    converged = check_convergence()
11:    alpha = exp((energy_old - energy_new)/T)
12:    r = min(1, alpha) ← Расчет энергии
13:    u = Unif[0,1]
14:    if u < r:
15:        x_old, y_old = x_new, y_new
16:        energy_old = energy_new → Принятие предложенного состояния
17:    end if
18:    T = temperature_schedule()
19: end while
20: x_opt, y_opt = x_old, y_old
21: return x_opt, y_opt

```

Рис. 9.8. Псевдокод имитации отжига

Листинг 9.5. Имитация отжига

```

import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

class simulated_annealing():
    def __init__(self):
        self.max_iter = 1000
        self.conv_thresh = 1e-4
        self.conv_window = 10

        self.samples = np.zeros((self.max_iter, 2))
        self.energies = np.zeros(self.max_iter)
        self.temperatures = np.zeros(self.max_iter)

    def target(self, x, y): ← Энергетический ландшафт
        z = 3*(1-x)**2 * np.exp(-x**2 - (y+1)**2) \
            - 10*(x/5 - x**3 - y**5) * np.exp(-x**2 - y**2) \
            - (1/3)*np.exp(-(x+1)**2 - y**2)
        return z

    def proposal(self, x, y):
        mean = np.array([x, y])

```

```

cov = 1.1 * np.eye(2)
x_new, y_new = np.random.multivariate_normal(mean, cov)
return x_new, y_new

def temperature_schedule(self, T, iter):
    return 0.9 * T

def run(self, x_init, y_init):

    converged = False
    T = 1
    self.temperatures[0] = T
    num_accepted = 0
    x_old, y_old = x_init, y_init
    energy_old = self.target(x_init, y_init)

    iter = 1
    while not converged:
        print("iter: {:4d}, temp: {:.4f}, energy = {:.6f}".format(iter,
            ➡ T, energy_old))
        x_new, y_new = self.proposal(x_old, y_old)
        energy_new = self.target(x_new, y_new)

        if iter > 2*self.conv_window:      ← Проверка на сходимость
            vals = self.energies[iter-self.conv_window : iter-1]
            if (np.std(vals) < self.conv_thresh):
                converged = True
            # конец if
        # конец if

        alpha = np.exp((energy_old - energy_new)/T)
        r = np.minimum(1, alpha)
        u = np.random.uniform(0, 1)
        if u < r:
            x_old, y_old = x_new, y_new
            num_accepted += 1
            energy_old = energy_new
        # конец if
        self.samples[iter, :] = np.array([x_old, y_old])
        self.energies[iter] = energy_old

        T = self.temperature_schedule(T, iter)
        self.temperatures[iter] = T

        iter = iter + 1

        if (iter > self.max_iter): converged = True
    # конец while

    niter = iter - 1
    acceptance_rate = num_accepted / niter
    print("acceptance rate: ", acceptance_rate)

```

```

x_opt, y_opt = x_old, y_old

return x_opt, y_opt, self.samples[:niter,:], self.energies[:niter],
    ➤ self.temperatures[:niter]

if __name__ == "__main__":

    SA = simulated_annealing()

    nx, ny = (1000, 1000)
    x = np.linspace(-2, 2, nx)
    y = np.linspace(-2, 2, ny)
    xv, yv = np.meshgrid(x, y)

    z = SA.target(xv, yv)
    plt.figure()
    plt.contourf(x, y, z)
    plt.title("Энергетический ландшафт")
    plt.show()

    # поиск глобального минимума с помощью исчерпывающего поиска
    min_search = np.min(z)
    argmin_search = np.argwhere(z == min_search)
    xmin, ymin = argmin_search[0][0], argmin_search[0][1]
    print("global minimum (exhaustive search): ", min_search)
    print("located at (x, y): ", x[xmin], y[ymin])

    # поиск глобального минимума с помощью имитационного отжига
    x_init, y_init = 0, 0
    x_opt, y_opt, samples, energies, temperatures = SA.run(x_init, y_init)
    print("global minimum (simulated annealing): ", energies[-1])
    print("located at (x, y): ", x_opt, y_opt)

    plt.figure()
    plt.plot(energies)
    plt.title("SA-моделированные значения энергии")
    plt.show()

    plt.figure()
    plt.plot(temperatures)
    plt.title("Temperature Schedule")
    plt.show()

```

На рис. 9.9 показан итоговый энергетический ландшафт (слева) и SA-моделированные значения энергии (справа).

В примере на рис. 9.9 нам удалось, используя метод имитации отжига, найти глобальный минимум энергетического ландшафта, который соответствует глобальному минимуму, найденному путем исчерпывающего поиска. В следующем разделе мы рассмотрим генетический алгоритм, разработанный на основе методов эволюционной биологии.

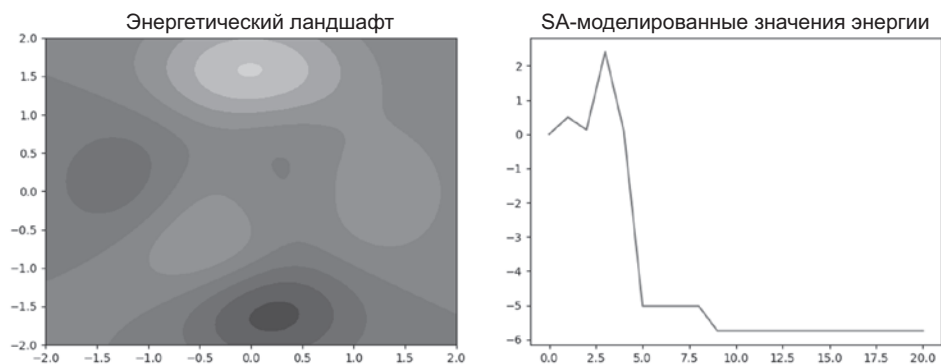


Рис. 9.9. Энергетический ландшафт (слева) и SA-моделированные значения энергии (справа)

9.5. Генетический алгоритм

Генетические алгоритмы (ГА) подсказаны эволюционной биологией и созданы по ее образцу. Они моделируют популяцию (случайно инициализированных) геномов особей, которые оцениваются на предмет их приспособленности (fitness) в соответствии с заданной целью. Две особи скрещиваются, объединяя свои геномы. За этапом скрещивания следует мутация, при которой геномы особей потомства могут изменяться в соответствии с частотой мутаций. Потомки добавляются в популяцию и оцениваются с помощью функции приспособленности (fitness function), которая определяет, выживет ли данная особь в популяции или нет. Рассмотрим псевдокод на рис. 9.10. В этом примере процессу эволюции будет подвергнута случайная строка, с задачей добиться ее соответствия целевой строке.

Класс `GeneticAlgorithm` состоит из следующих функций: `calculate_fitness`, `mutate`, `crossover` и `run`. В функции `calculate_fitness` мы вычисляем степень приспособленности как $1/\text{loss}$, где `loss` — расстояние между символами строки особи и символами целевой ASCII-строки. В функции `mutate` мы случайным образом меняем символы строки особи в соответствии с частотой мутаций (mutation rate). В функции `crossover` выбираем индекс, по которому будем случайным образом скрещивать геномы родителей, а затем проводим скрещивание геномов родителей по выбранному индексу, в результате чего получаются два дочерних гена. Наконец, в функции `run` инициализируем популяцию и вычисляем ее приспособленность. После этого находим наиболее приспособленную особь в популяции и сравниваем ее с целевой. Если приспособленности двух особей совпадают, то выходим из цикла и возвращаем наиболее приспособленную из них. В противном случае выбираем двух родителей в соответствии с их вероятностями, ранжированными по приспособленности; проводим скрещивание для получения потомства;

мутируем каждое потомство; а затем добавляем потомство обратно в новую популяцию. Повторяем этот процесс заданное количество раз или до тех пор, пока цель не будет найдена. Давайте теперь подробно рассмотрим реализацию генетического алгоритма:

```

1: class GeneticAlgorithm
2:   function calculate_fitness(population, target):
3:     for individual in population:
4:       рассчитать loss как расстояние между приспособленностью особи и целью
5:       вычислить приспособленность: fitness = 1 / (loss + epsilon)
6:     конец for
7:   return population_fitness
8:   function mutate(individual, mutation_rate):
9:     с вероятностью mutation_rate изменить символы случайным образом
10:  return new_individual
11:  function crossover(parent1, parent2):
12:    cross_idx = random_integer(0, len(parent1))
13:    child1 = parent1[:cross_idx] + parent2[cross_idx:]
14:    child2 = parent2[:cross_idx] + parent1[cross_idx:]
15:  return child1, child2
16:  function run(target, population_size, mutation_rate):
17:    init_population()
18:    for epoch in num_iter:
19:      population_fitness = calculate_fitness(population, target)
20:      fittest_individual = population(argmax(population_fitness))
21:      if fittest_individual == target:
22:        return fittest_individual
23:      конец if
24:      parent_probabilities = fitness / sum(population_fitness)
25:      new_population = []
26:      for i in population_size:
27:        parent1, parent2 = random_choice(population, p= parent_probabilities)
28:        child1, child2 = crossover(parent1, parent2)
29:        new_population += [mutate(child1), mutate(child2)]
30:      конец for
31:    конец for

```

Рис. 9.10. Псевдокод генетического алгоритма

Листинг 9.6. Реализация генетического алгоритма

```

import numpy as np
import string

class GeneticAlgorithm():

    def __init__(self, target_string, population_size, mutation_rate):

```

```

self.target = target_string
self.population_size = population_size
self.mutation_rate = mutation_rate
self.letters = [" "] + list(string.ascii_letters)

def initialize(self):  ← Инициализация популяции случайными строками
    self.population = []
    for _ in range(self.population_size):
        individual = "".join(np.random.choice(self.letters,
        ➡ size=len(self.target)))
        self.population.append(individual)

def calculate_fitness(self):  ← Расчет приспособленности особей популяции
    population_fitness = []
    for individual in self.population:
        loss = 0  ← Вычисление потерь (loss) как расстояния между символами
        for i in range(len(individual)):
            letter_i1 = self.letters.index(individual[i])
            letter_i2 = self.letters.index(self.target[i])
            loss += abs(letter_i1 - letter_i2)
        fitness = 1 / (loss + 1e-6)
        population_fitness.append(fitness)
    return population_fitness

def mutate(self, individual):  ← Случайная замена символов с вероятностью,
                                равной частоте мутаций
    individual = list(individual)
    for j in range(len(individual)):
        if np.random.random() < self.mutation_rate:
            individual[j] = np.random.choice(self.letters)
    return "".join(individual)

def crossover(self, parent1, parent2):  ← Создание потомков от родителей
                                        путем скрещивания
    cross_i = np.random.randint(0, len(parent1))
    child1 = parent1[:cross_i] + parent2[cross_i:]
    child2 = parent2[:cross_i] + parent1[cross_i:]
    return child1, child2

def run(self, iterations):
    self.initialize()

    for epoch in range(iterations):
        population_fitness = self.calculate_fitness()

        fittest_individual =
        ➡ self.population[np.argmax(population_fitness)]
        highest_fitness = max(population_fitness)
        if fittest_individual == self.target:
            break
        parent_probabilities = [fitness / sum(population_fitness) for
        ➡ fitness in population_fitness]  ← Выбор родителей пропорционально их приспособленности

        new_population = []  ← Следующее поколение
        for i in np.arange(0, self.population_size, 2):
            parent1, parent2 = np.random.choice(self.population,
            ➡ size=2, p=parent_probabilities, replace=False)  ← Выбор двух родителей

```



```

        child1, child2 = self.crossover(parent1, parent2)
        new_population += [self.mutate(child1), self.mutate(child2)]
        print("iter %d, closest candidate: %s, fitness: %.4f" %(epoch,
            ➡ fittest_individual, highest_fitness))
        self.population = new_population
        print("iter %d, final candidate: %s" %(epoch, fittest_individual))

    Сохранение мутировавшего потомства
    для следующего поколения

if __name__ == "__main__":

    target_string = "Genome"
    population_size = 50
    mutation_rate = 0.1

    ga = GeneticAlgorithm(target_string, population_size, mutation_rate)
    ga.run(iterations = 1000)

```

Скрещивание для
получения потомства

Как показывают выходные данные, мы смогли создать целевую последовательность `Genome` посредством эволюции случайно инициализированных буквенных последовательностей. В следующем разделе мы подробнее остановимся на изученных темах по материалам анализа исследовательской литературы, посвященной обучению без учителя.

9.6. Исследования в области ML: обучение без учителя

В этом разделе содержатся дополнительные соображения и материалы исследований по темам, обсуждаемым ранее в данной главе. Мы впервые столкнулись с байесовской непараметрической моделью на материалах метода *K* средних с процессом Дирихле. Количество параметров в таких моделях увеличивается с увеличением объема данных, и, следовательно, байесовские непараметрические модели лучше подходят для моделирования задач из реальной жизни. *DP-means* можно рассматривать как асимптотическую аппроксимацию с малой дисперсией (small variance asymptotics, SVA) модели смеси процессов Дирихле. Одним из главных преимуществ байесовских непараметрических моделей является то, что они могут применяться для моделирования бесконечных смесей, а иерархические расширения — для совместного использования кластеров несколькими группами данных.

Мы рассмотрели ЕМ-алгоритм, который представляет собой мощную оптимизационную методику, широко используемую в машинном обучении. Существует несколько расширений ЕМ-алгоритма. Среди них можно упомянуть такие, как онлайн-ЕМ, который работает с онлайн- или потоковыми данными. Другим примером является ЕМ с применением имитации отжига (annealed EM),

который использует температурный параметр для сглаживания энергетического ландшафта во время оптимизации для отслеживания глобального оптимума. Еще одно расширение — вариационный ЕМ (variational ЕМ), который заменяет точный вывод на этапе Е вариационным выводом. Наконец, существует Монте-Карло ЕМ, который на Е-шаге производит семплирование из неразрешимого распределения (intractable distribution), и несколько других методов.

Мы рассмотрели два способа снижения размерности с целью отбора признаков и визуализации данных. Существует несколько других способов моделирования базового многообразия данных, таких как Isomap, который является расширением метода Kernel PCA, стремящимся сохранить геодезические расстояния между всеми точками. Также стоит упомянуть локально-линейное вложение (locally linear embedding, LLE), которое можно рассматривать как серию локальных PCA, сравниваемых глобально для поиска наилучшего нелинейного вложения. Еще одним методом является спектральное встраивание, основанное на декомпозиции лапласиана графа. И наконец, используется многомерное шкалирование (multidimensional scaling, MDS), при котором расстояния во вложении хорошо отражают расстояния в исходном многомерном пространстве. Заметьте, что некоторые из этих методов можно объединить друг с другом, например PCA, который можно использовать для предварительной обработки данных и инициализации t-SNE с целью снижения вычислительной сложности.

Мы рассмотрели мощный метод поиска тем в текстовых документах с использованием вариационного байесовского алгоритма для скрытого размещения Дирихле, для которого также существует несколько расширений. К примеру, коррелированная тематическая модель (correlated topic model) фиксирует взаимосвязи между темами, динамическая тематическая модель (dynamic topic model) отслеживает эволюцию тем с течением времени, а контролируемая модель LDA (supervised LDA) может использоваться для выставления оценок документам с целью определения их качества.

Мы изучили проблему оценки плотности с использованием ядер и обнаружили влияние параметра сглаживания на результирующую оценку. Метод KDE можно реализовать более эффективно, если использовать дерево шаров (ball tree) или k-d-дерево для сокращения временных затрат на запрос данных.

Что касается моделирования структуры, то мы обнаружили экспоненциальное число возможных топологий графов и затронули тему причинной связи. Мы познакомились с тем, как можно построить более простые древовидные графы, используя взаимную информацию между узлами в качестве весов ребер в основном дереве с максимальным весом. Мы также увидели, как регуляризация обратной ковариационной матрицы для общих графов приводит к созданию проще интерпретируемых топологий с меньшим количеством ребер в выводимом графе.

Наконец, мы рассмотрели два алгоритма обучения без учителя, взятые из статистической физики (имитация отжига) и эволюционной биологии (генетический

алгоритм). Мы увидели, как, используя температурный параметр и режим охлаждения, можем изменить энергетический ландшафт и как краткосрочный выбор неблагоприятных вариантов может привести к улучшению долгосрочных оптимумов. Существует множество NP-трудных задач, которые могут быть аппроксимированы с помощью имитационного отжига, включая задачу коммивояжера (traveling salesman problem, TSP). Чтобы достичь оптимальных результатов, можно прибегнуть к повторным перезапускам и использованию разных точек инициализации. С другой стороны, генетические алгоритмы, хотя и подкупают присущим им параллелизмом, могут быть при этом медленно сходящимися. Но все же они могут найти некоторые интересные применения, такие как поиск архитектуры нейронных сетей.

9.7. Упражнения

- 9.1. Покажите, что латентное размещение Дирихле можно интерпретировать как неотрицательное матричное разложение.
- 9.2. Объясните, почему разреженность желательна при выводе структуры общего графа.
- 9.3. Перечислите несколько NP-трудных задач, которые можно аппроксимировать с помощью алгоритма имитации отжига.
- 9.4. Проведите мозговой штурм по поиску задач, которые можно успешно решить с помощью генетического алгоритма.

Итоги

- Латентное размещение Дирихле (LDA) — это тематическая модель, которая представляет каждый документ в виде конечного набора тем, где тема является распределением по словам. Цель данного метода состоит в том, чтобы найти распределение общих тем и пропорции тем для каждого документа.
- Распространенным методом корректировки количества употреблений слов является показатель $tf-idf$, который логарифмически сводит к нулю количество слов, часто встречающихся в документах: $A \log(D/n_t)$, где D — общее количество документов в корпусе, а n_t — количество документов, в которых встречается термин t .
- Целью оценки плотности является моделирование плотности вероятности данных. При ядерной оценке плотности (KDE) для каждой точки данных выделяется один центр кластера.
- Целью анализа средней доходности и вариации является максимизация ожидаемой доходности портфеля при заданном уровне риска, измеряемом стандартным отклонением прошлых показателей доходности.

- Поскольку задача моделирования структуры общих графов является NP-трудной, мы сосредоточились на приближенных методах, а именно: мы рассмотрели алгоритм Чоу — Лю для древовидных графов, а также оценку обратной ковариационной матрицы для общих графов.
- Имитационный отжиг (SA) — это эвристический метод поиска, который позволяет время от времени переходить в менее благоприятные состояния, чтобы избежать локальных оптимумов.
- Мы можем сформулировать имитационный отжиг как задачу минимизации энергии с температурным параметром T , с помощью которой можем изменить энергетический ландшафт и выбрать действия, неблагоприятные в краткосрочной перспективе, но способные привести к лучшим долгосрочным оптимумам.
- Генетические алгоритмы (ГА) основаны на принципах эволюционной биологии и созданы по ее образцу. Они моделируют (случайно инициализированные) геномы отдельных особей популяции, которые оцениваются на предмет их приспособленности к заданным условиям.
- В генетических алгоритмах две особи объединяют свои геномы в процессе скрещивания. За этим этапом следует мутация, посредством которой основания геномов особей могут изменяться в соответствии с частотой мутаций. Полученное в результате потомство добавляется в популяцию и оценивается по уровню своей приспособленности.

Часть 4

Глубокое обучение

В четвертой части книги вы познакомитесь с алгоритмами глубокого обучения, являющимися подвидом машинного обучения с учителем, который мы рассмотрели во второй части книги. Они произвели своего рода революцию и сделали пригодными для машинного обучения некоторые исследовательские и бизнес-задачи, которые ранее считались недоступными для классических алгоритмов ML.

В главе 10 мы начнем с основ, таких как многослойный перцептрон и сверточные модели LeNet для классификации цифр MNIST, а затем перейдем к более продвинутым задачам, например поиску изображений на базе сверточной нейронной сети ResNet50. Мы рассмотрим рекуррентные нейронные сети, применяемые для классификации последовательностей с использованием LSTM, и с нуля реализуем модель с несколькими входами для определения сходства последовательностей. Наконец, мы проведем сравнительный анализ различных алгоритмов оптимизации, используемых для обучения глубоких нейросетей.

В главе 11 мы изучим еще более передовые алгоритмы глубокого обучения. Мы углубимся в генеративные модели, построенные на базе вариационных автоэнкодеров, и самостоятельно реализуем детектор аномалий для данных временных рядов. Мы также познакомимся с интригующей комбинацией нейронных сетей и вероятностных графовых моделей и с нуля реализуем сети смешанной плотности (mixture density network). Далее мы обсудим мощную архитектуру трансформера и применим ее к задаче классификации текстов. Наконец, мы рассмотрим графовые нейронные сети и используем одну из них для классификации узлов в графе цитирования. В заключение мы проведем обзор исследований в области ML, сделав акцент на глубоком обучении.

10

Фундаментальные алгоритмы глубокого обучения

В этой главе

- ✓ Многослойный перцептрон
- ✓ Сверточные нейронные сети: LeNet на датасете MNIST и ResNet для поиска изображений
- ✓ Рекуррентные нейронные сети: LSTM-классификация последовательностей и нейросеть с несколькими входами
- ✓ Оптимизаторы нейронных сетей

В предыдущей главе мы обсуждали отдельные алгоритмы ML без учителя, которые помогают находить закономерности в данных. В этой главе вы познакомитесь с алгоритмами глубокого обучения. Они являются частью обучения с учителем, которое мы рассмотрели в главах 5, 6 и 7. Алгоритмы глубокого обучения произвели революцию в отрасли и позволили распространить использование ML на прежде считавшиеся неподвластными классическим алгоритмам исследовательские и бизнес-задачи. Мы начнем эту главу с основных архитектур, таких как многослойный перцептрон (МСП) и сверточная модель LeNet для классификации цифр MNIST. Далее продолжим изучение этих тем на примере более сложных задач, например поиска изображений с помощью сверточной нейронной сети (convolutional neural network, CNN) ResNet50. Мы подробно рассмотрим рекуррентные нейронные сети (recurrent neural network, RNN) на примере классификации последовательностей с использованием архитектуры

долгой краткосрочной памяти (long short-term memory, LSTM) и с нуля реализуем многоходовую модель схождения последовательностей. Затем обсудим различные алгоритмы оптимизации, используемые для обучения нейронных сетей, и проведем их сравнительное исследование. На протяжении всей этой главы мы будем задействовать библиотеку глубокого обучения Keras/TensorFlow.

10.1. Многослойный перцептрон

Многослойный перцептрон (MLP, multi-layer perceptron) обычно используется для задач классификации или регрессии. MLP состоит из множества плотно соединенных слоев, которые выполняют следующую линейную (аффинную) операцию:

$$f(x; \theta) = Wx + b. \quad (10.1)$$

Здесь x — входной вектор, W — матрица весов, а b — член смещения. Хотя линейность операции упрощает вычисления, она становится ограничивающим фактором, когда дело доходит до сборки из нескольких слоев. Введение нелинейности между слоями при помощи функций активации позволяет придать модели большую выразительную силу. Одной из часто используемых нелинейностей является функция ReLU, определяемая следующим образом:

$$\text{ReLU}(x) = \max(0, x). \quad (10.2)$$

Мы можем определить MLP с L слоями как композицию функций $f_l(x; \theta) = \text{ReLU}(W_{lx} + b_l)$, как представлено ниже:

$$\text{MLP}(x; \theta) = f_L(f_{L-1}(\dots(f_1(x; \theta_1))\dots)). \quad (10.3)$$

Функция активации последнего слоя f_L будет зависеть от поставленной задачи: многопеременная логистическая функция (softmax) для классификации или функция тождества для регрессии. На основе вещественнозначных входных данных активация softmax вычисляет вероятности классов, которые в сумме дают 1. Она определяется следующим образом:

$$\text{softmax}(x) = \frac{e^{x_i}}{\sum_{i=1}^K e^{x_i}}. \quad (10.4)$$

Здесь K — это количество классов. Рисунок 10.1 демонстрирует архитектуру MLP.

Чтобы обучить нейронную сеть, нам нужно ввести функцию потерь. *Функция потерь* (loss function) показывает, насколько далеко выходные данные сети находятся от ожидаемых эталонных значений. Выбор функции потерь зависит от задачи, для решения которой мы проводим оптимизацию, и в случае MLP это

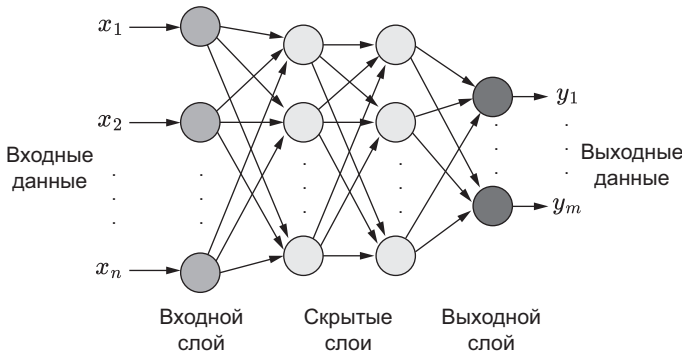


Рис. 10.1. Архитектура MLP

может быть, например, кросс-энтропия $H(p, q)$ для классификации или средне-квадратичная ошибка (MSE) для регрессии:

$$\begin{aligned}
 LH(y, \hat{y}) = H(p, q) &= -\sum_{x \in X} p(x) \log q(x) = -\sum_{k=1}^K y_k \log \hat{y}_k; \\
 L_{MSE}(y, \hat{y}) &= MSE(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2.
 \end{aligned} \tag{10.5}$$

Здесь y — истинная метка, а $p(x)$ — истинное распределение; $\hat{y}$ — предсказанная метка, а $q(x)$ — предсказанное распределение.

MLP известны как сети с прямой связью (feed-forward network), поскольку они от входа к выходу описываются прямым вычислительным графом. Обучение нейронной сети состоит из прямого и обратного проходов. На этапе вывода (прямой проход) каждый слой выдает прямое сообщение (forward message) $z = f(x; \theta)$ в качестве выходных данных (с учетом текущих весов сети), за которым следует вычисление функции потерь. В процессе обратного прохода каждый слой принимает обратное сообщение (backward message) dL/dz последующего слоя и выдает два обратных сообщения на выходе: градиент dL/dx относительно предыдущего слоя x и градиент $dL/d\theta$ по параметрам текущего слоя.

Этот алгоритм обратного распространения основан на цепном правиле (chain rule) и может быть в обобщенном виде представлен следующим образом:

$$\begin{aligned}
 \frac{\partial L}{\partial x_i} &= \sum_j \frac{\partial L}{\partial z_j} \frac{\partial z_j}{\partial x_i}; \\
 \frac{\partial L}{\partial \theta_i} &= \sum_j \frac{\partial L}{\partial z_j} \frac{\partial z_j}{\partial \theta_i}.
 \end{aligned} \tag{10.6}$$

Как только мы знаем величину градиента по параметрам для каждого слоя, мы можем обновить параметры слоя следующим образом:

$$\theta_i^t = \theta_i^{t-1} - \lambda \frac{\partial L}{\partial \theta_i}. \quad (10.7)$$

Здесь λ — это скорость обучения. Обратите внимание, что градиенты вычисляются автоматически библиотеками глубокого обучения, такими как TensorFlow или PyTorch.

Во время обучения имеет смысл придерживаться графика изменения скорости обучения (learning rate schedule), чтобы избежать локальных оптимумов и достичь сходимости. Обычно мы начинаем с некоторой постоянной скорости обучения и снижаем ее в соответствии с графиком после определенного количества эпох, где одна эпоха — это один проход по обучающему набору данных. На протяжении всей этой главы мы будем использовать экспоненциальный график скорости обучения. Среди других вариантов можно назвать кусочно-линейный график с последовательным сокращением вдвое, график косинусного отжига (cosine annealing) и график по принципу «один цикл» (one-cycle schedule).

Прежде чем мы перейдем к реализации MLP, полезно поговорить о емкости модели (model capacity) и о регуляризации для предотвращения переобучения. Регуляризация может происходить на разных уровнях, поскольку применяются различные техники, такие как уменьшение весов (weight decay), ранний останов или прореживание (dropout). На верхнем уровне, если обнаруживается, что модель недообучается или не достигает высокой доли верных результатов на этапе валидации, то возникает желание увеличить емкость модели, поменяв ее архитектуру (например, увеличив количество слоев и количество скрытых узлов на слой). С другой стороны, чтобы избежать переобучения, мы можем ввести уменьшение весов, или l2-регуляризацию, применяемую к весам помимо смещения (W , но не b) каждого слоя. Эта методика способствует уменьшению значений весов и, следовательно, созданию более простых моделей. Другим видом регуляризации является ранний останов обучения в момент, когда становится заметно, что валидационные потери начинают увеличиваться по сравнению с потерями обучения. Подобный критерий раннего останова в комбинации с сохранением контрольной точки модели может применяться для экономии времени и вычислительных ресурсов. Наконец, прореживание — это эффективный метод регуляризации, при котором плотные соединения выбрасываются или обнуляются случайным образом в соответствии с фиксированной долей, как показано на рис. 10.2. Прореживание позволяет улучшить обобщающую способность модели — см. статью Нитиша Шриваставы (Nitish Srivastava) и др. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting» (Journal of Machine Learning Research, 2014):

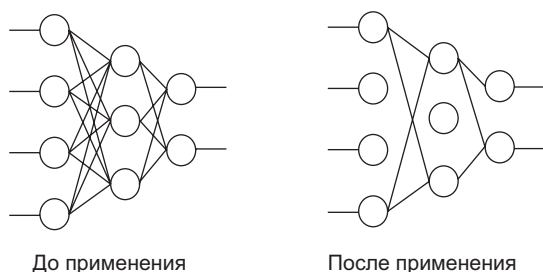


Рис. 10.2. Прореживание, примененное к многослойному перцептрону, на одной эпохе обучения

Давайте объединим принципы, которые мы изучили к настоящему времени, в нашей первой реализации многослойного перцептрона (MLP) с помощью библиотеки Keras/TensorFlow! За дополнительной информацией о библиотеке Keras читатель может обратиться на сайт <https://keras.io/> или ознакомиться с книгой Франсуа Шолле (François Chollet) «Deep Learning with Python» (Manning, 2021)¹.

Листинг 10.1. Многослойный перцептрон

```
import numpy as np
import tensorflow as tf
from tensorflow import keras

from keras.models import Sequential
from keras.layers import Dense, Dropout

from keras.callbacks import ModelCheckpoint
from keras.callbacks import TensorBoard  ← Для визуализации метрик
from keras.callbacks import LearningRateScheduler
from keras.callbacks import EarlyStopping

import math
import matplotlib.pyplot as plt

tf.keras.utils.set_random_seed(42)

SAVE_PATH = "/content/drive/MyDrive/Colab Notebooks/data/"

def scheduler(epoch, lr):  ← Изменение скорости обучения
    if epoch < 4:
        return lr
    else:
        return lr * tf.math.exp(-0.1)

if __name__ == "__main__":

    (x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
    x_train = x_train.reshape(x_train.shape[0], x_train.shape[1] *
    ⇒ x_train.shape[2]).astype("float32") / 255
```

¹ Шолле Ф. «Глубокое обучение на Python». СПб., издательство «Питер».

```

x_test = x_test.reshape(x_test.shape[0], x_test.shape[1] *
    ➔ x_test.shape[2]).astype("float32") / 255

y_train_label = keras.utils.to_categorical(y_train)
y_test_label = keras.utils.to_categorical(y_test)
num_classes = y_train_label.shape[1]

batch_size = 64 | Параметры
num_epochs = 16 | обучения

model = Sequential()
model.add(Dense(128, input_shape=(784, ), activation='relu'))
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(10, activation='softmax'))

model.compile(
    loss=keras.losses.CategoricalCrossentropy(),
    optimizer=tf.keras.optimizers.RMSprop(),
    metrics=["accuracy"]
)

model.summary()

# Определение обратных вызовов
file_name = SAVE_PATH + 'mlp-weights-checkpoint.h5'
checkpoint = ModelCheckpoint(file_name, monitor='val_loss', verbose=1,
    ➔ save_best_only=True, mode='min')
reduce_lr = LearningRateScheduler(scheduler, verbose=1)
early_stopping = EarlyStopping(monitor='val_loss', min_delta=0.01,
    ➔ patience=16, verbose=1)
#tensor_board = TensorBoard(log_dir='./logs', write_graph=True)
callbacks_list = [checkpoint, reduce_lr, early_stopping]

hist = model.fit(x_train, y_train_label,
    ➔ batch_size=batch_size, epochs=num_epochs,
    ➔ callbacks=callbacks_list, validation_split=0.2)    ← Обучение модели

test_scores = model.evaluate(x_test, y_test_label,
    ➔ verbose=2)    ← Оценка модели

print("Test loss:", test_scores[0])
print("Test accuracy:", test_scores[1])

plt.figure()
plt.plot(hist.history['loss'], 'b', lw=2.0, label='Обучение')
plt.plot(hist.history['val_loss'], '--r', lw=2.0, label='Валидация')
plt.title('МСП-модель')
plt.xlabel('Эпоха')
plt.ylabel('Потери кросс-энтропии')
plt.legend(loc='upper right')
plt.show()

plt.figure()
plt.plot(hist.history['accuracy'], 'b', lw=2.0, label='Обучение')

```

```
plt.plot(hist.history['val_accuracy'], '--r', lw=2.0, label='Валидация')
plt.title('МСП-модель')
plt.xlabel('Эпоха')
plt.ylabel('Доля верных результатов')
plt.legend(loc='upper left')
plt.show()

plt.figure()
plt.plot(hist.history['lr'], lw=2.0, label='learning rate')
plt.title('MLP model')
plt.xlabel('Epochs')
plt.ylabel('Learning Rate')
plt.legend()
plt.show()
```

Рисунок 10.3 демонстрирует графики потерь кросс-энтропии и показателя доли верных результатов для обучающего и валидационного наборов данных. Модель MLP достигает 98%-ной достоверности на тестовом наборе данных всего после 16 эпох обучения. Обратите внимание, что потери при обучении ниже, чем при валидации, и, аналогично, верность на обучающем наборе выше, чем на валидационном. Поскольку нас интересует способность модели делать предсказания на незнакомых данных, именно валидационный набор является реальным показателем ее производительности. В следующем разделе мы рассмотрим класс нейронных сетей, лучше подходящий для обработки данных изображений и потому широко используемый в компьютерном зрении.

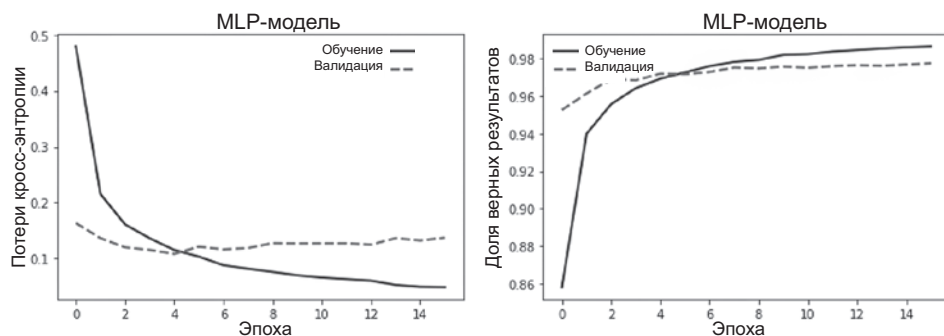


Рис. 10.3. Потери кросс-энтропии и доля верных результатов MLP для обучающего и валидационного наборов данных

10.2. Сверточные нейронные сети

Сверточные нейронные сети (СНС) используют операции свертки и объединения (pooling) вместо векторизованного умножения матриц. В этом разделе мы расскажем, для чего используются эти операции, и опишем две архитектуры: LeNet для классификации набора цифр MNIST и ResNet50 для поиска изображений.

СНС отлично работают с данными изображений. Изображения представляют собой высокоразмерные объекты объемом $W \times H \times C$, где W — ширина, H — высота, а C — количество каналов (например, $C = 3$ для изображения RGB и $C = 1$ для полутонового изображения). СНС состоят из обучаемых фильтров, или ядер, в результате свертки которых со входными данными получается карта признаков. Это приводит к значительному сокращению числа параметров и обеспечивает инвариантность относительно сдвига во входных данных (поскольку нам нужно уметь классифицировать объект в любом месте изображения). Операция свертки выполняется в 2D для изображений, в 1D для временных рядов и в 3D для пространственных данных. Двумерная (2D) свертка определяется следующим образом:

$$(f * g)(i, j) = \sum_a \sum_b f(a, b) g(i - a, j - b). \quad (10.8)$$

Другими словами, ядро скользит по всем возможным позициям входной матрицы. В каждой позиции в результате выполнения операции скалярного произведения получается соответствующее скалярное значение. В конечном итоге эти скаляры образуют выходную карту признаков.

Сверточные слои можно ставить друг за другом в цепочку. Поскольку свертки являются линейными операторами, мы помещаем нелинейные функции активации между ними точно так же, как мы это делали в полносвязных слоях.

Многие популярные архитектуры СНС включают в себя слой объединения данных (pooling layer). Слои объединения понижают разрешение (down-sample) карт признаков путем агрегирования информации. Например, объединение по максимальному значению (max pooling) находит максимум во входных значениях, тогда как объединение по среднему (average pooling) заменяет операцию взятия максимума на вычисление среднего значения. Аналогичным образом, объединение по глобальному среднему (global average pooling) можно использовать для уменьшения карты признаков $W \times H \times D$ до агрегата размером $1 \times 1 \times D$, который затем для дальнейшей обработки может быть преобразован в D -мерный вектор. Давайте рассмотрим некоторые примеры применения СНС.

10.2.1. LeNet на датасете MNIST

В этом разделе мы рассмотрим классическую архитектуру LeNet, разработанную Яном Лекуном для классификации рукописных цифр и обученную на наборе данных MNIST, — см. статью Яна Лекуна (Yann LeCun) и др. «GradientBased Learning Applied to Document Recognition» (Proceedings of the IEEE, 1998). Как показано на рис. 10.4, она построена по схеме: сверточный слой — ReLU-активация — операция объединения по максимальному значению.

Эта последовательность повторяется, и количество фильтров увеличивается по мере того, как мы движемся от входа к выходу СНС. Обратите внимание, что

одновременно увеличивается количество обучаемых признаков за счет увеличения числа фильтров и уменьшается размерность карты признаков благодаря операциям объединения по максимальному значению. На последнем слое карта признаков преобразуется (flatten) в вектор и подается на вход полносвязного слоя, за которым следует softmax-классификатор.

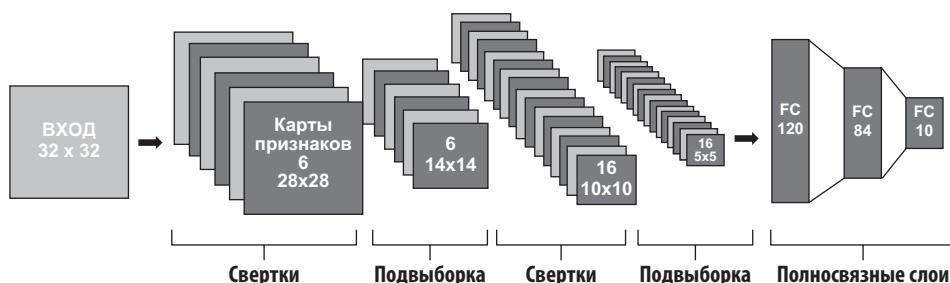


Рис. 10.4. Архитектура LeNet для классификации цифр MNIST

В следующей реализации мы начинаем с загрузки набора данных MNIST и формирования нужного размера изображений. Далее устанавливаем параметры обучения и параметры модели, а затем определяем архитектуру СНС. Обратите внимание на последовательность операций свертки, ReLU и объединения с поиском максимума. После этого компилируем модель и определяем набор функций обратного вызова, которые будут исполняться во время обучения модели. Мы запоминаем историю обучения и оцениваем производительность модели на тестовом наборе данных. Наконец, сохраняем результаты предсказания и создаем графики доли верных результатов и функции потерь. Теперь мы готовы к реализации несложной архитектуры MNIST СНС с нуля, используя Keras/TensorFlow.

Листинг 10.2. Простая СНС для классификации набора MNIST

```
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow import keras

from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten
from keras.layers import Conv2D, MaxPooling2D, Activation

from keras.callbacks import ModelCheckpoint
from keras.callbacks import TensorBoard
from keras.callbacks import LearningRateScheduler
from keras.callbacks import EarlyStopping

import math
import matplotlib.pyplot as plt
```

```

tf.keras.utils.set_random_seed(42)

SAVE_PATH = "/content/drive/MyDrive/Colab Notebooks/data/"

def scheduler(epoch, lr):  ← Изменение скорости обучения
    if epoch < 4:
        return lr
    else:
        return lr * tf.math.exp(-0.1)

if __name__ == "__main__":

    img_rows, img_cols = 28, 28
    (x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
    x_train = x_train.reshape(x_train.shape[0], img_rows, img_cols,
    ➡ 1).astype("float32") / 255
    x_test = x_test.reshape(x_test.shape[0], img_rows, img_cols,
    ➡ 1).astype("float32") / 255

    y_train_label = keras.utils.to_categorical(y_train)
    y_test_label = keras.utils.to_categorical(y_test)
    num_classes = y_train_label.shape[1]

    batch_size = 128 | Параметры
    num_epochs = 8   | обучения

    num_filters_l1 = 32 | Параметры
    num_filters_l2 = 64 | модели

    # Архитектура CHC
    cnn = Sequential()
    cnn.add(Conv2D(num_filters_l1, kernel_size = (5, 5),
    ➡ input_shape=(img_rows, img_cols, 1), padding='same'))
    cnn.add(Activation('relu'))
    cnn.add(MaxPooling2D(pool_size=(2,2), strides=(2,2)))

    cnn.add(Conv2D(num_filters_l2, kernel_size = (5, 5), padding='same'))
    cnn.add(Activation('relu'))
    cnn.add(MaxPooling2D(pool_size=(2,2), strides=(2,2)))

    cnn.add(Flatten())
    cnn.add(Dense(128))
    cnn.add(Activation('relu'))

    cnn.add(Dense(num_classes))
    cnn.add(Activation('softmax'))
    cnn.compile(
        loss=keras.losses.CategoricalCrossentropy(),
        optimizer=tf.keras.optimizers.Adam(),
        metrics=["accuracy"]
    )

    cnn.summary()

    # Определение обратных вызовов

```

```

file_name = SAVE_PATH + 'lenet-weights-checkpoint.h5'
checkpoint = ModelCheckpoint(file_name, monitor='val_loss', verbose=1,
    ➤ save_best_only=True, mode='min')
reduce_lr = LearningRateScheduler(scheduler, verbose=1)
early_stopping = EarlyStopping(monitor='val_loss', min_delta=0.01,
    ➤ patience=16, verbose=1)
#tensor_board = TensorBoard(log_dir='./logs', write_graph=True)
callbacks_list = [checkpoint, reduce_lr, early_stopping]

hist = cnn.fit(x_train, y_train_label, batch_size=batch_size,
    ➤ epochs=num_epochs, callbacks=callbacks_list,
    ➤ validation_split=0.2)    ← Обучение модели

test_scores = cnn.evaluate(x_test, y_test_label,
    ➤ verbose=2)    ← Оценка модели

print("Test loss:", test_scores[0])
print("Test accuracy:", test_scores[1])

y_prob = cnn.predict(x_test)
y_pred = y_prob.argmax(axis=-1)

submission = pd.DataFrame(index=pd.RangeIndex(start=1, stop=10001,
    ➤ step=1), columns=['Label'])
submission['Label'] = y_pred.reshape(-1,1)
submission.index.name = "ImageId"
submission.to_csv(SAVE_PATH + '/lenet_pred.csv', index=True, header=True)

plt.figure()
plt.plot(hist.history['loss'], 'b', lw=2.0, label='Обучение')
plt.plot(hist.history['val_loss'], '--r', lw=2.0, label='Валидация')
plt.title('Модель LeNet')
plt.xlabel('Эпоха')
plt.ylabel('Потери кросс-энтропии')
plt.legend(loc='upper right')
plt.show()

plt.figure()
plt.plot(hist.history['accuracy'], 'b', lw=2.0, label='Обучение')
plt.plot(hist.history['val_accuracy'], '--r', lw=2.0, label='Валидация')
plt.title('Модель LeNet')
plt.xlabel('Эпоха')
plt.ylabel('Доля верных результатов')
plt.legend(loc='upper left')
plt.show()

plt.figure()
plt.plot(hist.history['lr'], lw=2.0, label='learning rate')
plt.title('LeNet model')
plt.xlabel('Epochs')
plt.ylabel('Learning Rate')
plt.legend()
plt.show()

```


С помощью архитектуры СНС нам удалось достичь впечатляющей достоверности результатов в 99 % на тестовом наборе данных! Обратите внимание на различия в кривых обучения и валидации как в случае потерь кросс-энтропии, так и в отношении показателя правильных результатов. Ранний останов позволяет нам прекратить обучение, если валидационные потери не уменьшаются на протяжении нескольких эпох:

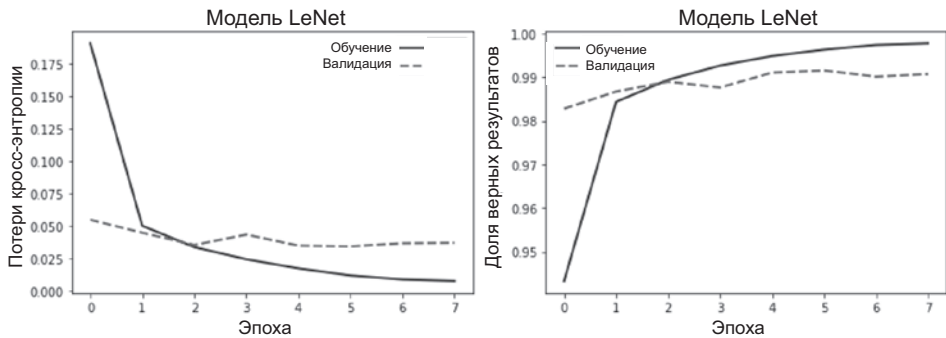


Рис. 10.5. Графики потерь и доли верных результатов СНС LeNet для обучающего и валидационного наборов данных

10.2.2. ResNet для поиска изображений

Поиском изображений называется извлечение из базы данных изображений, аналогичных запрашиваемому. В этом разделе мы будем использовать предварительно обученную СНС ResNet50 для кодирования коллекции изображений в плотные векторы. Это позволит, вычисляя расстояния между векторами, находить похожие изображения.

Архитектура ResNet50 задействует обходные соединения (skip connection) для избежания проблемы затухающего градиента — см. статью Кайминга Хе (Kaiming He) и др. «Deep Residual Learning for Image Recognition» (Conference on Computer Vision and Pattern Recognition, 2016). Обходные соединения позволяют обойти некоторые слои сети, как показано на рис. 10.6.

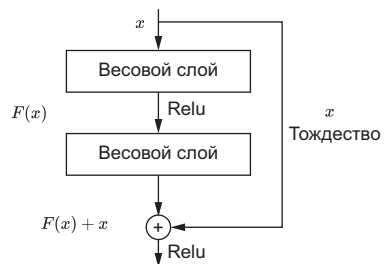


Рис. 10.6. Обходное соединение

Основная идея ResNet заключается в сохранении градиента посредством обратного распространения через функцию тождества. Градиент умножается на единицу, и поэтому его значение сохраняется на предшествующих слоях. Это дает нам

возможность объединять множество таких слоев и создавать очень глубокие архитектуры. Пусть $H = F(x) + x$. Тогда мы получаем следующее уравнение:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial H} \frac{\partial H}{\partial x} = \frac{\partial L}{\partial H} \left(\frac{\partial F}{\partial x} + 1 \right) = \frac{\partial L}{\partial H} \frac{\partial F}{\partial x} + \frac{\partial L}{\partial H}. \quad (10.9)$$

В этом разделе мы используем предварительно обученный на датасете ImageNet базовый вариант СНС ResNet50 для кодирования всего набора изображений Caltech 101. Начинаем с загрузки датасета Caltech 101 с сайта <http://www.vision.caltech.edu/datasets/>. Затем берем предобученную на ImageNet модель ResNet50 в качестве базовой модели и устанавливаем в качестве выходного слой объединения по среднему, который фактически кодирует входное изображение в 2048-мерный вектор. Мы осуществляем ResNet50-кодирование всего набора изображений и сохраняем результаты в списке активаций. Для дальнейшей экономии места сжимаем закодированные ResNet50 2048-мерные векторы с помощью метода главных компонент (PCA) до 300-мерных векторов. Сортируя векторы в соответствии с косинусным сходством, получаем изображения ближайших соседей. Теперь мы можем приступить к знакомству с деталями реализации алгоритма поиска изображений с использованием библиотеки Keras/TensorFlow:

Листинг 10.3. ResNet50 для поиска изображений

```
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow import keras

from keras import Model
from keras.applications.resnet50 import ResNet50
from keras.preprocessing import image
from keras.applications.resnet50 import preprocess_input

from keras.callbacks import ModelCheckpoint
from keras.callbacks import TensorBoard
from keras.callbacks import LearningRateScheduler
from keras.callbacks import EarlyStopping

import os
import random
from PIL import Image
from scipy.spatial import distance
from sklearn.decomposition import PCA

import matplotlib.pyplot as plt

tf.keras.utils.set_random_seed(42)

SAVE_PATH = "/content/drive/MyDrive/Colab Notebooks/data/"
DATA_PATH = "/content/drive/MyDrive/data/101_ObjectCategories/"
```

```

def get_closest_images(acts, query_image_idx, num_results=5):
    num_images, dim = acts.shape
    distances = []
    for image_idx in range(num_images):
        distances.append(distance.euclidean(acts[query_image_idx, :],
        ➡ acts[image_idx, :]))
    # конец for
    idx_closest = sorted(range(len(distances)), key=lambda k:
    ➡ distances[k])[1:num_results+1]

    return idx_closest

def get_concatenated_images(images, indexes, thumb_height):
    thumbs = []
    for idx in indexes:
        img = Image.open(images[idx])
        img = img.resize((int(img.width * thumb_height / img.height),
        ➡ int(thumb_height)), Image.ANTIALIAS)
        if img.mode != "RGB":
            img = img.convert("RGB")
        thumbs.append(img)
    concat_image = np.concatenate([np.asarray(t) for t in thumbs], axis=1)

    return concat_image

if __name__ == "__main__":
    num_images = 5000
    images = [os.path.join(dp,f) for dp, dn, filenames in
    ➡ os.walk(DATA_PATH) for f in filenames \
        if os.path.splitext(f)[1].lower() in ['.jpg', '.png', '.jpeg']]
    images = [images[i] for i in sorted(random.sample(range(len(images)),
    ➡ num_images))]

    # СНС-кодирование
    base_model = ResNet50(weights='imagenet')
    model = Model(inputs=base_model.input,
    outputs=base_model.get_layer('avg_pool').output)

    activations = []
    for idx, image_path in enumerate(images):
        if idx % 100 == 0:
            print('getting activations for %d/%d image...'
            ➡ %(idx, len(images)))
            img = image.load_img(image_path, target_size=(224, 224))
            x = image.img_to_array(img)
            x = np.expand_dims(x, axis=0)
            x = preprocess_input(x)
            features = model.predict(x)
            activations.append(features.flatten().reshape(1,-1))

    print('computing PCA...')
    acts = np.concatenate(activations, axis=0)

```

Предобученная на ImageNet модель ResNet50

```

pca = PCA(n_components=300)  ← Уменьшение размерности активаций
pca.fit(acts)
acts = pca.transform(acts)

print('image search...')
query_image_idx = int(num_images*random.random())
idx_closest = get_closest_images(acts, query_image_idx)
query_image = get_concatenated_images(images, [query_image_idx], 300)
results_image = get_concatenated_images(images, idx_closest, 300)

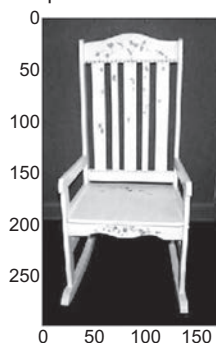
plt.figure()
plt.imshow(query_image)
plt.title("Запрашиваемое изображение (%d)" %query_image_idx)
plt.show()

plt.figure()
plt.imshow(results_image)
plt.title("Найденные изображения")
plt.show()

```

На рис. 10.7 показано запрашиваемое изображение стула (слева) и найденные изображения ближайших соседей (справа). Полученные изображения очень похожи на запрашиваемое. В следующем разделе мы познакомим вас с нейронными сетями для обработки последовательных данных.

Запрашиваемое изображение (1314)



Найденные изображения

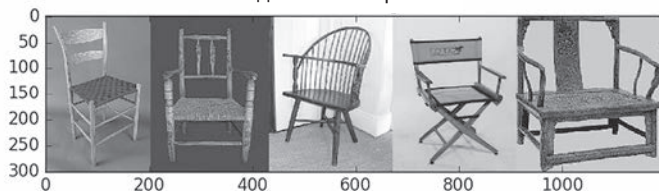


Рис. 10.7. Результаты поиска изображений СНС ResNet50

10.3. Рекуррентные нейронные сети

Рекуррентные нейронные сети (РНС) предназначены для обработки последовательностей данных. РНС хранят внутреннее состояние прошлого и поэтому подходят для кодирования последовательности (Seq) в вектор (Vec) и наоборот. Области применения РНС варьируют от генерации языковых последовательностей (Vec2Seq) до классификации (Seq2Vec) и перевода (Seq2Seq) последовательностей. В этом разделе мы сосредоточимся на задачах классификации последовательностей с использованием двунаправленной РНС архитектуры

LSTM и определения сходства последовательностей с использованием предварительно обученных эмбедингов слов.

РНС используют скрытые слои, которые включает в себя текущие входные данные x_t и предыдущее состояние h_{t-1} :

$$h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b_h). \quad (10.10)$$

Здесь W_{hh} — это веса «скрытое состояние — скрытое состояние», W_{xh} — веса «вход — скрытое состояние», а b_h — свободный член. Опционально на каждом шаге можно генерировать выходные данные y_t :

$$y_t = g(W_{hy}h_t + b_y). \quad (10.11)$$

Общая схема архитектуры LSTM представлена на рис. 10.8. В следующих подразделах мы рассмотрим примеры применения РНС.

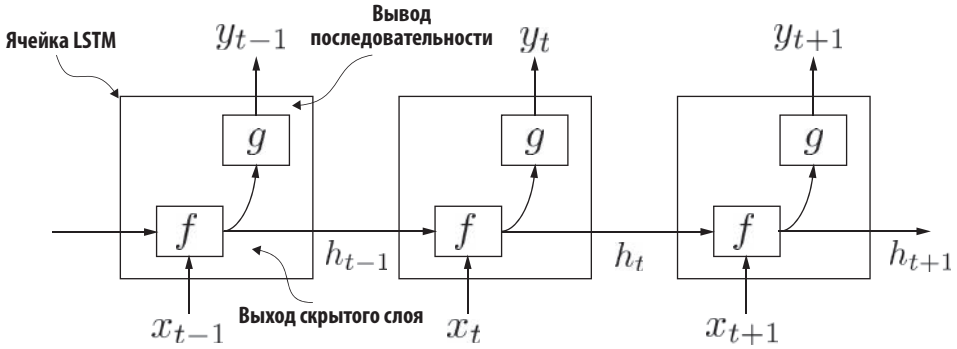


Рис. 10.8. Архитектура рекуррентной нейронной сети LSTM

10.3.1. LSTM-классификация последовательностей

В этом разделе наша задача — определение тональности обзоров фильмов из базы IMDb. Мы начинаем с того, что токенизируем (tokenize) каждое слово в обзоре и преобразуем его в вектор с помощью обучаемого эмбединга-слоя. Последовательность векторов слов подается на вход прямой LSTM-модели, которая кодирует ее в вектор. Параллельно мы подаем нашу последовательность на обратную LSTM-модель и объединяем ее векторный вывод с выходными данными прямой модели, получая скрытое представление обзора фильма:

$$\begin{aligned} h_t^{\rightarrow} &= f(W_{hh}^{\rightarrow}h_{t-1}^{\rightarrow} + W_{xh}^{\rightarrow}x_t + b_h^{\rightarrow}); \\ h_t^{\leftarrow} &= f(W_{hh}^{\leftarrow}h_{t-1}^{\leftarrow} + W_{xh}^{\leftarrow}x_t + b_h^{\leftarrow}); \\ h_t &= [h_t^{\rightarrow}, h_t^{\leftarrow}]. \end{aligned} \quad (10.12)$$

Здесь h_t содержит в себе информацию как из прошлого, так и из будущего. Сочетание таких двух LSTM-моделей называется *двунаправленной LSTM*, причем входные данные они обрабатывают так, как будто время течет вперед и назад. На рис. 10.9 показана архитектура двунаправленной РНС.

После кодирования текста обзора в вектор мы усредняем несколько полносвязных слоев с применением регуляризации l2 и прореживания, а затем классифицируем тональность как положительную или отрицательную при помощи сигмоидной функции активации выходного слоя. Давайте теперь рассмотрим код LSTM-классификации последовательностей, написанный с использованием библиотеки Keras/TensorFlow:

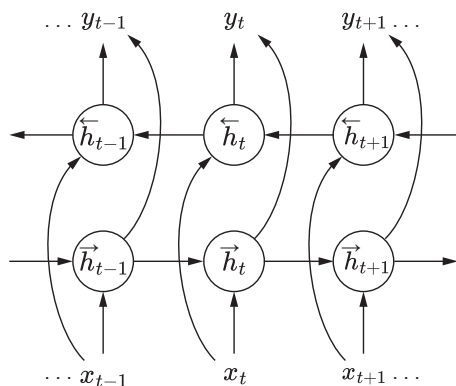


Рис. 10.9. Архитектура двунаправленной РНС

Листинг 10.4. Двунаправленная LSTM для определения тональности

```
import numpy as np
import pandas as pd

import tensorflow as tf
from tensorflow import keras

from keras.models import Sequential
from keras.layers import LSTM, Bidirectional
from keras.layers import Dense, Dropout, Activation, Embedding

from keras import regularizers
from keras.preprocessing import sequence
from keras.utils import np_utils

from keras.callbacks import ModelCheckpoint
from keras.callbacks import TensorBoard
```

```

from keras.callbacks import LearningRateScheduler
from keras.callbacks import EarlyStopping

import matplotlib.pyplot as plt

tf.keras.utils.set_random_seed(42)

SAVE_PATH = "/content/drive/MyDrive/Colab Notebooks/data/"

def scheduler(epoch, lr):  ← Изменение скорости обучения
    if epoch < 4:
        return lr
    else:
        return lr * tf.math.exp(-0.1)

if __name__ == "__main__"
    # Загрузка набора данных
    max_words = 20000
    seq_len = 200
    (x_train, y_train), (x_val, y_val) = keras.datasets.imdb.load_data(
        ➡ num_words=max_words)

    # Топ-20 тысяч самых часто встречающихся слов
    # Первые 200 слов каждой рецензии на фильм

    x_train = keras.utils.pad_sequences(x_train, maxlen=seq_len)
    x_val = keras.utils.pad_sequences(x_val, maxlen=seq_len)

    batch_size = 256
    num_epochs = 8
    # Параметры обучения

    hidden_size = 64
    embed_dim = 128
    lstm_dropout = 0.2
    dense_dropout = 0.5
    weight_decay = 1e-3
    # Параметры модели

    # Архитектура LSTM
    model = Sequential()
    model.add(Embedding(max_words, embed_dim, input_length=seq_len))
    model.add(Bidirectional(LSTM(hidden_size, dropout=lstm_dropout,
        ➡ recurrent_dropout=lstm_dropout)))
    model.add(Dense(hidden_size,
        kernel_regularizer=regularizers.l2(weight_decay),
        ➡ activation='relu'))
    model.add(Dropout(dense_dropout))
    model.add(Dense(hidden_size/4,
        ➡ kernel_regularizer=regularizers.l2(weight_decay), activation='relu'))
    model.add(Dense(1, activation='sigmoid'))

    model.compile(
        loss=keras.losses.BinaryCrossentropy(),
        optimizer=tf.keras.optimizers.Adam(),
        metrics=["accuracy"]

```

```

)

model.summary()

# Определение обратных вызовов
file_name = SAVE_PATH + 'lstm-weights-checkpoint.h5'
checkpoint = ModelCheckpoint(file_name, monitor='val_loss', verbose=1,
    ➔ save_best_only=True, mode='min')
reduce_lr = LearningRateScheduler(scheduler, verbose=1)
early_stopping = EarlyStopping(monitor='val_loss', min_delta=0.01,
    ➔ patience=16, verbose=1)
#tensor_board = TensorBoard(log_dir='./logs', write_graph=True)
callbacks_list = [checkpoint, reduce_lr, early_stopping]

hist = model.fit(x_train, y_train, batch_size=
    ➔ batch_size, epochs=num_epochs, callbacks=
    ➔ callbacks_list, validation_data=(x_val, y_val))    ← Обучение модели

test_scores = model.evaluate(x_val, y_val,
    ➔ verbose=2)    ← Оценка модели

print("Test loss:", test_scores[0])
print("Test accuracy:", test_scores[1])

plt.figure()
plt.plot(hist.history['loss'], 'b', lw=2.0, label='Обучение')
plt.plot(hist.history['val_loss'], '--r', lw=2.0, label='Валидация')
plt.title('LSTM-модель')
plt.xlabel('Эпоха')
plt.ylabel('Потери кросс-энтропии')
plt.legend(loc='upper right')
plt.show()

plt.figure()
plt.plot(hist.history['accuracy'], 'b', lw=2.0, label='Обучение')
plt.plot(hist.history['val_accuracy'], '--r', lw=2.0, label='Валидация')
plt.title('LSTM-модель')
plt.xlabel('Эпоха')
plt.ylabel('Доля верных результатов')
plt.legend(loc='upper left')
plt.show()

plt.figure()
plt.plot(hist.history['lr'], lw=2.0, label='learning rate')
plt.title('LSTM model')
plt.xlabel('Epochs')
plt.ylabel('Learning Rate')
plt.legend()
plt.show()

```

Как видно из рис. 10.10, мы достигли тестовой верности классификации в 85 % на базе данных IMDb обзоров фильмов. В следующем разделе мы рассмотрим нейросетевые модели с несколькими входами.

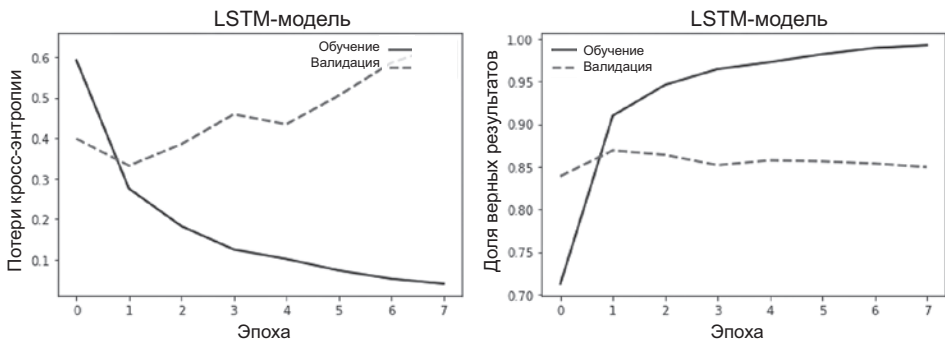


Рис. 10.10. Двухнаправленная LSTM для определения тональности: потери и верность классификации на обучающем и валидационном наборах данных

10.3.2. Модель с несколькими входами

В этом разделе вы познакомитесь с моделью, имеющей несколько входов, на примере определения сходства последовательностей. Модель сравнивает два вопроса (по одному на каждую входную ветвь) и решает, имеют ли они схожее значение. Сходство помогает понять, являются ли вопросы избыточными, то есть дублируют ли они друг друга. Эта задача была поставлена в рамках соревнования по data science «Quora Questions Pairs» на платформе Kaggle. Приведенную здесь модель можно было бы использовать как составную часть ансамбля моделей с высокими показателями качества.

В нашем случае для кодирования отдельного слова в 300-мерный плотный вектор используется предварительно обученная модель эмбедингов GloVe — см. работу Джеффри Пеннингтона (Jeffrey Pennington), Ричарда Сочера (Richard Socher), Кристофера Мэннинга (Christopher Manning) «GloVe: Global Vectors for Word Representation» (EMNLP, 2014). В каждой входной ветви для обработки последовательности данных используются одномерные свертки и операции объединения по максимальному значению, результаты работы которых объединяются (concatenate) в один вектор. После нескольких полносвязных слоев вычисляем вероятность дублирования вопросов с помощью сигмоидной функции активации.

Преимущество использования одномерных сверток заключается в том, что они быстрее вычисляются и могут выполняться параллельно в силу отсутствия рекуррентного цикла для разворачивания (unrolling). Их точность зависит от того, является ли информация о недавнем прошлом более важной, чем о далеком прошлом (тогда LSTM дает более достоверные результаты), или же информация из прошлого одинаково важна независимо от периода времени (в таком случае одномерная свертка будет более точной).

В нашей реализации используется набор пар вопросов Quora, который можно загрузить с сайта Kaggle: <https://www.kaggle.com/datasets/quora/question-pairs-dataset>.

Давайте теперь посмотрим, как может выглядеть реализация на Keras/TensorFlow модели с несколькими входами для определения сходства последовательностей.

Листинг 10.5. Модель нейронной сети с несколькими входами для определения сходства последовательностей

```
import numpy as np
import pandas as pd

import tensorflow as tf
from tensorflow import keras

import os
import re
import csv
import codecs

from keras.models import Model
from keras.layers import Input, Flatten, Concatenate, LSTM, Lambda, Dropout
from keras.layers import Dense, Dropout, Activation, Embedding
from keras.layers import Conv1D, MaxPooling1D
from keras.layers import TimeDistributed, Bidirectional, BatchNormalization

from keras import backend as K
from keras.preprocessing.text import Tokenizer
from keras.preprocessing.sequence import pad_sequences

from nltk.corpus import stopwords
from nltk.stem import SnowballStemmer

from keras import regularizers
from keras.preprocessing import sequence
from keras.utils import np_utils

from keras.callbacks import ModelCheckpoint
from keras.callbacks import TensorBoard
from keras.callbacks import LearningRateScheduler
from keras.callbacks import EarlyStopping

import matplotlib.pyplot as plt

tf.keras.utils.set_random_seed(42)

SAVE_PATH = "/content/drive/MyDrive/Colab Notebooks/data/"
DATA_PATH = "/content/drive/MyDrive/data/"

GLOVE_DIR = DATA_PATH
TRAIN_DATA_FILE = DATA_PATH + 'quora_train.csv'
TEST_DATA_FILE = DATA_PATH + 'quora_test.csv'
MAX_SEQUENCE_LENGTH = 30
MAX_NB_WORDS = 200000
EMBEDDING_DIM = 300
VALIDATION_SPLIT = 0.01
```

```

def scheduler(epoch, lr):
    if epoch < 4:
        return lr
    else:
        return lr * tf.math.exp(-0.1)

def text_to_wordlist(row, remove_stopwords=False, stem_words=False):

    text = row['question']
    if type(text) is str:
        text = text.lower().split()  ← Перевод строки в нижний
                                     регистр и разбиение на слова
    else:
        return " "

    if remove_stopwords:
        stops = set(stopwords.words("english"))
        text = [w for w in text if not w in stops]  ← Удаление стоп-слов

    text = " ".join(text)

    # Очистка текста
    text = re.sub(r"^[A-Za-z0-9^,!.\\/'+-=]", " ", text)

    if stem_words:
        text = text.split()
        stemmer = SnowballStemmer('english')
        stemmed_words = [stemmer.stem(word) for word in text]  ← Усечение слов
                                                                до их основы
        text = " ".join(stemmed_words)

    return(text)

if __name__ == "__main__"

    print('Indexing word vectors...')
    embeddings_index = {}
    f = codecs.open(os.path.join(GLOVE_DIR, 'glove.6B.
    ➔ 300d.txt'), encoding='utf-8')  ← Загрузка эмбедингов

    for line in f:
        values = line.split(' ')
        word = values[0]
        coefs = np.asarray(values[1:], dtype='float32')
        embeddings_index[word] = coefs
    f.close()
    print('Found %s word vectors.' % len(embeddings_index))

    train_df = pd.read_csv(TRAIN_DATA_FILE)  | Загрузка набора
    test_df  = pd.read_csv(TEST_DATA_FILE)  | данных

    q1df = train_df['question1'].reset_index()
    q2df = train_df['question2'].reset_index()
    q1df.columns = ['index', 'question']
    q2df.columns = ['index', 'question']
    texts_1 = q1df.apply(text_to_wordlist, axis=1, raw=False).tolist()
    texts_2 = q2df.apply(text_to_wordlist, axis=1, raw=False).tolist()

```

```

labels = train_df['is_duplicate'].astype(int).tolist()
print('Found %s texts.' % len(texts_1))
del q1df
del q2df

q1df = test_df['question1'].reset_index()
q2df = test_df['question2'].reset_index()
q1df.columns = ['index', 'question']
q2df.columns = ['index', 'question']
test_texts_1 = q1df.apply(text_to_wordlist, axis=1, raw=False).tolist()
test_texts_2 = q2df.apply(text_to_wordlist, axis=1, raw=False).tolist()
test_labels = np.arange(0, test_df.shape[0])
print('Found %s texts.' % len(test_texts_1))
del q1df
del q2df

tokenizer = Tokenizer(nb_words=MAX_NB_WORDS)
tokenizer.fit_on_texts(texts_1 + texts_2 + test_texts_1 + test_texts_2)
sequences_1 = tokenizer.texts_to_sequences(texts_1)
sequences_2 = tokenizer.texts_to_sequences(texts_2)
word_index = tokenizer.word_index
print('Found %s unique tokens.' % len(word_index))

test_sequences_1 = tokenizer.texts_to_sequences(test_texts_1)
test_sequences_2 = tokenizer.texts_to_sequences(test_texts_2)
data_1 = pad_sequences(sequences_1, maxlen=MAX_SEQUENCE_LENGTH)
data_2 = pad_sequences(sequences_2, maxlen=MAX_SEQUENCE_LENGTH)
labels = np.array(labels)
print('Shape of data tensor:', data_1.shape)
print('Shape of label tensor:', labels.shape)

test_data_1 = pad_sequences(test_sequences_1, maxlen=MAX_SEQUENCE_LENGTH)
test_data_2 = pad_sequences(test_sequences_2, maxlen=MAX_SEQUENCE_LENGTH)
test_labels = np.array(test_labels)
del test_sequences_1
del test_sequences_2
del sequences_1
del sequences_2

print('Preparing embedding matrix...')
nb_words = min(MAX_NB_WORDS, len(word_index))

embedding_matrix = np.zeros((nb_words, EMBEDDING_DIM))
for word, i in word_index.items():
    if i >= nb_words:
        continue
    embedding_vector = embeddings_index.get(word)
    if embedding_vector is not None:
        # слова, не найденные в индексе эмбедингов, будут состоять
        # только из нулей
        embedding_matrix[i] = embedding_vector
print('Null word embeddings: %d' % np.sum(np.sum(embedding_matrix,
    ➤ axis=1) == 0))

embedding_layer = Embedding(nb_words,
    EMBEDDING_DIM,

```

```

weights=[embedding_matrix],
input_length=MAX_SEQUENCE_LENGTH,
trainable=False)

sequence_1_input = Input(shape=(MAX_SEQUENCE_LENGTH,), dtype='int32')
embedded_sequences_1 = embedding_layer(sequence_1_input)
x1 = Conv1D(128, 3, activation='relu')(embedded_sequences_1)
x1 = MaxPooling1D(10)(x1)
x1 = Flatten()(x1)
x1 = Dense(64, activation='relu')(x1)
x1 = Dropout(0.2)(x1)

sequence_2_input = Input(shape=(MAX_SEQUENCE_LENGTH,), dtype='int32')
embedded_sequences_2 = embedding_layer(sequence_2_input)
y1 = Conv1D(128, 3, activation='relu')(embedded_sequences_2)
y1 = MaxPooling1D(10)(y1)
y1 = Flatten()(y1)
y1 = Dense(64, activation='relu')(y1)
y1 = Dropout(0.2)(y1)

merged = Concatenate()([x1, y1])
merged = BatchNormalization()(merged)
merged = Dense(64, activation='relu')(merged)
merged = Dropout(0.2)(merged)
merged = BatchNormalization()(merged)
preds = Dense(1, activation='sigmoid')(merged)

model = Model(inputs=[sequence_1_input, sequence_2_input], outputs=preds) ←
model.compile(
    loss=keras.losses.BinaryCrossentropy(),
    optimizer=tf.keras.optimizers.Adam(),
    metrics=["accuracy"]
)

model.summary()

file_name = SAVE_PATH + 'multi-input-weights-checkpoint.h5'
checkpoint = ModelCheckpoint(file_name, monitor='val_loss', verbose=1,
    ➤ save_best_only=True, mode='min')
reduce_lr = LearningRateScheduler(scheduler, verbose=1)
early_stopping = EarlyStopping(monitor='val_loss', min_delta=0.01,
    ➤ patience=16, verbose=1)
#tensor_board = TensorBoard(log_dir='./logs', write_graph=True)
callbacks_list = [checkpoint, reduce_lr, early_stopping]

hist = model.fit([data_1, data_2], labels,
    ➤ batch_size=1024, epochs=10, callbacks=callbacks_list,
    ➤ validation_split=VALIDATION_SPLIT) ← Обучение модели

num_test = 100000
preds = model.predict([test_data_1[:num_test,:],
    ➤ test_data_2[:num_test,:]]) ← Оценка модели

quora_submission = pd.DataFrame({"test_id":test_labels[:num_test],
    ➤ "is_duplicate":preds.ravel()})

```

Архитектура
с несколькими
входами

```

quora_submission.to_csv(SAVE_PATH + "quora_submission.csv", index=False)

plt.figure()
plt.plot(hist.history['loss'], 'b', lw=2.0, label='Обучение')
plt.plot(hist.history['val_loss'], '--r', lw=2.0, label='Валидация')
plt.title('Модель с несколькими входами')
plt.xlabel('Эпоха')
plt.ylabel('Потери кросс-энтропии')
plt.legend(loc='upper right')
plt.show()
#plt.savefig('./figures/lstm_loss.png')

plt.figure()
plt.plot(hist.history['accuracy'], 'b', lw=2.0, label='Обучение')
plt.plot(hist.history['val_accuracy'], '--r', lw=2.0, label='Валидация')
plt.title('Модель с несколькими входами')
plt.xlabel('Эпоха')
plt.ylabel('Доля верных результатов')
plt.legend(loc='upper left')
plt.show()
#plt.savefig('./figures/lstm_acc.png')

plt.figure()
plt.plot(hist.history['lr'], lw=2.0, label='learning rate')
plt.title('Multi-Input model')
plt.xlabel('Epochs')
plt.ylabel('Learning Rate')
plt.legend()
plt.show()
#plt.savefig('./figures/lstm_learning_rate.png')

```

Из рис. 10.11 видно, что доля верных результатов на валидационной выборке колеблется в районе 69 %, и очевидно, что есть возможность улучшить эти результаты. Теоретически можно было бы увеличить емкость нашей модели или улучшить представление данных.

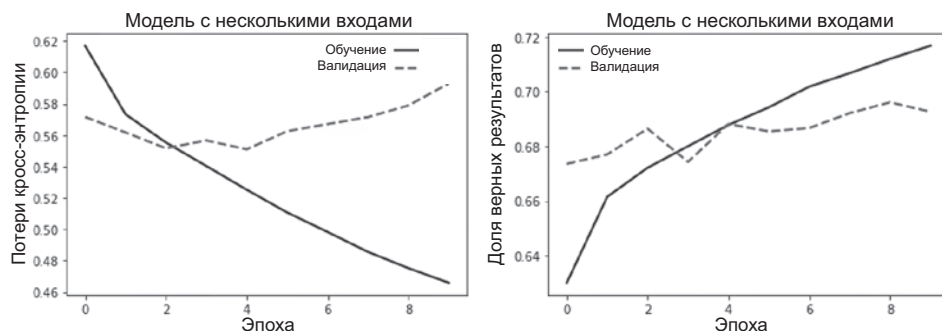


Рис. 10.11. Функция потерь и верность модели с несколькими входами на тестовых и валидационных данных

В этом разделе мы сосредоточились на обсуждении важного класса нейросетевых моделей — рекуррентных нейронных сетей. Мы изучили, как они работают, и рассмотрели два примера возможного применения: классификацию последовательностей и определение сходства последовательностей. В следующем разделе мы рассмотрим различные нейросетевые оптимизаторы.

10.4. Оптимизаторы нейронных сетей

Каковы самые популярные алгоритмы оптимизации, используемые для обучения нейронных сетей? Мы попытаемся ответить на этот вопрос, используя сверточную нейронную сеть (СНС), обученную с помощью пакета Keras/TensorFlow на наборе данных CIFAR-100.

Стохастический градиентный спуск (stochastic gradient descent, SGD) обновляет параметры модели θ в отрицательном направлении градиента g , беря подмножество, или мини-пакет (mini-batch), данных размером m :

$$g = \frac{1}{m} \nabla_{\theta} \sum_i L(f(x^{(i)}; \theta), y^{(i)}); \quad (10.13)$$

$$\theta = \theta - \varepsilon_k \times g.$$

Здесь $f(x; \theta)$ — это нейросеть, обученная на примерах x_i и метках y_i , а L — функция потерь. Градиент потерь L вычисляется по параметрам модели θ . Скорость обучения ε_k определяет размер шага, который алгоритм делает вдоль градиента (в отрицательном направлении в случае минимизации или в положительном направлении в случае максимизации).

Скорость обучения зависит от номера итерации k и является единственным важнейшим гиперпараметром. Слишком высокая скорость обучения (например, > 0.1) может привести к тому, что обновление пропустит оптимальное значение параметров. Слишком низкая скорость обучения (например, $< 1e-5$) приводит к неоправданно долгому времени обучения. Оптимальная стратегия состоит в том, чтобы начать со значения $1e-3$ и использовать график изменения скорости обучения, согласно которому скорость обучения снижается с ростом числа итераций. В общем случае желательно, чтобы скорость обучения удовлетворяла условиям Роббинса — Монро:

$$\sum_k \varepsilon_k = \infty; \quad (10.14)$$

$$\sum_k \varepsilon_k^2 < \infty.$$

Первое условие гарантирует, что алгоритм сможет найти локально оптимальное решение независимо от начальной точки, а второе позволяет избежать колебаний.

Импульс накапливает экспоненциально затухающее скользящее среднее прошлых градиентов и помогает двигаться в их направлении. Таким образом, размер шага зависит от того, насколько последовательность градиентов велика и насколько она выровнена. Типичные значения параметра импульса α равны 0.5 и 0.9:

$$v = \alpha v - \varepsilon \nabla_{\theta} \left(\frac{1}{m} \sum_i L(f(x^{(i)}; \theta), y^{(i)}) \right); \quad (10.15)$$

$$\theta = \theta + v.$$

Нестеровский импульс (Nesterov momentum) основан на методе ускоренного градиента Нестерова. Разница между нестеровским и стандартным импульсом заключается в том, где вычисляется градиент: в случае нестеровского момента градиент вычисляется после применения текущей скорости. Таким образом, импульс Нестерова добавляет поправочный коэффициент к градиенту:

$$v = \alpha v - \varepsilon \nabla_{\theta} \left(\frac{1}{m} \sum_i L(f(x^{(i)}; \theta + \alpha \times v), y^{(i)}) \right); \quad (10.16)$$

$$\theta = \theta + v.$$

AdaGrad — это адаптивный метод настройки скорости обучения — см. статью Джона Дучи (John Duchi), Элада Хазана (Elad Hazan) и Йорама Сингера (Yoram Singer) «Adaptive Subgradient Methods for Online Learning and Stochastic Optimization» (Journal of Machine Learning Research, 2011). Рассмотрим два сценария, представленных на рис. 10.12.

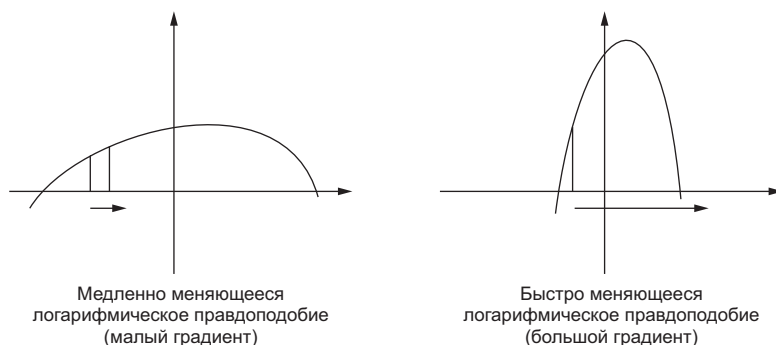


Рис. 10.12. Медленно (слева) и быстро (справа) меняющееся логарифмическое правдоподобие

В случае медленно изменяющейся целевой функции (слева) градиент обычно (в большинстве точек) имеет малую величину. Как результат, нам нужна большая скорость обучения, чтобы быстро достичь оптимума. В случае быстро меняющейся целевой функции (справа) градиент, как правило, очень большой.

Использование высокой скорости обучения привело бы к очень большим шагам, колеблющимся вокруг, но не достигающим оптимума. Эти две ситуации возникают, когда скорость обучения устанавливается независимо от градиента. AdaGrad решает эту проблему, суммируя квадраты норм градиентов, наблюдаемых до текущего момента времени, и деля скорость обучения на квадратный корень суммы, представленной в формуле ниже:

$$\begin{aligned} g &= \frac{1}{m} \nabla_{\theta} \sum_i L(f(x^{(i)}; \theta), y^{(i)}); \\ s &= s + g^T g; \\ \theta &= \theta - \epsilon_k \times \frac{g}{\sqrt{s + eps}}. \end{aligned} \quad (10.17)$$

Таким образом, эффективная скорость обучения для параметров с большим градиентом становится меньше, а для параметров с малым градиентом — больше. Конечным результатом является более быстрое продвижение в пологих направлениях пространства параметров и более осторожные обновления при наличии больших градиентов.

RMSProp является модификацией AdaGrad, в которой накопление градиента преобразуется в экспоненциально взвешенное скользящее среднее (история из далекого прошлого отбрасывается):

$$\begin{aligned} g &= \frac{1}{m} \nabla_{\theta} \sum_i L(f(x^{(i)}; \theta), y^{(i)}); \\ s &= \text{скорость_затухания} \times s + (1 - \text{скорость_затухания}) g^T g; \\ \theta &= \theta - \epsilon_k \times \frac{g}{\sqrt{s + eps}}. \end{aligned} \quad (10.18)$$

Обратите внимание, что AdaGrad неявно предполагает снижение скорости обучения, даже в случае постоянных градиентов, из-за накопления градиентов за период с начала обучения. За счет ведения экспоненциально взвешенного скользящего среднего придается большой вес недавнему прошлому по сравнению с далеким. Как следствие, было показано, что RMSProp является эффективным и удобным в применении алгоритмом оптимизации для глубоких нейронных сетей.

Название алгоритма *Adam* происходит от словосочетания *адаптивные импульсы* (adaptive moments). Adam можно рассматривать как разновидность сочетания метода импульса и алгоритма RMSProp, причем его обновление выглядит как RMSProp, за исключением того, что вместо «сырого» стохастического градиента используется его гладкая версия. Полное обновление Adam включает также в себя механизм коррекции смещения — см. работу Дидерика П. Кингмы

(Diederik P. Kingma), Джимми Ба (Jimmy Ba) «Adam: A Method for Stochastic Optimization» (International Conference on Learning Representations, 2015):

$$\begin{aligned}
 g &= \frac{1}{m} \nabla_{\theta} \sum_i L(x^{(i)}; \theta, y^{(i)}); \\
 m &= \beta_1 m + (1 - \beta_1) g; \\
 s &= \beta_2 s + (1 - \beta_2) g^T g; \\
 \theta &= \theta - \epsilon_k \times \frac{m}{\sqrt{s + \text{eps}}}.
 \end{aligned} \tag{10.19}$$

В статье рекомендуются следующие значения гиперпараметров: $\epsilon = 1e-8$, $\beta_1 = 0.9$, $\beta_2 = 0.999$. Давайте теперь оценим производительность различных оптимизаторов, используя Keras/TensorFlow!

Листинг 10.6. Оптимизаторы нейронных сетей

```

import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow import keras

from keras import backend as K
from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten
from keras.layers import Conv2D, MaxPooling2D, Activation

from keras.callbacks import ModelCheckpoint
from keras.callbacks import TensorBoard
from keras.callbacks import LearningRateScheduler
from keras.callbacks import EarlyStopping

import math
import matplotlib.pyplot as plt

tf.keras.utils.set_random_seed(42)

SAVE_PATH = "/content/drive/MyDrive/Colab Notebooks/data/"

def scheduler(epoch, lr):
    if epoch < 4:
        return lr
    else:
        return lr * tf.math.exp(-0.1)

if __name__ == "__main__":
    img_rows, img_cols = 32, 32
    (x_train, y_train), (x_test, y_test) =
    ➤ keras.datasets.cifar100.load_data()  ← cifar100 dataset
    x_train = x_train.reshape(x_train.shape[0], img_rows, img_cols,
    ➤ 3).astype("float32") / 255

```

```

x_test = x_test.reshape(x_test.shape[0], img_rows, img_cols,
➡ 3).astype("float32") / 255

y_train_label = keras.utils.to_categorical(y_train)
y_test_label = keras.utils.to_categorical(y_test)
num_classes = y_train_label.shape[1]

batch_size = 256      | Параметры
num_epochs = 32      | обучения

num_filters_l1 = 64    | Параметры
num_filters_l2 = 128   | модели

# Архитектура CNN
cnn = Sequential()
cnn.add(Conv2D(num_filters_l1, kernel_size = (5, 5),
➡ input_shape=(img_rows, img_cols, 3), padding='same'))
cnn.add(Activation('relu'))
cnn.add(MaxPooling2D(pool_size=(2,2), strides=(2,2)))

cnn.add(Conv2D(num_filters_l2, kernel_size = (5, 5), padding='same'))
cnn.add(Activation('relu'))
cnn.add(MaxPooling2D(pool_size=(2,2), strides=(2,2)))

cnn.add(Flatten())
cnn.add(Dense(128))
cnn.add(Activation('relu'))

cnn.add(Dense(num_classes))
cnn.add(Activation('softmax'))

opt1 = tf.keras.optimizers.SGD()
opt2 = tf.keras.optimizers.SGD(momentum=0.9,
➡ nesterov=True)
opt3 = tf.keras.optimizers.RMSprop()
opt4 = tf.keras.optimizers.Adam()

optimizer_list = [opt1, opt2, opt3, opt4]

history_list = []

for idx in range(len(optimizer_list)):

    K.clear_session()

    opt = optimizer_list[idx]

    cnn.compile(
        loss=keras.losses.CategoricalCrossentropy(),
        optimizer=opt,
        metrics=["accuracy"]
    )

    # Определение обратных вызовов
    reduce_lr = LearningRateScheduler(scheduler, verbose=1)

```

Оптимизаторы

```

callbacks_list = [reduce_lr]

# цикл обучения
hist = cnn.fit(x_train, y_train_label, batch_size=batch_size,
    ⇒ epochs=num_epochs, callbacks=callbacks_list, validation_split=0.2)
history_list.append(hist)

# конец for

plt.figure()
plt.plot(history_list[0].history['loss'], 'b', lw=2.0, label='SGD')
plt.plot(history_list[1].history['loss'], '--r', lw=2.0, label='SGD
    ⇒ Нестерова')
plt.plot(history_list[2].history['loss'], ':g', lw=2.0, label='RMSProp')
plt.plot(history_list[3].history['loss'], '-.k', lw=2.0, label='ADAM')
plt.title('LeNet, CIFAR-100, оптимизаторы')
plt.xlabel('Эпоха')
plt.ylabel('Потери кросс-энтропии при обучении')
plt.legend(loc='upper right')
plt.show()
#plt.savefig('./figures/lenet_loss.png')

plt.figure()
plt.plot(history_list[0].history['val_accuracy'], 'b', lw=2.0,
    ⇒ label='SGD')
plt.plot(history_list[1].history['val_accuracy'], '--r', lw=2.0,
    ⇒ label='SGD Нестерова')
plt.plot(history_list[2].history['val_accuracy'], ':g', lw=2.0,
    ⇒ label='RMSProp')
plt.plot(history_list[3].history['val_accuracy'], '-.k', lw=2.0,
    ⇒ label='ADAM')
plt.title('LeNet, CIFAR-100, оптимизаторы')
plt.xlabel('Эпоха')
plt.ylabel('Валидационная доля верных результатов')
plt.legend(loc='upper right')
plt.show()
#plt.savefig('./figures/lenet_loss.png')

plt.figure()
plt.plot(history_list[0].history['lr'], 'b', lw=2.0, label='SGD')
plt.plot(history_list[1].history['lr'], '--r', lw=2.0, label='SGD
    ⇒ Nesterov')
plt.plot(history_list[2].history['lr'], ':g', lw=2.0, label='RMSProp')
plt.plot(history_list[3].history['lr'], '-.k', lw=2.0, label='ADAM')
plt.title('LeNet, CIFAR-100, Optimizers')
plt.xlabel('Epochs')
plt.ylabel('Learning Rate Schedule')
plt.legend(loc='upper right')
plt.show()
#plt.savefig('./figures/lenet_loss.png')

```

На рис. 10.13 показаны графики потерь и доли верных результатов на валидационном наборе для нейросети LeNet, обученной на наборе данных CIFAR-100 с использованием различных оптимизаторов:

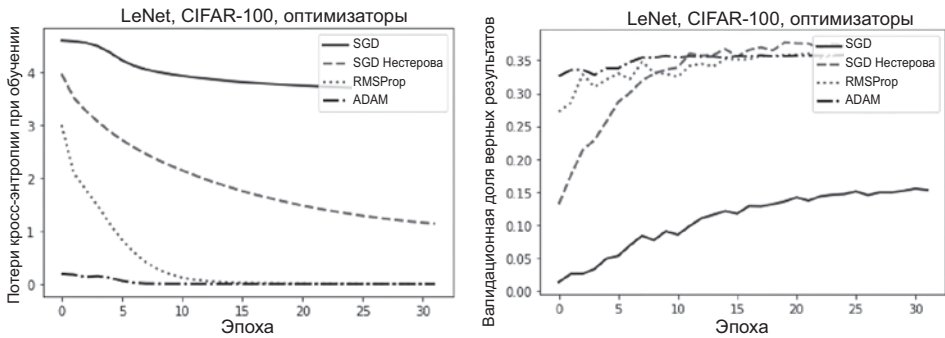


Рис. 10.13. Потеря кросс-энтропии и валидационная верность нейросети LeNet, обученной на CIFAR-100 с использованием различных оптимизаторов

Результаты показывают, что в нашем эксперименте оптимизатор Adam и импульс Нестерова продемонстрировали наивысшую верность результатов на валидационном наборе. В следующей главе мы сосредоточимся на продвинутых алгоритмах глубокого обучения. Мы научимся обнаруживать аномалии в данных временных рядов с помощью вариационного автоэнкодера (variational autoencoder, VAE), определять кластеры с помощью сетей смешанной плотности (mixture density network, MDN), классифицировать тексты с помощью трансформеров и графы цитирования — с помощью графовых нейронных сетей (graph neural network, GNN).

10.5. Упражнения

- 10.1. Объясните назначение нелинейностей в нейронных сетях.
- 10.2. Объясните проблему затухания и взрыва градиента.
- 10.3. Опишите способы увеличить емкость нейронной модели и избежать при этом переобучения.
- 10.4. Почему количество фильтров в архитектуре LeNet увеличивается по мере продвижения от входа к выходу?
- 10.5. Объясните, как работает оптимизатор Adam.

Итоги

- Многослойный перцептрон состоит из набора плотно соединенных слоев, за которыми следует нелинейность.
- Потеря кросс-энтропии используется в задачах классификации, тогда как потеря среднеквадратичной ошибки применяется в задачах регрессии.

- Оптимизация нейросетей происходит с использованием алгоритма обратного распространения ошибки, построенного на основе цепного правила.
- Увеличение емкости модели для борьбы с недообучением может быть достигнуто путем изменения ее архитектуры (например, увеличения количества слоев и скрытых элементов на слой).
- Регуляризация, позволяющая избежать переобучения, может происходить на нескольких уровнях, причем используются такие приемы, как уменьшение весов, ранний останов и прореживание.
- Сверточные нейронные сети исключительно хорошо показали себя в работе с данными изображений. Они используют слои свертки и объединения вместо векторизованного матричного умножения.
- Классическая архитектура СНС включает в себя сверточный слой, за которым следует нелинейная функция активации ReLU и слой объединения по максимальному значению.
- Предварительно обученные СНС можно использовать, например, для извлечения векторов признаков изображений или поиска изображений.
- Рекуррентные нейронные сети предназначены для обработки последовательных данных.
- Области применения РНС включают в себя генерацию языка, классификацию последовательностей и перевод последовательностей.
- Нейронные сети с несколькими входами могут быть построены путем объединения векторов признаков из отдельных входных ветвей.
- Оптимизаторы нейронных сетей включают в себя стохастический градиентный спуск, метод импульса, импульс Нестерова, AdaGrad, RMSProp и Adam.

11

Передовые алгоритмы глубокого обучения

В этой главе

- ✓ Вариационные автоэнкодеры для обнаружения аномалий временных рядов
- ✓ Сети смешанной плотности, использующие амортизированный вариационный вывод
- ✓ Механизм внимания и трансформеры
- ✓ Графовые нейронные сети
- ✓ Исследования в области ML: глубокое обучение

В предыдущей главе мы рассматривали фундаментальные алгоритмы глубокого обучения, которые помогают работать с числовыми, графическими и текстовыми данными. В этой главе мы поговорим о передовых алгоритмах глубокого обучения. Они были подобраны с учетом соответствия их архитектуры современным стандартам качества и широты спектра применения. В этой главе мы изучим генеративные модели, основанные на вариационных автоэнкодерах (variational autoencoders, VAE), и рассмотрим полноценную реализацию детектора аномалий для данных временных рядов. Мы продолжим наше путешествие знакомством с интригующей комбинацией нейронных сетей и классических моделей гауссовой смеси с использованием амортизированного вариационного вывода и взглянем на реализацию сети смешанной плотности. Затем мы сосредоточимся на концепции

внимания и изучим реализацию с чистого листа архитектуры трансформера для задачи классификации. Наконец, мы рассмотрим графовые нейронные сети и используем одну из них для классификации вершин в графе цитирования. На протяжении всей этой главы мы будем пользоваться библиотекой глубокого обучения Keras/TensorFlow.

11.1. Автоэнкодеры

Автоэнкодер — это обучаемая в режиме без учителя нейронная сеть, которая используется для уменьшения размерности или выделения признаков. Она состоит из энкодера, скрытого слоя — так называемого узкого места (bottleneck) — и декодера, где энкодер и декодер являются обучаемыми нейронными сетями, как показано на рис. 11.1.

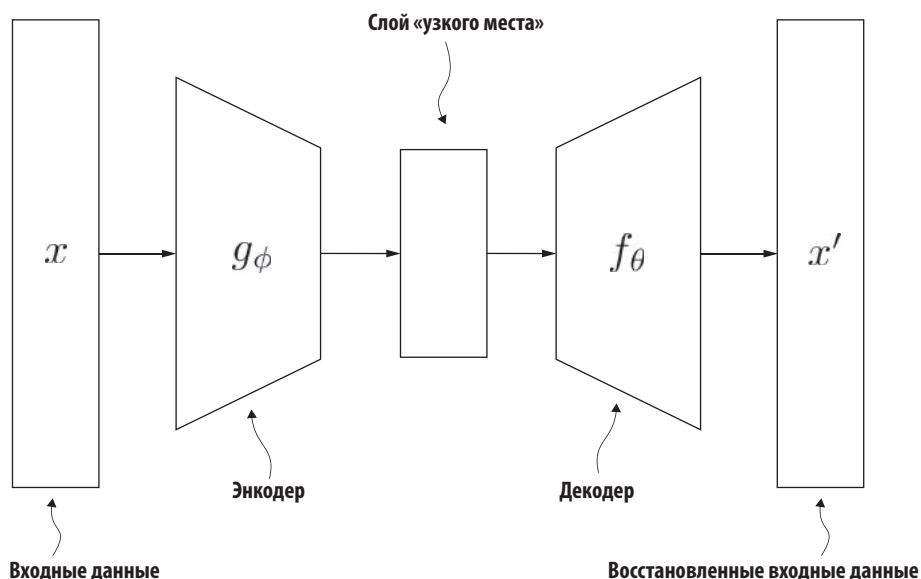


Рис. 11.1. Архитектура автоэнкодера, состоящая из энкодера, слоя «узкого места» и декодера

Автоэнкодеры обучаются восстанавливать свои входные данные. Другими словами, выходные данные автоэнкодера должны максимально точно соответствовать его входным данным. Чтобы обучение нейросети не свелось к выучиванию ею тривиальной функции тождества, скрытый слой в середине должен быть действительно узким местом. При обучении автоэнкодера минимизируется ошибка восстановления (reconstruction error), гарантируя, что скрытые модули отражают наиболее существенную информацию во входных данных.

На практике автоэнкодеры используются для извлечения признаков и не способны к созданию хорошо структурированных скрытых пространств. Вот тут-то в дело и вступают VAE — см. диссертацию Д. П. Кингмы (D. P. Kingma) «Variational Inference and Deep Learning: A New Synthesis» (University of Amsterdam, 2017). Архитектура VAE также содержит энкодер и декодер. Однако вместо сжатия входного изображения до размеров узкого скрытого слоя VAE преобразует изображение в параметры статистического распределения (например, среднее значение и дисперсию гауссовой случайной величины). Затем VAE из этого распределения генерирует выборочное значение, которое используется для декодирования обратно в исходное представление. Рисунок 11.2 демонстрирует архитектуру VAE.

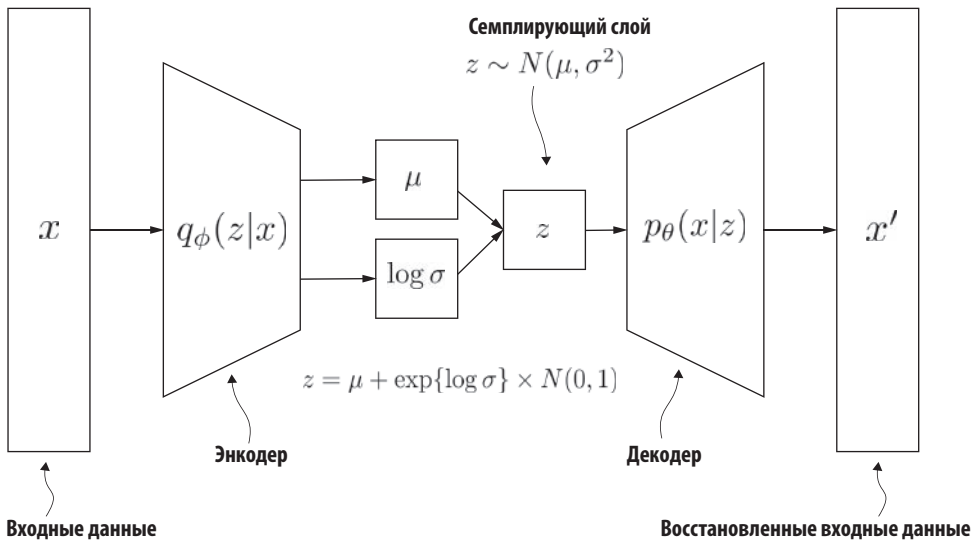


Рис. 11.2. Архитектура вариационного автоэнкодера: энкодер, семплирующий слой и декодер

Обучение VAE структурирует скрытое пространство таким образом, что каждая точка может быть декодирована в корректный выходной формат. Целевой функцией вариационного автоэнкодера является нижняя вариационная граница (ELBO). Пусть x представляет собой входное пространство, а z — скрытое пространство. Пусть $p(x|z)$ — распределение декодера с параметрами θ , которое, при условии, что выборочное значение из скрытого пространства равно z , восстанавливает исходное значение входных данных x . Аналогично, пусть $q(z|x)$ — это распределение энкодера с параметрами ϕ , которое принимает входные данные x и кодирует их в скрытую переменную z . Обратите внимание, что параметры θ и ϕ получаются при обучении. Наконец, пусть $p(z)$ — априорное распределение скрытого пространства. Поскольку мы стремимся к тому, чтобы вариационное

апостериорное распределение было как можно ближе к истинному апостериорному, то для обучения VAE используется следующая функция потерь:

$$\min_{\theta, \phi} \text{Loss}(x; \theta, \phi) = E_{q_{\phi}(z|x)} [\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x) \| p_{\theta}(z)). \quad (11.1)$$

Первое слагаемое определяет, насколько хорошо VAE восстанавливает точку данных x из выборочного значения z вариационного апостериорного распределения, а второе — насколько вариационное апостериорное распределение $q(z|x)$ близко к априорному распределению $p(z)$. Если предположить, что априорное распределение $p(z)$ является гауссовым, то член KL-дивергенции можно переписать следующим образом (см. формулу 11.2). В следующем разделе мы увидим, как VAE можно использовать для обнаружения аномалий во временных рядах.

$$D_{KL}(q_{\phi}(z|x) \| p_{\theta}(z)) = -\frac{1}{2} \sum_{d=1}^D (1 + \log \sigma_d^2 - \mu_d^2 - \sigma_d^2). \quad (11.2)$$

11.1.1. VAE: обнаружение аномалий во временных рядах

Давайте рассмотрим модель VAE, которая оперирует данными временных рядов с целью обнаружения аномалий. Эта архитектура показана на рис. 11.3:

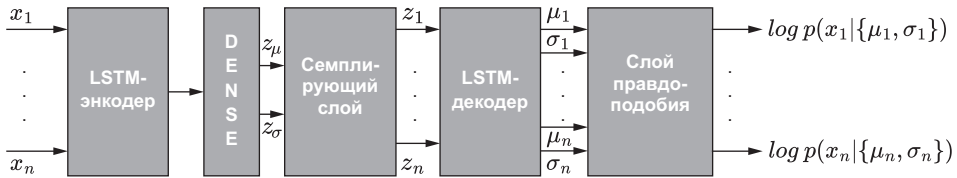


Рис. 11.3. Архитектура детектора аномалий LSTM-VAE

Входные данные состоят из n сигналов $x_1, \dots, x_n$, а выходные представляют собой логарифмическую вероятность наблюдения входных данных x_i при нормальных (неаномальных) параметрах обучения μ_i, σ_i . Это означает, что модель обучается на неаномальных данных в режиме без учителя, и когда в заданных входных данных x_i возникает аномалия, соответствующий логарифм правдоподобия $p(x_i | \{\mu_i, \sigma_i\})$ падает, и по пересечению порогового значения можно судить о наличии аномалии.

Исходя из предположения гауссова правдоподобия, каждый датчик для представления аномалии имеет две степени свободы: (μ, σ) . В результате для n входных датчиков мы обучаем $2n$ выходных параметров (среднее значение и дисперсию), которые позволяют отличить аномальное поведение от нормального.

Хотя входные сигналы независимы, в VAE они вкладываются (embedded) в совместное скрытое пространство семплирующего слоя. Структура скрытого

пространства приближается к стандартному нормальному распределению $N(0,1)$ путем минимизации KL -дивергенции. Модель обучается в режиме без учителя с помощью целевой функции, которая помогает достичь следующих двух целей: (1) максимизации логарифмического правдоподобия модели, усредненного по датчикам, и (2) структурирования скрытого пространства для приближения к $N(0, 1)$:

$$\min_{\theta} Loss(\theta) = \min_{\theta} D_{KL}(N(z_{\mu}, z_{\sigma}) \| N(0, 1)) - \frac{1}{n} \sum_{i=1}^n \log p(x_i | \mu_i, \sigma_i). \quad (11.3)$$

Теперь мы можем приступить к знакомству с полноценной реализацией детектора аномалий LSTM-VAE, построенной на базе библиотеки Keras/TensorFlow. В приведенном ниже коде мы загружаем набор данных NAB (который можно найти в папке данных репозитория кода) и подготавливаем их для обучения. Numenta Anomaly Benchmark (NAB) — это новый тест для оценки алгоритмов обнаружения аномалий в потоковых и онлайн-приложениях. Далее мы определяем архитектуру детектора аномалий вместе с пользовательской функцией потерь и обучаем модель. Вам стоит попробовать запустить этот код в режиме блокнота на платформе Google Colab (доступна по ссылке <https://colab.research.google.com/>), чтобы посмотреть на результаты пошагового исполнения, а также ускорить процесс обучения модели при помощи графического процессора.

Листинг 11.1. Детектор аномалий LSTM-VAE

```
import numpy as np
import pandas as pd

import tensorflow as tf
from tensorflow import keras
import tensorflow_probability as tfp

from keras.layers import Input, Dense, Lambda, Layer
from keras.layers import LSTM, RepeatVector
from keras.models import Model
from keras import backend as K
from keras import metrics
from keras import optimizers

import math
import json
from scipy.stats import norm
from sklearn.model_selection import train_test_split
from sklearn import preprocessing
from sklearn.metrics import confusion_matrix
from sklearn.preprocessing import StandardScaler

from keras.callbacks import ModelCheckpoint
from keras.callbacks import TensorBoard
from keras.callbacks import LearningRateScheduler
from keras.callbacks import EarlyStopping
```

286 Глава 11. Передовые алгоритмы глубокого обучения

```
import matplotlib.pyplot as plt

tf.keras.utils.set_random_seed(42)

SAVE_PATH = "/content/drive/MyDrive/Colab Notebooks/data/"
DATA_PATH = "/content/drive/MyDrive/data/"

def scheduler(epoch, lr):
    if epoch < 4:
        return lr
    else:
        return lr * tf.math.exp(-0.1)

nab_path = DATA_PATH + 'NAB/'
nab_data_path = nab_path

labels_filename = '/labels/combined_labels.json'
train_file_name = 'artificialNoAnomaly/art_daily_no_noise.csv'
test_file_name = 'artificialWithAnomaly/art_daily_jumpsup.csv'
#train_file_name = 'realAWScloudwatch/rds_cpu_utilization_cc0c53.csv'
#test_file_name = 'realAWScloudwatch/rds_cpu_utilization_e47b3b.csv'

labels_file = open(nab_path + labels_filename, 'r')
labels = json.loads(labels_file.read())
labels_file.close()

def load_data_frame_with_labels(file_name):
    data_frame = pd.read_csv(nab_data_path + file_name)
    data_frame['anomaly_label'] = data_frame['timestamp'].isin(
        labels[file_name]).astype(int)
    return data_frame

train_data_frame = load_data_frame_with_labels(train_file_name)
test_data_frame = load_data_frame_with_labels(test_file_name)

plt.plot(train_data_frame.loc[0:3000, 'value'])
plt.plot(test_data_frame['value'])

train_data_frame_final = train_data_frame.loc[0:3000, :]
test_data_frame_final = test_data_frame

data_scaler = StandardScaler()
data_scaler.fit(train_data_frame_final[['value']].values)
train_data = data_scaler.transform(train_data_frame_final[['value']].values)
test_data = data_scaler.transform(test_data_frame_final[['value']].values)

def create_dataset(dataset, look_back=64):
    dataX, dataY = [], []
    for i in range(len(dataset)-look_back-1):
        dataX.append(dataset[i:(i+look_back), :])
        dataY.append(dataset[i+look_back, :])

    return np.array(dataX), np.array(dataY)

X_data, y_data = create_dataset(train_data, look_back=64) #look_back =
⇒ window_size
```

```
X_train, X_val, y_train, y_val = train_test_split(X_data, y_data,
    ➤ test_size=0.1, random_state=42)
X_test, y_test = create_dataset(test_data, look_back=64) #look_back =
    ➤ window_size
```

```
batch_size = 256
num_epochs = 32
```

Параметры
обучения

```
timesteps = X_train.shape[1]
input_dim = X_train.shape[-1]
intermediate_dim = 16
latent_dim = 2
epsilon_std = 1.0
```

Параметры
модели

```
class Sampling(Layer):  ← Семплирующий слой
```

```
    def call(self, inputs):
        z_mean, z_log_var = inputs
        batch = tf.shape(z_mean)[0]
        dim = tf.shape(z_mean)[1]
        epsilon = tf.keras.backend.random_normal(shape=(batch, dim))
        return z_mean + tf.exp(0.5 * z_log_var) * epsilon
```

```
class Likelihood(Layer):  ← Слой правдоподобия
```

```
    def call(self, inputs):
        x, x_decoded_mean, x_decoded_scale = inputs
        dist = tfp.distributions.MultivariateNormalDiag(x_decoded_mean,
    ➤ x_decoded_scale)
        likelihood = dist.log_prob(x)
        return likelihood
```

```
# Архитектура VAE
```

```
# кодер
```

```
x = Input(shape=(timesteps, input_dim,))
h = LSTM(intermediate_dim)(x)
```

```
z_mean = Dense(latent_dim)(h)
z_log_sigma = Dense(latent_dim, activation='softplus')(h)
```

```
# семплирование
```

```
z = Sampling()(z_mean, z_log_sigma)
```

```
# декодер
```

```
decoder_h = LSTM(intermediate_dim, return_sequences=True)
decoder_loc = LSTM(input_dim, return_sequences=True)
decoder_scale = LSTM(input_dim, activation='softplus', return_sequences=True)
```

```
h_decoded = RepeatVector(timesteps)(z)
h_decoded = decoder_h(h_decoded)
```

```
x_decoded_mean = decoder_loc(h_decoded)
x_decoded_scale = decoder_scale(h_decoded)
```

```
# логарифмическое правдоподобие
```

```
llh = Likelihood()(x, x_decoded_mean, x_decoded_scale)]
```

288 Глава 11. Передовые алгоритмы глубокого обучения

```
vae = Model(inputs=x, outputs=llh)  ← Определение VAE-модели

# Потери: KL-дивергенция плюс логарифмическое правдоподобие
kl_loss = - 0.5 * K.mean(1 + z_log_sigma - K.square(z_mean) -
    ⇒ K.exp(z_log_sigma))
tot_loss = -K.mean(llh - kl_loss)
vae.add_loss(tot_loss)

# Функция потерь и оптимизатор
loss_fn = tf.keras.losses.MeanSquaredError()
optimizer = tf.keras.optimizers.Adam()

@tf.function
def training_step(x):
    with tf.GradientTape() as tape:
        reconstructed = vae(x) # Восстановление входных данных
        # Вычисление потерь
        loss = 0 #loss_fn(x, reconstructed)
        loss += sum(vae.losses)
    # Обновление весов VAE
    grads = tape.gradient(loss, vae.trainable_weights)
    optimizer.apply_gradients(zip(grads, vae.trainable_weights))
    return loss

losses = [] # Регистрация потерь во времени
dataset = tf.data.Dataset.from_tensor_slices(X_train).batch(batch_size)
for epoch in range(num_epochs):
    for step, x in enumerate(dataset):
        loss = training_step(x)
        losses.append(float(loss))
    print("Epoch:", epoch, "Loss:", sum(losses) / len(losses))

plt.figure()
plt.plot(losses, c='b', lw=2.0, label='Обучение')
plt.title('Модель LSTM-VAE')
plt.xlabel('Эпоха')
plt.ylabel('Общие потери')
plt.legend(loc='upper right')
plt.show()

pred_test = vae.predict(X_test)

plt.plot(pred_test[:,0])

is_anomaly = pred_test[:,0] < -1e1
plt.figure()
plt.plot(test_data, color='b')
plt.figure()
plt.plot(is_anomaly, color='r')
```

Рисунок 11.4 демонстрирует, что в простом прямоугольном входном сигнале удается обнаружить падение логарифмической вероятности и после пересечения порогового значения выдать сигнал о наличии аномалии. Уменьшение общих потерь в процессе обучения показано на рис. 11.5.

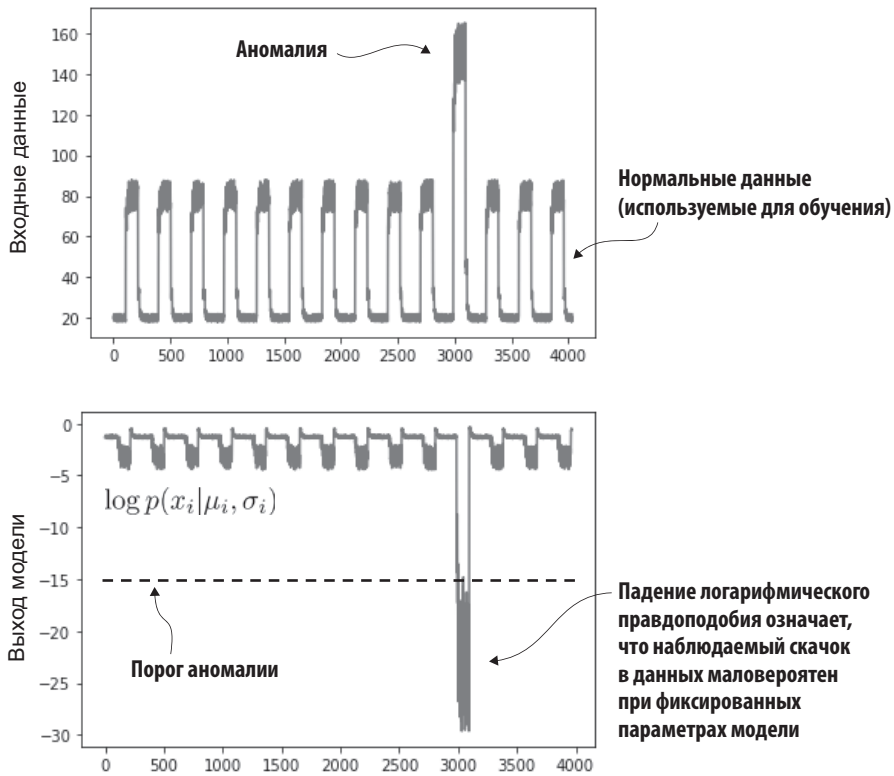


Рис. 11.4. Результат обнаружения аномалии при помощи LSTM-VAE

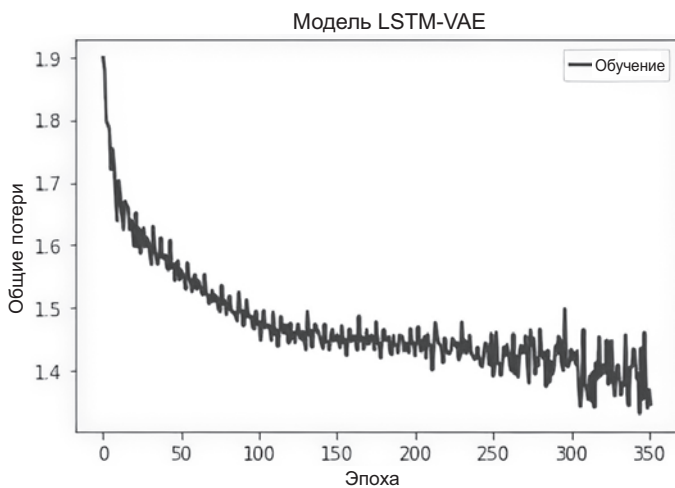


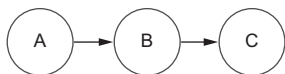
Рис. 11.5. Потери при обучении LSTM-VAE

В следующем разделе мы рассмотрим амортизированный вариационный вывод применительно к сетям смешанной плотности.

11.2. Амортизированный вариационный вывод

Амортизированный вариационный вывод (VI) является воплощением идеи, что вместо оптимизации набора свободных параметров мы можем ввести параметризованную функцию, которая сопоставляет пространство наблюдений с параметрами приближенного апостериорного распределения. В практическом плане наблюдения могут подаваться на вход нейросети, которая выводит параметры среднего значения и дисперсии для скрытой переменной, связанной с этими наблюдениями (с чем мы столкнулись в архитектуре VAE). Затем можно оптимизировать параметры этой нейросети вместо отдельных параметров каждого наблюдения.

Одним из преимуществ амортизированного VI является повторное использование предыдущих выводов по аналогии с динамическим программированием, в котором мы храним решения ранее вычисленных подзадач. В качестве примера рассмотрим два запроса, приведенные на рис. 11.6, — см. книгу Сэмюэл Дж. Гершмана (Samuel J. Gershman), Ноа Д. Гудмана (Noah D. Goodman) «Amortized Inference in Probabilistic Reasoning» (Cognitive Science, 2014):



Запрос 1: $P(B|C) = P(C|B)P(B)/P(C)$

Запрос 2: $P(A|C) = \sum_B P(A|B)P(B|C)$

Рис. 11.6. Простая байесовская сеть.

Запрос 1 является подзапросом запроса 2

Видно, что запрос 1 является подзапросом запроса 2. Тем самым условное распределение, вычисленное для запроса 1, может быть повторно использовано для ответа на запрос 2. Еще одним преимуществом амортизированного VI является то, что в нем отсутствует необходимость аналитического вывода ELBO, поскольку оптимизация происходит с помощью стохастического градиентного спуска (SGD). Ограничение амортизированного VI состоит в том, что пробел в обобщающей способности (generalization gap) модели зависит от емкости выбранной нейронной сети как стохастической функции.

Давайте рассмотрим один из примеров амортизированного VI, а именно сеть смешанной плотности, в которой мы будем использовать многослойный перцептрон (МСП) для параметризации модели гауссовой смеси.

11.2.1. Сети смешанной плотности

Сети смешанной плотности (mixture density networks, MDN) — это модели смеси, в которых параметры, такие как среднее значение, ковариация и пропорция

смеси, получаются в результате обучения нейронной сети. Сети MDN объединяют в себе структурированное представление данных (смесь плотностей) с неструктурированным выводом параметров (нейронная сеть МСП). При обучении MDN параметры смеси получаются за счет максимизации логарифмического правдоподобия или, что эквивалентно, минимизации потерь отрицательного логарифмического правдоподобия.

Предполагая модель гауссовой смеси с K компонентами, мы можем записать вероятность тестовой точки данных y_i при условии обучающих данных x следующим образом:

$$p(y_i | x) = \sum_{k=1}^K \pi_k(x) N(y_i | \mu_k(x), \Sigma_k(x)). \quad (11.4)$$

Здесь параметры μ_k , σ_k , π_k выучиваются нейронной сетью (например, МСП) с параметрами θ :

$$\mu_k, \sigma_k, \pi_k = NN(x; \theta). \quad (11.5)$$

Архитектура MDN представлена на рис. 11.7.

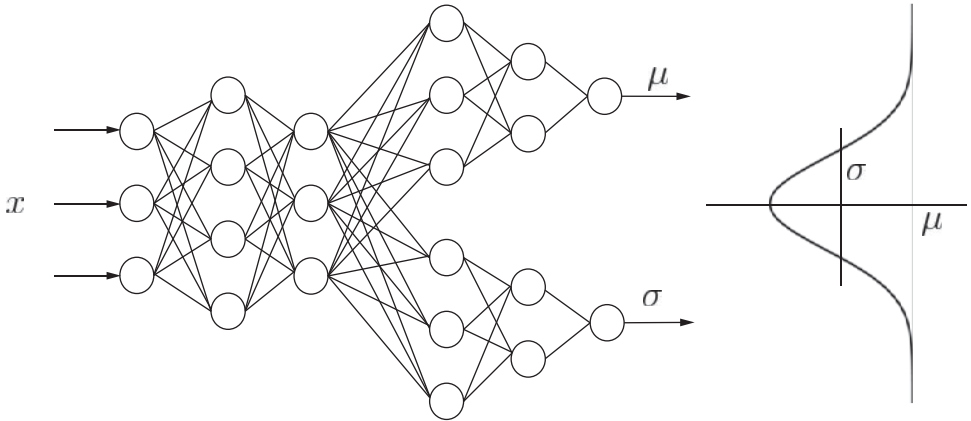


Рис. 11.7. Нейронная сеть с несколькими выходами для получения гауссовых параметров

Как результат, нейронная сеть (НС) представляет собой модель с несколькими выходами, на которые накладываются следующие ограничения:

$$\begin{aligned} \forall k \sigma_k(x) &> 0. \\ \sum_{k=1}^K \pi_k(x) &= 1. \end{aligned} \quad (11.6)$$

Другими словами, мы накладываем ограничения, что дисперсия является строго положительной величиной, а весовые коэффициенты смеси в сумме дают 1.

Первое ограничение может быть достигнуто с помощью экспоненциальных активаций, а второе — с помощью активаций softmax. Наконец, используя предположение о независимости и одинаковой распределенности, мы можем попытаться минимизировать следующую функцию потерь:

$$\begin{aligned} \min_{\theta} L(\theta) &= NLLLoss(\theta) = -\log \prod_{i=1}^n p(y_i | x) = -\sum_{i=1}^n \log p(y_i | x) = \\ &= -\sum_{i=1}^n \log \left[\sum_{k=1}^K \pi_k(x_i, \theta) N(y_i | \mu_k(x_i, \theta), \Sigma_k(x_i, \theta)) \right]. \end{aligned} \quad (11.7)$$

В нашем примере используется предположение об изотропности ковариационной матрицы: $\Sigma_k = \sigma_k^2 I$. Таким образом, мы можем записать d -мерный гауссиан в виде произведения. С учетом формулы для многомерного гауссиана мы получаем следующее:

$$N(y_i | \mu_k, \Sigma_k) = \frac{1}{(2\pi)^{\frac{d}{2}} |\Sigma_k|^{\frac{1}{2}}} \exp \left(-\frac{1}{2} (y_i - \mu_k)^T \Sigma_k^{-1} (y_i - \mu_k) \right). \quad (11.8)$$

Поскольку ковариационная матрица изотропна, мы можем переписать формулу 11.8 следующим образом:

$$\begin{aligned} N(y_i | \mu_k, \Sigma_k) &= \frac{1}{(2\pi\sigma_k^2)^{\frac{d}{2}}} \exp \left[-\frac{1}{2\sigma_k^2} \sum_{d=1}^D (y_{i,d} - \mu_{k,d})^2 \right] = \\ &= \prod_{d=1}^D \frac{1}{\sigma_k \sqrt{2\pi}} \exp \left[-\frac{1}{2\sigma_k^2} (y_{i,d} - \mu_{k,d})^2 \right]. \end{aligned} \quad (11.9)$$

Давайте теперь познакомимся с гауссовой MDN, реализованной с использованием Keras/TensorFlow. В нашем примере мы используем синтетические данные с эталонными значениями среднего и дисперсии. Данные генерируются при помощи семплирования из многомерного распределения. Затем мы определяем архитектуру MDN с несколькими выходами с ограничениями на пропорции смеси и дисперсию. Наконец, мы вычисляем потерю отрицательного логарифмического правдоподобия, обучаем модель и отображаем результаты предсказания на тестовых данных. Вам стоит попробовать запустить этот код в режиме блокнота на платформе Google Colab (доступна по ссылке <https://colab.research.google.com/>), чтобы посмотреть на результаты пошагового исполнения, а также ускорить процесс обучения модели при помощи графического процессора.

Листинг 11.2. Сеть плотности гауссовой смеси

```
import numpy as np
import pandas as pd

import tensorflow as tf
```

```

from tensorflow import keras

from keras.models import Model
from keras.layers import concatenate, Input
from keras.layers import Dense, Activation, Dropout, Flatten
from keras.layers import BatchNormalization

from keras import regularizers
from keras import backend as K
from keras.utils import np_utils

from keras.callbacks import ModelCheckpoint
from keras.callbacks import TensorBoard
from keras.callbacks import LearningRateScheduler
from keras.callbacks import EarlyStopping

from sklearn.datasets import make_blobs
from sklearn.metrics import adjusted_rand_score
from sklearn.metrics import normalized_mutual_info_score
from sklearn.model_selection import train_test_split

import math
import matplotlib.pyplot as plt
import matplotlib.cm as cm

tf.keras.utils.set_random_seed(42)

SAVE_PATH = "/content/drive/MyDrive/Colab Notebooks/data/"

def scheduler(epoch, lr):  ← Изменение скорости обучения
    if epoch < 4:
        return lr
    else:
        return lr * tf.math.exp(-0.1)

def generate_data(N):  ← Синтетические эталонные данные
    pi = np.array([0.2, 0.4, 0.3, 0.1])
    mu = [[2,2], [-2,2], [-2,-2], [2,-2]]
    std = [[0.5,0.5], [1.0,1.0], [0.5,0.5], [1.0,1.0]]
    x = np.zeros((N,2), dtype=np.float32)
    y = np.zeros((N,2), dtype=np.float32)
    z = np.zeros((N,1), dtype=np.int32)
    for n in range(N):
        k = np.argmax(np.random.multinomial(1, pi))
        x[n,:] = np.random.multivariate_normal(mu[k], np.diag(std[k]))
        y[n,:] = mu[k]
        z[n,:] = k
    # конец for
    z = z.flatten()
    return x, y, z, pi, mu, std

def tf_normal(y, mu, sigma):  ← Изотропный многомерный гауссиан
    y_tile = K.stack([y]*num_clusters, axis=1) #[batch_size, K, D]
    result = y_tile - mu
    sigma_tile = K.stack([sigma]*data_dim, axis=-1) #[batch_size, K, D]
    result = result * 1.0/(sigma_tile+1e-8)
    result = -K.square(result)/2.0
    oneDivSqrtTwoPI = 1.0/math.sqrt(2*math.pi)

```

294 Глава 11. Передовые алгоритмы глубокого обучения

```
result = K.exp(result) * (1.0/(sigma_tile + 1e-8))*oneDivSqrtTwoPI
result = K.prod(result, axis=-1) #[batch_size, K] независимые
    ➡ и одинаково распределенные гауссианы
return result

def NLLLoss(y_true, y_pred):  ← Потеря отрицательного логарифмического правдоподобия
    out_mu = y_pred[:, :num_clusters*data_dim]
    out_sigma = y_pred[:, num_clusters*data_dim : num_clusters*(data_dim+1)]
    out_pi = y_pred[:, num_clusters*(data_dim+1):]

    out_mu = K.reshape(out_mu, [-1, num_clusters, data_dim])

    result = tf_normal(y_true, out_mu, out_sigma)
    result = result * out_pi
    result = K.sum(result, axis=1, keepdims=True)
    result = -K.log(result + 1e-8)
    result = K.mean(result)
    return tf.maximum(result, 0)

# генерация данных
X_data, y_data, z_data, pi_true, mu_true, sigma_true = generate_data(4096)

data_dim = X_data.shape[1]
num_clusters = len(mu_true)

num_train = 3500
X_train, X_test, y_train, y_test = X_data[:num_train,:],
    ➡ X_data[num_train:,:], y_data[:num_train:], y_data[num_train:,:]
z_train, z_test = z_data[:num_train], z_data[num_train:]

# визуализация данных
plt.figure()
plt.scatter(X_train[:,0], X_train[:,1], c=z_train, cmap=cm.bwr)
plt.title('training data')
plt.show()
#plt.savefig(SAVE_PATH + '/mdn_training_data.png')

batch_size = 128 | Параметры
num_epochs = 128 | обучения

hidden_size = 32 | Параметры
weight_decay = 1e-4 | модели

# Архитектура MDN
input_data = Input(shape=(data_dim,))
x = Dense(32, activation='relu')(input_data)
x = Dropout(0.2)(x)
x = BatchNormalization()(x)
x = Dense(32, activation='relu')(x)
x = Dropout(0.2)(x)
x = BatchNormalization()(x)

mu = Dense(num_clusters * data_dim, activation
    ➡ ='linear')(x)  ← Кластерные средние

sigma = Dense(num_clusters, activation=K.exp)(x)  ← Диагональная ковариационная матрица
```

```

pi = Dense(num_clusters, activation='softmax')(x)  ← Пропорции смеси
out = concatenate([mu, sigma, pi], axis=-1)

model = Model(input_data, out)

model.compile(
    loss=NLLLoss,
    optimizer=tf.keras.optimizers.Adam(),
    metrics=["accuracy"]
)

model.summary()

# Определение обратных вызовов
file_name = SAVE_PATH + 'mdn-weights-checkpoint.h5'
checkpoint = ModelCheckpoint(file_name, monitor='val_loss', verbose=1,
    ➔ save_best_only=True, mode='min')
reduce_lr = LearningRateScheduler(scheduler, verbose=1)
early_stopping = EarlyStopping(monitor='val_loss', min_delta=0.01,
    ➔ patience=16, verbose=1)
#tensor_board = TensorBoard(log_dir='./logs', write_graph=True)
callbacks_list = [checkpoint, reduce_lr, early_stopping]

hist = model.fit(X_train, y_train, batch_size
    ➔ =batch_size, epochs=num_epochs, callbacks
    ➔ =callbacks_list, validation_split=0.2,
    ➔ shuffle=True, verbose=2)  ← Обучение модели

y_pred = model.predict(X_test)  ← Оценка модели

mu_pred = y_pred[:, num_clusters*data_dim]
mu_pred = np.reshape(mu_pred, [-1, num_clusters, data_dim])
sigma_pred = y_pred[:, num_clusters*data_dim : num_clusters*(data_dim+1)]
pi_pred = y_pred[:, num_clusters*(data_dim+1):]
z_pred = np.argmax(pi_pred, axis=-1)

rand_score = adjusted_rand_score(z_test, z_pred)
print("adjusted rand score: ", rand_score)

nmi_score = normalized_mutual_info_score(z_test, z_pred)
print("normalized MI score: ", nmi_score)

mu_pred_list = []
sigma_pred_list = []
for label in np.unique(z_pred):
    z_idx = np.where(z_pred == label)[0]
    mu_pred_lbl = np.mean(mu_pred[z_idx, label, :], axis=0)
    mu_pred_list.append(mu_pred_lbl)

    sigma_pred_lbl = np.mean(sigma_pred[z_idx, label], axis=0)
    sigma_pred_list.append(sigma_pred_lbl)
# конец for

print("true means:")
print(np.array(mu_true))

```

```

print("predicted means:")
print(np.array(mu_pred_list))

print("true sigmas:")
print(np.array(sigma_true))

print("predicted sigmas:")
print(np.array(sigma_pred_list))

# создание графиков
plt.figure()
plt.scatter(X_test[:,0], X_test[:,1], c=z_pred, cmap=cm.bwr)
plt.scatter(np.array(mu_pred_list)[:,0], np.array(mu_pred_list)[:,1],
            s=100, marker='x', lw=4.0, color='k')
plt.title('Тестовые данные')

plt.figure()
plt.plot(hist.history['loss'], 'b', lw=2.0, label='Обучение')
plt.plot(hist.history['val_loss'], '--r', lw=2.0, label='Валидация')
plt.title('Сеть смешанной плотности')
plt.xlabel('Эпоха')
plt.ylabel('Потеря отрицательного логарифмического правдоподобия')
plt.legend(loc='upper left')

```

На рис. 11.8 показаны кластерные центроиды, наложенные поверх тестовых данных, а также потери при обучении и валидации.

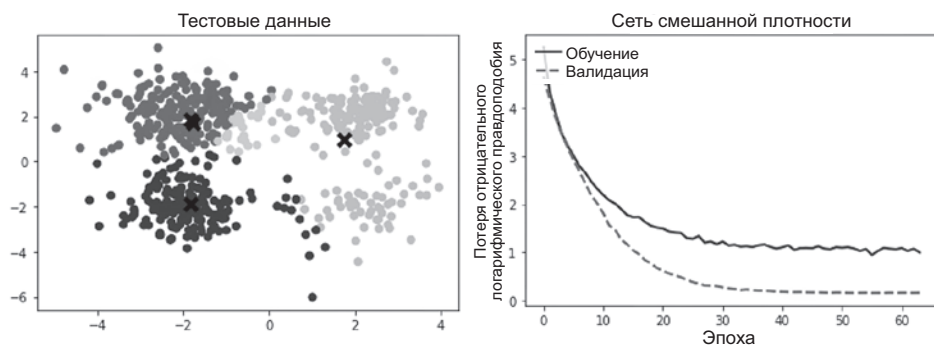


Рис. 11.8. Сеть смешанной плотности: центроиды кластеров и потери при обучении и валидации

Можно видеть, что прогнозируемые средние значения близки к центрам кластеров. Интересно заметить, что для приведенного примера два центра кластера совпадают. Читатель может поэкспериментировать с различными значениями начального числа (seed) и количеством обучающих точек, чтобы понять поведение модели. Кроме того, из графиков видно, что потери как при обучении, так и при валидации уменьшаются с увеличением количества эпох. В следующем

разделе мы рассмотрим мощную архитектуру трансформера, построенную по принципу обучения с самоконтролем (self-supervised learning).

11.3. Внимание и трансформеры

Механизм *внимания* позволяет модели адаптивно, при помощи весовых коэффициентов внимания, регулировать степень внимания к различным частям входных данных. Внимание может применяться во многих видах нейронных сетей, но впервые оно было использовано в контексте рекуррентных нейронных сетей (РНЧ). В РНЧ-моделях Seq2Seq, подобных используемым для машинного перевода, выходной контекстный вектор, который суммирует входное предложение, не имеет доступа к входным словам по отдельности. Это приводит к снижению показателей производительности, измеряемых метрикой BLEU. Мы можем избавиться от этого недочета, позволив выходным данным напрямую взвешенным образом уделять внимание словам на входе. Другими словами, мы можем представить вектор контекста как взвешенную сумму векторов входных слов h_s^{enc} :

$$c_t = \sum_s a_{ts} h_s^{enc}. \quad (11.10)$$

Здесь a_{ts} — это обучаемые значения весов внимания, определяемые следующим образом:

$$a_{ts} = \frac{\exp\left\{ \text{score}\left(h_{t-1}^{dec}, h_s^{enc}\right) \right\}}{\sum_{s'} \exp\left\{ \text{score}\left(h_{t-1}^{dec}, h_{s'}^{enc}\right) \right\}}. \quad (11.11)$$

Существует несколько способов обучения функции оценки (score), например мультипликативный стиль (multiplicative style):

$$\text{score}(a, b) = a^T W b. \quad (11.12)$$

Здесь W — промежуточная обучаемая матрица. Внимание можно обобщенно представить себе как мягкий поиск по словарю (soft dictionary lookup). Мягкий поиск по словарю относится к типу поиска, при котором точное соответствие не найдено. Это полезно при поиске слов, которые могли быть написаны с ошибками или связаны с искомым термином. Мы можем рассматривать механизм внимания как сравнение набора целевых векторов, или запросов, q_i с набором векторов-кандидатов, или ключей, k_j . Для каждого запроса мы оцениваем, насколько он связан с каждым ключом, а затем используем эти оценки для взвешивания и суммирования значений v_j , связанных с каждым ключом. Таким образом, матрицу внимания A можно определить следующим образом:

$$A_{ij} = \text{score}(q_i, k_j). \quad (11.13)$$

Учитывая весовые коэффициенты внимания A_{ij} , вычисляем взвешенную комбинацию значений v_j , связанных с каждым ключом. В результате для i -го запроса имеем следующее:

$$r_i = \sum_j A_{ij} v_j = \sum_j \text{score}(q_i, k_j) v_j. \quad (11.14)$$

Здесь можно выбрать нормированную мультипликативную оценку с $W = 1$:

$$\text{score}(q_i, k_j) = \frac{q_i^T k_j}{\sqrt{D}}. \quad (11.15)$$

Если имеется N запросов (Q размера $N \times D$) и N пар ключ — значение (K размером $N \times D$), то можем записать результат (взвешенный набор значений V , ключи которого K больше всего напоминают запрос Q) в матричной форме:

$$\begin{aligned} R = \text{attention}(Q, K, V) &= AV = \\ &= \text{score}(Q, K) V = \text{softmax} \left(\frac{QK^T}{\sqrt{D}} \right) V. \end{aligned} \quad (11.16)$$

Множитель softmax в произведении гарантирует, что распределение дает в сумме 1, и его можно рассматривать как указатель на место, куда нужно смотреть в матрице значений V .

На рис. 11.9 показана тепловая карта матрицы внимания для нейронного машинного перевода (neural machine translation, NMT). В нем запрос — это целевая

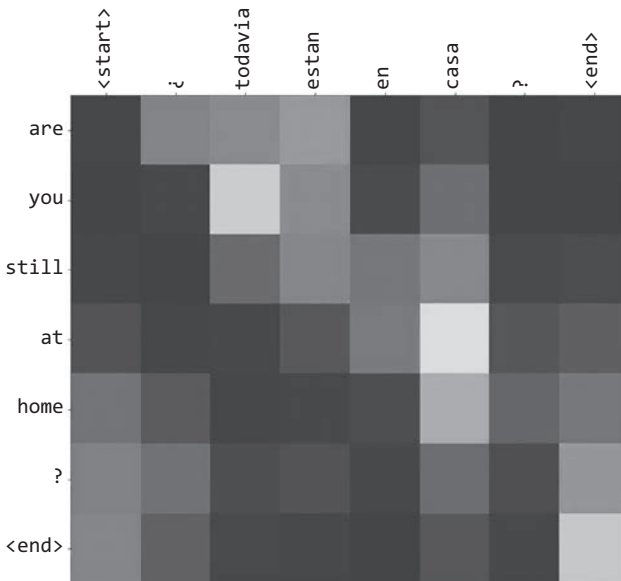


Рис. 11.9. Тепловая карта матрицы внимания для нейронного машинного перевода

последовательность, в то время как ключи и значения — исходная последовательность. Диаграмма показывает, на какие исходные английские слова модель обращает внимание при создании целевой переводной последовательности на испанском языке.

Самовнимание — ключевой компонент архитектуры трансформера. *Трансформер* — это модель Seq2Seq, которая использует внимание как в энкодере, так и в декодере, что устраняет необходимость в РНС. На рис. 11.10 показана архитектура трансформера — см. статью Ашиша Васвани (Ashish Vaswani) и др. «Attention Is All You Need» (NeurIPS, 2017). Давайте более детально рассмотрим строительные блоки трансформера.

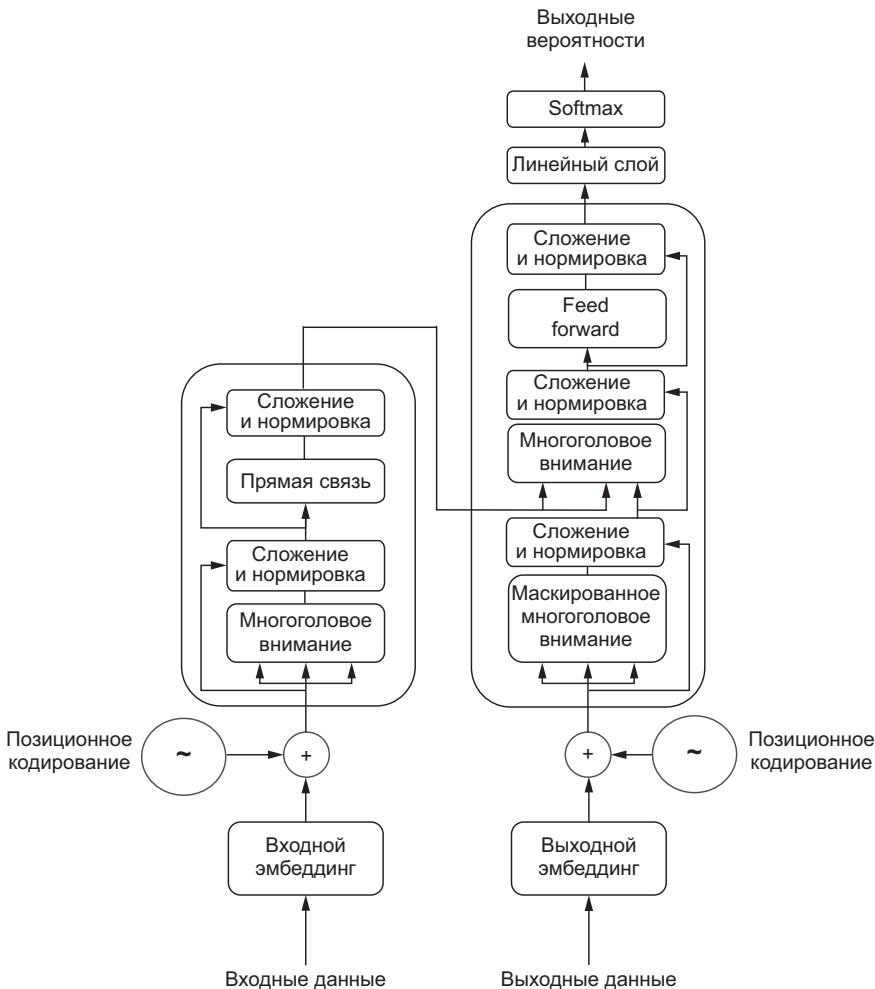


Рис. 11.10. Архитектура трансформера

На верхнем уровне мы видим архитектуру «энкодер — декодер» с входами в левой ветви и выходами в правой ветви. Также можно выделить такие элементы, как многоголовое внимание (multihead attention), позиционное кодирование, плотные слои и слои нормализации (normalization layer), а также остаточные связи (residual connection) для облегчения обратного распространения ошибки.

Идею самовнимания можно распространить на многоголовое внимание. По сути, создавая несколько «голов», мы распараллеливаем механизм внимания, как показано на рис. 11.11. Выходы нескольких голов затем объединяются (concatenate) и преобразуются при помощи плотного слоя. Благодаря многоголовому вниманию модель имеет несколько независимых способов понимания входных данных.

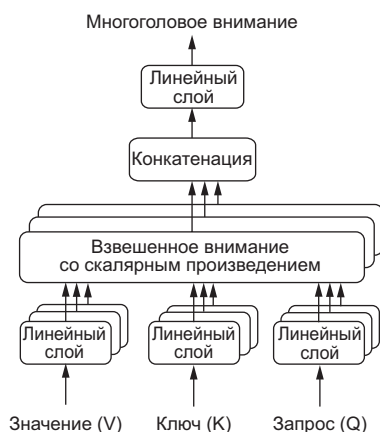


Рис. 11.11. Многоголовое внимание

Чтобы модель могла использовать порядок следования, нужно ввести некоторую информацию о расположении токенов в последовательности. Именно для этого требуется позиционное кодирование, информация о котором добавляется к входным эмбедингам в нижней части ветвей энкодера и декодера. Позиционные кодировки имеют те же размеры, что и эмбединги, так что их можно складывать. Таким образом, если одно и то же слово появляется в нескольких позициях, фактическое представление слова будет отличаться в зависимости от того, где оно стоит в предложении.

Наконец, нормализация входных данных производится путем вычисления среднего значения и дисперсии по каналам и пространственным измерениям, а для завершения работы энкодера добавляется линейный (плотный) уровень. Линейный слой появляется после многоголового самовнимания, чтобы спроецировать представление в пространство более высокой размерности, а затем вернуться к исходному пространству. Это помогает решать проблемы со стабильностью и предотвращает неправильную инициализацию.

Если мы внимательно посмотрим на декодер, то заметим, что он содержит все те же компоненты и, в дополнение к маскированному многоголовому слою самовнимания, новый многоголовый слой самовнимания, известный как *внимание «энкодер — декодер»*. Конечный результат работы декодера преобразуется с помощью заключительного линейного слоя, а выходные вероятности, которые предсказывают следующий токен в выходной последовательности, вычисляются с помощью стандартной функции softmax.

Цель маскированного внимания (masked attention) — соблюдать принцип причинности при генерации выходного предложения. Поскольку полное предложение еще недоступно и генерируется по одному токenu за раз, мы маскируем выходные данные, вводя матрицу масок M , которая содержит только два типа значений: ноль и отрицательную бесконечность:

$$\text{MaskedAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^T + M}{\sqrt{D}}\right)V. \quad (11.17)$$

В конечном счете нули будут преобразованы в единицы с помощью softmax, тогда как отрицательные значения бесконечности станут нулевыми, тем самым удаляя соответствующее соединение из выхода.

Внимание «энкодер — декодер» — это всего лишь известное нам многоголовое самовнимание, за исключением того, что запрос Q поступает из другого источника, чем ключи K и значения V . Такое внимание также известно в литературе как *перекрестное внимание* (cross-attention). Стоит запомнить, что в примере с машинным переводом наша целевая последовательность или запрос Q поступает из декодера, в то время как энкодер действует как база данных и предоставляет ключи K и значения V . Интуитивный подход, лежащий в основе слоя внимания «энкодер — декодер», заключается в соединении входных и выходных предложений. Таким образом, внимание «энкодер — декодер» обучается связывать входное предложение с соответствующим выходным словом, определяя, насколько каждое целевое слово связано с входными словами.

И это все! Теперь мы готовы к знакомству с классификатором на основе трансформера, реализованным с нуля. Вам стоит попробовать запустить этот код в режиме блокнота на платформе Google Colab (доступна по ссылке <https://colab.research.google.com/>), чтобы посмотреть на результаты пошагового исполнения, а также ускорить процесс обучения модели при помощи графического процессора.

Листинг 11.3. Трансформер для текстовой классификации

```
import numpy as np
import pandas as pd

import tensorflow as tf
from tensorflow import keras
```

302 Глава 11. Передовые алгоритмы глубокого обучения

```
from keras.models import Model, Sequential
from keras.layers import Layer, Dense, Dropout, Activation
from keras.layers import LayerNormalization, MultiHeadAttention
from keras.layers import Input, Embedding, GlobalAveragePooling1D

from keras import regularizers
from keras.preprocessing import sequence
from keras.utils import np_utils

from keras.callbacks import ModelCheckpoint
from keras.callbacks import TensorBoard
from keras.callbacks import LearningRateScheduler
from keras.callbacks import EarlyStopping

import matplotlib.pyplot as plt

tf.keras.utils.set_random_seed(42)

SAVE_PATH = "/content/drive/MyDrive/Colab Notebooks/data/"

def scheduler(epoch, lr):  ← Изменение скорости обучения
    if epoch < 4:
        return lr
    else:
        return lr * tf.math.exp(-0.1)

# Загрузка набора данных
max_words = 20000  ← Топ-20 тысяч самых частотных слов
seq_len = 200      ← Первые 200 слов каждой рецензии на фильм
(x_train, y_train), (x_val, y_val) =
    ↪ keras.datasets.imdb.load_data(num_words=max_words)

x_train = keras.utils.pad_sequences(x_train, maxlen=seq_len)
x_val = keras.utils.pad_sequences(x_val, maxlen=seq_len)

class TransformerBlock(Layer):
    def __init__(self, embed_dim, num_heads, ff_dim, rate=0.1):
        super(TransformerBlock, self).__init__()
        self.att = MultiHeadAttention(num_heads=num_heads, key_dim=embed_dim)
        self.ffn = Sequential(
            [Dense(ff_dim, activation="relu"), Dense(embed_dim)]
        )
        self.layernorm1 = LayerNormalization(epsilon=1e-6)
        self.layernorm2 = LayerNormalization(epsilon=1e-6)
        self.dropout1 = Dropout(rate)
        self.dropout2 = Dropout(rate)

    def call(self, inputs, training):
        attn_output = self.att(inputs, inputs)
        attn_output = self.dropout1(attn_output, training=training)
        out1 = self.layernorm1(inputs + attn_output)
        ffn_output = self.ffn(out1)
        ffn_output = self.dropout2(ffn_output, training=training)
        return self.layernorm2(out1 + ffn_output)
```

```

class TokenAndPositionEmbedding(Layer):
    def __init__(self, maxlen, vocab_size, embed_dim):
        super(TokenAndPositionEmbedding, self).__init__()
        self.token_emb = Embedding(input_dim=vocab_size,
            ➡ output_dim=embed_dim)
        self.pos_emb = Embedding(input_dim=maxlen, output_dim=embed_dim)

    def call(self, x):
        maxlen = tf.shape(x)[-1]
        positions = tf.range(start=0, limit=maxlen, delta=1)
        positions = self.pos_emb(positions)
        x = self.token_emb(x)
        return x + positions

batch_size = 32 | Параметры
num_epochs = 8 | обучения

embed_dim = 32 | Параметры
num_heads = 2 | модели
ffdim = 32

# архитектура трансформера
inputs = Input(shape=(seq_len,))
embedding_layer = TokenAndPositionEmbedding(seq_len, max_words, embed_dim)
x = embedding_layer(inputs)
transformer_block = TransformerBlock(embed_dim, num_heads, ff_dim)
x = transformer_block(x)
x = GlobalAveragePooling1D()(x)
x = Dropout(0.1)(x)
x = Dense(20, activation="relu")(x)
x = Dropout(0.1)(x)
outputs = Dense(2, activation="softmax")(x)

model = Model(inputs=inputs, outputs=outputs)

model.compile(
    loss=keras.losses.SparseCategoricalCrossentropy(),
    optimizer=tf.keras.optimizers.Adam(),
    metrics=["accuracy"]
)

# Определение обратных вызовов
file_name = SAVE_PATH + 'transformer-weights-checkpoint.h5'
#checkpoint = ModelCheckpoint(file_name, monitor='val_loss', verbose=1,
➡ save_best_only=True, mode='min')
reduce_lr = LearningRateScheduler(scheduler, verbose=1)
early_stopping = EarlyStopping(monitor='val_loss', min_delta=0.01,
➡ patience=16, verbose=1)
#tensor_board = TensorBoard(log_dir='./logs', write_graph=True)
callbacks_list = [reduce_lr, early_stopping]

hist = model.fit(x_train, y_train, batch_size=batch_size,
➡ epochs=num_epochs, callbacks=callbacks_list, validation_data=(x_val,
➡ y_val)) ◀ Оценка модели

```

```

test_scores = model.evaluate(x_val, y_val, verbose=2)  ← Обучение модели

print("Test loss:", test_scores[0])
print("Test accuracy:", test_scores[1])

plt.figure()
plt.plot(hist.history['loss'], 'b', lw=2.0, label='Обучение')
plt.plot(hist.history['val_loss'], '--r', lw=2.0, label='Валидация')
plt.title('Трансформерная модель')
plt.xlabel('Эпоха')
plt.ylabel('Потери кросс-энтропии')
plt.legend(loc='upper right')
plt.show()

plt.figure()
plt.plot(hist.history['accuracy'], 'b', lw=2.0, label='Обучение')
plt.plot(hist.history['val_accuracy'], '--r', lw=2.0, label='Валидация')
plt.title('Трансформерная модель')
plt.xlabel('Эпоха')
plt.ylabel('Доля верных результатов')
plt.legend(loc='upper left')
plt.show()

plt.figure()
plt.plot(hist.history['lr'], lw=2.0, label='learning rate')
plt.title('Transformer model')
plt.xlabel('Epochs')
plt.ylabel('Learning Rate')
plt.legend()
plt.show()

```

На рис. 11.12 заметны ранние признаки переобучения. Сохраняя модель с наименьшими валидационными потерями, мы избегаем проблемы переобучения.

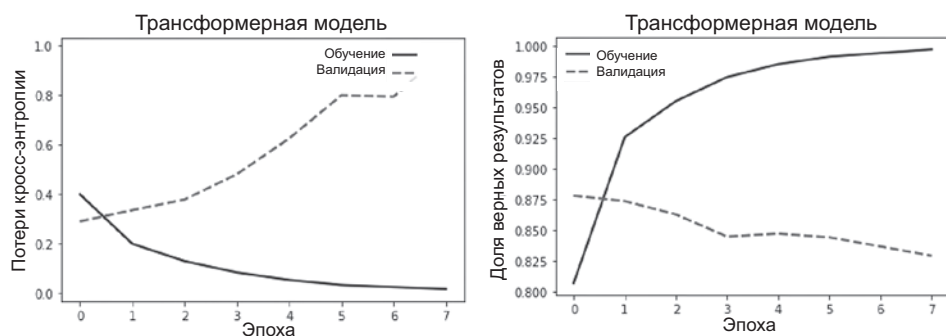


Рис. 11.12. Классификатор на базе трансформера: потери (слева) и доля верных результатов (справа) на обучающем и валидационном наборах данных

В этом разделе мы познакомились с тем, как самовнимание может быть использовано в архитектуре трансформера для классификации тональности

текстов. В следующем разделе мы рассмотрим нейронные сети в применении к графовым данным.

11.4. Графовые нейронные сети

Самые разнообразные виды информации могут быть представлены в виде графов. Среди примеров можно назвать графы знаний, социальные сети, молекулярные структуры и сети цитирования документов. Графовые нейронные сети (ГНС) работают с графами и реляционными данными. В этом разделе мы изучим графовые сверточные сети (ГСС) для классификации вершин в наборе данных сети цитирования CORA. Но сначала давайте рассмотрим основы ГНС.

Пусть $G = (V, E)$ — граф с вершинами V и ребрами E . Чтобы отразить реберную связь, можно построить матрицу смежности A , причем $A_{ij} = 1$, если ребро существует между вершинами i и j , и 0 в противном случае. Для неориентированных графов матрица смежности будет симметричной: $A = A^T$. А еще при обучении ГНС будет полезна матрица признаков вершин X . Если у нас есть N вершин и F признаков на вершину, то размер X равен $N \times F$.

Например, в наборе данных CORA каждая вершина представляет собой документ, а каждое ребро — цитату, которая образует направленное ребро между двумя вершинами. Мы можем зафиксировать отношения цитирования с помощью матрицы смежности A . Поскольку каждый документ представляет собой набор слов, то можно ввести признаки — индикаторы, которые сигнализируют нам, присутствует ли в документе то или иное словарное слово. В этом случае N будет количеством документов, а F — размером словаря. Таким образом, мы можем представить текстовую информацию в каждом документе с помощью двоичной матрицы X размером $N \times F$.

Важно заметить, что ребра также могут иметь свой собственный набор признаков. В этом случае, если размер признаков ребер равен S , а количество ребер равно M , мы можем построить матрицу признаков ребер E размером $M \times S$. Также важно провести различие между классификацией всего графа как целого (например, при классификации молекул) и классификацией вершин внутри графа (например, как в сети цитирования CORA). В первом случае мы имеем дело с *классификацией в пакетном режиме* (batch mode classification), а во втором — с *последовательной классификацией* (single mode classification).

Интересно провести параллель между ГСС и СНС. Изображения также можно рассматривать в виде графов, хотя и с регулярной структурой. Например, вершины могут обозначать пиксели, признаки вершин могут представлять значения пикселей, а признаки ребер — евклидово расстояние между всеми пикселями графа. В этом свете ГСС можно рассматривать как обобщение СНС, поскольку они работают на произвольно связных графах.

Мы можем рассматривать распространение информации в спектральной ГСС как распространение сигнала вдоль вершин. Спектральная ГСС использует разложение по собственным векторам матрицы Лапласа графа для распространения сигнала с помощью следующего ключевого уравнения прямого распространения:

$$X' = D^{-\frac{1}{2}}(A + I)D^{-\frac{1}{2}}XW + b. \quad (11.18)$$

Давайте подробно разберем это уравнение. Если мы возьмем матрицу смежности A и умножим ее на матрицу признаков X , то произведение AX будет представлять собой сумму признаков соседних вершин. Однако суммирование производится по всем соседям, за исключением самой вершины. Чтобы исправить это, мы добавляем петлю в матрицу смежности, складывая ее с единичной матрицей. Мы получаем $(A + I)X$, но нам не хватает нормировки. Чтобы ввести ее, мы добавляем деление на степени всех вершин. Таким образом, мы формируем диагональную степенную матрицу D и умножаем наше выражение на D в степени $-1/2$ с левой и правой стороны. Затем мы производим знакомые нам операции: добавляем умножение на обучаемую матрицу весов W и слагаемое смещения b . Мы добавляем поверх этого нелинейность — и вуаля! Уравнение прямого распространения ГСС готово.

Теперь у нас есть все составляющие для реализации графовой нейронной сети с использованием библиотеки Spektral Keras/Tensorflow. В листинге 11.4 мы начинаем с импорта набора данных, который можно найти в папке данных репозитория кода на GitHub. Мы подготавливаем данные к обработке и определяем архитектуру графовой нейронной сети. Далее мы обучаем модель и отображаем результаты. Вам стоит попробовать запустить этот код в режиме блокнота на платформе Google Colab (доступна по ссылке <https://colab.research.google.com/>), чтобы посмотреть на результаты пошагового исполнения, а также ускорить процесс обучения модели при помощи графического процессора.

Листинг 11.4. Графовая сверточная нейронная сеть для классификации графов цитирования

```
import numpy as np
import pandas as pd

import tensorflow as tf
from tensorflow import keras

import networkx as nx
from tensorflow.keras.utils import to_categorical
from sklearn.preprocessing import LabelEncoder
from sklearn.utils import shuffle
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split

from spektral.layers import GCNConv
```



```

from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dropout, Dense
from tensorflow.keras import Sequential
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import TensorBoard, EarlyStopping
from tensorflow.keras.regularizers import l2

import os
from collections import Counter
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

tf.keras.utils.set_random_seed(42)

SAVE_PATH = "/content/drive/MyDrive/Colab Notebooks/data/"
DATA_PATH = "/content/drive/MyDrive/data/cora/"

column_names = ["paper_id"] + [f"term_{idx}" for idx in range(1433)] +
    ➤ ["subject"]
node_df = pd.read_csv(DATA_PATH + "cora.content", sep="\t", header=None,
    ➤ names=column_names)
print("Node df shape:", node_df.shape)

edge_df = pd.read_csv(DATA_PATH + "cora.cites", sep="\t", header=None,
    ➤ names=["target", "source"])
print("Edge df shape:", edge_df.shape)

nodes = nodedf.iloc[:,0].tolist()
labels = node_df.iloc[:,-1].tolist()
X = node_df.iloc[:,1:-1].values

```

Парсинг данных
вершин

```

X = np.array(X, dtype=int)
N = X.shape[0] # количество вершин
F = X.shape[1] # размер признаков вершин

edge_list = [(x, y) for x, y in zip(edge_df['target'],
    ➤ edge_df['source'])]

```

Парсинг данных ребер

```

num_classes = len(set(labels))

print('Number of nodes:', N)
print('Number of features of each node:', F)
print('Labels:', set(labels))
print('Number of classes:', num_classes)

def sample_data(labels, limit=20, val_num=500, test_num=1000):
    label_counter = dict((l, 0) for l in labels)
    train_idx = []

    for i in range(len(labels)):
        label = labels[i]
        if label_counter[label]<limit:
            # добавление примера к обучающим данным
            train_idx.append(i)
            label_counter[label]+=1

```

308 Глава 11. Передовые алгоритмы глубокого обучения

```
# выход из цикла при нахождении 20 примеров каждого класса
if all(count == limit for count in label_counter.values()):
    break

# получение индексов, не попадающих в обучающие данные
rest_idx = [x for x in range(len(labels)) if x not in train_idx]
# получение val_num первых индексов
val_idx = rest_idx[:val_num]
test_idx = rest_idx[val_num:(val_num+test_num)]
return train_idx, val_idx, test_idx

train_idx, val_idx, test_idx = sample_data(labels)

# назначение маски
train_mask = np.zeros((N,), dtype=bool)
train_mask[train_idx] = True

val_mask = np.zeros((N,), dtype=bool)
val_mask[val_idx] = True

test_mask = np.zeros((N,), dtype=bool)
test_mask[test_idx] = True

print("Training data distribution:\n{}".format(Counter([labels[i] for i in
    ➤ train_idx])))
print("Validation data distribution:\n{}".format(Counter([labels[i] for i
    ➤ in val_idx])))

def encode_label(labels):
    label_encoder = LabelEncoder()
    labels = label_encoder.fit_transform(labels)
    labels = to_categorical(labels)
    return labels, label_encoder.classes_

labels_encoded, classes = encode_label(labels)

G = nx.Graph()
G.add_nodes_from(nodes)
G.add_edges_from(edge_list)
# Построение графа

A = nx.adjacency_matrix(G)
# ← Получение матрицы смежности

print('Graph info: ', nx.info(G))

# Параметры
channels = 16
# ← Количество каналов в первом слое
dropout = 0.5
# ← Коэффициент прореживания признаков
l2_reg = 5e-4
# ← Коэффициент регуляризации L2
learning_rate = 1e-2
# ← Скорость обучения
epochs = 200
# ← Количество эпох обучения
es_patience = 10
# ← Параметр терпения для раннего останова

# Операции предварительной обработки
```

```

A = GCNConv.preprocess(A).astype('f4')

# Определение модели
X_in = Input(shape=(F, ))
fltr_in = Input((N, ), sparse=True)

dropout_1 = Dropout(dropout)(X_in)
graph_conv_1 = GCNConv(channels,
                        activation='relu',
                        kernel_regularizer=l2(l2_reg),
                        use_bias=False)([dropout_1, fltr_in])

dropout_2 = Dropout(dropout)(graph_conv_1)
graph_conv_2 = GCNConv(num_classes,
                        activation='softmax',
                        use_bias=False)([dropout_2, fltr_in])

# Построение модели
model = Model(inputs=[X_in, fltr_in], outputs=graph_conv_2)
model.compile(optimizer=Adam(learning_rate=learning_rate),
              loss='categorical_crossentropy',
              weighted_metrics=['accuracy'])
model.summary()

# Обучение модели
validation_data = ([X, A], labels_encoded, val_mask)
hist = model.fit([X, A],
                 labels_encoded,
                 sample_weight=train_mask,
                 epochs=epochs,
                 batch_size=N,
                 validation_data=validation_data,
                 shuffle=False,
                 callbacks=[
                     EarlyStopping(patience=es_patience, restore_best_weights=True)
                 ])  ← Обучение модели

# Оценка модели
X_test = X
A_test = A
y_test = labels_encoded

y_pred = model.predict([X_test, A_test], batch_size=N)  ← Оценка модели
report = classification_report(np.argmax(y_test,axis=1),
                               np.argmax(y_pred,axis=1), target_names=classes)
print('GCN Classification Report: \n {}'.format(report))

layer_outputs = [layer.output for layer in model.layers]
activation_model = Model(inputs=model.input, outputs=layer_outputs)
activations = activation_model.predict([X,A],batch_size=N)

# Получение результатов t-SNE
# получение представления скрытого слоя после первого слоя ГСС

```

```

x_tsne = TSNE(n_components=2).fit_transform(activations[3])

def plot_tsne(labels_encoded, x_tsne):
    color_map = np.argmax(labels_encoded, axis=1)
    plt.figure(figsize=(10,10))
    for c1 in range(num_classes):
        indices = np.where(color_map==c1)
        indices = indices[0]
        plt.scatter(x_tsne[indices,0], x_tsne[indices, 1], label=c1)
    plt.legend()
    plt.show()

plot_tsne(labels_encoded, x_tsne)

plt.figure()
plt.plot(hist.history['loss'], 'b', lw=2.0, label='Обучение')
plt.plot(hist.history['val_loss'], '--r', lw=2.0, label='Валидация')
plt.title('ГНС-модель')
plt.xlabel('Эпоха')
plt.ylabel('Потери кросс-энтропии')
plt.legend(loc='upper right')
plt.show()

plt.figure()
plt.plot(hist.history['accuracy'], 'b', lw=2.0, label='Обучение')
plt.plot(hist.history['val_accuracy'], '--r', lw=2.0, label='Валидация')
plt.title('ГНС-модель')
plt.xlabel('Эпоха')
plt.ylabel('Доля верных результатов')
plt.legend(loc='upper left')
plt.show()

```

На рис. 11.13 показаны потери классификации и верность результатов для обучающего и валидационного наборов данных:

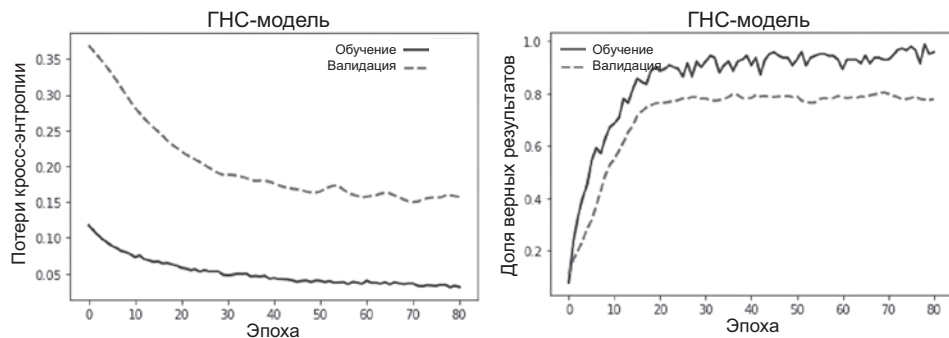


Рис. 11.13. Потери и доля верных результатов ГНС-модели для обучающего и валидационного наборов данных

На рис. 11.14 показано представление скрытого слоя ГСС, визуализированное с помощью метода t-SNE, для набора данных CORA:

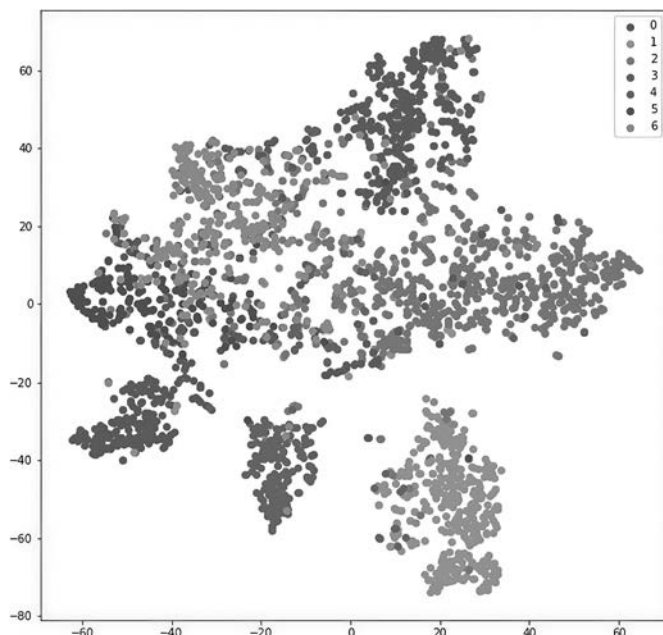


Рис. 11.14. t-SNE-визуализация представления скрытого слоя после первого слоя ГСС для набора данных CORA

В следующем разделе мы сделаем обзор исследований по глубокому обучению в сферах компьютерного зрения и обработки естественного языка.

11.5. Исследования в области ML: глубокое обучение

В компьютерном зрении (computer vision, CV) архитектура СНС прошла путь развития от операций свертки, ReLU и объединения по максимуму архитектуры LeNet — см. работу Яна Лекуна (Yann LeCun) и др. «Gradient-based Learning Applied to Document Recognition» (Proceedings of the IEEE, 1998) — до inception-модулей сети GoogLeNet — см. работу Кристиана Середи (Christian Szegedy) и др. «Going Deeper with Convolutions» (Computer Vision and Pattern Recognition, 2015). Модуль inception создает переменные поля восприятия, применяя ядра разного размера. Это, в свою очередь, позволяет фиксировать разреженные корреляционные закономерности в новом наборе

карт признаков. GoogLeNet обеспечивает высокое качество результатов на наборе данных ImageNet с меньшим количеством параметров, чем AlexNet, — см. работу Алекса Крижевского (Alex Krizhevsky), Ильи Суцкевера (Ilya Sutskever) и Джеффри Хинтона (Geoffrey E. Hinton) «ImageNet Classification with Deep Convolutional Neural Networks» (Conference on Neural Information Processing Systems, 2012) — или VGG — см. работу Карена Симоняна (Karen Simonyan) и Эндрю Зиссермана (Andrew Zisserman) «Very Deep Convolutional Networks for Large-Scale Image Recognition» (International Conference on Learning Representations, 2015). Мы познакомились с механизмом остаточных связей в модели ResNet, которая стала стандартным архитектурным решением для современных нейронных сетей произвольной глубины — см. работу Кайминга Хе (Kaiming He) и др. «Deep Residual Learning for Image Recognition» (Conference on Computer Vision and Pattern Recognition, 2015). Переходя к визуальному трансформеру (vision transformer, ViT), эта модель для обработки фрагментов изображения задействует архитектуру, подобную трансформерной, — см. работу Алексея Досовицкого (Alexey Dosovitskiy) и др. «An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale» (International Conference on Learning Representations, 2021). Изображение делится на фрагменты фиксированного размера, каждый из которых проходит линейное и позиционное кодирование, и результирующая последовательность векторов подается на стандартный трансформерный энкодер, а затем на голову МСП для классификации.

В области генеративных моделей генеративно-сопоставительные сети (generative adversarial network, GAN), представленные Яном Гудфеллоу (Ian Goodfellow) и соавторами в работе «Generative Adversarial Networks» (Conference and Workshop on Neural Information Processing Systems, 2014), основаны на идеях теории игр и используют следующие две сети, обученные для аппроксимации распределения данных: генератор для генерации изображений и дискриминатор для распознавания реальных и поддельных изображений. С другой стороны, генеративные модели на основе правдоподобия моделируют распределение данных, используя соответствующую функцию правдоподобия. Наиболее популярным подклассом генеративных моделей на основе правдоподобия является вариационный автоэнкодер (VAE), представленный Дидериком П. Кингмой (Diederik P. Kingma) и Максом Веллингмом (Max Welling) в работе «Auto-Encoding Variational Bayes» (International Conference on Learning Representations, 2014). VAE состоит из энкодера, преобразующего входные данные в скрытое представление, и декодера, восстанавливающего на выходе данные из скрытого состояния. beta-VAE — это модификация VAE, в которой особое внимание уделяется выявлению разделенных (disentangled) скрытых факторов, см. работу Ирины Хиггинс (Irina Higgins) и др. «beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework» (International Conference on Learning Representations, 2017).

Переключим свое внимание на диффузионные модели, которые приобрели популярность благодаря своей способности генерировать изображения высокого разрешения. Они представляют собой параметризованные цепи Маркова, которые проходят обучение с использованием вариационного вывода для получения выборок, соответствующих входным данным. Диффузионный процесс состоит из двух проходов: прямого, в котором изображение становится все более зашумленным благодаря добавлению гауссовского шума, и обратного, преобразующего шум обратно в пример из целевого распределения, см. статью Джонатана Хо (Jonathan Ho), Аджая Джайна (Ajay Jain) и Питера Эббела (Pieter Abbeel) «Denoising Diffusion Probabilistic Models» (Conference and Workshop on Neural Information Processing Systems, 2020).

В области обработки естественного языка (natural language processing, NLP), к примеру, архитектура машинного перевода прошла путь от моделей Seq2Seq LSTM до моделей на основе трансформера, таких как BERT, — см. статью Джейкоба Девлина (Jacob Devlin) и др. «BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding» (North American Chapter of the Association for Computational Linguistics, 2019). Достаточно к предобученной модели BERT пристыковать всего лишь один дополнительный выходной слой, чтобы получить наивысшую модель для решения широкого спектра задач, таких как ответы на вопросы и логический вывод (language inference), без необходимости существенных архитектурных изменений, специфичных для конкретной задачи. Вместе с большими трансформерными моделями, такими как GPT-3, приобрело популярность обучение с нулевым (zero-shot learning) и небольшим (few-shot learning) количеством примеров, представленное Томом Б. Брауном (Tom B. Brown) и др. в статье «Language Models are Few-Shot Learners» (Conference and Workshop on Neural Information Processing Systems, 2020). Области применения GPT-3 включают такие задачи, как завершение текста (text completion) и завершение кода (code completion). Например, берущая начало из серии GPT-3 модель OpenAI Codex, обученная как на текстах естественного языка, так и на миллиардах строк кода, выполняет функции ИИ программиста-напарника, который может превращать комментарии в код и, исходя из контекста, завершать следующую строку или функцию кода.

В ряде задач ML требуется обработка графовых данных, содержащих информацию, богатую взаимосвязями, — от моделирования социальных сетей до предсказания структуры белка. Это говорит о потребности в моделях, которые могут работать с такими данными. Графовые нейронные сети (ГНС) — это нейронные модели, которые отражают отношения зависимости посредством передачи сообщений между вершинами графа, — см. работу Томаса Н. Кипфа (Thomas N. Kipf) и Макса Уэллинга (Max Welling) «Semi-Supervised Classification with Graph Convolutional Networks» (International Conference on Learning Representations, 2017). В последние годы такие разновидности ГНС, как графовые сверточные

сети (ГСС), графовые сети внимания (graph attention network, GAT) и графовые рекуррентные сети (graph recurrent networks, GRN), совершили прорыв в производительности на многих задачах глубокого обучения.

И наконец, амортизированный вариационный вывод — интересная область исследований, поскольку она сочетает в себе выразительную силу и обучение представлениям глубоких нейронных сетей со знанием предметной области вероятностных графовых моделей, — см. диссертацию Д. П. Кингмы (D. P. Kingma) «Variational Inference and Deep Learning: A New Synthesis» (University of Amsterdam, 2017). Мы познакомились с примером применения MDN, когда использовали нейронную сеть для отображения пространства наблюдений на параметры приближенного апостериорного распределения.

Глубокое обучение — очень активно развивающаяся область исследований. Для знакомства с самыми современными разработками и сферами применения рекомендуется изучить публикации Conference and Workshop on Neural Information Processing Systems, International Conference on Learning Representations и других исследовательских конференций, упомянутых в приложении А.

11.6. Упражнения

11.1. Что такое поле восприятия (receptive field) в СНС?

11.2. Объясните преимущества остаточных связей путем вывода формулы обратного прохода (backward pass).

11.3. Сравните плюсы и минусы использования разных типов нейросетей: СНС, РНС и трансформера.

11.4. Приведите пример нейронной сети, использующей амортизированный вариационный вывод.

11.5. Покажите на примере интуитивную основу прямого уравнения ГСС: $D^{-1/2}(A+I)D^{-1/2}XW + b$.

Итоги

- Автоэнкодеры — это нейронные сети в режиме без учителя, обучаемые восстанавливать входные данные. Слой «узкого места» автоэнкодера может быть использован для уменьшения размерности или выделения признаков.
- Вариационный автоэнкодер состоит из энкодера, преобразующего входные данные в скрытое представление, и декодера, который принимает скрытое представление и возвращает восстановленные данные.
- Амортизированный вариационный вывод реализует идею, что вместо оптимизации набора свободных параметров можно ввести параметризованную

функцию, которая отображает пространство наблюдений на параметры приближенного апостериорного распределения.

- Сети смешанной плотности — это модели смеси, в которых параметры, такие как средние значения, ковариации и пропорции смеси, получаются при обучении нейронной сети.
- Внимание позволяет модели адаптивно фокусироваться на различных частях входных данных, регулируя весовые коэффициенты внимания.
- Трансформер — это модель Seq2Seq, которая использует механизм внимания как в энкодере, так и в декодере. Самовнимание — ключевой компонент архитектуры трансформера.
- Графовые нейронные сети — это нейронные модели, которые фиксируют отношения зависимости посредством передачи сообщений между вершинами графа.

Дополнительные материалы и веб-ресурсы

А.1. Спортивное программирование

В деле освоения алгоритмов читателю окажет помощь целый ряд отличных книг и веб-ресурсов. Я настоятельно рекомендую книгу Стивена Халима (Steven Halim) «Competitive Programming» в дополнение к классическим «Algorithm Design Manual»¹ Стивена Скиены (Steven Skiena) и «Introduction to Algorithms» Томаса Х. Кормена (Thomas H. Cormen) и др.

Также существует множество прекрасных веб-сайтов с задачами по программированию, некоторые из них перечислены ниже:

LeetCode: <https://leetcode.com>

TopCoder: <https://www.topcoder.com>

CodeForces: <https://codeforces.com>

HackerRank: <https://www.hackerrank.com>

GeeksForGeeks: <https://www.geeksforgeeks.org>

uVAOnlineJudge: <https://onlinejudge.org>

Я надеюсь, эти ресурсы будут способствовать вашему профессиональному росту в качестве программистов.

А.2. Рекомендуемая литература

Овладение навыками машинного обучения требует глубокого понимания базовых концепций. Новые алгоритмы ML разрабатываются на основе как фундаментальных принципов, так и сочетания новых тенденций с классическими

¹ Скиена С. «Алгоритмы. Руководство по разработке».

достижениями отрасли. В этом разделе представлены некоторые из ключевых книг по машинному обучению, обязательные к прочтению каждым, кто стремится совершенствоваться в этой области. Среди книг, показанных на рис. А.1, есть как теоретические, так и прикладные. Они охватывают такие темы, как статистика, машинное обучение, оптимизация, теория информации, алгоритмы и структуры данных.



Рис. А.1. Рекомендуемая литература

Для совершенствования навыков в области машинного обучения я крайне рекомендую ознакомиться со следующими книгами, обложки которых представлены на рис. А.1:

- Кевин Мэрфи (Kevin Murphy), «Machine Learning: A Probabilistic Perspective» (MIT Press, 2012).
- Кристофер Бишоп (Christopher Bishop), «Pattern Recognition and Machine Learning»¹ (Springer, 2011).
- Дэвид Маккей (David MacKay), «Information Theory, Inference, and Learning Algorithms» (Cambridge University Press, 2003).
- Стивен Скиена (Steven Skiena), «Algorithm Design Manual» (Springer, 2011)²
- Томас Кормен (Thomas Cormen), Чарльз Лейзерсон (Charles Leiserson), Рональд Ривест (Ronald Rivest) и Клиффорд Штайн (Clifford Stein), «Introduction to Algorithms» (MIT Press, 2009)³.

¹ Бишоп К. М. «Распознавание образов и машинное обучение».

² Скиена С. С. «Алгоритмы. Руководство по разработке».

³ Кормен Т. Х., Лейзерсон Ч. И., Ривест Р. Л., Штайн К. «Алгоритмы: построение и анализ».

- Стивен Халим (Steven Halim), «Competitive Programming» (lulu, 2013)¹.
- Франсуа Шолле (François Chollet), «Deep Learning with Python» (Manning Publications, 2017)².
- Тревор Хасты (Trevor Hastie), Роберт Тибширани (Robert Tibshirani) и Джером Фридман (Jerome Friedman), «The Elements of Statistical Learning: Data Mining, Inference, and Prediction»³ (Springer, 2016).
- Стивен Бойд (Stephen Boyd) и Ливен Ванденберг (Lieven Vandenberghe), «Convex Optimization» (Cambridge University Press, 2004).
- Томас Кавер (Thomas Cover) и Джой А. Томас (Joy A. Thomas), «Elements of Information Theory» (Wiley, 1991).

А.3. Научно-исследовательские конференции

Машинное обучение — динамично развивающаяся область, и лучший способ быть в курсе новейших исследований — знакомиться с материалами научных конференций. В этом разделе представлена краткая информация о ведущих конференциях в области машинного обучения, компьютерного зрения, обработки естественного языка и теоретической информатики.

А.3.1. Машинное обучение

NeurIPS — Neural Information Processing Systems: <https://nips.cc/>

ICLR — International Conference on Learning Representations: <https://iclr.cc/>

ICML — International Conference on Machine Learning: <https://icml.cc/>

AISTATS — Artificial Intelligence and Statistics: <https://www.aistats.org/>

UAI — Uncertainty in Artificial Intelligence: <https://www.auai.org/>

А.3.2. Компьютерное зрение

CVPR — Computer Vision and Pattern Recognition

ICCV — International Conference on Computer Vision

ECCV — European Conference on Computer Vision

А.3.3. Обработка естественного языка

EMNLP — Empirical Methods in Natural Language Processing

NAACL — North American chapter of the ACL

¹ Халим С., Халим Ф. «Спортивное программирование».

² Шолле Ф. «Глубокое обучение на Python». СПб., издательство «Питер».

³ Хасты Т., Тибширани Р., Фридман Д. «Основы статистического обучения. Интеллектуальный анализ данных, логический вывод и прогнозирование».

А.3.4. Теоретическая информатика

STOC — ACM Symposium on Theory of Computing: <http://acm-stoc.org/>

FOCS — IEEE Symposium on Foundations of Computer Science: <http://ieee-focs.org/>

Чтобы быть экспертом в своей области, важно быть в курсе результатов новейших исследований. Все материалы доступны в режиме онлайн на соответствующих официальных сайтах, а также на сайте arxiv.org. Краткую информацию о конференциях по искусственному интеллекту и сроки их проведения можно найти на сайте <https://aideadlin.es/?sub=ML,CV,NLP,RO,SP>. И наконец, воспроизвести новейшие результаты машинного обучения можно, если обратиться к ресурсу <https://paperswithcode.com/>.

Б

Ответы на упражнения

2.1. Выведите условные распределения вероятностей $p(x_A | x_B)$ для многомерного распределения Гаусса, где A и B — подмножества совместно распределенных нормальных случайных величин $x_1, x_2, \dots, x_n$.

Давайте разобьем наш вектор x на два подмножества, x_A и x_B . Тогда матрицу средних и ковариационную матрицу можно переписать в блочном виде:

$$\mu = \begin{pmatrix} \mu_A \\ \mu_B \end{pmatrix};$$
$$\Sigma = \begin{pmatrix} \Sigma_{AA} & \Sigma_{AB} \\ \Sigma_{BA} & \Sigma_{BB} \end{pmatrix}.$$

Мы получаем следующие формулы для вычисления условных распределений:

$$p(x_A | x_B) = N(x_A | \mu_A + \Sigma_{AB} \Sigma_{BB}^{-1} (x_B - \mu_B), \Sigma_{AA} - \Sigma_{AB} \Sigma_{BB}^{-1} \Sigma_{BA});$$
$$p(x_B | x_A) = N(x_B | \mu_B + \Sigma_{BA} \Sigma_{AA}^{-1} (x_A - \mu_A), \Sigma_{BB} - \Sigma_{BA} \Sigma_{AA}^{-1} \Sigma_{AB}).$$

2.2. Выведите маргинальные распределения $p(x_A)$ и $p(x_B)$ для многомерного нормального распределения, где A и B — подмножества совместно распределенных нормальных случайных величин $x_1, x_2, \dots, x_n$.

Давайте разобьем наш вектор x на два подмножества, x_A и x_B . Тогда матрицу средних и ковариационную матрицу можно переписать в блочном виде:

$$\mu = \begin{pmatrix} \mu_A \\ \mu_B \end{pmatrix}$$
$$\Sigma = \begin{pmatrix} \Sigma_{AA} & \Sigma_{AB} \\ \Sigma_{BA} & \Sigma_{BB} \end{pmatrix}.$$

Чтобы вычислить маргинальные распределения, просто возьмем соответствующие строки и столбцы из матрицы средних значений и ковариационной матрицы:

$$\begin{aligned} p(x_A) &= \int N(x | \mu, \Sigma) dx_B = N(x_A | \mu_A, \Sigma_{AA}); \\ p(x_B) &= \int N(x | \mu, \Sigma) dx_A = N(x_B | \mu_B, \Sigma_{BB}). \end{aligned}$$

2.3. Пусть $y \sim N(\mu, \Sigma)$, где $\Sigma = LL^T$. Покажите, что выборку y можно получить следующим образом: $x \sim N(0, I)$; $y = Lx + \mu$.

Пусть $y = Lx + \mu$. Тогда $E[y] = LE[x] + \mu = 0 + \mu = \mu$. Аналогично $\text{cov}(y) = L \text{cov}(x) L^T + 0 = LL^T = \Sigma$. Поскольку y является аффинным преобразованием гауссовой СВ, y также нормально распределена: $y \sim N(\mu, \Sigma)$.

3.1. Получите выражение для KL-дивергенции между двумя одномерными гауссианами: $p(x) \sim N(\mu_1, \sigma_1^2)$ и $q(x) \sim N(\mu_2, \sigma_2^2)$.

$$\begin{aligned} KL(p \| q) &= -\int p(x) \log q(x) dx + \int p(x) \log p(x) dx = \\ &= \frac{1}{2} \log(2\pi\sigma_2^2) + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} \\ &\quad - \frac{1}{2} (1 + \log 2\pi\sigma_1^2) = \\ &= \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}. \end{aligned}$$

3.2. Выведите $E[X]$, $\text{Var}(X)$ и $H(X)$ для распределения Бернулли.

Распределение Бернулли определяется как $p(x) = p^x(1-p)^{1-x}$, где $x \in \{0, 1\}$. Пользуясь определениями математического ожидания, дисперсии и энтропии, можно записать следующие формулы:

$$\begin{aligned} E[X] &= \sum_{x \in \{0, 1\}} p_x(x) x = p_x(0) \times 0 + p_x(1) \times 1 = p. \\ \text{var}(X) &= E[X^2] - E[X]^2 = \sum_{x \in \{0, 1\}} p_x(x) x^2 - p^2 = \\ &= p_x(0) \times 0^2 + p_x(1) \times 1^2 - p^2 = p(1-p). \\ H(X) &= - \sum_{x \in \{0, 1\}} p_x(x) \log p_x(x) = \\ &= -p_x(0) \log p_x(0) - p_x(1) \log p_x(1) = \\ &= -(1-p) \log(1-p) - p \log p. \end{aligned}$$

322 Приложение Б. Ответы на упражнения

3.3. Выведите формулы для математического ожидания, моды и дисперсии распределения $\text{Beta}(a, b)$.

Бета-распределение имеет носитель на интервале $[0, 1]$ и определено следующим образом:

$$\text{Beta}(x | a, b) = \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} x^{a-1} (1-x)^{b-1}.$$

Для математического ожидания, моды и дисперсии получаем следующие выражения:

$$E[X] = \frac{a}{a+b}; \quad \text{mode}[X] = \frac{a-1}{a+b-2}; \quad \text{VAR}(X) = \frac{ab}{(a+b)^2(a+b+1)}.$$

4.1. Докажите следующее биномиальное тождество: $C(n, k) = C(n-1, k-1) + C(n-1, k)$.

Используем алгебраическое доказательство. Пользуясь определением и раскрывая правую часть равенства, получаем:

$$\begin{aligned} \binom{n-1}{k-1} + \binom{n-1}{k} &= \frac{(n-1)!}{(n-1)!(k-1)!} + \frac{(n-1)!}{(n-k-1)!(k)!} = \\ &= \frac{(n-1)!}{(n-k-1)!(k-1)!} \cdot \left(\frac{1}{n-k} + \frac{1}{k} \right) = \\ &= \frac{n!}{(n-k)! \cdot k!} = \\ &= \binom{n}{k}. \end{aligned}$$

4.2. Выведите неравенство Гиббса: $H(p, q) \geq H(q)$, где $H(p, q) = -\sum p(x) \log q(x)$ — кросс-энтропия, а $H(q) = -\sum q(x) \log q(x)$ — энтропия.

Сначала покажем, что KL-дивергенция неотрицательна:

$$\begin{aligned} -KL(p \| q) &= -\sum_x p(x) \log \frac{p(x)}{q(x)} dx = \sum_x p(x) \log \frac{q(x)}{p(x)} dx \leq \\ &\leq \log \sum_x p(x) \frac{q(x)}{p(x)} dx = \log \sum_x q(x) \leq \\ &\leq \log 1 = 0. \end{aligned}$$

Используя доказанное выше, к неравенству Гиббса можно прийти следующим образом:

$$\begin{aligned} KL(p \| q) &\geq 0; \\ \sum_x p(x) \log p(x) - \sum_x p(x) \log q(x) &\geq 0; \\ -H(p) + H(p, q) &\geq 0; \\ H(p, q) &\geq H(p). \end{aligned}$$

4.3. Используйте неравенство Йенсена с $f(x) = \log(x)$, чтобы доказать неравенство $AM \geq GM$.

Поскольку $f(x) = \log(x)$ является вогнутой функцией, неравенство Йенсена можно записать следующим образом:

$$\begin{aligned} f(E[x]) &\geq E[f(x)]; \\ \log E[x] &\geq E[\log(x)]; \\ \log\left(\frac{x_1 + \dots + x_n}{n}\right) &\geq \frac{\log x_1 + \dots + \log x_n}{n}; \\ \log\left(\frac{x_1 + \dots + x_n}{n}\right) &\geq \log(x_1 \times \dots \times x_n)^{\frac{1}{n}}; \\ AM &\geq GM. \end{aligned}$$

4.4. Докажите, что $I(x; y) = H(x) - H(x|y) = H(y) - H(y|x)$.

Согласно определению взаимной информации, получаем:

$$\begin{aligned} I(X; Y) &= \sum_x \sum_y p(x, y) \log \frac{p(x, y)}{p(x)p(y)} = \\ &= E_{p(x, y)} \left[\log \frac{p(x|y)}{p(x)} \right] = E_{p(x, y)} \left[\log \frac{p(y|x)}{p(y)} \right] = \\ &= H(X) - H(X|Y) = \\ &= H(Y) - H(Y|X). \end{aligned}$$

5.1. Даны точка $y \in R^d$ и гиперплоскость $\theta \cdot x + \theta_0 = 0$. Вычислите евклидово расстояние от точки до гиперплоскости.

Пусть x — точка на гиперплоскости, то есть она удовлетворяет равенству $\theta \cdot x + \theta_0 = 0$. Чтобы найти евклидово расстояние от точки до гиперплоскости,

построим вектор $y - x$ и спроецируем его на единственно возможный вектор, перпендикулярный плоскости. В результате получаем:

$$\begin{aligned} d &= \|proj_{\theta}(y - x)\| = \\ &= \left\| \frac{(y - x) \cdot \theta}{\theta \cdot \theta} \theta \right\| = |y \cdot \theta - x \cdot \theta| \frac{\|\theta\|}{\|\theta\|^2} = \\ &= \frac{|y \cdot \theta - x \cdot \theta|}{\|\theta\|} = \frac{|y \cdot \theta + \theta_0|}{\|\theta\|}. \end{aligned}$$

5.2. Дана прямая задача линейного программирования (ЛП): $\min c^T x$ при условии, что $Ax \leq b, x \geq 0$. Сформулируйте двойственную задачу ЛП.

Каждая переменная в прямой задаче ЛП соответствует ограничению в двойственной задаче, и наоборот. Прямая задача — минимизационная, поэтому двойственная — задача максимизации. Ограничение прямой задачи содержит знак меньше или равно, поэтому переменные двойственной задачи будут сопровождаться знаком больше или равно. Переменные прямой задачи неотрицательны, поэтому ограничение двойственной задачи также будет иметь знак больше или равно. Другими словами, двойственная задача формулируется следующим образом:

$$\begin{aligned} \max b^T w, \\ \text{при условии что } A^T w \geq c \\ w \geq 0. \end{aligned}$$

5.3. Покажите, что использование радиальной базисной функции в качестве ядра эквивалентно вычислению меры сходства между двумя бесконечномерными векторами признаков.

Используя полиномиальную теорему, ядро радиальной базисной функции можно разложить следующим образом. Предполагая, что σ равна 1, получаем:

$$\begin{aligned} k(x, x') &= \exp\left(-\frac{1}{2}\|x - x'\|^2\right) = \\ &= \exp\left(x^T x' - \frac{1}{2}\|x\|^2 - \frac{1}{2}\|x'\|^2\right) = \\ &= \exp(x^T x') \exp\left(-\frac{1}{2}\|x\|^2\right) \exp\left(-\frac{1}{2}\|x'\|^2\right) = \\ &= \sum_{k=0}^{\infty} \frac{(x^T x')^k}{k!} \exp\left(-\frac{1}{2}\|x\|^2\right) \exp\left(-\frac{1}{2}\|x'\|^2\right). \end{aligned}$$

Это означает, что разложение RBF имеет бесконечное число измерений.

5.4. Убедитесь, что график изменения скорости обучения $\eta_k = (\tau_0 + k)^{-\kappa}$ удовлетворяет условиям Роббинса — Монро.

Чтобы показать, что скорость обучения удовлетворяет условиям Роббинса — Монро, нам нужно показать, что следующее верно для неотрицательных τ_0 и κ из $(0,5,1]$:

$$\sum_{k=1}^{\infty} \eta_k = \sum_{k=1}^{\infty} \frac{1}{(\tau_0 + k)^{\kappa}} = \infty;$$

$$\sum_{k=1}^{\infty} \eta_k^2 = \sum_{k=1}^{\infty} \frac{1}{(\tau_0 + k)^{2\kappa}} < \infty.$$

Условия выполняются вследствие сходимости степенных рядов.

5.5. Найдите производную сигмоидной функции $\sigma(a) = [1 + \exp(-a)]^{-1}$.

Производную можно найти следующим образом:

$$\begin{aligned} \frac{d}{dx} \sigma(x) &= \frac{d}{dx} (1 + e^{-x})^{-1} = \\ &= -(1 + e^{-x})^{-2} (-e^{-x}) = \\ &= \frac{e^{-x}}{(1 + e^{-x})^2} = \\ &= \frac{1}{1 + e^{-x}} \frac{e^{-x}}{1 + e^{-x}} = \\ &= \sigma(x)(1 - \sigma(x)). \end{aligned}$$

5.6. Оцените временную и пространственную сложность наивного байесовского алгоритма Бернулли.

Наивный байесовский алгоритм Бернулли описывается следующим псевдокодом:

```

1: Обучение:
2:  $N_c = 0, N_{jc} = 0$ 
3: for  $i = 1, 2, \dots, n$  do
4:    $c = y_i$  // метка класса для  $i$ -го примера
5:    $N_c = N_c + 1$ 
6:   for  $j = 1, \dots, D$  do
7:     if  $x_{ij} = 1$  then
8:        $N_{jc} = N_{jc} + 1$ 
9:   end for
10: end for
```

```

11:  $\hat{\pi}_c = \frac{N_c}{N}$ ,  $\hat{\theta}_{jc} = \frac{N_{jc}}{N}$ 
12: return  $\hat{\pi}_c$ ,  $\hat{\theta}_{jc}$ 
13: Тестирование (для одного тестового документа):
14: for  $c = 1, 2, \dots, C$  do
15:    $\log p[c] = \log \pi_c$ 
16:   for  $j = 1, 2, \dots, D$  do
17:     if  $x_j = 1$  then
18:        $\log p[c] += \log \hat{\theta}_{jc}$ 
19:     else
20:        $\log p[c] += \log(1 - \hat{\theta}_{jc})$ 
21:   end for
22: end for
23:  $c = \arg \max_c \log p[c]$ 
24: return  $c$ 

```

Учитывая, что у нас имеется два вложенных цикла **for**, временная сложность равна $O(ND)$, где N — количество обучающих документов, а D — размер словаря. Временная сложность процесса тестирования составляет $O(TCD)$, где T — количество тестовых документов, C — количество классов, а D — размер словаря. Аналогично, пространственная сложность, то есть размер массивов, необходимых для хранения параметров модели, возрастает как $O(DC)$.

6.1. Оцените временную и пространственную сложность KNN-регрессора.

Следующий псевдокод описывает алгоритм KNN-регрессии:

```

1: class KNN:
2:   function knn_search(K, X, y, Q)
3:   for query in Q:
4:      $\text{idx} = \text{argsort}(\text{euclidean\_dist}(\text{query}, X))[:K]$ 
5:      $\text{knn\_labels} = [y[i] \text{ for } i \text{ in idx}]$ 
6:      $y\_pred = \text{mean}(\text{knn\_labels})$ 
7:   end for
8:   return  $y\_pred$ 

```

В коде присутствует операция сортировки, которая требует $O(N \log N)$ времени, где N — количество точек обучающих данных. Таким образом, временная сложность для каждого запроса q из Q составляет $O(N \log N + K) = O(N \log N)$. KNN-регрессор является непараметрической моделью, следовательно, его пространственная сложность относительно параметров равна $O(1)$.

6.2. Выведите формулы обновления гауссовского процесса (ГП) на основе правил для условного распределения многомерных нормальных случайных величин.

Рассмотрим следующее совместное ГП-распределение:

$$\begin{pmatrix} f \\ f_* \end{pmatrix} \sim \left(\begin{pmatrix} \mu \\ \mu_* \end{pmatrix}, \begin{pmatrix} K & K_* \\ K_*^T & K_{**} \end{pmatrix} \right).$$

Здесь нам надо предсказать выходные данные функции $y^* = f(x^*)$, то есть переменные со звездочкой X^* представляют собой предсказание на тестовых данных, а $K = \kappa(X, X)$, $K^* = \kappa(X, X^*)$ и $K^{**} = \kappa(X^*, X^*)$. Используя правила для условных распределений, мы получаем следующее:

$$\begin{aligned} p(x_A | x_B) &= N(x_A | \mu_A + \Sigma_{AB} \Sigma_{BB}^{-1} (x_B - \mu_B), \Sigma_{AA} - \Sigma_{AB} \Sigma_{BB}^{-1} \Sigma_{BA}); \\ p(x_B | x_A) &= N(x_B | \mu_B + \Sigma_{BA} \Sigma_{AA}^{-1} (x_A - \mu_A), \Sigma_{BB} - \Sigma_{BA} \Sigma_{AA}^{-1} \Sigma_{AB}). \end{aligned}$$

Здесь множество A соответствует X , а множество B — X^* . Мы приходим к уравнениям обновления гауссовского процесса:

$$\begin{aligned} p(f_* | X_*, X, f) &\sim N(f_* | \mu_*, \Sigma_*); \\ \mu_* &= \mu(X_*) + K_*^T K^{-1} (f - \mu(X)); \\ \Sigma_* &= K_{**} - K_*^T K^{-1} K_*. \end{aligned}$$

7.1. Объясните, как работает температурный softmax для различных значений параметра температуры T .

Температурный softmax определяется следующим образом:

$$p_i = \text{softmax}(x, T) = \frac{\exp\left(\frac{x_i}{T}\right)}{\sum_{j=1}^N \exp\left(\frac{x_j}{T}\right)}.$$

При $T = 1$ мы имеем обычную функцию softmax. При движении T в сторону бесконечности результирующее распределение все более становится похожим на равномерное (исходы все более равновероятны). Таким образом, «нагрев» распределения увеличивает его энтропию. По мере приближения T к 0 энтропия итогового распределения уменьшается, тем самым более четко выделяя высоковероятные значения.

7.2. В алгоритме «вперед-назад» СММ сохраните переменную скрытого состояния z (как часть класса НММ) и сравните выведенное значение z с эталонным z .

Чтобы сохранить переменную скрытого состояния z , инициализируйте ее в конструкторе класса НММ при помощи `self.z=np.zeros(self.n)` и в последующем коде замените `z[idx]` на `self.z[idx]`.

8.1. Покажите, что распределение Дирихле $\text{Dir}(\theta | \alpha)$ является сопряженным априорным распределением для мультиномиального правдоподобия, получив формулу для апостериорного распределения. Как изменяется форма апостериорного распределения в зависимости от числа событий?

Представьте, что мы наблюдаем N бросков кубика $D = \{x_1, \dots, x_n\}$, где x_i принимает значения из $\{1, \dots, K\}$. Если предположить, что величины независимы и одинаково распределены, то функция правдоподобия имеет следующий вид:

$$p(\mathcal{D} | \theta) = \prod_{k=1}^K \theta_k^{N_k}.$$

Здесь N_k — это количество раз, когда происходило событие k . Поскольку вектор параметров лежит на K -мерном вероятностном симплексе, нам требуется априорное распределение, которое имеет носитель на этом симплексе. Распределение Дирихле удовлетворяет этому критерию:

$$\text{Dir}(\theta | \alpha) = \frac{1}{B(\alpha)} \prod_{k=1}^K \theta_k^{\alpha_k - 1}.$$

Перемножая правдоподобие и априорное распределение, мы убеждаемся в том, что апостериорное также является распределением Дирихле:

$$\begin{aligned} p(\theta | \mathcal{D}) &\propto p(\mathcal{D} | \theta) p(\theta) \propto \\ &\propto \prod_{k=1}^K \theta_k^{N_k} \theta_k^{\alpha_k - 1} = \prod_{k=1}^K \theta_k^{\alpha_k + N_k - 1} = \\ &= \text{Dir}(\theta | \alpha_1 + N_1, \dots, \alpha_K + N_K). \end{aligned}$$

Видно, что число событий апостериорного распределения получается путем сложения числа событий в априорном распределении и наблюдаемого числа событий. Поскольку апостериорное распределение имеет ту же форму, что и априорное, распределение Дирихле называется сопряженным априорным распределением для мультиномиального правдоподобия. По мере увеличения числа наблюдаемых событий апостериорное распределение Дирихле становится более концентрированным.

8.2. Объясните принцип, лежащий в основе инициализации метода K-means++.

Инициализация K-means++ решает проблему неудачных исходных значений центроидов, приводящих к плохим результатам кластеризации. Основная идея состоит в том, что исходные центроиды кластеров должны находиться далеко друг от друга. Принцип, лежащий в основе инициализации K-means++, заключается в том, чтобы сначала произвольно выбрать положение центроида одного кластера, а затем разыграть центроиды остальных кластеров с вероятностью, пропорциональной расстоянию от существующих центроидов. Такой подход

гарантирует рассредоточение центроидов и представляет собой лучшую стратегию инициализации.

8.3. Докажите свойство циклической перестановки следа: $tr(ABC) = tr(BCA) = tr(CAB)$.

Используя определение следа, мы можем раскрыть произведение следующим образом:

$$tr(ABC) = \sum_i (ABC)_{ii} = \sum_i \sum_j A_{ij} (BC)_{ji} = \sum_i \sum_j \sum_k A_{ij} B_{jk} C_{ki}.$$

Теперь, чтобы доказать свойство перестановки, можно переставить местами матрицы в произведении:

$$tr(ABC) = \sum_i \sum_j \sum_k A_{ij} B_{jk} C_{ki} = \sum_i \sum_j \sum_k B_{jk} C_{ki} A_{ij} = tr(BCA).$$

8.4. Оцените временную сложность алгоритма анализа главных компонент (PCA).

Псевдокод анализа главных компонент выглядит следующим образом:

```

1: class PCA
2: function transform( $X, K$ ):
3:    $\Sigma = \text{covariance\_matrix}(X)$ 
4:    $V, \lambda = \text{eig}(\Sigma)$ 
5:    $\text{idx} = \text{argsort}(\lambda)$ 
6:    $V_{pca} = V[\text{idx}][:K]$ 
7:    $\lambda_{pca} = \lambda[\text{idx}][:K]$ 
8:    $X_{pca} = XV_{pca}$ 
9: return  $X_{pca}$ 
```

Алгоритм PCA состоит из трех вычислительно затратных этапов: вычисления ковариационной матрицы, разложения по собственным значениям и сортировки. Пусть N — количество точек данных, каждая из которых представлена D признаками, а вычисление ковариационной матрицы занимает $O(ND^2)$ в предположении, что $D < N$. Сложность разложения по собственным значениям составляет $O(D^3)$, тогда как операция сортировки по времени занимает $O(D \log D)$. Таким образом, временная сложность алгоритма PCA составляет $O(ND^2 + D^3 + D \log D) = O(ND^2 + D^3)$.

9.1. Покажите, что латентное размещение Дирихле (LDA) можно интерпретировать как неотрицательное матричное разложение.

Вернемся к терм-документной матрице A_{vd} , где V — словарь, а D — количество документов. Поскольку нас интересует определение количества тем K , LDA

можно интерпретировать как задачу матричной факторизации, в которой терм-документная матрица A_{vd} раскладывается на произведение тем W_{vk} и пропорций тем H_{kd} : $A = WH$. Поскольку темы и пропорции тем являются вероятностными распределениями, результирующие матрицы должны быть неотрицательными:

$$\min_{W \geq 0, H \geq 0} \|A - WH\|_F.$$

Таким образом, LDA можно интерпретировать как задачу неотрицательного матричного разложения, которую можно решить с помощью алгоритма чередующихся наименьших квадратов (alternating least squares, ALS).

9.2. Объясните, почему разреженность желательна при выводе структуры общего графа.

Разреженность желательна при выводе структуры графа, поскольку нам хотелось бы, чтобы наши результаты поддавались интерпретации. Полностью связный граф не дает нам существенной информации; поэтому имеет смысл подобрать пороговое значение в методе оценки обратной ковариационной матрицы таким образом, чтобы на выходе получалась разреженная структура графа, из которой можно сделать содержательные выводы.

9.3. Перечислите несколько NP-трудных задач, которые можно аппроксимировать с помощью алгоритма имитации отжига.

Метод имитации отжига используется для решения таких задач, как задача коммивояжера, определение максимального разреза графа, задача нахождения независимого множества и многих других.

9.4. Проведите мозговой штурм по поиску задач, которые можно успешно решить с помощью генетического алгоритма.

Примерами задач, которые могут быть решены с помощью генетических алгоритмов, являются задача о восьми ферзях, задача коммивояжера, проблема отбора признаков, поиск топологии нейронной сети и многие другие.

10.1. Объясните назначение нелинейностей в нейронных сетях.

Нелинейности в нейронных сетях требуются для моделирования нелинейных взаимосвязей между входными и выходными данными отдельного слоя нейросети. Без нелинейностей нейронная сеть будет вести себя как однослойный перцептрон.

10.2. Объясните проблему затухания и взрыва градиента.

Взрыв градиента происходит, когда при обратном распространении градиент становится все больше и больше с каждым слоем. Проблема затухающего

градиента возникает, когда в процессе обратного распространения градиент стремится к нулю.

10.3. Опишите способы увеличить емкость нейронной модели и избежать при этом переобучения.

Чтобы увеличить емкость нейронной модели, можно увеличить размер и количество слоев. Такие изменения могут привести к переобучению. Для борьбы с переобучением можно использовать методики уменьшения весов, прореживания и раннего останова.

10.4. Почему в архитектуре LeNet количество фильтров увеличивается по мере продвижения от входа к выходу?

Количество фильтров увеличивается для того, чтобы улавливать более сложные закономерности в данных изображения. Начальные слои используются для извлечения геометрических признаков, таких как грани, углы и т. п., в то время как последующие слои объединяют эти структуры в более сложные конфигурации.

10.5. Объясните, как работает оптимизатор Adam.

Adam можно рассматривать как вариант комбинации оптимизаторов импульса и RMSProp. Обновление выглядит как RMSProp, за исключением того, что вместо «сырого» стохастического градиента используется его гладкая версия. Полное обновление Adam также включает механизм коррекции смещений:

$$\begin{aligned} g &= \frac{1}{m} \nabla_{\theta} \sum_i L(x^{(i)}; \theta), y(i); \\ m &= \beta_1 m + (1 - \beta_1) g; \\ s &= \beta_2 v + (1 - \beta_2) g^T g; \\ \theta &= \theta - \varepsilon_k \times \frac{m}{\sqrt{s + eps}}. \end{aligned}$$

11.1. Что такое поле восприятия (receptive field) в СНС?

Поле восприятия — это количество входных пикселей, которые отвечают за создание карты признаков. Это мера связи выходного признака (любого слоя) с входной областью.

11.2. Объясните преимущества остаточных связей путем вывода формулы обратного прохода (backward pass).

Преимущество остаточных связей заключается в том, что они позволяют градиентам передаваться по сети напрямую, минуя нелинейные функции активации,

которые приводят к резкому увеличению или затуханию градиентов (в зависимости от весовых коэффициентов).

Пусть $H = F(x) + x$. Тогда мы получаем следующее выражение:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial H} \frac{\partial H}{\partial x} = \frac{\partial L}{\partial H} \left(\frac{\partial F}{\partial x} + 1 \right) = \frac{\partial L}{\partial H} \frac{\partial F}{\partial x} + \frac{\partial L}{\partial H}.$$

11.3. Сравните плюсы и минусы использования разных типов нейросетей: СНС, РНС и трансформера.

СНС легко параллелизуются и, следовательно, высокопроизводительны. Они отлично обучаются и хорошо справляются с выявлением закономерностей в сигналах, но, тем не менее, они игнорируют историю сигналов. РНС хранят внутреннюю модель прошлого, но ограничены в параллельных вычислениях. Трансформеры как модели обладают высокой емкостью, что, однако, может приводить к переобучению.

11.4. Приведите пример нейронной сети, использующей амортизированный вариационный вывод.

В примере, рассматриваемом в этой книге, мы обучали многослойный перцептрон параметрам модели смеси. Таким образом, МСП является одним из примеров нейронной сети, способной распределять затраты вариационного вывода между несколькими точками данных.

11.5. Покажите на примере интуитивную основу прямого уравнения ГСС: $D^{(-1/2)}(A + I)D^{(-1/2)}XW + b$.

В формуле произведение AX представляет собой суммирование признаков соседних вершин в матрице X . После добавления единичной матрицы к A мы получаем суммирование, включающее саму вершину: $(A + I)X$. Чтобы нормализовать это произведение, мы формируем диагональную степенную матрицу D и с двух сторон умножаем наше произведение на D в степени $(-1/2)$. В заключение мы умножаем наше выражение на весовую матрицу W и добавляем слагаемое смещения b .

Вадим Смоляков

Алгоритмы машинного обучения

Перевел с английского Е. Суворкин

Научный редактор В. Бородина

Руководитель дивизиона	<i>Ю. Сергиенко</i>
Руководитель проекта	<i>А. Питиримов</i>
Ведущий редактор	<i>Е. Строганова</i>
Литературный редактор	<i>Ю. Широкина</i>
Художественный редактор	<i>В. Мостипан</i>
Корректоры	<i>Н. Викторова, М. Котова</i>
Верстка	<i>Л. Егорова</i>

Изготовлено в России. Изготовитель: ООО «Прогресс книга». Место нахождения и фактический адрес: 194044, Россия, г. Санкт-Петербург, Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 11.2025. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 — Книги печатные профессиональные, технические и научные.

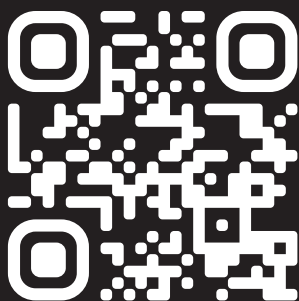
Импортёр в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 17.10.25. Формат 70×100/16. Бумага офсетная. Усл. п. л. 27,090. Тираж 700. Заказ 0000.

Read IT Club

Комьюнити рецензентов и переводчиков ИТ-литературы

Миссия участников клуба – обеспечить высокое качество профессиональной переводной литературы в России. «Книжные дебагеры» проверяют корректность терминологии и подписей на схемах и иллюстрациях, чтобы сделать книги более понятными русскоязычному читателю. Стать участником Read IT Club может любой ИТ-специалист, готовый поделиться опытом с сообществом.



присоединиться к нам

Кришнанду Чаудхури

МАТЕМАТИКА И АРХИТЕКТУРА ГЛУБОКОГО ОБУЧЕНИЯ



Узнайте, что происходит внутри черного ящика! Для использования глубокого обучения вам придется подготовить данные, выбрать правильную модель, обучить ее, оценить качество и точность и предусмотреть обработку неопределенности и изменчивости в выходных данных развернутого решения. Эта книга шаг за шагом знакомит с основными математическими концепциями, которые пригодятся вам как специалисту по данным, — с векторным исчислением, линейной алгеброй и байесовским выводом, представляя их с точки зрения глубокого обучения.

Авторы объясняют математику, теорию и принципы построения моделей глубокого обучения, а затем демонстрируют применение теории на практике, приводя фрагменты программного кода на Python с подробными комментариями. В книге вы пройдете путь от основ алгебры, исчисления и статистики до современных архитектур глубокого обучения, ставших результатом новейших исследований.

КУПИТЬ

Алексей Григорьев

МАШИННОЕ ОБУЧЕНИЕ. ПОРТФОЛИО РЕАЛЬНЫХ ПРОЕКТОВ



Изучите ключевые концепции машинного обучения, работая над реальными проектами! Машинное обучение — то, что поможет вам в анализе поведения клиентов, прогнозировании тенденций движения цен, оценке рисков и многом другом. Чтобы освоить машинное обучение, вам нужны отличные примеры, четкие объяснения и много практики. В книге все это есть!

Автор описывает реалистичные, практические сценарии машинного обучения, а также предельно понятно раскрывает ключевые концепции. Вы разберете интересные проекты, такие как сервис прогнозирования цен на автомобили с использованием линейной регрессии и сервис прогнозирования оттока клиентов. Вы выйдете за рамки алгоритмов и изучите важные техники, например развертывание приложений в бессерверных системах и запуск моделей с помощью Kubernetes и Kubeflow. Пришло время закатать рукава и прокачать свои навыки в области машинного обучения!

КУПИТЬ